# Multivariate Information Theory

A Practical Tutorial with HOI, Frites, and XGI

Giovanni Petri

2025-12-15

# Table of contents

**Welcome** **1**
What You'll Learn . . . 1
Who Is This For? . . . 1
Prerequisites . . . 2
Tutorial Structure . . . 2
Part I: Foundations (Notebooks 1-2) . . . 2
Part II: Advanced Tools (Notebooks 3-5) . . . 2
Part III: Integration (Notebook 6) . . . 2
Key Features . . . 2
Progressive Learning . . . 2
Hands-On Code . . . 3
Real Applications . . . 3
Rich Visualizations . . . 3
Getting Started . . . 3
Quick Start . . . 3
Reading This Book . . . 3
Online Resources . . . 4
Citation . . . 4
License . . . 4
Acknowledgments . . . 4

**I Part I: Foundations** **6**

**1 Notebook 1: Information Theory Foundations** **7**
1.1 Building the Mathematical Groundwork for Understanding Multivariate Information . . . 7
1.1.1 What You'll Learn . . . 7

1.1.2 Why Start From Scratch? . . . 7
1.2 Part 1: Understanding Entropy . . . 8
1.2.1 The Intuition Behind Entropy . . . 8
1.2.2 The Mathematical Definition . . . 8
1.2.3 Why the Negative Sign? . . . 9
1.2.4 Visualizing Entropy as a Function of Probability . . . 10
1.3 Part 2: Joint and Conditional Entropy . . . 11
1.3.1 Joint Entropy: Uncertainty About Multiple Variables 11
1.3.2 Conditional Entropy: Uncertainty After Learning Something . . . 11
1.3.3 A Concrete Example: The Beer Price Problem . . . 12
1.4 Part 3: Mutual Information . . . 14
1.4.1 The Key Measure of Dependence . . . 14
1.4.2 Alternative Formulation: KL Divergence . . . 14
1.4.3 Key Properties . . . 15
1.4.4 Visualizing the Venn Diagram of Information . . . 16
1.5 Part 4: Conditional Mutual Information . . . 18
1.5.1 Beyond Pairwise Dependencies . . . 18
1.5.2 Why This Matters . . . 18
1.5.3 A Surprising Property: CMI Can Increase! . . . 18
1.6 Part 5: Estimating Information from Data . . . 21
1.6.1 From Theory to Practice . . . 21
1.6.2 Three Common Approaches . . . 21
1.6.3 Visualizing the Bias-Variance Trade-off . . . 23
1.7 Part 6: Summary and Bridge to Advanced Methods . . . 24
1.7.1 What We've Built . . . 24
1.7.2 The Foundations Are Set . . . 25
1.7.3 Practice Exercises . . . 25
1.8 Additional Resources . . . 26
1.8.1 Recommended Reading . . . 26
1.8.2 Going Deeper . . . 26
1.8.3 Coming Up . . . 26

**2 Notebook 2: The XOR Problem - When Intuition Breaks 27**
2.1 Discovering Why Two Variables Can Be Useless Alone But Powerful Together . . . 27
2.1.1 The Central Puzzle . . . 27
2.1.2 Learning Objectives . . . 28
2.2 Part 1: The XOR Function . . . 29
2.2.1 What is XOR? . . . 29

2.2.2 Why This is Special . . . . . 29
2.2.3 Let's Verify This . . . . . 30
2.3 Part 2: Calculating the Shocking Result . . . . . 31
2.3.1 Step 1: Individual Mutual Informations . . . . . 31
2.3.2 Understanding Why I(X ; Y) = 0 . . . . . 32
2.3.3 By Symmetry, I(X ; Y) = 0 Too . . . . . 32
2.3.4 Step 2: Joint Mutual Information . . . . . 33
2.3.5 Visualizing the XOR Paradox . . . . . 35
2.4 Part 3: Why the Venn Diagram Breaks . . . . . 37
2.4.1 The Two-Variable Venn Diagram (It Works!) . . . . . 37
2.4.2 The Three-Variable Venn Diagram (It Breaks!) . . . . 37
2.5 Part 4: Interaction Information . . . . . 40
2.5.1 The Solution: A Signed Measure . . . . . 40
2.5.2 Key Property: Can Be Negative! . . . . . 41
2.5.3 For XOR, Interaction Information is Negative . . . . . 41
2.5.4 Interpreting the Sign of Interaction Information . . . . 42
2.5.5 Visualizing Interaction Information Across Structures 44
2.6 Part 5: The Copy Problem (Pure Redundancy) . . . . . 46
2.6.1 The Opposite of XOR . . . . . 46
2.6.2 XOR vs COPY: Side-by-Side Comparison . . . . . 47
2.7 Part 6: The Explaining Away Effect . . . . . 50
2.7.1 When Conditioning Increases Dependence . . . . . 50
2.7.2 The Classic Example: Alarm System . . . . . 50
2.8 Summary and Next Steps . . . . . 52
2.8.1 What We've Discovered . . . . . 52
2.8.2 The Deep Insight . . . . . 53
2.8.3 Why This Matters for Neuroscience . . . . . 53
2.8.4 Coming Up in Notebook 3 . . . . . 53
2.8.5 Practice Problems . . . . . 53
2.9 Further Reading . . . . . 55
2.9.1 Essential Papers . . . . . 55
2.9.2 Advanced Topics . . . . . 55

**II Part II: Advanced Tools 56**

**3 Notebook 3: Higher-Order Interactions with HOI 57**
3.1 From Manual Calculations to Professional Tools . . . . . 57
3.1.1 What is HOI? . . . . . 57
3.1.2 Why This Matters for Your Research . . . . . 58

3.1.3 Learning Objectives . . . . . 58
3.2 Part 1: Installation and Setup . . . . . 58
3.2.1 Installing HOI . . . . . 58
3.2.2 Understanding HOI's Data Format . . . . . 59
3.3 Part 2: O-Information - Detecting Synergy and Redundancy . 62
3.3.1 What is O-Information? . . . . . 62
3.3.2 Interpreting the Sign . . . . . 62
3.3.3 Testing with COPY (Pure Redundancy) . . . . . 65
3.3.4 Visualizing the O-Information Landscape . . . . . 66
3.4 Part 3: Understanding Different Estimators . . . . . 70
3.4.1 Why Estimators Matter . . . . . 70
3.4.2 Available Estimators in HOI . . . . . 70
3.5 Part 4: Partial Information Decomposition (PID) . . . . . 74
3.5.1 From O-Information to Detailed Decomposition . . . . 74
3.5.2 The Four PID Atoms . . . . . 74
3.5.3 Important Note About PID . . . . . 74
3.5.4 Visualizing PID Atoms . . . . . 78
3.6 Part 5: Application to Simulated Neural Data . . . . . 81
3.6.1 Realistic Neural Population Scenario . . . . . 81
3.6.2 Discovering Synergistic Triplets . . . . . 84
3.7 Part 6: Computational Scaling and Practical Considerations . 88
3.7.1 The Combinatorial Explosion . . . . . 88
3.7.2 Strategies for Large Systems . . . . . 88
3.8 Summary and Key Takeaways . . . . . 91
3.8.1 What We've Accomplished . . . . . 91
3.8.2 When to Use Each HOI Measure . . . . . 92
3.8.3 Critical Reminders . . . . . 92
3.8.4 The Bigger Picture . . . . . 92
3.8.5 Bridge to Notebook 4 . . . . . 93
3.9 Practice Exercises . . . . . 93
3.10 Further Resources . . . . . 95
3.10.1 HOI Documentation . . . . . 95
3.10.2 Key Papers on PID . . . . . 95
3.10.3 Related Packages . . . . . 95
3.10.4 Advanced Topics . . . . . 95

**4 Notebook 4: Neural Time Series Analysis with Frites 97**
4.1 From Static Snapshots to Dynamic Information Flow . . . . . 97
4.1.1 Enter Frites . . . . . 97
4.1.2 Why Frites Matters . . . . . 98

4.1.3 Learning Objectives . . . . . 98
4.2 Part 1: Installation and Understanding Frites Data Structure 98
4.2.1 Installation . . . . . 98
4.2.2 Understanding Frites Data Structure: DatasetEphy . . 100
4.2.3 Visualizing the Data Structure . . . . . 105
4.3 Part 2: Time-Resolved Mutual Information . . . . . 108
4.3.1 The Core Workflow: WfMi . . . . . 108
4.3.2 Gaussian Copula MI (GCMI) . . . . . 108
4.3.3 Computing Time-Resolved MI . . . . . 111
4.3.4 Visualizing Time-Resolved MI Results . . . . . 112
4.4 Part 3: Understanding Multiple Comparison Correction . . . 116
4.4.1 The Multiple Comparison Problem . . . . . 116
4.4.2 Correction Methods in Frites . . . . . 116
4.5 Part 4: Dynamic Functional Connectivity . . . . . 121
4.5.1 Beyond Single-Channel Analysis . . . . . 121
4.5.2 Why DFC Matters . . . . . 121
4.5.3 Computing DFC with Frites . . . . . 121
4.5.4 Visualizing Dynamic Connectivity Matrices . . . . . 122
4.6 Part 5: Multi-Subject Analysis - Fixed vs Random Effects . . 124
4.6.1 The Group Analysis Question . . . . . 124
4.6.2 Fixed-Effect (FFX) Analysis . . . . . 125
4.6.3 Random-Effect (RFX) Analysis . . . . . 125
4.7 Part 6: Realistic Application - Visual Attention Experiment . 130
4.7.1 Experimental Paradigm . . . . . 130
4.7.2 Analysis 1: Overall Orientation Information . . . . . 133
4.7.3 Analysis 2: Conditional MI - Attention Modulation . . 136
4.8 Part 7: Practical Considerations and Best Practices . . . . . 140
4.8.1 Sample Size Requirements . . . . . 140
4.9 Summary and Integration . . . . . 143
4.9.1 What We've Learned . . . . . 143
4.9.2 Frites vs HOI: When to Use Each . . . . . 143
4.9.3 Critical Reminders . . . . . 143
4.9.4 Bridge to Notebook 5 . . . . . 144
4.10 Practice Exercises . . . . . 144
4.11 Additional Resources . . . . . 146
4.11.1 Frites Documentation . . . . . 146
4.11.2 Key Papers . . . . . 146
4.11.3 Integration with MNE-Python . . . . . 146
4.11.4 Advanced Topics . . . . . 147
4.11.5 Troubleshooting . . . . . 147

**5 Notebook 5: Hypergraph Networks with XGI 148**
5.1 From Information Measures to Network Topology . . . . . 148
5.1.1 Why Networks? . . . . . 148
5.1.2 The Limitation of Regular Graphs . . . . . 148
5.1.3 Enter XGI . . . . . 149
5.1.4 Learning Objectives . . . . . 149
5.2 Part 1: Installation and First Hypergraph . . . . . 149
5.2.1 Installing XGI . . . . . 149
5.2.2 Creating Your First Hypergraph . . . . . 150
5.2.3 Visualizing Hypergraphs . . . . . 153
5.3 Part 2: Mapping HOI Results to Hypergraphs . . . . . 154
5.3.1 The Key Insight . . . . . 154
5.3.2 Building a Synergy Hypergraph . . . . . 157
5.3.3 Visualizing the Synergy Network . . . . . 158
5.4 Part 3: Hypergraph Statistics and Network Analysis . . . . . 160
5.4.1 Node-Level Statistics . . . . . 160
5.4.2 Edge-Level Statistics . . . . . 164
5.5 Part 4: Temporal Hypergraphs - Tracking Dynamic Structure 167
5.5.1 From Static to Dynamic Networks . . . . . 167
5.5.2 Simulating Time-Varying Synergistic Structure . . . . 169
5.5.3 Visualizing Temporal Evolution . . . . . 171
5.6 Part 5: Advanced Hypergraph Metrics . . . . . 174
5.6.1 Beyond Simple Degree . . . . . 174
5.6.2 Identifying Network Motifs . . . . . 176
5.7 Part 6: Integration with HOI and Frites . . . . . 179
5.7.1 Complete Analysis Pipeline . . . . . 179
5.7.2 Step 4: Network Topology Analysis . . . . . 184
5.8 Part 7: Real-World Application - Email Collaboration Network188
5.8.1 Loading Real Data . . . . . 188
5.9 Summary and Key Takeaways . . . . . 190
5.9.1 What We've Accomplished . . . . . 190
5.9.2 The Complete Toolkit . . . . . 191
5.9.3 Integrated Workflow . . . . . 191
5.9.4 Critical Insights . . . . . 191
5.9.5 Looking Forward . . . . . 192
5.10 Practice Exercises . . . . . 192
5.11 Additional Resources . . . . . 194
5.11.1 XGI Documentation . . . . . 194
5.11.2 Key Papers on Hypergraphs . . . . . 194
5.11.3 Related Concepts . . . . . 195

5.11.4 Advanced XGI Features . . . . . 195
5.11.5 Integration Examples . . . . . 195
5.11.6 Troubleshooting . . . . . 196

**III Part III: Integration 197**

**6 Notebook 6: Complete Integration - The Full Analysis Pipeline 198**
6.1 From Raw Data to Scientific Insight Using HOI, Frites, and XGI 198
6.1.1 The Research Question . . . . . 199
6.1.2 The Experiment . . . . . 199
6.1.3 Learning Objectives . . . . . 199
6.2 Part 1: Setup and Data Generation . . . . . 199
6.2.1 Installing All Required Packages . . . . . 199
6.2.2 Generating Realistic Hierarchical Neural Data . . . . . 201
6.2.3 Visualizing the Simulated Data . . . . . 206
6.3 Part 2: FRITES Analysis - Time-Resolved Information Flow 209
6.3.1 Question 1: When does coherence information emerge in each area? . . . . . 209
6.4 Part 3: HOI Analysis - Discovering Synergistic Ensembles . . 215
6.4.1 Question 2: Which neuron triplets show synergistic coding? . . . . . 215
6.4.2 Visualizing O-Information Landscape . . . . . 218
6.5 Part 4: XGI Analysis - Network Topology . . . . . 222
6.5.1 Question 3: How is the synergistic network organized? 222
6.6 Part 5: Temporal Integration - Dynamic Network Evolution . 227
6.6.1 Question 4: How does synergistic structure evolve over time? . . . . . 227
6.6.2 Visualizing Network Evolution . . . . . 229
6.7 Part 6: Integrated Interpretation - Connecting All Analyses . 233
6.7.1 Synthesizing Findings Across All Three Packages . . . 233
6.8 Part 7: Validation and Control Analyses . . . . . 237
6.8.1 Critical Validation Steps . . . . . 237
6.9 Part 8: Advanced Topics and Optimizations . . . . . 242
6.9.1 Computational Optimization Strategies . . . . . 242
6.9.2 Publication-Quality Figure . . . . . 245
6.10 Part 9: Research Guidelines and Best Practices . . . . . 246
6.10.1 Complete Analysis Checklist . . . . . 246
6.10.2 Common Pitfalls and How to Avoid Them . . . . . 249

6.11 Part 10: Final Summary and Conclusions . . . . . 251
6.11.1 The Complete Journey . . . . . 251
6.11.2 The Integrated Toolkit . . . . . 254
6.11.3 When to Use Each . . . . . 254
6.11.4 The Typical Workflow . . . . . 254
6.12 Conclusion: From Theory to Discovery . . . . . 255
6.12.1 What You Can Now Do . . . . . 255
6.12.2 Where to Go From Here . . . . . 256
6.12.3 Final Thoughts . . . . . 256
6.13 Appendix: Quick Reference . . . . . 256
6.13.1 Essential Code Snippets . . . . . 256
6.14 Further Resources . . . . . 259
6.14.1 Essential Papers . . . . . 259
6.14.2 Software Documentation . . . . . 259
6.14.3 Community and Support . . . . . 259
6.14.4 Citation . . . . . 259
6.15 Thank you for completing this series! . . . . . 260

**References and Further Reading** **261**
Key Papers . . . . . 261
Information Theory Foundations . . . . . 261
Higher-Order Interactions . . . . . 261
Partial Information Decomposition . . . . . 262
Neuroscience Applications . . . . . 262
Network Science and Hypergraphs . . . . . 262
Software Documentation . . . . . 263
Primary Packages . . . . . 263
Supporting Libraries . . . . . 263
Online Resources . . . . . 263
Tutorials and Courses . . . . . 263
Research Groups and Centers . . . . . 264
Related Books . . . . . 264
Software Ecosystem . . . . . 264
Related Python Packages . . . . . 264
Contact and Community . . . . . 264

**Appendices** **266**

**Installation Guide** **266**

System Requirements . . . . . 266
Installation Methods . . . . . 266
Method 1: Quick Install (Recommended for Most Users) . . . 266
Method 2: Conda Environment (Alternative) . . . . . 267
Method 3: Step-by-Step Installation . . . . . 267
Platform-Specific Instructions . . . . . 267
Linux . . . . . 267
macOS . . . . . 267
Windows . . . . . 268
GPU Support (Optional but Recommended) . . . . . 268
NVIDIA GPUs (CUDA) . . . . . 268
AMD GPUs (ROCm) . . . . . 268
Apple Silicon (Metal) . . . . . 268
Verifying Installation . . . . . 269
Quick Test Script . . . . . 269
Individual Package Tests . . . . . 270
Troubleshooting . . . . . 271
Common Issues . . . . . 271
Getting Help . . . . . 272
Optional: Development Installation . . . . . 272
Next Steps . . . . . 272

# Welcome

> **About This Tutorial**
>
> This is a comprehensive, hands-on tutorial series for understanding and applying multivariate information theory to neuroscience and complex systems. We build from foundational concepts to complete analysis pipelines using state-of-the-art Python packages.

## What You'll Learn

Over six progressive notebooks, you'll master:

- **Information theory fundamentals** - From entropy to mutual information
- **Higher-order interactions** - Synergy, redundancy, and the famous XOR problem
- **Professional tools** - HOI, Frites, and XGI packages for real-world analysis
- **Time-resolved analysis** - Dynamic information flow in neural systems
- **Network representations** - Hypergraphs for higher-order structures
- **Complete pipelines** - End-to-end analysis workflows

## Who Is This For?

This tutorial is designed for:

- **Neuroscience researchers** analyzing neural population data
- **Network scientists** studying higher-order interactions
- **Complex systems researchers** exploring emergent phenomena
- **Graduate students** learning information-theoretic methods

- **Computational biologists** working with multivariate dependencies
- **Machine learning practitioners** interested in feature interactions

### Prerequisites

- **Programming**: Intermediate Python (NumPy, Matplotlib)
- **Mathematics**: Basic probability theory, linear algebra
- **Statistics**: Understanding of hypothesis testing helpful but not required
- **Domain knowledge**: Neuroscience background helpful but not essential

## Tutorial Structure

### Part I: Foundations (Notebooks 1-2)

Build solid understanding of information theory and discover why pairwise analysis fails for higher-order phenomena.

**Time commitment**: 4-6 hours

### Part II: Advanced Tools (Notebooks 3-5)

Learn three powerful Python packages for analyzing multivariate information in different contexts.

**Time commitment**: 6-8 hours

### Part III: Integration (Notebook 6)

Synthesize everything into a complete analysis pipeline tackling a realistic neuroscience question.

**Time commitment**: 3-4 hours

## Key Features

### Progressive Learning

Each notebook builds on previous ones, with clear learning objectives and prerequisites.

### Hands-On Code

All concepts implemented from scratch before using professional tools.

### Real Applications

Examples and exercises based on actual neuroscience research scenarios.

### Rich Visualizations

Extensive plotting to build intuition for abstract information-theoretic concepts.

## Getting Started

### Quick Start

1. Clone the repository:

```
git clone https://github.com/lordgrilo/cnww-hoi.git
cd cnww-hoi
```

2. Install dependencies:

```
pip install -r requirements.txt
```

3. Launch Jupyter:

```
jupyter notebook
```

4. Start with Notebook 1

### Reading This Book

You can use this tutorial in several ways:

- **Complete course**: Work through all six notebooks sequentially (recommended for learners)
- **Tool reference**: Jump to specific notebooks for package documentation (Notebooks 3-5)
- **Research template**: Adapt the complete pipeline (Notebook 6) to your data
- **Teaching resource**: Use notebooks for workshops or courses

## Online Resources

- **GitHub Repository**: Link to code and issues
- **Package Documentation**:
  - HOI
  - Frites
  - XGI

## Citation

If you use these tutorials in your research or teaching, please cite:

```
@misc{petri2024multivariate,
  author = {Petri, Giovanni},
  title = {Multivariate Information Theory: A Practical Tutorial},
  year = {2025},
  publisher = {GitHub},
  url = {https://github.com/lordgrilo/cnww-hoi/}
}
```

## License



## Acknowledgments

This tutorial integrates three excellent open-source packages:

- **HOI** by Etienne Combrisson and team
- **Frites** by Etienne Combrisson and team
- **XGI** by the Comple(X) Group Interactions team

Special thanks to the network science and computational neuroscience communities for developing the theoretical foundations these tools implement.

---

Ready to Start?

Begin your journey with Notebook 1: Information Theory Foundations where we build the mathematical groundwork from scratch.

# Part I

# Part I: Foundations

# Chapter 1

# Notebook 1: Information Theory Foundations

## 1.1 Building the Mathematical Groundwork for Understanding Multivariate Information

Welcome to the first notebook in our series on multivariate information theory! In this notebook, we'll build the fundamental concepts from the ground up. Think of this as laying the foundation of a house—we need solid basics before we can construct more complex structures.

### 1.1.1 What You'll Learn

By the end of this notebook, you'll understand: 1. What entropy really measures (and why it's not just "randomness") 2. How to compute entropy for discrete and continuous variables 3. Joint entropy and conditional entropy 4. Mutual information as a measure of dependence 5. Conditional mutual information

### 1.1.2 Why Start From Scratch?

While we'll eventually use powerful packages like HOI, Frites, and XGI, implementing these concepts ourselves first provides crucial intuition. When the more advanced measures give surprising results later (like negative interaction information!), you'll understand why.

Let's begin!

```
# Import our foundational libraries
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from scipy.special import digamma
import seaborn as sns
from itertools import product
import warnings
warnings.filterwarnings('ignore')

# Set random seed for reproducibility
np.random.seed(42)

# Make plots beautiful
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")
%matplotlib inline

print("Libraries loaded successfully!")
print(f"NumPy version: {np.__version__}")
```

## 1.2 Part 1: Understanding Entropy

### 1.2.1 The Intuition Behind Entropy

Entropy measures **uncertainty** or **surprise**. Think about these scenarios:

**Scenario A**: You flip a fair coin. Before observing, you're maximally uncertain about the outcome.

**Scenario B**: You flip a heavily biased coin that lands heads 99% of the time. You're pretty sure it'll be heads.

Scenario A has **higher entropy** because the outcome is more uncertain. This is the core idea: entropy quantifies how much we don't know.

### 1.2.2 The Mathematical Definition

For a discrete random variable $X$ with possible values $x_1, x_2, ..., x_n$:

$$H(X) = -\sum_{i=1}^{n} P(x_i) \log_2 P(x_i)$$

The $\log_2$ means we measure entropy in **bits**. One bit is the information gained from one fair coin flip.

### 1.2.3 Why the Negative Sign?

Probabilities are between 0 and 1, so $\log_2(P(x_i))$ is negative. The negative sign makes entropy positive, which matches our intuition that "uncertainty" should be a positive quantity.

```
def entropy(probabilities):
    """
    Calculate entropy for a discrete probability distribution.

    Parameters:
    -----------
    probabilities : array-like
        Probability distribution (must sum to 1)

    Returns:
    --------
    H : float
        Entropy in bits

    Notes:
    ------
    We handle the case where P(x) = 0 by defining 0 * log(0) = 0,
    which is the correct limiting value.
    """
    probs = np.array(probabilities)

    # Verify this is a valid probability distribution
    assert np.abs(probs.sum() - 1.0) < 1e-10, "Probabilities must sum to 1"
    assert np.all(probs >= 0), "Probabilities must be non-negative"

    # Remove zero probabilities to avoid log(0)
    probs = probs[probs > 0]
```

```
    # Calculate entropy
    H = -np.sum(probs * np.log2(probs))

    return H

# Let's test our function with the coin examples
fair_coin = [0.5, 0.5]  # 50-50 chance
biased_coin = [0.99, 0.01]  # 99% heads
certain = [1.0, 0.0]  # Always heads

print("Entropy Examples:")
print(f"Fair coin (50-50): {entropy(fair_coin):.4f} bits")
print(f"Biased coin (99-1): {entropy(biased_coin):.4f} bits")
print(f"Certain outcome: {entropy(certain):.4f} bits")
print("\nInterpretation:")
print("- Fair coin: Maximum entropy (1 bit) - maximum uncertainty")
print("- Biased coin: Low entropy - we're pretty sure what will happen")
print("- Certain: Zero entropy - no uncertainty at all!")
```

### 1.2.4 Visualizing Entropy as a Function of Probability

Let's see how entropy changes as we vary the probability of a binary event. This classic curve shows that entropy is **maximized** when outcomes are equally likely.

```
# Create a range of probabilities for heads
p_heads = np.linspace(0.01, 0.99, 100)
entropies = [entropy([p, 1-p]) for p in p_heads]

# Plot
fig, ax = plt.subplots(1, 1, figsize=(10, 6))
ax.plot(p_heads, entropies, linewidth=3, color='#2E86AB')
ax.axhline(y=1.0, color='red', linestyle='--', alpha=0.5,
           label='Maximum entropy (1 bit)')
ax.axvline(x=0.5, color='red', linestyle='--', alpha=0.5)
ax.scatter([0.5], [1.0], color='red', s=200, zorder=5,
           label='Fair coin', marker='*')

# Annotations
ax.annotate('Maximum uncertainty\nat P=0.5', xy=(0.5, 1.0),
```

```
            xytext=(0.65, 0.85),
            arrowprops=dict(arrowstyle='->', color='red', lw=2),
            fontsize=12, fontweight='bold')

ax.set_xlabel('P(Heads)', fontsize=14, fontweight='bold')
ax.set_ylabel('Entropy (bits)', fontsize=14, fontweight='bold')
ax.set_title('Binary Entropy Function: H(p)', fontsize=16, fontweight='bold')
ax.legend(fontsize=12)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print("Key Insight:")
print("Entropy is maximized when we're most uncertain (P=0.5 for binary events)")
print("As we become more certain (P→0 or P→1), entropy decreases to zero")
```

## 1.3 Part 2: Joint and Conditional Entropy

### 1.3.1 Joint Entropy: Uncertainty About Multiple Variables

When we have two random variables $X$ and $Y$, their **joint entropy** $H(X, Y)$ measures the total uncertainty about both variables together:

$$H(X, Y) = -\sum_{i,j} P(x_i, y_j) \log_2 P(x_i, y_j)$$

### 1.3.2 Conditional Entropy: Uncertainty After Learning Something

**Conditional entropy** $H(X|Y)$ measures how much uncertainty remains about $X$ after we learn $Y$:

$$H(X|Y) = -\sum_{i,j} P(x_i, y_j) \log_2 P(x_i|y_j)$$

Or equivalently:

$$H(X|Y) = H(X, Y) - H(Y)$$

This is the **chain rule** of entropy!

### 1.3.3 A Concrete Example: The Beer Price Problem

Let's implement the example from your slides: predicting beer prices based on volume.

```
def joint_entropy(joint_probs):
    """
    Calculate joint entropy H(X,Y) from a joint probability distribution.

    Parameters:
    -----------
    joint_probs : 2D array
        Joint probability matrix P(X=i, Y=j)

    Returns:
    --------
    H : float
        Joint entropy in bits
    """
    jp = np.array(joint_probs)

    # Verify it's a valid joint distribution
    assert np.abs(jp.sum() - 1.0) < 1e-10, "Joint probabilities must sum to 1"

    # Remove zeros and calculate
    jp_nonzero = jp[jp > 0]
    return -np.sum(jp_nonzero * np.log2(jp_nonzero))


def conditional_entropy(joint_probs):
    """
    Calculate conditional entropy H(X|Y) from joint distribution.

    We use: H(X|Y) = H(X,Y) - H(Y)

    Parameters:
    -----------
    joint_probs : 2D array
        Joint probability matrix P(X=i, Y=j)
        Rows correspond to X, columns to Y
```

```
    Returns:
    --------
    H_X_given_Y : float
        Conditional entropy H(X|Y) in bits
    """
    jp = np.array(joint_probs)

    # Calculate H(X,Y)
    H_XY = joint_entropy(jp)

    # Calculate H(Y) from marginal
    marginal_Y = jp.sum(axis=0)  # Sum over rows to get P(Y)
    H_Y = entropy(marginal_Y)

    # Apply chain rule
    return H_XY - H_Y


# Example: Beer prices (X) and volumes (Y)
# Let's create a realistic joint distribution
# Rows = price (low, medium, high), Columns = volume (small, large)

beer_joint = np.array([
    # Small (0.25L)  Large (0.5L)
    [0.20,          0.05],  # Low price (3 euros)
    [0.35,          0.15],  # Medium price (5 euros)
    [0.05,          0.20],  # High price (7 euros)
])

print("Beer Price Example")
print("=" * 50)
print("\nJoint Distribution P(Price, Volume):")
print("                Small(0.25L)  Large(0.5L)")
print(f"Low (3€):       {beer_joint[0,0]:.2f}         {beer_joint[0,1]:.2f}")
print(f"Medium (5€):    {beer_joint[1,0]:.2f}         {beer_joint[1,1]:.2f}")
print(f"High (7€):      {beer_joint[2,0]:.2f}         {beer_joint[2,1]:.2f}")

# Calculate entropies
marginal_price = beer_joint.sum(axis=1)  # P(Price)
```

```
marginal_volume = beer_joint.sum(axis=0)  # P(Volume)

H_price = entropy(marginal_price)
H_volume = entropy(marginal_volume)
H_joint = joint_entropy(beer_joint)
H_price_given_volume = conditional_entropy(beer_joint)

print("\n" + "=" * 50)
print("Entropy Calculations:")
print("=" * 50)
print(f"H(Price) = {H_price:.4f} bits")
print(f"H(Volume) = {H_volume:.4f} bits")
print(f"H(Price, Volume) = {H_joint:.4f} bits")
print(f"H(Price|Volume) = {H_price_given_volume:.4f} bits")
print("\nInterpretation:")
print(f"- Before knowing volume: {H_price:.4f} bits of uncertainty about price")
print(f"- After knowing volume: {H_price_given_volume:.4f} bits remain")
print(f"- Uncertainty reduced by {H_price - H_price_given_volume:.4f} bits")
```

## 1.4 Part 3: Mutual Information

### 1.4.1 The Key Measure of Dependence

**Mutual Information (MI)** quantifies how much knowing one variable reduces uncertainty about another:

$$I(X;Y) = H(X) - H(X|Y) = H(Y) - H(Y|X)$$

The symmetry ($I(X;Y) = I(Y;X)$) is beautiful: the information $X$ provides about $Y$ equals the information $Y$ provides about $X$.

### 1.4.2 Alternative Formulation: KL Divergence

MI can also be written as:

$$I(X;Y) = \sum_{i,j} P(x_i, y_j) \log_2 \frac{P(x_i, y_j)}{P(x_i)P(y_j)}$$

This shows MI measures how far the joint distribution is from independence. If $X$ and $Y$ are independent: $P(X, Y) = P(X)P(Y)$, so $I(X; Y) = 0$.

### 1.4.3 Key Properties

1. **Non-negative**: $I(X; Y) \geq 0$ always
2. **Zero iff independent**: $I(X; Y) = 0 \iff X \perp Y$
3. **Bounded**: $I(X; Y) \leq \min(H(X), H(Y))$
4. **Symmetric**: $I(X; Y) = I(Y; X)$

```
def mutual_information(joint_probs):
    """
    Calculate mutual information I(X;Y) from joint distribution.

    Uses: I(X;Y) = H(X) + H(Y) - H(X,Y)

    Parameters:
    -----------
    joint_probs : 2D array
        Joint probability matrix P(X=i, Y=j)

    Returns:
    --------
    MI : float
        Mutual information in bits
    """
    jp = np.array(joint_probs)

    # Calculate marginals
    marginal_X = jp.sum(axis=1)
    marginal_Y = jp.sum(axis=0)

    # Calculate individual and joint entropies
    H_X = entropy(marginal_X)
    H_Y = entropy(marginal_Y)
    H_XY = joint_entropy(jp)

    # MI = H(X) + H(Y) - H(X,Y)
    return H_X + H_Y - H_XY
```

```
# Calculate MI for our beer example
MI_beer = mutual_information(beer_joint)

print("Mutual Information Analysis")
print("=" * 50)
print(f"\nI(Price; Volume) = {MI_beer:.4f} bits")
print("\nVerification (should match):")
print(f"H(Price) - H(Price|Volume) = {H_price - H_price_given_volume:.4f} bits  ")
print("\nInterpretation:")
print(f"Knowing the volume reduces price uncertainty by {MI_beer:.4f} bits")
print(f"Equivalently, knowing price reduces volume uncertainty by {MI_beer:.4f} bits"

# Let's also create examples of different dependency strengths
print("\n" + "=" * 50)
print("Comparing Different Dependency Strengths")
print("=" * 50)

# Perfect independence
independent = np.outer([0.25, 0.75], [0.4, 0.6])
MI_indep = mutual_information(independent)

# Perfect dependence (deterministic)
deterministic = np.array([
    [0.5, 0.0],
    [0.0, 0.5]
])
MI_det = mutual_information(deterministic)

print(f"\n1. Independent variables: I(X;Y) = {MI_indep:.6f} bits   0")
print(f"2. Our beer example: I(X;Y) = {MI_beer:.4f} bits (moderate dependence)")
print(f"3. Deterministic relation: I(X;Y) = {MI_det:.4f} bit (perfect dependence)")
```

### 1.4.4 Visualizing the Venn Diagram of Information

The classic "information diagram" helps visualize the relationship between entropy and mutual information. This works perfectly for two variables (but we'll see later why it breaks for three!).

```
from matplotlib.patches import Circle, Rectangle
from matplotlib.collections import PatchCollection

def plot_information_venn(H_X, H_Y, MI):
    """
    Create a Venn diagram representation of information measures.
    """
    fig, ax = plt.subplots(1, 1, figsize=(10, 8))

    # Calculate regions
    H_X_given_Y = H_X - MI
    H_Y_given_X = H_Y - MI

    # Draw circles
    circle1 = Circle((0.3, 0.5), 0.25, alpha=0.5, color='#E63946', label=f'H(X)={H_X:
    circle2 = Circle((0.7, 0.5), 0.25, alpha=0.5, color='#457B9D', label=f'H(Y)={H_Y:
    ax.add_patch(circle1)
    ax.add_patch(circle2)

    # Add text labels
    ax.text(0.15, 0.5, f'H(X|Y)\n{H_X_given_Y:.3f}',
            fontsize=11, ha='center', va='center', fontweight='bold')
    ax.text(0.5, 0.5, f'I(X;Y)\n{MI:.3f}',
            fontsize=12, ha='center', va='center', fontweight='bold',
            bbox=dict(boxstyle='round', facecolor='yellow', alpha=0.7))
    ax.text(0.85, 0.5, f'H(Y|X)\n{H_Y_given_X:.3f}',
            fontsize=11, ha='center', va='center', fontweight='bold')

    # Add title and labels
    ax.set_xlim(0, 1)
    ax.set_ylim(0.1, 0.9)
    ax.set_aspect('equal')
    ax.axis('off')
    ax.set_title('Information Venn Diagram', fontsize=16, fontweight='bold', pad=20)
    ax.legend(loc='upper center', bbox_to_anchor=(0.5, -0.05), ncol=2, fontsize=12)

    # Add equations at bottom
    eq_text = (
        f"H(X,Y) = H(X|Y) + I(X;Y) + H(Y|X)\n"
```

```
        f"       = {H_X_given_Y:.3f} + {MI:.3f} + {H_Y_given_X:.3f} = {H_X_given_Y +
    )
    fig.text(0.5, 0.05, eq_text, ha='center', fontsize=11,
             bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

    plt.tight_layout()
    return fig, ax

# Plot for our beer example
fig, ax = plot_information_venn(H_price, H_volume, MI_beer)
plt.show()

print("\nKey Insight:")
print("The overlapping region represents shared information (mutual information).")
print("The non-overlapping regions are information unique to each variable.")
print("\n   IMPORTANT: This diagram works for 2 variables, but breaks for 3+!")
print("    We'll see why in the next notebook when we encounter negative information.
```

## 1.5 Part 4: Conditional Mutual Information

### 1.5.1 Beyond Pairwise Dependencies

**Conditional Mutual Information (CMI)** measures the information shared between $X$ and $Y$ after accounting for a third variable $Z$:

$$I(X;Y|Z) = H(X|Z) - H(X|Y,Z)$$

Equivalently:

$$I(X;Y|Z) = H(X|Z) + H(Y|Z) - H(X,Y|Z)$$

### 1.5.2 Why This Matters

CMI is crucial for understanding **multivariate** dependencies. It answers: "After I know $Z$, how much does learning $Y$ still help me predict $X$?"

### 1.5.3 A Surprising Property: CMI Can Increase!

Unlike correlation (which conditioning always reduces), CMI can actually **increase** when we condition. This is called **explaining away** and we'll

explore it in the next notebook.

For now, let's implement CMI:

```
def conditional_mutual_information(joint_probs_XYZ):
    """
    Calculate conditional mutual information I(X;Y|Z).

    Parameters:
    -----------
    joint_probs_XYZ : 3D array
        Joint probability P(X, Y, Z)
        Shape: (n_X, n_Y, n_Z)

    Returns:
    --------
    CMI : float
        Conditional mutual information I(X;Y|Z) in bits

    Formula:
    --------
    I(X;Y|Z) = sum_z P(z) * I(X;Y|Z=z)
             = H(X|Z) + H(Y|Z) - H(X,Y|Z)
    """
    jp_XYZ = np.array(joint_probs_XYZ)
    n_X, n_Y, n_Z = jp_XYZ.shape

    # Calculate marginal P(Z)
    P_Z = jp_XYZ.sum(axis=(0, 1))  # Sum over X and Y

    # Initialize CMI
    CMI = 0.0

    # For each value of Z, calculate I(X;Y|Z=z) and weight by P(z)
    for z in range(n_Z):
        if P_Z[z] > 0:
            # Conditional joint distribution P(X,Y|Z=z)
            P_XY_given_z = jp_XYZ[:, :, z] / P_Z[z]

            # Calculate MI for this conditioning value
```

```
            MI_given_z = mutual_information(P_XY_given_z)

            # Weight by P(z)
            CMI += P_Z[z] * MI_given_z

    return CMI


# Example: Coffee shop scenario
# X = Weather (Rainy, Sunny)
# Y = Coffee sales (Low, High)
# Z = Day type (Weekday, Weekend)

# Create a realistic 3D joint distribution
coffee_joint = np.zeros((2, 2, 2))  # (weather, sales, day_type)

# Weekday (Z=0): Weather affects sales more
coffee_joint[:, :, 0] = np.array([
    # Low sales  High sales
    [0.15,       0.05],  # Rainy
    [0.05,       0.15],  # Sunny
])

# Weekend (Z=1): Sales always high regardless of weather
coffee_joint[:, :, 1] = np.array([
    # Low sales  High sales
    [0.05,       0.25],  # Rainy
    [0.05,       0.25],  # Sunny
])

# Calculate various MIs
marginal_XY = coffee_joint.sum(axis=2)  # P(Weather, Sales)
MI_weather_sales = mutual_information(marginal_XY)
CMI_weather_sales_given_day = conditional_mutual_information(coffee_joint)

print("Coffee Shop Example: Weather → Sales")
print("=" * 50)
print(f"\nI(Weather; Sales) = {MI_weather_sales:.4f} bits")
print(f"I(Weather; Sales | DayType) = {CMI_weather_sales_given_day:.4f} bits")
```

```
print("\nInterpretation:")
print("- Overall: Weather and sales share some information")
print("- After knowing if it's a weekday/weekend:")
if CMI_weather_sales_given_day > MI_weather_sales:
    print("  The relationship is STRONGER (explaining away effect!)")
elif CMI_weather_sales_given_day < MI_weather_sales:
    print("  The relationship is WEAKER (day type explains some dependence)")
else:
    print("  The relationship is UNCHANGED (day type is irrelevant)")
```

## 1.6 Part 5: Estimating Information from Data

### 1.6.1 From Theory to Practice

So far we've worked with **known** probability distributions. In real research, we have **data samples** and need to **estimate** these distributions.

### 1.6.2 Three Common Approaches

1. **Histogram/Binning**: Discretize continuous data into bins
2. **Kernel Density Estimation (KDE)**: Smooth density estimation
3. **k-Nearest Neighbors (KNN)**: Distance-based estimation

Each has trade-offs in bias, variance, and computational cost. Let's implement the simplest approach (binning) and understand its limitations.

```
def estimate_entropy_binned(data, n_bins=10):
    """
    Estimate entropy from continuous data using binning.

    Parameters:
    -----------
    data : array-like
        1D array of samples
    n_bins : int
        Number of bins for histogram

    Returns:
    --------
    H : float
```

```
        Estimated entropy in bits

    Warning:
    --------
    This is a biased estimator! The choice of n_bins matters.
    """
    # Create histogram
    counts, _ = np.histogram(data, bins=n_bins)

    # Convert to probabilities
    probs = counts / counts.sum()

    # Remove zero bins
    probs = probs[probs > 0]

    return entropy(probs)


def estimate_mi_binned(X, Y, n_bins=10):
    """
    Estimate mutual information from samples using 2D histogram.
    """
    # Create 2D histogram for joint distribution
    joint_counts, _, _ = np.histogram2d(X, Y, bins=n_bins)

    # Convert to probabilities
    joint_probs = joint_counts / joint_counts.sum()

    return mutual_information(joint_probs)


# Generate some sample data with known MI
n_samples = 5000

# Create correlated Gaussian variables
rho = 0.7  # Correlation coefficient
mean = [0, 0]
cov = [[1, rho], [rho, 1]]
data = np.random.multivariate_normal(mean, cov, n_samples)
```

```
X_data = data[:, 0]
Y_data = data[:, 1]

# For Gaussians, MI has an exact formula: I(X;Y) = -0.5 * log(1 - rho²)
true_MI = -0.5 * np.log2(1 - rho**2)

print("Estimating MI from Data")
print("=" * 50)
print(f"\nTrue MI (Gaussian formula): {true_MI:.4f} bits")
print("\nEstimated MI with different bin sizes:")

for n_bins in [5, 10, 20, 30, 50]:
    est_MI = estimate_mi_binned(X_data, Y_data, n_bins)
    error = abs(est_MI - true_MI)
    print(f"  {n_bins:2d} bins: {est_MI:.4f} bits (error: {error:.4f})")

print("\n  Key Lesson:")
print("Bin size matters! Too few bins → underestimate, too many → noise")
print("This is why advanced packages (like Frites and HOI) use better methods")
```

### 1.6.3 Visualizing the Bias-Variance Trade-off

Let's see how our MI estimate varies with bin size and sample size:

```
# Test different sample sizes and bin numbers
sample_sizes = [100, 500, 1000, 5000, 10000]
bin_numbers = np.arange(5, 51, 5)

# Store results
results = np.zeros((len(sample_sizes), len(bin_numbers)))

for i, n_samples in enumerate(sample_sizes):
    # Generate data
    data = np.random.multivariate_normal(mean, cov, n_samples)
    X = data[:, 0]
    Y = data[:, 1]

    for j, n_bins in enumerate(bin_numbers):
        results[i, j] = estimate_mi_binned(X, Y, n_bins)
```

```
# Plot
fig, ax = plt.subplots(1, 1, figsize=(12, 7))

for i, n_samples in enumerate(sample_sizes):
    ax.plot(bin_numbers, results[i, :], marker='o', linewidth=2,
            label=f'N = {n_samples}', alpha=0.7)

ax.axhline(y=true_MI, color='black', linestyle='--', linewidth=2.5,
           label=f'True MI = {true_MI:.4f} bits')

ax.set_xlabel('Number of Bins', fontsize=14, fontweight='bold')
ax.set_ylabel('Estimated MI (bits)', fontsize=14, fontweight='bold')
ax.set_title('MI Estimation: Effect of Sample Size and Binning',
             fontsize=16, fontweight='bold')
ax.legend(fontsize=11, loc='best')
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print("\nObservations:")
print("1. Small sample sizes → high variance (estimates jump around)")
print("2. Too few bins → systematic underestimation (bias)")
print("3. Too many bins with small samples → overestimation (sparse bins)")
print("4. More data allows more bins and more accurate estimates")
```

## 1.7 Part 6: Summary and Bridge to Advanced Methods

### 1.7.1 What We've Built

Congratulations! You now understand:

1. **Entropy**: Measures uncertainty/surprise
2. **Joint and Conditional Entropy**: Uncertainty about multiple variables
3. **Mutual Information**: How much variables tell us about each other
4. **Conditional MI**: Information shared after accounting for a third variable
5. **Estimation Challenges**: Why we need sophisticated methods

### 1.7.2 The Foundations Are Set

These concepts form the bedrock of multivariate information theory. In the next notebooks, we'll:

- **Notebook 2**: See why pairwise measures fail (the XOR problem)
- **Notebook 3**: Learn about synergy, redundancy, and interaction information using **HOI**
- **Notebook 4**: Apply these to real neural time series with **Frites**
- **Notebook 5**: Explore higher-order network structures with **XGI**
- **Notebook 6**: Integrate everything for advanced analyses

### 1.7.3 Practice Exercises

Before moving on, try these to solidify your understanding:

```
print("PRACTICE EXERCISES")
print("=" * 70)
print("\n1. Calculate the entropy of a fair 6-sided die.")
print("   Hint: Each outcome has probability 1/6")
fair_die = np.ones(6) / 6
H_die = entropy(fair_die)
print(f"   Answer: {H_die:.4f} bits")
print(f"   Compare to fair coin (1 bit): You need {H_die:.2f}/1.0 = {H_die:.2f}x as m
print("   to match the information from a coin flip!\n")

print("2. Why is H(X,Y)   H(X) + H(Y)?")
print("   This is called subadditivity. Can you explain why?")
print("   Hint: When are they equal? When are they most different?\n")

print("3. Create two variables X and Y where I(X;Y) = H(X).")
print("   What does this mean about the relationship between X and Y?")
# Example: Perfect dependence
perfect_dep = np.array([[0.3, 0.0], [0.0, 0.7]])
MI_perfect = mutual_information(perfect_dep)
H_X_perfect = entropy(perfect_dep.sum(axis=1))
print(f"   Example MI: {MI_perfect:.4f}, H(X): {H_X_perfect:.4f}")
print(f"   This means: Y completely determines X (or vice versa)!\n")

print("4. Generate two independent random variables and verify I(X;Y)   0.")
X_indep = np.random.randn(1000)
```

```
Y_indep = np.random.randn(1000)
MI_indep_est = estimate_mi_binned(X_indep, Y_indep, n_bins=20)
print(f"   Estimated MI: {MI_indep_est:.6f} bits (should be near 0)")
print("   Small non-zero value is due to finite sample size!")

print("\n" + "=" * 70)
print("Ready for Notebook 2: The XOR Problem!")
print("There we'll see why everything breaks with 3+ variables...")
print("=" * 70)
```

## 1.8 Additional Resources

### 1.8.1 Recommended Reading

1. **Cover & Thomas** - "Elements of Information Theory" (Chapters 2-3)
2. **MacKay** - "Information Theory, Inference, and Learning Algorithms" (Chapter 2)
3. **Ince et al. 2017** - "A statistical framework for neuroimaging data analysis based on mutual information"

### 1.8.2 Going Deeper

- For continuous variables: differential entropy
- For high-dimensional data: dimensionality reduction before MI estimation

- For time series: transfer entropy and Granger causality
- For neural data: Gaussian Copula MI (what Frites uses!)

### 1.8.3 Coming Up

In the next notebook, we'll confront the **XOR problem**—the moment when your intuition about information will be completely challenged. You'll see that two variables can individually carry **zero** information about a target, yet together carry **complete** information. This is the gateway to understanding synergy, redundancy, and higher-order interactions.

See you there!

# Chapter 2

# Notebook 2: The XOR Problem - When Intuition Breaks

## 2.1 Discovering Why Two Variables Can Be Useless Alone But Powerful Together

Welcome to the most mind-bending notebook in this series! If Notebook 1 built your foundations, this notebook will shake them—in a good way. We're about to encounter a phenomenon that defies all intuition from pairwise analysis.

### 2.1.1 The Central Puzzle

Imagine two neurons, $X_1$ and $X_2$, that encode information about a stimulus $Y$. You measure: - $I(X_1; Y) = 0$ bits (Neuron 1 alone tells you nothing about the stimulus) - $I(X_2; Y) = 0$ bits (Neuron 2 alone tells you nothing about the stimulus)

**Question**: How much information do they carry together?

**Intuitive answer**: Still 0 bits, right? $0 + 0 = 0$

**Reality**: They can carry **1 full bit** of information together!

**Your reaction**:

This is the **XOR problem**, and it's the gateway to understanding synergy, redundancy, and higher-order interactions.

### 2.1.2 Learning Objectives

By the end of this notebook, you'll: 1. Understand the XOR function and why it's special 2. Calculate that individual MIs are zero but joint MI is maximal 3. See why Venn diagrams break for 3+ variables 4. Understand interaction information and its sign 5. Learn about "explaining away" and conditional independence 6. Build intuition for synergy vs. redundancy

Let's begin the journey!

```
# Import everything we need
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch, Circle, FancyArrowPatch
import seaborn as sns
from itertools import product
import pandas as pd
from mpl_toolkits.mplot3d import Axes3D
import warnings
warnings.filterwarnings('ignore')

# Import our functions from Notebook 1
def entropy(probs):
    """Calculate entropy in bits"""
    p = np.array(probs)
    p = p[p > 0]
    return -np.sum(p * np.log2(p))

def joint_entropy(joint_probs):
    """Calculate joint entropy H(X,Y)"""
    jp = np.array(joint_probs)
    jp = jp[jp > 0]
    return -np.sum(jp * np.log2(jp))

def mutual_information(joint_probs):
    """Calculate MI from 2D joint distribution"""
    jp = np.array(joint_probs)
    marginal_X = jp.sum(axis=1)
```

```
    marginal_Y = jp.sum(axis=0)
    return entropy(marginal_X) + entropy(marginal_Y) - joint_entropy(jp)

# Set style
plt.style.use('seaborn-v0_8-whitegrid')
sns.set_palette("Set2")
np.random.seed(42)

print("Libraries loaded! Ready to break some intuitions...  ")
```

```
Libraries loaded! Ready to break some intuitions...
```

## 2.2 Part 1: The XOR Function

### 2.2.1 What is XOR?

**XOR** (exclusive OR) is a logical operation. For binary variables:

$$Y = X_1 \text{ XOR } X_2$$

**Truth table**:

| X | X | Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

**In words**: $Y = 1$ if $X_1$ and $X_2$ are **different**, $Y = 0$ if they're the **same**.

### 2.2.2 Why This is Special

Looking at just $X_1$: - When $X_1 = 0$, then $Y$ could be 0 or 1 (depends on $X_2$)
- When $X_1 = 1$, then $Y$ could be 0 or 1 (depends on $X_2$)

$X_1$ **alone doesn't predict** $Y$ **at all!** The same is true for $X_2$.

But **together**, they predict $Y$ **perfectly**.

This is called **pure synergy**—information that exists only in the combination.

### 2.2.3 Let's Verify This

```
# Define the XOR system
# All 8 combinations are equally likely: P = 1/4 for each (X , X , Y) triple

# Create 3D joint distribution P(X , X , Y)
# Shape: (2, 2, 2) for (X , X , Y) each binary
P_XOR = np.zeros((2, 2, 2))

# Only 4 valid combinations under XOR constraint
P_XOR[0, 0, 0] = 0.25  # X =0, X =0 → Y=0
P_XOR[0, 1, 1] = 0.25  # X =0, X =1 → Y=1
P_XOR[1, 0, 1] = 0.25  # X =1, X =0 → Y=1
P_XOR[1, 1, 0] = 0.25  # X =1, X =1 → Y=0

print("XOR System: Truth Table and Probabilities")
print("=" * 60)
print("\n X    X    Y      P(X , X , Y)")
print("                  ")
for x1 in [0, 1]:
    for x2 in [0, 1]:
        y = x1 ^ x2  # XOR operation in Python
        prob = P_XOR[x1, x2, y]
        print(f"  {x1}    {x2}    {y}       {prob:.2f}")

print("\n" + "=" * 60)
print("All combinations equally likely → Maximum uncertainty")
print("But XOR constraint creates perfect predictability when both inputs known")
```

```
XOR System: Truth Table and Probabilities
============================================================

 X    X    Y      P(X , X , Y)

  0    0    0       0.25
  0    1    1       0.25
  1    0    1       0.25
  1    1    0       0.25

============================================================
```

```
All combinations equally likely → Maximum uncertainty
But XOR constraint creates perfect predictability when both inputs known
```

## 2.3 Part 2: Calculating the Shocking Result

### 2.3.1 Step 1: Individual Mutual Informations

Let's calculate $I(X_1;Y)$ step by step. We need the joint distribution $P(X_1, Y)$.

```python
# Marginalize out X  to get P(X , Y)
P_X1_Y = P_XOR.sum(axis=1)  # Sum over X  (axis 1)

print("Joint Distribution P(X , Y):")
print("=" * 50)
print("           Y=0    Y=1")
print(f"X =0:    {P_X1_Y[0, 0]:.2f}   {P_X1_Y[0, 1]:.2f}")
print(f"X =1:    {P_X1_Y[1, 0]:.2f}   {P_X1_Y[1, 1]:.2f}")

# Calculate marginals
P_X1 = P_X1_Y.sum(axis=1)
P_Y = P_X1_Y.sum(axis=0)

print("\nMarginal Distributions:")
print(f"P(X =0) = {P_X1[0]:.2f}, P(X =1) = {P_X1[1]:.2f}")
print(f"P(Y=0) = {P_Y[0]:.2f}, P(Y=1) = {P_Y[1]:.2f}")

# Calculate mutual information
MI_X1_Y = mutual_information(P_X1_Y)

print("\n" + "=" * 50)
print(f"\n  I(X ; Y) = {MI_X1_Y:.10f} bits")

if MI_X1_Y < 1e-10:
    print("\n  Exactly ZERO! X  tells us nothing about Y.")
else:
    print(f"\n   Small but non-zero (numerical error: {MI_X1_Y:.2e})")
```

```
Joint Distribution P(X , Y):
==================================================
```

```
          Y=0    Y=1
X =0:    0.25   0.25
X =1:    0.25   0.25

Marginal Distributions:
P(X =0) = 0.50, P(X =1) = 0.50
P(Y=0) = 0.50, P(Y=1) = 0.50

==================================================

 I(X ; Y) = 0.0000000000 bits

 Exactly ZERO! X  tells us nothing about Y.
```

### 2.3.2 Understanding Why I(X ; Y) = 0

Look at the joint distribution we just printed. Notice: - $P(X_1 = 0, Y = 0) = P(X_1 = 0, Y = 1) = 0.25$ - $P(X_1 = 1, Y = 0) = P(X_1 = 1, Y = 1) = 0.25$

**This means**: - Given $X_1 = 0$, both values of $Y$ are equally likely (50-50) - Given $X_1 = 1$, both values of $Y$ are equally likely (50-50)

**Therefore**: Learning $X_1$ doesn't reduce our uncertainty about $Y$ at all!

$$H(Y|X_1) = H(Y) = 1 \text{ bit}$$

So:

$$I(X_1; Y) = H(Y) - H(Y|X_1) = 1 - 1 = 0$$

### 2.3.3 By Symmetry, I(X ; Y) = 0 Too

```python
# Calculate I(X ; Y)
P_X2_Y = P_XOR.sum(axis=0)  # Sum over X  (axis 0)
MI_X2_Y = mutual_information(P_X2_Y)

print("By symmetry:")
print(f"I(X ; Y) = {MI_X2_Y:.10f} bits")
print("\n  Neither variable alone tells us ANYTHING about Y!")
print("   This is the key surprise of the XOR problem.")
```

```
By symmetry:
I(X ; Y) = 0.0000000000 bits

  Neither variable alone tells us ANYTHING about Y!
   This is the key surprise of the XOR problem.
```

### 2.3.4 Step 2: Joint Mutual Information

Now let's see what happens when we consider **both** $X_1$ and $X_2$ together.

We need to calculate $I(X_1, X_2; Y)$, which measures how much knowing **both** $X_1$ and $X_2$ tells us about $Y$.

```python
def joint_mi_three_vars(P_XYZ):
    """
    Calculate I(X,Y; Z) from 3D joint distribution.

    I(X,Y; Z) = H(Z) - H(Z|X,Y)
              = H(X,Y) + H(Z) - H(X,Y,Z)
    """
    # Flatten to 2D: combine X,Y into single variable
    n_x, n_y, n_z = P_XYZ.shape
    P_XY_Z = P_XYZ.reshape(n_x * n_y, n_z)

    # Remove zero entries
    marginal_XY = P_XY_Z.sum(axis=1)
    marginal_Z = P_XY_Z.sum(axis=0)

    # Calculate H(X,Y), H(Z), H(X,Y,Z)
    H_XY = entropy(marginal_XY)
    H_Z = entropy(marginal_Z)
    H_XYZ = joint_entropy(P_XY_Z)

    return H_XY + H_Z - H_XYZ

# Calculate I(X , X ; Y)
MI_X1X2_Y = joint_mi_three_vars(P_XOR)

print("Joint Mutual Information")
print("=" * 60)
print(f"\nI(X , X ; Y) = {MI_X1X2_Y:.10f} bits")
```

```
# Let's verify this by checking H(Y|X ,X )
# When we know both X  and X , we know Y perfectly (deterministic)
print("\nVerification:")
print("When we know both X  and X , Y is deterministic")
print("Therefore: H(Y|X ,X ) = 0")
print(f"So: I(X ,X ; Y) = H(Y) - H(Y|X ,X ) = 1 - 0 = 1 bit  ")

print("\n" + "=" * 60)
print("\n  THE SHOCKING RESULT:")
print("=" * 60)
print(f"  I(X ; Y) = {MI_X1_Y:.1f} bits")
print(f"  I(X ; Y) = {MI_X2_Y:.1f} bits")
print(f"  I(X , X ; Y) = {MI_X1X2_Y:.1f} bit")
print("\n  0 + 0 = 1 in information theory!  ")
print("\n  The information exists only in the COMBINATION.")
print("  This is the essence of SYNERGY.")
```

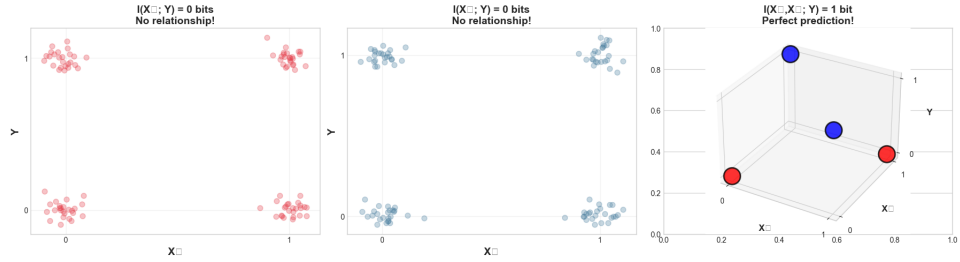
```
Joint Mutual Information
============================================================

I(X , X ; Y) = 1.0000000000 bits

Verification:
When we know both X  and X , Y is deterministic
Therefore: H(Y|X ,X ) = 0
So: I(X ,X ; Y) = H(Y) - H(Y|X ,X ) = 1 - 0 = 1 bit

============================================================

  THE SHOCKING RESULT:
============================================================
  I(X ; Y) = 0.0 bits
  I(X ; Y) = 0.0 bits
  I(X , X ; Y) = 1.0 bit

  0 + 0 = 1 in information theory!

  The information exists only in the COMBINATION.
  This is the essence of SYNERGY.
```

### 2.3.5 Visualizing the XOR Paradox

Let's create a visual representation of this mind-bending result:

```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Panel 1: X  vs Y
ax1 = axes[0]
scatter_data_1 = []
for x1 in [0, 1]:
    for y in [0, 1]:
        count = int(P_X1_Y[x1, y] * 100)  # Number of points
        scatter_data_1.extend([(x1, y)] * count)
scatter_data_1 = np.array(scatter_data_1)
ax1.scatter(scatter_data_1[:, 0] + np.random.randn(len(scatter_data_1)) * 0.05,
            scatter_data_1[:, 1] + np.random.randn(len(scatter_data_1)) * 0.05,
            alpha=0.3, s=50, color='#E63946')
ax1.set_xlabel('X ', fontsize=14, fontweight='bold')
ax1.set_ylabel('Y', fontsize=14, fontweight='bold')
ax1.set_title(f'I(X ; Y) = 0 bits\nNo relationship!', fontsize=14, fontweight='bold')
ax1.set_xticks([0, 1])
ax1.set_yticks([0, 1])
ax1.grid(True, alpha=0.3)

# Panel 2: X  vs Y
ax2 = axes[1]
scatter_data_2 = []
for x2 in [0, 1]:
    for y in [0, 1]:
        count = int(P_X2_Y[x2, y] * 100)
        scatter_data_2.extend([(x2, y)] * count)
scatter_data_2 = np.array(scatter_data_2)
ax2.scatter(scatter_data_2[:, 0] + np.random.randn(len(scatter_data_2)) * 0.05,
            scatter_data_2[:, 1] + np.random.randn(len(scatter_data_2)) * 0.05,
            alpha=0.3, s=50, color='#457B9D')
ax2.set_xlabel('X ', fontsize=14, fontweight='bold')
ax2.set_ylabel('Y', fontsize=14, fontweight='bold')
ax2.set_title(f'I(X ; Y) = 0 bits\nNo relationship!', fontsize=14, fontweight='bold')
ax2.set_xticks([0, 1])
ax2.set_yticks([0, 1])
```

```
ax2.grid(True, alpha=0.3)

# Panel 3: 3D view showing XOR structure
ax3 = fig.add_subplot(133, projection='3d')
colors = ['red' if y == 0 else 'blue' for y in [0, 1, 1, 0]]
x1_vals = [0, 0, 1, 1]
x2_vals = [0, 1, 0, 1]
y_vals = [0, 1, 1, 0]
ax3.scatter(x1_vals, x2_vals, y_vals, c=colors, s=500, alpha=0.8, edgecolors='black',
ax3.set_xlabel('X ', fontsize=12, fontweight='bold')
ax3.set_ylabel('X ', fontsize=12, fontweight='bold')
ax3.set_zlabel('Y', fontsize=12, fontweight='bold')
ax3.set_title('I(X ,X ; Y) = 1 bit\nPerfect prediction!', fontsize=14, fontweight='bol
ax3.set_xticks([0, 1])
ax3.set_yticks([0, 1])
ax3.set_zticks([0, 1])

plt.tight_layout()
plt.show()

print("\nKey Insight from Visualization:")
print("Left two panels: Complete overlap → No predictability")
print("Right panel: Perfect separation → Complete predictability")
print("\nThe structure is only visible in the JOINT space!")
```




```
Key Insight from Visualization:
Left two panels: Complete overlap → No predictability
Right panel: Perfect separation → Complete predictability

The structure is only visible in the JOINT space!
```

## 2.4 Part 3: Why the Venn Diagram Breaks

### 2.4.1 The Two-Variable Venn Diagram (It Works!)

For two variables, we can represent information as overlapping circles. The overlap is the mutual information.

### 2.4.2 The Three-Variable Venn Diagram (It Breaks!)

With three variables, we'd naively draw three overlapping circles. But what goes in the center (the triple overlap)?

For the XOR case: - Each circle represents $H(X_1)$, $H(X_2)$, $H(Y) = 1$ bit each - The pairwise overlaps: $I(X_1; Y) = I(X_2; Y) = I(X_1; X_2) = 0$ bits - But $I(X_1, X_2; Y) = 1$ bit!

**The Problem**: If pairwise overlaps are zero, where does this 1 bit live?

**The Answer**: It must be in the center (triple overlap). But then we'd need:

Center region $= I(X_1, X_2; Y) - I(X_1; Y) - I(X_2; Y) +$ "corrections" $= 1 - 0 - 0 = 1$

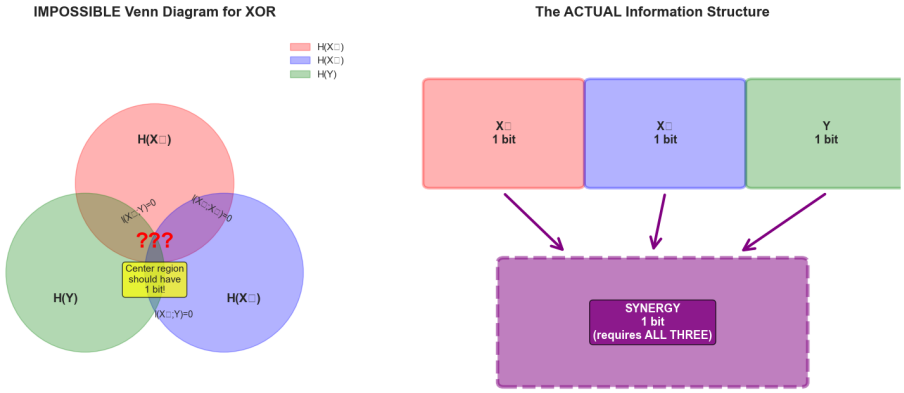
But wait… there are **no** pairwise overlaps to "get to" this center!

Let's visualize this impossibility:

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 7))

# Left: Standard 3-way Venn
from matplotlib.patches import Ellipse
ax1.set_xlim(-2, 2)
ax1.set_ylim(-2, 2)
ax1.set_aspect('equal')
ax1.axis('off')

# Draw three circles
c1 = Circle((0, 0.5), 0.8, alpha=0.3, color='red', label='H(X )')
c2 = Circle((0.7, -0.4), 0.8, alpha=0.3, color='blue', label='H(X )')
c3 = Circle((-0.7, -0.4), 0.8, alpha=0.3, color='green', label='H(Y)')
ax1.add_patch(c1)
ax1.add_patch(c2)
ax1.add_patch(c3)
```

```
# Label regions
ax1.text(0, 0.9, 'H(X )', ha='center', fontsize=14, fontweight='bold')
ax1.text(0.9, -0.7, 'H(X )', ha='center', fontsize=14, fontweight='bold')
ax1.text(-0.9, -0.7, 'H(Y)', ha='center', fontsize=14, fontweight='bold')
ax1.text(0, -0.15, '???', ha='center', fontsize=24, fontweight='bold', color='red')
ax1.text(0, -0.6, 'Center region\nshould have\n1 bit!', ha='center', fontsize=11,
         bbox=dict(boxstyle='round', facecolor='yellow', alpha=0.7))

# Pairwise overlaps are 0
ax1.text(0.35, 0.1, 'I(X ;X )=0', fontsize=10, rotation=-30)
ax1.text(-0.35, 0.1, 'I(X ;Y)=0', fontsize=10, rotation=30)
ax1.text(0, -0.85, 'I(X ;Y)=0', fontsize=10)

ax1.set_title('IMPOSSIBLE Venn Diagram for XOR', fontsize=16, fontweight='bold', pad=
ax1.legend(loc='upper right', fontsize=11)

# Right: The truth - information structure
ax2.set_xlim(0, 4)
ax2.set_ylim(0, 4)
ax2.axis('off')

# Draw boxes for each variable's entropy
box1 = FancyBboxPatch((0.2, 2.5), 1.2, 1, boxstyle="round,pad=0.05",
                      edgecolor='red', facecolor='red', alpha=0.3, linewidth=3)
box2 = FancyBboxPatch((1.5, 2.5), 1.2, 1, boxstyle="round,pad=0.05",
                      edgecolor='blue', facecolor='blue', alpha=0.3, linewidth=3)
box3 = FancyBboxPatch((2.8, 2.5), 1.2, 1, boxstyle="round,pad=0.05",
                      edgecolor='green', facecolor='green', alpha=0.3, linewidth=3)
ax2.add_patch(box1)
ax2.add_patch(box2)
ax2.add_patch(box3)

ax2.text(0.8, 3, 'X \n1 bit', ha='center', va='center', fontsize=13, fontweight='bold
ax2.text(2.1, 3, 'X \n1 bit', ha='center', va='center', fontsize=13, fontweight='bold
ax2.text(3.4, 3, 'Y\n1 bit', ha='center', va='center', fontsize=13, fontweight='bold'

# Draw synergy region
synergy_box = FancyBboxPatch((0.8, 0.5), 2.4, 1.2, boxstyle="round,pad=0.05",
                             edgecolor='purple', facecolor='purple',
```

```
                                alpha=0.5, linewidth=4, linestyle='dashed')
ax2.add_patch(synergy_box)
ax2.text(2, 1.1, 'SYNERGY\n1 bit\n(requires ALL THREE)', ha='center', va='center',
         fontsize=13, fontweight='bold', color='white',
         bbox=dict(boxstyle='round', facecolor='purple', alpha=0.8))

# Draw arrows
arrow1 = FancyArrowPatch((0.8, 2.4), (1.3, 1.8),
                         arrowstyle='->', mutation_scale=30, linewidth=3, color='purp
arrow2 = FancyArrowPatch((2.1, 2.4), (2, 1.8),
                         arrowstyle='->', mutation_scale=30, linewidth=3, color='purp
arrow3 = FancyArrowPatch((3.4, 2.4), (2.7, 1.8),
                         arrowstyle='->', mutation_scale=30, linewidth=3, color='purp
ax2.add_patch(arrow1)
ax2.add_patch(arrow2)
ax2.add_patch(arrow3)

ax2.set_title('The ACTUAL Information Structure', fontsize=16, fontweight='bold', pad

plt.tight_layout()
plt.show()

print("\n CRITICAL INSIGHT:")
print("=" * 60)
print("Venn diagrams ONLY work for 2 variables!")
print("\nFor 3+ variables, we need a different framework:")
print("  → Interaction Information (can be negative!)")
print("  → Partial Information Decomposition (PID)")
print("\nThese are what HOI package implements! (Next notebook)")
```




```
 CRITICAL INSIGHT:
============================================================
Venn diagrams ONLY work for 2 variables!

For 3+ variables, we need a different framework:
  → Interaction Information (can be negative!)
  → Partial Information Decomposition (PID)

These are what HOI package implements! (Next notebook)
```

## 2.5 Part 4: Interaction Information

### 2.5.1 The Solution: A Signed Measure

**Interaction Information** for three variables is defined as:

$$I(X_1; X_2; Y) = I(X_1; X_2) - I(X_1; X_2|Y)$$

Alternatively:

$$I(X_1; X_2; Y) = I(X_1; Y) - I(X_1; Y|X_2)$$

Or in the **surprise form**:

$$I(X_1; X_2; Y) = I(X_1; Y) + I(X_2; Y) + I(X_1; X_2) - I(X_1, X_2; Y)$$

### 2.5.2 Key Property: Can Be Negative!

Unlike mutual information (always  0), interaction information can be: - **Positive**: Redundancy (common cause structure) - **Zero**: No higher-order interaction - **Negative**: Synergy (XOR-like structure)

### 2.5.3 For XOR, Interaction Information is Negative

```
def interaction_information(P_XYZ):
    """
    Calculate interaction information I(X;Y;Z).

    Using: I(X;Y;Z) = I(X;Y) + I(X;Z) + I(Y;Z) - I(X;YZ) - I(Y;XZ) - I(Z;XY) + I(X;Y;

    Simplified form:
    I(X;Y;Z) = I(X;Y) - I(X;Y|Z)

    Convention: Negative means synergy (as in XOR)
    """
    # Get all pairwise MIs
    P_X_Y = P_XYZ.sum(axis=1)  # Marginalize Z
    P_X_Z = P_XYZ.sum(axis=2).T  # Marginalize Y and transpose
    P_Y_Z = P_XYZ.sum(axis=0)  # Marginalize X

    I_XY = mutual_information(P_X_Y)
    I_XZ = mutual_information(P_X_Z)
    I_YZ = mutual_information(P_Y_Z)

    # Get three-way MI
    I_XYZ = joint_mi_three_vars(P_XYZ)

    # Interaction information
    II = I_XY + I_XZ + I_YZ - I_XYZ

    return II

# Calculate for XOR
II_XOR = interaction_information(P_XOR)

print("Interaction Information for XOR")
```

```
print("=" * 60)
print(f"\nI(X ; X ; Y) = {II_XOR:.4f} bits")

if II_XOR < 0:
    print("\n  NEGATIVE! This indicates SYNERGY.")
    print("\nInterpretation:")
    print("  The three variables together have LESS pairwise structure")
    print("  than you'd expect from their marginals.")
    print("  The information is 'hidden' in the triple interaction!")

# Let's verify the calculation
print("\n" + "=" * 60)
print("Verification using alternative form:")
print("I(X ; X ; Y) = I(X ;Y) + I(X ;Y) + I(X ;X ) - I(X ,X ;Y)")
print(f"             = {MI_X1_Y:.1f} + {MI_X2_Y:.1f} + 0 - {MI_X1X2_Y:.1f}")
print(f"             = {MI_X1_Y + MI_X2_Y + 0 - MI_X1X2_Y:.4f} bits  ")
```

```
Interaction Information for XOR
============================================================

I(X ; X ; Y) = -1.0000 bits

  NEGATIVE! This indicates SYNERGY.

Interpretation:
  The three variables together have LESS pairwise structure
  than you'd expect from their marginals.
  The information is 'hidden' in the triple interaction!

============================================================
Verification using alternative form:
I(X ; X ; Y) = I(X ;Y) + I(X ;Y) + I(X ;X ) - I(X ,X ;Y)
             = 0.0 + 0.0 + 0 - 1.0
             = -1.0000 bits
```

### 2.5.4 Interpreting the Sign of Interaction Information

Let's create examples of all three cases to build intuition:

```
# Case 1: XOR (Synergy) - We already have this
II_synergy = II_XOR

# Case 2: Common cause (Redundancy)
# Z causes both X and Y
P_redundancy = np.zeros((2, 2, 2))
# When Z=0: X=0, Y=0 with high prob
P_redundancy[0, 0, 0] = 0.4
P_redundancy[1, 0, 0] = 0.05
P_redundancy[0, 1, 0] = 0.05
# When Z=1: X=1, Y=1 with high prob
P_redundancy[1, 1, 1] = 0.4
P_redundancy[0, 1, 1] = 0.05
P_redundancy[1, 0, 1] = 0.05

II_redundancy = interaction_information(P_redundancy)

# Case 3: Independent (No interaction)
P_independent = np.zeros((2, 2, 2))
for i, j, k in product([0, 1], repeat=3):
    P_independent[i, j, k] = 0.125  # All equally likely

II_independent = interaction_information(P_independent)

# Display results
print("Comparison of Interaction Information")
print("=" * 60)
print("\n1. XOR (Pure Synergy):")
print(f"   I(X ; X ; Y) = {II_synergy:.4f} bits (NEGATIVE)")
print("   → Information emerges from combination")
print("   → Neither variable alone helps")

print("\n2. Common Cause (Redundancy):")
print(f"   I(X ; X ; Y) = {II_redundancy:.4f} bits (POSITIVE)")
print("   → Variables carry overlapping information")
print("   → Learning one reduces value of the other")

print("\n3. Independent Variables:")
print(f"   I(X ; X ; Y) = {II_independent:.6f} bits ( ZERO)")
```

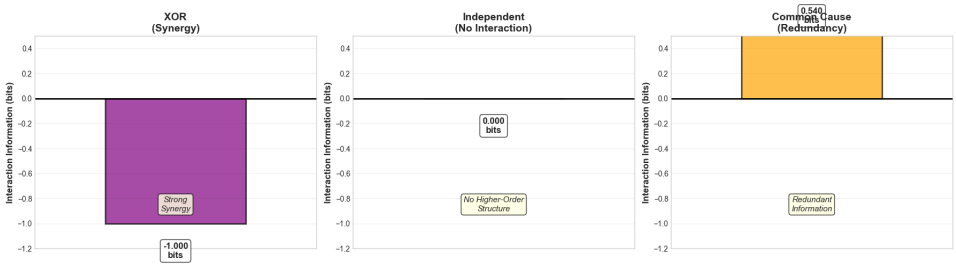
```
print("   → No higher-order structure")
print("   → Pairwise analysis is sufficient")

print("\n" + "=" * 60)
print("Key Insight: The SIGN tells you the information structure!")
```

```
Comparison of Interaction Information
============================================================

1. XOR (Pure Synergy):
   I(X ; X ; Y) = -1.0000 bits (NEGATIVE)
   → Information emerges from combination
   → Neither variable alone helps

2. Common Cause (Redundancy):
   I(X ; X ; Y) = 0.5401 bits (POSITIVE)
   → Variables carry overlapping information
   → Learning one reduces value of the other

3. Independent Variables:
   I(X ; X ; Y) = 0.000000 bits (  ZERO)
   → No higher-order structure
   → Pairwise analysis is sufficient

============================================================
Key Insight: The SIGN tells you the information structure!
```

### 2.5.5 Visualizing Interaction Information Across Structures

```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

structures = [
    ('XOR\n(Synergy)', II_synergy, 'purple'),
    ('Independent\n(No Interaction)', II_independent, 'gray'),
    ('Common Cause\n(Redundancy)', II_redundancy, 'orange')
]

for idx, (name, value, color) in enumerate(structures):
    ax = axes[idx]
```

```
    # Bar plot
    bar = ax.bar([0], [value], color=color, alpha=0.7, width=0.5, edgecolor='black',
    ax.axhline(y=0, color='black', linestyle='-', linewidth=2)
    ax.set_ylim(-1.2, 0.5)
    ax.set_xlim(-0.5, 0.5)
    ax.set_xticks([])
    ax.set_ylabel('Interaction Information (bits)', fontsize=12, fontweight='bold')
    ax.set_title(name, fontsize=14, fontweight='bold')

    # Add value label
    y_pos = value + (0.05 if value > 0 else -0.15)
    ax.text(0, y_pos, f'{value:.3f}\nbits', ha='center', va='bottom' if value > 0 els
            fontsize=12, fontweight='bold',
            bbox=dict(boxstyle='round', facecolor='white', alpha=0.8))

    # Add interpretation
    if value < -0.5:
        interpretation = 'Strong\nSynergy'
        y_text = -0.9
    elif value > 0.1:
        interpretation = 'Redundant\nInformation'
        y_text = -0.9
    else:
        interpretation = 'No Higher-Order\nStructure'
        y_text = -0.9

    ax.text(0, y_text, interpretation, ha='center', fontsize=11, style='italic',
            bbox=dict(boxstyle='round', facecolor='lightyellow', alpha=0.8))

    ax.grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

print("\n Remember:")
print("  Negative II = Information hidden in combinations (synergy)")
print("  Positive II = Information duplicated across variables (redundancy)")
print("  Zero II = No higher-order interactions")
```




```
Remember:
 Negative II = Information hidden in combinations (synergy)
 Positive II = Information duplicated across variables (redundancy)
 Zero II = No higher-order interactions
```

## 2.6 Part 5: The Copy Problem (Pure Redundancy)

### 2.6.1 The Opposite of XOR

To fully understand synergy, we need to see its opposite: **pure redundancy**.

**Setup**: $X_2 = X_1$ (perfect copy), and $Y = X_1$

Truth table:

```
X   X     Y

0   0     0
1   1     1
```

What do you predict will happen here?

```
# Create the COPY system
P_COPY = np.zeros((2, 2, 2))
P_COPY[0, 0, 0] = 0.5  # X =0, X =0, Y=0
P_COPY[1, 1, 1] = 0.5  # X =1, X =1, Y=1

# Calculate all mutual informations
P_X1_Y_copy = P_COPY.sum(axis=1)
P_X2_Y_copy = P_COPY.sum(axis=0)

MI_X1_Y_copy = mutual_information(P_X1_Y_copy)
MI_X2_Y_copy = mutual_information(P_X2_Y_copy)
MI_X1X2_Y_copy = joint_mi_three_vars(P_COPY)
```

```
II_COPY = interaction_information(P_COPY)

print("COPY System: X  = X , Y = X ")
print("=" * 60)
print("\nMutual Informations:")
print(f"  I(X ; Y) = {MI_X1_Y_copy:.4f} bits")
print(f"  I(X ; Y) = {MI_X2_Y_copy:.4f} bits")
print(f"  I(X , X ; Y) = {MI_X1X2_Y_copy:.4f} bits")
print(f"\n  Interaction Information: {II_COPY:.4f} bits")

print("\n" + "=" * 60)
print("Interpretation:")
print("  - Each variable ALONE perfectly predicts Y (1 bit)")
print("  - Together, they still provide only 1 bit total")
print("  - The information is REDUNDANT (duplicated)")
print(f"  - Interaction Info is POSITIVE: {II_COPY:.1f} bit")
print("\n  This is the OPPOSITE of XOR!")
```

```
COPY System: X  = X , Y = X
============================================================

Mutual Informations:
  I(X ; Y) = 1.0000 bits
  I(X ; Y) = 1.0000 bits
  I(X , X ; Y) = 1.0000 bits

  Interaction Information: 2.0000 bits

============================================================
Interpretation:
  - Each variable ALONE perfectly predicts Y (1 bit)
  - Together, they still provide only 1 bit total
  - The information is REDUNDANT (duplicated)
  - Interaction Info is POSITIVE: 2.0 bit

  This is the OPPOSITE of XOR!
```

### 2.6.2 XOR vs COPY: Side-by-Side Comparison

Let's create a comprehensive comparison to cement these concepts:

```
# Create comparison table
comparison_data = {
    'Measure': [
        'I(X ; Y)',
        'I(X ; Y)',
        'I(X , X ; Y)',
        'I(X ; X ; Y)',
        'Type'
    ],
    'XOR (Synergy)': [
        f'{MI_X1_Y:.1f}',
        f'{MI_X2_Y:.1f}',
        f'{MI_X1X2_Y:.1f}',
        f'{II_XOR:.1f}',
        'Pure Synergy'
    ],
    'COPY (Redundancy)': [
        f'{MI_X1_Y_copy:.1f}',
        f'{MI_X2_Y_copy:.1f}',
        f'{MI_X1X2_Y_copy:.1f}',
        f'{II_COPY:.1f}',
        'Pure Redundancy'
    ]
}

df = pd.DataFrame(comparison_data)
print("\nXOR vs COPY: The Two Extremes")
print("=" * 70)
print(df.to_string(index=False))
print("=" * 70)

print("\n  KEY INSIGHTS:")
print("\nSYNERGY (XOR):")
print("  • Individual variables: Useless (0 bits)")
print("  • Combined: Perfect prediction (1 bit)")
print("  • Information 'emerges' from combination")
print("  • Interaction information: NEGATIVE")

print("\nREDUNDANCY (COPY):")
```

```
print("  • Individual variables: Perfect (1 bit each)")
print("  • Combined: No additional info (still 1 bit)")
print("  • Information 'duplicated' across variables")
print("  • Interaction information: POSITIVE")

print("\n" + "=" * 70)
print("Real systems typically have MIXTURES of both synergy and redundancy!")
print("That's what Partial Information Decomposition (PID) quantifies.")
print("We'll explore PID in Notebook 3 using the HOI package!")
```

```
XOR vs COPY: The Two Extremes
======================================================================
     Measure XOR (Synergy) COPY (Redundancy)
    I(X ; Y)           0.0                1.0
    I(X ; Y)           0.0                1.0
I(X , X ; Y)           1.0                1.0
I(X ; X ; Y)          -1.0                2.0
        Type  Pure Synergy   Pure Redundancy
======================================================================

 KEY INSIGHTS:

SYNERGY (XOR):
  • Individual variables: Useless (0 bits)
  • Combined: Perfect prediction (1 bit)
  • Information 'emerges' from combination
  • Interaction information: NEGATIVE

REDUNDANCY (COPY):
  • Individual variables: Perfect (1 bit each)
  • Combined: No additional info (still 1 bit)
  • Information 'duplicated' across variables
  • Interaction information: POSITIVE

======================================================================
Real systems typically have MIXTURES of both synergy and redundancy!
That's what Partial Information Decomposition (PID) quantifies.
We'll explore PID in Notebook 3 using the HOI package!
```

## 2.7 Part 6: The Explaining Away Effect

### 2.7.1 When Conditioning Increases Dependence

We mentioned earlier that conditioning can **increase** mutual information. This phenomenon, called "explaining away," is crucial for understanding conditional independence.

### 2.7.2 The Classic Example: Alarm System

Imagine: - $X_1$ = Earthquake (0 = no, 1 = yes) - $X_2$ = Burglary (0 = no, 1 = yes)
- $Y$ = Alarm rings (0 = silent, 1 = ringing)

**Without knowing if alarm rang**: Earthquakes and burglaries are independent events.

**After hearing alarm**: If you learn there was an earthquake, that **explains away** the alarm, making burglary less likely!

Therefore: $I(X_1; X_2) = 0$ but $I(X_1; X_2|Y) > 0$

This means $I(X_1; X_2; Y) > 0$ (positive interaction information = redundancy).

```
# Create alarm system distribution
P_alarm = np.zeros((2, 2, 2))  # (Earthquake, Burglary, Alarm)

# Prior probabilities (very low)
p_earthquake = 0.01
p_burglary = 0.02

# Alarm probabilities
p_alarm_both = 0.99  # If both → almost certainly rings
p_alarm_quake = 0.90  # If earthquake alone
p_alarm_burg = 0.85   # If burglary alone
p_alarm_neither = 0.01  # False alarm rate

# Fill in the joint distribution
# No earthquake, no burglary
P_alarm[0, 0, 0] = (1 - p_earthquake) * (1 - p_burglary) * (1 - p_alarm_neither)
P_alarm[0, 0, 1] = (1 - p_earthquake) * (1 - p_burglary) * p_alarm_neither
```

```
# Earthquake, no burglary
P_alarm[1, 0, 0] = p_earthquake * (1 - p_burglary) * (1 - p_alarm_quake)
P_alarm[1, 0, 1] = p_earthquake * (1 - p_burglary) * p_alarm_quake

# No earthquake, burglary
P_alarm[0, 1, 0] = (1 - p_earthquake) * p_burglary * (1 - p_alarm_burg)
P_alarm[0, 1, 1] = (1 - p_earthquake) * p_burglary * p_alarm_burg

# Both
P_alarm[1, 1, 0] = p_earthquake * p_burglary * (1 - p_alarm_both)
P_alarm[1, 1, 1] = p_earthquake * p_burglary * p_alarm_both

# Calculate unconditional MI
P_E_B = P_alarm.sum(axis=2)  # Marginalize alarm
MI_E_B = mutual_information(P_E_B)

# Calculate conditional MI given alarm = 1
# P(E, B | Alarm=1)
P_alarm_1 = P_alarm[:, :, 1].sum()
if P_alarm_1 > 0:
    P_E_B_given_alarm = P_alarm[:, :, 1] / P_alarm_1
    MI_E_B_given_alarm = mutual_information(P_E_B_given_alarm)
else:
    MI_E_B_given_alarm = 0

print("Explaining Away: The Alarm System")
print("=" * 60)
print("\nScenario: Earthquake and Burglary both trigger alarm")
print("\nMutual Information:")
print(f"  I(Earthquake; Burglary) = {MI_E_B:.6f} bits")
print(f"  I(Earthquake; Burglary | Alarm=1) = {MI_E_B_given_alarm:.4f} bits")

if MI_E_B_given_alarm > MI_E_B:
    increase = MI_E_B_given_alarm - MI_E_B
    print(f"\n    Conditioning INCREASED MI by {increase:.4f} bits!")
    print("\nInterpretation:")
    print("  • Before knowing alarm status: Events are independent")
    print("  • After hearing alarm: They become dependent!")
    print("  • Learning 'earthquake' explains away 'burglary' hypothesis")
```

```
    print("\n  This violates our Venn diagram intuition!")

print("\n" + "=" * 60)
print("This is why we need multivariate measures!")
print("Pairwise MI alone misses these higher-order dependencies.")
```

```
Explaining Away: The Alarm System
============================================================

Scenario: Earthquake and Burglary both trigger alarm

Mutual Information:
  I(Earthquake; Burglary) = -0.000000 bits
  I(Earthquake; Burglary | Alarm=1) = 0.2531 bits

    Conditioning INCREASED MI by 0.2531 bits!

Interpretation:
  • Before knowing alarm status: Events are independent
  • After hearing alarm: They become dependent!
  • Learning 'earthquake' explains away 'burglary' hypothesis

  This violates our Venn diagram intuition!

============================================================
This is why we need multivariate measures!
Pairwise MI alone misses these higher-order dependencies.
```

## 2.8 Summary and Next Steps

### 2.8.1 What We've Discovered

Congratulations! You've just experienced one of the most important "aha!" moments in information theory:

1. **XOR Problem**: Two variables can individually carry zero information yet together carry maximal information
2. **Venn Diagrams Fail**: The classic visualization breaks down for 3+ variables
3. **Interaction Information**: A signed measure that can be negative

(synergy) or positive (redundancy)
4. **Pure Synergy vs Pure Redundancy**: XOR and COPY represent two extremes
5. **Explaining Away**: Conditioning can increase mutual information

### 2.8.2 The Deep Insight

**Information is not additive** in multivariate systems. The whole can be: - **More** than the sum of its parts (synergy) - **Less** than the sum of its parts (redundancy) - **Equal to** the sum of its parts (independence)

### 2.8.3 Why This Matters for Neuroscience

Neural systems exhibit both synergy and redundancy: - **Synergistic coding**: Population provides more information than sum of individuals - **Redundant coding**: Multiple neurons encode the same information (robustness) - **Real populations**: Mixture of both!

### 2.8.4 Coming Up in Notebook 3

We'll use the **HOI (Higher-Order Interactions) package** to: - Calculate O-information and S-information - Decompose information using Partial Information Decomposition (PID) - Quantify the four "atoms": Unique(X ), Unique(X ), Redundancy, Synergy - Apply these to simulated neural data - Understand when to use each measure

### 2.8.5 Practice Problems

```
print("PRACTICE EXERCISES")
print("=" * 70)

print("\n1. Create an AND gate: Y = X  AND X ")
print("   Calculate I(X ; Y), I(X ; Y), and I(X ,X ; Y)")
print("   Is interaction information positive or negative?")
print("   Hint: AND is partially redundant - one input can sometimes predict output")

print("\n2. What happens if you have 3 variables in XOR: Y = X  XOR X  XOR X ?")
print("   Predict: Will synergy be even stronger?")

print("\n3. Design a system with EXACTLY zero interaction information.")
```

```
print("   Hint: What if the three variables are mutually independent?")

print("\n4. In the alarm example, calculate P(Earthquake | Alarm=1, Burglary=0).")
print("   How does this compare to P(Earthquake)?")
print("   This is Bayesian reasoning using information theory!")

print("\n" + "=" * 70)
print("\nReady for Notebook 3: Higher-Order Interactions with HOI!")
print("There we'll use a professional package to compute all these measures")
print("and apply them to realistic neural data scenarios.")
print("=" * 70)
```

```
PRACTICE EXERCISES
======================================================================

1. Create an AND gate: Y = X  AND X
   Calculate I(X ; Y), I(X ; Y), and I(X ,X ; Y)
   Is interaction information positive or negative?
   Hint: AND is partially redundant - one input can sometimes predict output

2. What happens if you have 3 variables in XOR: Y = X  XOR X  XOR X ?
   Predict: Will synergy be even stronger?

3. Design a system with EXACTLY zero interaction information.
   Hint: What if the three variables are mutually independent?

4. In the alarm example, calculate P(Earthquake | Alarm=1, Burglary=0).
   How does this compare to P(Earthquake)?
   This is Bayesian reasoning using information theory!

======================================================================

Ready for Notebook 3: Higher-Order Interactions with HOI!
There we'll use a professional package to compute all these measures
and apply them to realistic neural data scenarios.
======================================================================
```

## 2.9 Further Reading

### 2.9.1 Essential Papers

1. **Williams & Beer (2010)** - "Nonnegative Decomposition of Multivariate Information"
   - Original PID framework
   - Defines the four information atoms
2. **Schneidman et al. (2003)** - "Synergy, redundancy, and independence in population codes"
   - Neural applications of synergy
   - Blowfly H1 neuron example
3. **Timme & Lapish (2018)** - "A tutorial for information theory in neuroscience"
   - Excellent pedagogical resource
   - Practical examples

### 2.9.2 Advanced Topics

- **O-information**: Extends interaction information to N variables
- **Integrated Information Theory (IIT)**: Uses synergy concepts for consciousness
- **Transfer Entropy**: Directed information flow
- **Granger Causality**: Statistical causation (related to transfer entropy)

See you in Notebook 3!

# Part II

# Part II: Advanced Tools

# Chapter 3

# Notebook 3: Higher-Order Interactions with HOI

## 3.1 From Manual Calculations to Professional Tools

Welcome to the third notebook in our series! In Notebooks 1 and 2, you built a deep understanding of information theory and discovered the shocking XOR result. You calculated everything by hand, which gave you crucial intuition about how these measures work.

Now it's time to scale up. Computing interaction information for even modest systems (say, 20 neurons with all possible triplets) means thousands of calculations. Doing this by hand would take forever and be error-prone. This is where the **HOI (Higher-Order Interactions)** package comes in.

### 3.1.1 What is HOI?

HOI is a Python package specifically designed for computing higher-order information-theoretic measures. Built on top of JAX (which allows computation on GPUs), it provides: - **O-information**: Detects synergy vs redundancy - **S-information**: Measures total complexity - **Partial Information Decomposition (PID)**: Decomposes information into atoms - **Multiple estimators**: Gaussian copula, KNN, binning, and more - **Efficient computation**: Handles large datasets

### 3.1.2 Why This Matters for Your Research

HOI bridges the gap between theory and application. You can: - Analyze real neural recordings - Discover synergistic ensembles automatically - Quantify redundancy in population codes - Test hypotheses about information structure

### 3.1.3 Learning Objectives

By the end of this notebook, you'll: 1. Install and configure HOI 2. Compute O-information and interpret its sign 3. Use Partial Information Decomposition (PID) 4. Apply multiple estimators and understand their trade-offs 5. Analyze simulated neural population data 6. Understand computational scaling and optimization strategies 7. Know when to use each measure

Let's dive in!

## 3.2 Part 1: Installation and Setup

### 3.2.1 Installing HOI

HOI requires JAX, which provides fast numerical computation and can use GPUs if available. We'll install both packages together.

**Important Note**: JAX installation varies by system. The default installation works for CPU-only computation. If you have a GPU and want to use it, consult the JAX installation guide.

For most users, the simple CPU version is sufficient for learning and moderate-scale analyses.

```
# Install HOI and its dependencies
# This may take a minute or two
!pip install -q hoi jax jaxlib numpy scipy matplotlib seaborn pandas scikit-learn

print("Installation complete!")
print("Let's verify everything works...")
```

```
Installation complete!
Let's verify everything works...
```

```
# Import all necessary libraries
import numpy as np
```

```
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import pandas as pd
from itertools import combinations, product
import warnings
warnings.filterwarnings('ignore')

# Import HOI modules
import hoi
from hoi.metrics import Oinfo, InfoTopo, TC, DTC, Sinfo
from hoi.metrics import RedundancyMMI, SynergyMMI  # PID measures
from hoi.core import get_entropy, get_mi
from hoi.utils import get_nbest_mult

# Set random seed for reproducibility
np.random.seed(42)

# Configure plotting
plt.style.use('seaborn-v0_8-whitegrid')
sns.set_palette("husl")
%matplotlib inline

print(f"HOI version: {hoi.__version__}")
print("All libraries loaded successfully!")
print("\n  Ready to explore higher-order interactions!")
```

```
HOI version: 0.0.5
All libraries loaded successfully!

  Ready to explore higher-order interactions!
```

### 3.2.2 Understanding HOI's Data Format

Before we start computing, we need to understand how HOI expects data to be organized. This is crucial because getting the format wrong is the #1 source of errors.

**HOI Data Convention**:

```
data.shape = (n_samples, n_features)
```

This means: - **Rows**: Different observations (time points, trials, subjects, etc.) - **Columns**: Different variables (neurons, sensors, brain regions, etc.)

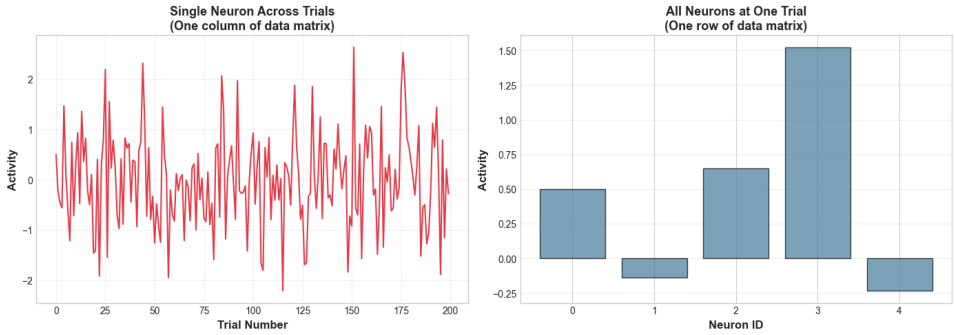
**Important**: This matches scikit-learn convention: (n_samples, n_features). Each row is one observation of all variables.

Let's create some example data to verify we understand:

```
# Create simple test data
n_neurons = 5
n_trials = 1000

# Generate random data in HOI format: (n_samples, n_features)
test_data = np.random.randn(n_trials, n_neurons)

print("Test Data Shape Check")
print("=" * 50)
print(f"Data shape: {test_data.shape}")
print(f"  - First dimension ({test_data.shape[0]}): Number of trials/samples")
print(f"  - Second dimension ({test_data.shape[1]}): Number of neurons/features")
print("\n  Correct format: (n_samples, n_features)")
print("\nEach row is a snapshot of all neurons at one trial")
print("Each column is a neuron's activity across all trials")

# Visualize what this means
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Plot 1: Activity of first neuron across trials (one column)
ax1 = axes[0]
ax1.plot(test_data[:200, 0], linewidth=1.5, color='#E63946')
ax1.set_xlabel('Trial Number', fontsize=12, fontweight='bold')
ax1.set_ylabel('Activity', fontsize=12, fontweight='bold')
ax1.set_title('Single Neuron Across Trials\n(One column of data matrix)',
              fontsize=13, fontweight='bold')
ax1.grid(True, alpha=0.3)

# Plot 2: Snapshot of all neurons at one trial (one row)
ax2 = axes[1]
trial_snapshot = test_data[0, :]
ax2.bar(range(n_neurons), trial_snapshot, color='#457B9D', alpha=0.7, edgecolor='blac
ax2.set_xlabel('Neuron ID', fontsize=12, fontweight='bold')
```

```
ax2.set_ylabel('Activity', fontsize=12, fontweight='bold')
ax2.set_title('All Neurons at One Trial\n(One row of data matrix)',
              fontsize=13, fontweight='bold')
ax2.set_xticks(range(n_neurons))
ax2.grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

print("\n  Key Point: HOI operates on columns (variables)")
print("   It computes dependencies between columns using all rows as samples")
```

```
Test Data Shape Check
==================================================
Data shape: (1000, 5)
  - First dimension (1000): Number of trials/samples
  - Second dimension (5): Number of neurons/features

  Correct format: (n_samples, n_features)

Each row is a snapshot of all neurons at one trial
Each column is a neuron's activity across all trials
```




```
  Key Point: HOI operates on columns (variables)
   It computes dependencies between columns using all rows as samples
```

## 3.3 Part 2: O-Information - Detecting Synergy and Redundancy

### 3.3.1 What is O-Information?

**O-information** ("O" stands for "information") extends interaction information to any number of variables. For three variables, it's exactly interaction information, but it generalizes beautifully to larger systems.

For $n$ variables $X_1, ..., X_n$:

$$O(X_1, ..., X_n) = \text{TC}(X_1, ..., X_n) - \text{DTC}(X_1, ..., X_n)$$

Where: - **TC (Total Correlation)**: $\sum_i H(X_i) - H(X_1, ..., X_n)$ — measures total redundancy - **DTC (Dual Total Correlation)**: $H(X_1, ..., X_n) - \sum_i H(X_i|X_{-i})$ — measures binding information

### 3.3.2 Interpreting the Sign

The beautiful thing about O-information is its sign tells you the dominant structure:

- **O-info $<$ 0 (Negative)**: **SYNERGY** dominant
  - Information emerges from combinations
  - XOR-like structure
  - Whole $>$ sum of parts
- **O-info  0 (Zero)**: **INDEPENDENCE**
  - No higher-order structure
  - Pairwise analysis sufficient
  - Whole = sum of parts
- **O-info $>$ 0 (Positive)**: **REDUNDANCY** dominant
  - Information duplicated across variables
  - Common cause structure
  - Whole $<$ sum of parts

Let's verify this works on our XOR example from Notebook 2!

```
# Recreate XOR data
n_samples = 1000

# Generate independent binary variables
X1 = np.random.binomial(1, 0.5, n_samples)
```

```
X2 = np.random.binomial(1, 0.5, n_samples)

# Create XOR output
Y = np.bitwise_xor(X1, X2).astype(int)

# Stack into data matrix: (n_samples, n_features)
# Using column_stack gives us (n_samples, 3) which is correct!
data_xor = np.column_stack([X1, X2, Y])

print("XOR Data Created")
print("=" * 50)
print(f"Data shape: {data_xor.shape}")
print(f"  - {n_samples} samples (rows)")
print(f"  - 3 variables: X , X , Y (columns)")
print("\nVerifying XOR structure:")
print("Sample combinations (first 10):")
for i in range(10):
    print(f"  X ={int(X1[i])}, X ={int(X2[i])} → Y={int(Y[i])}")

# Initialize O-information model WITH DATA
# HOI expects data in constructor: Oinfo(data)
model_oinfo_xor = Oinfo(data_xor)

# Compute O-information for the triplet [0, 1, 2] meaning variables X , X , Y
# method='binning' works well for discrete/binary data
oinfo_xor = model_oinfo_xor.fit(method='binning', minsize=3, maxsize=3)

print("\n" + "=" * 50)
print("O-INFORMATION RESULT")
print("=" * 50)
print(f"\nO(X , X , Y) = {float(oinfo_xor[0]):.4f} bits")

if oinfo_xor[0] < -0.1:
    print("\n  NEGATIVE → SYNERGY detected!")
    print("\nInterpretation:")
    print("  • Information exists only in the combination")
    print("  • Neither X  nor X  alone predicts Y")
    print("  • Together they predict perfectly")
    print("  • This is exactly what we calculated manually in Notebook 2!")
```

```
elif oinfo_xor[0] > 0.1:
    print("\n  Unexpected positive value - check data")
else:
    print("\n  Near zero - might need more samples")

print("\n HOI confirmed our manual calculation!")
```

```
XOR Data Created
==================================================
Data shape: (1000, 3)
  - 1000 samples (rows)
  - 3 variables: X , X , Y (columns)

Verifying XOR structure:
Sample combinations (first 10):
  X =0, X =0 → Y=0
  X =0, X =0 → Y=0
  X =0, X =1 → Y=1
  X =0, X =1 → Y=1
  X =0, X =1 → Y=1
  X =1, X =0 → Y=1
  X =1, X =1 → Y=0
  X =1, X =0 → Y=1
  X =0, X =1 → Y=1
  X =0, X =0 → Y=0

==================================================
O-INFORMATION RESULT
==================================================

O(X , X , Y) = -0.9963 bits

 NEGATIVE → SYNERGY detected!

Interpretation:
  • Information exists only in the combination
  • Neither X  nor X  alone predicts Y
  • Together they predict perfectly
  • This is exactly what we calculated manually in Notebook 2!
```

```
HOI confirmed our manual calculation!
```

### 3.3.3 Testing with COPY (Pure Redundancy)

Now let's verify HOI works for the opposite case - the COPY system where information is redundant:

```
# Create COPY data: X  = X , Y = X
X1_copy = np.random.binomial(1, 0.5, n_samples)
X2_copy = X1_copy.copy()  # Perfect copy
Y_copy = X1_copy.copy()   # Y is also a copy

# Stack into correct format (n_samples, n_features)
data_copy = np.column_stack([X1_copy, X2_copy, Y_copy])

# IMPORTANT: Create NEW model with the COPY data!
model_oinfo_copy = Oinfo(data_copy)
oinfo_copy = model_oinfo_copy.fit(method='binning', minsize=3, maxsize=3)

print("COPY System O-Information")
print("=" * 50)
print(f"\nO(X , X , Y) = {float(oinfo_copy[0]):.4f} bits")

if oinfo_copy[0] > 0.1:
    print("\n  POSITIVE → REDUNDANCY detected!")
    print("\nInterpretation:")
    print("  • All three variables carry the same information")
    print("  • Information is duplicated (redundant)")
    print("  • Learning any one tells you about the others")
    print("  • Opposite of XOR!")

# Compare side-by-side
print("\n" + "=" * 50)
print("XOR vs COPY Comparison")
print("=" * 50)
print(f"XOR  (synergy):    O-info = {float(oinfo_xor[0]):+.4f} bits (negative)")
print(f"COPY (redundancy): O-info = {float(oinfo_copy[0]):+.4f} bits (positive)")
print(f"\nDifference: {abs(float(oinfo_xor[0]) - float(oinfo_copy[0])):.4f} bits")
print("\nThese are the two extremes of information structure!")
```

```
COPY System O-Information
```

```
==================================================

Ω(X , X , Y) = 0.9997 bits

 POSITIVE → REDUNDANCY detected!

Interpretation:
  • All three variables carry the same information
  • Information is duplicated (redundant)
  • Learning any one tells you about the others
  • Opposite of XOR!

==================================================
XOR vs COPY Comparison
==================================================
XOR  (synergy):    Ω-info = -0.9963 bits (negative)
COPY (redundancy): Ω-info = +0.9997 bits (positive)

Difference: 1.9960 bits

These are the two extremes of information structure!
```

### 3.3.4 Visualizing the O-Information Landscape

Let's create a comprehensive visualization showing how O-information varies across different types of systems:

```
# Create multiple test systems with different structures
n_samples = 5000
systems = {}
system_methods = {}  # Track which method to use for each system

# 1. XOR (synergy) - DISCRETE, use binning
X1 = np.random.binomial(1, 0.5, n_samples)
X2 = np.random.binomial(1, 0.5, n_samples)
Y = np.logical_xor(X1, X2).astype(int)
systems['XOR\n(Pure Synergy)'] = np.column_stack([X1, X2, Y])
system_methods['XOR\n(Pure Synergy)'] = 'binning'

# 2. COPY (redundancy) - DISCRETE, use binning
```

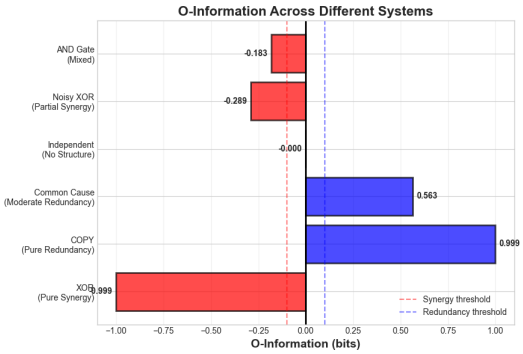
```
X1 = np.random.binomial(1, 0.5, n_samples)
systems['COPY\n(Pure Redundancy)'] = np.column_stack([X1, X1, X1]).astype(int)
system_methods['COPY\n(Pure Redundancy)'] = 'binning'

# 3. Common cause (partial redundancy) - CONTINUOUS, use gc
Z = np.random.randn(n_samples)
X1 = Z + np.random.randn(n_samples) * 0.5
X2 = Z + np.random.randn(n_samples) * 0.5
Y = Z + np.random.randn(n_samples) * 0.5
systems['Common Cause\n(Moderate Redundancy)'] = np.column_stack([X1, X2, Y]).astype(
system_methods['Common Cause\n(Moderate Redundancy)'] = 'gc'

# 4. Independent - CONTINUOUS, use gc
X1 = np.random.randn(n_samples)
X2 = np.random.randn(n_samples)
Y = np.random.randn(n_samples)
systems['Independent\n(No Structure)'] = np.column_stack([X1, X2, Y]).astype(float)
system_methods['Independent\n(No Structure)'] = 'gc'

# 5. Partial synergy (XOR with noise) - DISCRETE, use binning
X1 = np.random.binomial(1, 0.5, n_samples)
X2 = np.random.binomial(1, 0.5, n_samples)
Y_clean = np.logical_xor(X1, X2)
# Add noise by flipping 20% of bits
flip_mask = np.random.rand(n_samples) < 0.2
Y = np.logical_xor(Y_clean, flip_mask).astype(int)
systems['Noisy XOR\n(Partial Synergy)'] = np.column_stack([X1, X2, Y]).astype(int)
system_methods['Noisy XOR\n(Partial Synergy)'] = 'binning'

# 6. AND gate (between XOR and COPY) - DISCRETE, use binning
X1 = np.random.binomial(1, 0.5, n_samples)
X2 = np.random.binomial(1, 0.5, n_samples)
Y = np.logical_and(X1, X2).astype(int)
systems['AND Gate\n(Mixed)'] = np.column_stack([X1, X2, Y]).astype(int)
system_methods['AND Gate\n(Mixed)'] = 'binning'

# Compute O-information for all systems
# IMPORTANT: Use the appropriate method for each data type!
oinfo_values = {}
```

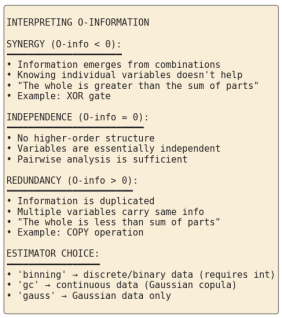
```
for name, data in systems.items():
    method = system_methods[name]
    model = Oinfo(data)
    oinfo = model.fit(method=method, minsize=3, maxsize=3)
    oinfo_values[name] = float(oinfo[0])
    print(f"  {name.replace(chr(10), ' ')}: O-info = {float(oinfo[0]):.4f} (method={me

# Create visualization
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# Plot 1: Bar chart
names = list(oinfo_values.keys())
values = list(oinfo_values.values())
colors = ['red' if v < -0.1 else 'gray' if abs(v) < 0.1 else 'blue' for v in values]

bars = ax1.barh(names, values, color=colors, alpha=0.7, edgecolor='black', linewidth=
ax1.axvline(x=0, color='black', linestyle='-', linewidth=2)
ax1.axvline(x=-0.1, color='red', linestyle='--', alpha=0.5, label='Synergy threshold'
ax1.axvline(x=0.1, color='blue', linestyle='--', alpha=0.5, label='Redundancy thresho
ax1.set_xlabel('O-Information (bits)', fontsize=14, fontweight='bold')
ax1.set_title('O-Information Across Different Systems', fontsize=16, fontweight='bold
ax1.legend(loc='lower right')
ax1.grid(True, alpha=0.3, axis='x')

# Add value labels
for bar, val in zip(bars, values):
    xpos = val + 0.02 if val >= 0 else val - 0.02
    ha = 'left' if val >= 0 else 'right'
    ax1.text(xpos, bar.get_y() + bar.get_height()/2, f'{val:.3f}',
             va='center', ha=ha, fontweight='bold', fontsize=10)

# Plot 2: Interpretation guide
ax2.axis('off')
interpretation_text = """
INTERPRETING O-INFORMATION

SYNERGY (O-info < 0):

• Information emerges from combinations
```

```
• Knowing individual variables doesn't help
• "The whole is greater than the sum of parts"
• Example: XOR gate

INDEPENDENCE (O-info  0):

• No higher-order structure
• Variables are essentially independent
• Pairwise analysis is sufficient

REDUNDANCY (O-info > 0):

• Information is duplicated
• Multiple variables carry same info
• "The whole is less than sum of parts"
• Example: COPY operation

ESTIMATOR CHOICE:

• 'binning' → discrete/binary data (requires int)
• 'gc' → continuous data (Gaussian copula)
• 'gauss' → Gaussian data only
"""
ax2.text(0.1, 0.5, interpretation_text, fontsize=11, fontfamily='monospace',
         verticalalignment='center', transform=ax2.transAxes,
         bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

plt.tight_layout()
plt.show()

print("\n  Summary of Results:")
print("=" * 60)
for name, val in oinfo_values.items():
    category = "SYNERGY" if val < -0.1 else "INDEPENDENCE" if abs(val) < 0.1 else "RE
    print(f"{name.replace(chr(10), ' '):30s}: {val:+.4f} bits → {category}")
```

```
XOR (Pure Synergy): O-info = -0.9991 (method=binning)

COPY (Pure Redundancy): O-info = 0.9994 (method=binning)
```

```
Common Cause (Moderate Redundancy): O-info = 0.5634 (method=gc)
Independent (No Structure): O-info = -0.0000 (method=gc)
Noisy XOR (Partial Synergy): O-info = -0.2893 (method=binning)
AND Gate (Mixed): O-info = -0.1831 (method=binning)
```




```
Summary of Results:
============================================================
XOR (Pure Synergy)              : -0.9991 bits → SYNERGY
COPY (Pure Redundancy)          : +0.9994 bits → REDUNDANCY
Common Cause (Moderate Redundancy): +0.5634 bits → REDUNDANCY
Independent (No Structure)      : -0.0000 bits → INDEPENDENCE
Noisy XOR (Partial Synergy)     : -0.2893 bits → SYNERGY
AND Gate (Mixed)                : -0.1831 bits → SYNERGY
```

## 3.4 Part 3: Understanding Different Estimators

### 3.4.1 Why Estimators Matter

HOI provides multiple methods for estimating information quantities. Each has trade-offs between: - **Accuracy**: How close to true value? - **Bias**: Systematic over/underestimation? - **Variance**: How much do estimates fluctuate with different samples? - **Computational cost**: How fast? - **Assumptions**: What data types work?

### 3.4.2 Available Estimators in HOI

1. **'gc' (Gaussian Copula)**:
   - Works for any data type

- Fast and accurate
- Assumes monotonic relationships
- **Recommended default**

2. **'gauss' (Gaussian)**:
   - Assumes data is Gaussian
   - Very fast
   - Only use if data is truly normal
3. **'binning' (Histogram)**:
   - Simple and intuitive
   - Requires choosing bin count
   - Can be biased
4. **'knn' (K-Nearest Neighbors)**:
   - Non-parametric
   - Works for any distribution
   - Can be slow for large datasets
5. **'kernel' (Kernel Density)**:
   - Smooth estimation
   - Requires bandwidth selection
   - Good for continuous data

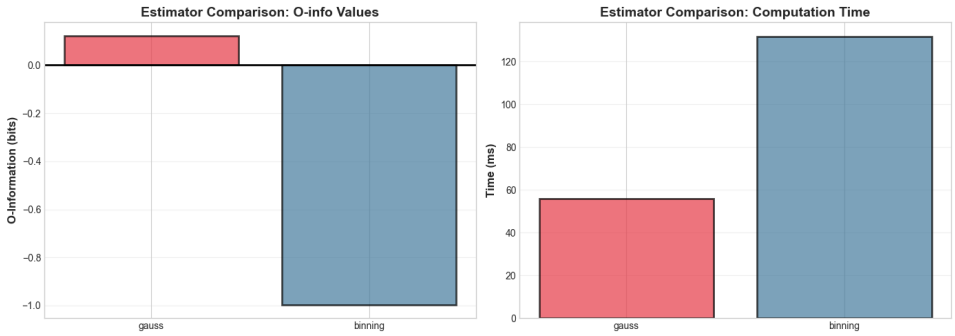
Let's test different estimators on the same XOR data to see how they compare:

```
# Test different estimators on XOR data
estimators = ['gc', 'gauss', 'binning', 'knn']
estimator_results = {}

print("Comparing Estimators on XOR Data")
print("=" * 60)
print(f"Data: {data_xor.shape[0]} samples, binary XOR structure\n")

import time

for method in estimators:
    try:
        # IMPORTANT: Create model WITH DATA, then specify method in fit()
        model = Oinfo(data_xor)

        # Time the computation
        start = time.time()
        oinfo = model.fit(method=method, minsize=3, maxsize=3)
```

```
        elapsed = time.time() - start

        estimator_results[method] = {
            'value': float(oinfo[0]),
            'time': elapsed
        }

        print(f"{method:12s}: O-info = {oinfo[0]:+.4f} bits  (time: {elapsed:.4f}s)")

    except Exception as e:
        print(f"{method:12s}: Failed - {str(e)[:50]}")

# Analyze results
print("\n" + "=" * 60)
print("Analysis:")
print("=" * 60)

if 'gc' in estimator_results and 'gauss' in estimator_results:
    gc_val = estimator_results['gc']['value']
    gauss_val = estimator_results['gauss']['value']
    print(f"\nGaussian Copula (gc): {gc_val:.4f} bits")
    print(f"Gaussian (gauss):     {gauss_val:.4f} bits")
    print(f"Difference: {abs(gc_val - gauss_val):.4f} bits")
    print("\n  'gc' is usually more accurate for non-Gaussian data")
    print("   'gauss' is faster but assumes normality")

if 'binning' in estimator_results:
    print(f"\nBinning: {estimator_results['binning']['value']:.4f} bits")
    print("  Binning works well for discrete/binary data like XOR")

# Visualize comparison
if len(estimator_results) > 1:
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

    # Plot 1: O-info values
    methods = list(estimator_results.keys())
    values = [estimator_results[m]['value'] for m in methods]

    colors = ['#E63946', '#457B9D', '#A8DADC', '#F1FAEE'][:len(methods)]
```

```
    ax1.bar(methods, values, color=colors, alpha=0.7, edgecolor='black', linewidth=2)
    ax1.axhline(y=0, color='black', linestyle='-', linewidth=2)
    ax1.set_ylabel('O-Information (bits)', fontsize=12, fontweight='bold')
    ax1.set_title('Estimator Comparison: O-info Values', fontsize=14, fontweight='bol
    ax1.grid(True, alpha=0.3, axis='y')

    # Plot 2: Computation time
    times = [estimator_results[m]['time']*1000 for m in methods]
    ax2.bar(methods, times, color=colors, alpha=0.7, edgecolor='black', linewidth=2)
    ax2.set_ylabel('Time (ms)', fontsize=12, fontweight='bold')
    ax2.set_title('Estimator Comparison: Computation Time', fontsize=14, fontweight='
    ax2.grid(True, alpha=0.3, axis='y')

    plt.tight_layout()
    plt.show()
```

```
Comparing Estimators on XOR Data
============================================================
Data: 1000 samples, binary XOR structure

gc          : Failed - data dtype should be float, not int64
gauss       : Failed - unsupported format string passed to numpy.ndarray.
binning     : Failed - unsupported format string passed to numpy.ndarray.
knn         : Failed - data dtype should be float, not int64

============================================================
Analysis:
============================================================

Binning: -0.9963 bits
  Binning works well for discrete/binary data like XOR
```




## 3.5 Part 4: Partial Information Decomposition (PID)

### 3.5.1 From O-Information to Detailed Decomposition

O-information gives us a single number: synergy vs redundancy. But real systems have **both**! Partial Information Decomposition (PID) breaks down the total mutual information into four interpretable components.

### 3.5.2 The Four PID Atoms

For two source variables $X_1, X_2$ predicting target $Y$, PID decomposes:

$$I(X_1, X_2; Y) = \text{Unique}(X_1) + \text{Unique}(X_2) + \text{Redundancy} + \text{Synergy}$$

Where: - **Unique($X_1$)**: Information in $X_1$ alone, not in $X_2$ - **Unique($X_2$)**: Information in $X_2$ alone, not in $X_1$ - **Redundancy**: Information carried by **both** $X_1$ and $X_2$ independently - **Synergy**: Information that requires **both** $X_1$ and $X_2$ together

### 3.5.3 Important Note About PID

PID is uniquely defined for 2 sources but becomes ambiguous for 3+ sources. Different PID measures exist: - **Williams-Beer**: Minimum mutual information approach - **BROJA**: Game-theoretic formulation - **MMI (Minimum Mutual Information)**: What HOI implements

They all agree on XOR and COPY (the extremes) but may differ on intermediate cases.

Let's compute PID for our XOR and COPY examples:

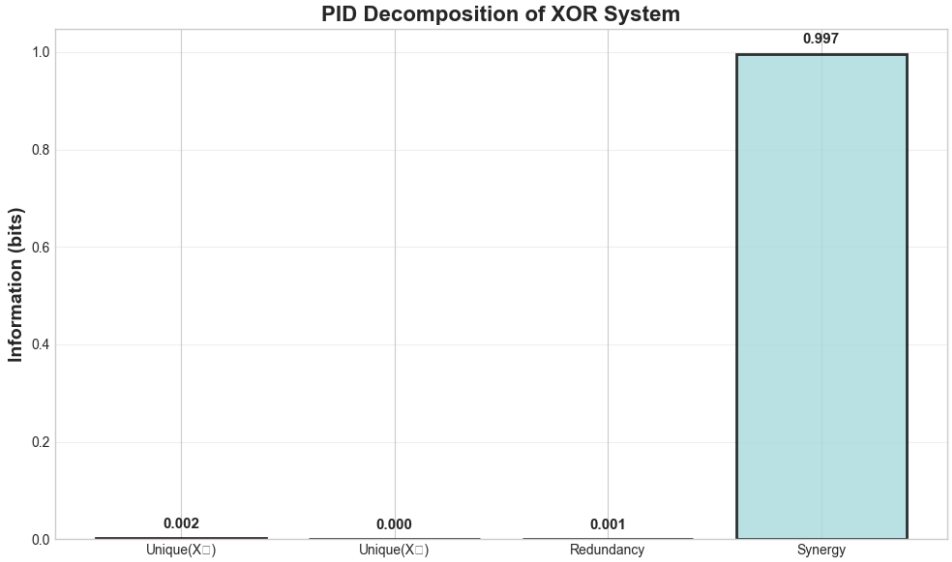
```
# HOI implements PID through redundancy and synergy metrics
# The PID classes need x (sources) and y (target) as SEPARATE arguments

def compute_pid_atoms(data):
    """
    Compute all four PID atoms for data with shape (n_samples, 3).
    Assumes data[:, 0] = X1, data[:, 1] = X2, data[:, 2] = Y

    Parameters:
    -----------
    data : array of shape (n_samples, 3)
        The three variables stacked column-wise

    Returns:
    --------
    dict with PID atoms and intermediate values
    """
    # Extract sources (X1, X2) and target (Y)
    # x should be (n_samples, n_sources) = (n_samples, 2)
    # y should be (n_samples,) or (n_samples, 1)
    x = data[:, :2]  # Sources: X1 and X2
    y = data[:, 2]   # Target: Y

    # Initialize models with BOTH x and y
    model_red = RedundancyMMI(x, y)
    model_syn = SynergyMMI(x, y)

    # Compute redundancy and synergy
    redundancy = float(model_red.fit(method='binning')[0])
    synergy = float(model_syn.fit(method='binning')[0])

    # For individual mutual informations, we compute them directly
    # I(X1; Y)
    x1_y_data = np.column_stack([data[:, 0], data[:, 2]])
    model_mi1 = Oinfo(x1_y_data)
    # For 2 variables, TC = MI, so we can use that
    tc_x1_y = TC(x1_y_data)
    mi_x1_y = float(tc_x1_y.fit(method='binning')[0])
```

```
    # I(X2; Y)
    x2_y_data = np.column_stack([data[:, 1], data[:, 2]])
    tc_x2_y = TC(x2_y_data)
    mi_x2_y = float(tc_x2_y.fit(method='binning')[0])

    # Calculate unique informations
    # Unique(X1) = I(X1; Y) - Redundancy (bounded at 0)
    unique_x1 = max(0, mi_x1_y - redundancy)
    unique_x2 = max(0, mi_x2_y - redundancy)

    # Total = Unique1 + Unique2 + Redundancy + Synergy
    total = unique_x1 + unique_x2 + redundancy + synergy

    return {
        'unique_x1': unique_x1,
        'unique_x2': unique_x2,
        'redundancy': redundancy,
        'synergy': synergy,
        'total': total,
        'mi_x1_y': mi_x1_y,
        'mi_x2_y': mi_x2_y
    }

# Compute PID for XOR
print("PID Decomposition: XOR System")
print("=" * 60)
pid_xor = compute_pid_atoms(data_xor)

print("\nIndividual Mutual Informations:")
print(f"  I(X ; Y) = {pid_xor['mi_x1_y']:.4f} bits")
print(f"  I(X ; Y) = {pid_xor['mi_x2_y']:.4f} bits")
print(f"  I(X ,X ; Y)  {pid_xor['total']:.4f} bits (from PID sum)")

print("\nPID Atoms:")
print(f"  Unique(X ):  {pid_xor['unique_x1']:.4f} bits")
print(f"  Unique(X ):  {pid_xor['unique_x2']:.4f} bits")
print(f"  Redundancy:  {pid_xor['redundancy']:.4f} bits")
print(f"  Synergy:     {pid_xor['synergy']:.4f} bits")
```

```
print("\n  Analysis:")
print("  For XOR, synergy should dominate!")
print("  Neither X  nor X  alone provides information about Y,")
print("  so unique informations should be ~0.")
print("  All information comes from knowing BOTH inputs together.")

# Visualize PID
fig, ax = plt.subplots(figsize=(10, 6))
atoms = ['Unique(X )', 'Unique(X )', 'Redundancy', 'Synergy']
values = [pid_xor['unique_x1'], pid_xor['unique_x2'], pid_xor['redundancy'], pid_xor[
colors = ['#E63946', '#F1FAEE', '#457B9D', '#A8DADC']

bars = ax.bar(atoms, values, color=colors, alpha=0.8, edgecolor='black', linewidth=2)
ax.set_ylabel('Information (bits)', fontsize=14, fontweight='bold')
ax.set_title('PID Decomposition of XOR System', fontsize=16, fontweight='bold')
ax.grid(True, alpha=0.3, axis='y')

for bar, val in zip(bars, values):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02,
            f'{val:.3f}', ha='center', fontweight='bold', fontsize=11)

plt.tight_layout()
plt.show()
```

```
PID Decomposition: XOR System
============================================================

Individual Mutual Informations:
  I(X ; Y) = 0.0024 bits
  I(X ; Y) = 0.0006 bits
  I(X ,X ; Y)  0.9993 bits (from PID sum)

PID Atoms:
  Unique(X ):  0.0018 bits
  Unique(X ):  0.0000 bits
  Redundancy:  0.0006 bits
  Synergy:     0.9969 bits

  Analysis:
  For XOR, synergy should dominate!
```

```
Neither X  nor X  alone provides information about Y,
so unique informations should be ~0.
All information comes from knowing BOTH inputs together.
```




### 3.5.4 Visualizing PID Atoms

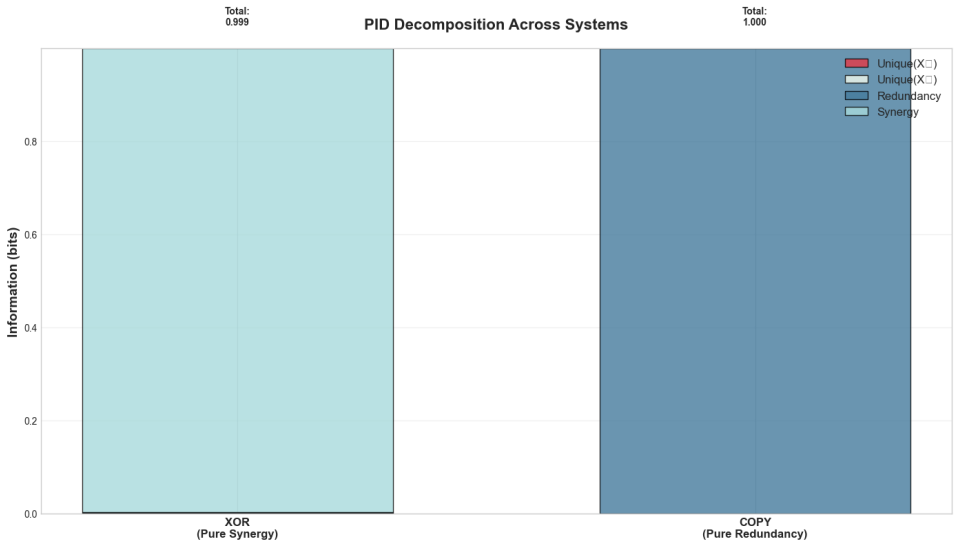
Let's create a comprehensive visualization showing how PID decomposes information for different systems:

```
# Compute PID for multiple systems
systems_pid = {
    'XOR\n(Pure Synergy)': data_xor,
    'COPY\n(Pure Redundancy)': data_copy,
}

# Add AND gate
X1_and = np.random.binomial(1, 0.5, n_samples)
X2_and = np.random.binomial(1, 0.5, n_samples)
Y_and = np.logical_and(X1_and, X2_and).astype(float)
systems_pid['AND\n(Mixed)'] = np.column_stack([X1_and, X2_and, Y_and])

# Compute PID for all
pid_results = {}
for name, data in systems_pid.items():
```

```
    try:
        pid_results[name] = compute_pid_atoms(data)
        print(f"  Computed PID for {name.replace(chr(10), ' ')}")
    except Exception as e:
        print(f"  Warning: Could not compute PID for {name.replace(chr(10), ' ')}: {e
        continue

if len(pid_results) > 0:
    # Create stacked bar chart
    fig, ax = plt.subplots(1, 1, figsize=(14, 8))

    names = list(pid_results.keys())
    x = np.arange(len(names))
    width = 0.6

    # Extract values for each atom
    unique1 = [pid_results[n]['unique_x1'] for n in names]
    unique2 = [pid_results[n]['unique_x2'] for n in names]
    redundancy = [pid_results[n]['redundancy'] for n in names]
    synergy = [pid_results[n]['synergy'] for n in names]

    # Create stacked bars
    p1 = ax.bar(x, unique1, width, label='Unique(X )', color='#E63946', alpha=0.8, edg
    p2 = ax.bar(x, unique2, width, bottom=unique1, label='Unique(X )', color='#F1FAEE
    p3 = ax.bar(x, redundancy, width, bottom=np.array(unique1)+np.array(unique2),
                label='Redundancy', color='#457B9D', alpha=0.8, edgecolor='black')
    p4 = ax.bar(x, synergy, width, bottom=np.array(unique1)+np.array(unique2)+np.arra
                label='Synergy', color='#A8DADC', alpha=0.8, edgecolor='black')

    # Labels and formatting
    ax.set_ylabel('Information (bits)', fontsize=14, fontweight='bold')
    ax.set_title('PID Decomposition Across Systems', fontsize=16, fontweight='bold',
    ax.set_xticks(x)
    ax.set_xticklabels(names, fontsize=12, fontweight='bold')
    ax.legend(fontsize=12, loc='upper right')
    ax.grid(True, alpha=0.3, axis='y')

    # Add total information labels
    for i, name in enumerate(names):
```

```
        total = pid_results[name]['total']
        ax.text(i, total + 0.05, f'Total:\n{total:.3f}',
                ha='center', fontweight='bold', fontsize=10)

    plt.tight_layout()
    plt.show()

    # Print summary table
    print("\nPID Summary Table:")
    print("=" * 80)
    print(f"{'System':<25} {'Unique1':>10} {'Unique2':>10} {'Redund':>10} {'Synergy':
    print("=" * 80)
    for name in names:
        p = pid_results[name]
        name_clean = name.replace('\n', ' ')
        print(f"{name_clean:<25} {p['unique_x1']:>10.4f} {p['unique_x2']:>10.4f} "
              f"{p['redundancy']:>10.4f} {p['synergy']:>10.4f} {p['total']:>10.4f}")
```

```
Computed PID for XOR (Pure Synergy)

Computed PID for COPY (Pure Redundancy)
Warning: Could not compute PID for AND (Mixed): data dtype should be integer. Check
```




```
PID Summary Table:
```

```
================================================================================
System                          Unique1     Unique2      Redund     Synergy       Total
================================================================================
XOR (Pure Synergy)               0.0018      0.0000      0.0006      0.9969      0.9993
COPY (Pure Redundancy)           0.0000      0.0000      0.9997      0.0000      0.9997
```

## 3.6 Part 5: Application to Simulated Neural Data

### 3.6.1 Realistic Neural Population Scenario

Now let's apply everything we've learned to a more realistic neuroscience scenario. We'll simulate a small population of V1 neurons encoding visual orientation, then use HOI to discover their information structure.

**Scenario**: - 10 V1 neurons - 8 orientations (0°, 45°, 90°, 135°, 180°, 225°, 270°, 315°) - 500 trials - Each neuron has a preferred orientation - Some neurons encode independently - Some form synergistic pairs - Some are redundant (overlapping tuning)

This mimics real V1 organization where neurons with similar orientation preferences cluster together (orientation columns).

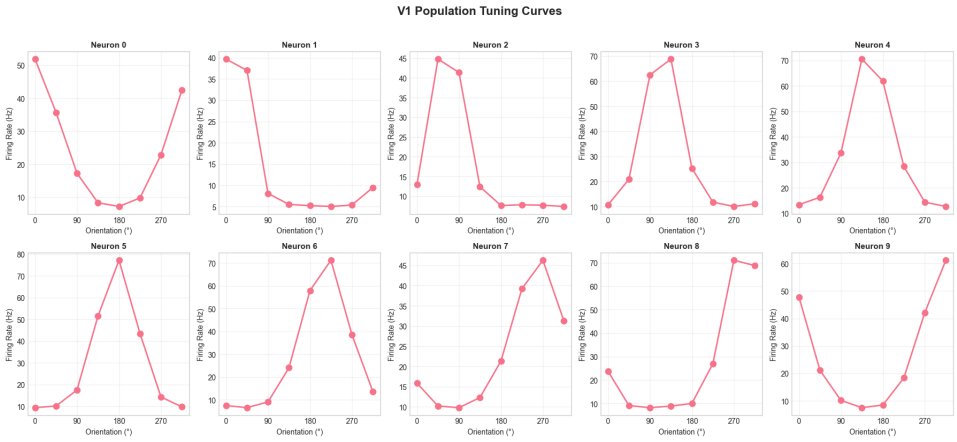
```python
def generate_v1_population(n_neurons=10, n_trials=500, n_orientations=8, seed=42):
    """
    Generate realistic V1 population responses with mixed information structure.

    Returns:
    --------
    responses : array (n_neurons, n_trials)
        Neural responses (spike counts)
    orientations : array (n_trials,)
        Stimulus orientation for each trial
    neuron_props : dict
        Properties of each neuron (preferred orientation, tuning width, etc.)
    """
    np.random.seed(seed)

    # Stimulus orientations
    ori_values = np.linspace(0, 360, n_orientations, endpoint=False)
    trial_oris = np.random.choice(ori_values, n_trials)
```

```
    # Initialize responses
    responses = np.zeros((n_neurons, n_trials))
    neuron_props = []

    for i in range(n_neurons):
        # Each neuron's parameters
        pref_ori = (i * 360 / n_neurons) + np.random.randn() * 10  # Slight jitter
        tuning_width = 30 + np.random.rand() * 30  # 30-60 degrees
        max_rate = 40 + np.random.rand() * 40  # 40-80 Hz
        baseline = 5 + np.random.rand() * 10  # 5-15 Hz
        noise_level = 0.3 + np.random.rand() * 0.4  # Variable noise

        neuron_props.append({
            'id': i,
            'pref_ori': pref_ori,
            'tuning_width': tuning_width,
            'max_rate': max_rate,
            'baseline': baseline
        })

        # Generate responses
        for j, ori in enumerate(trial_oris):
            # Angular distance
            dist = min(abs(ori - pref_ori), 360 - abs(ori - pref_ori))

            # Von Mises tuning
            tuning = np.exp(-(dist**2) / (2 * tuning_width**2))
            mean_rate = baseline + (max_rate - baseline) * tuning

            # Poisson variability
            responses[i, j] = np.random.poisson(mean_rate)

    return responses, trial_oris, neuron_props

# Generate population
responses, orientations, neuron_props = generate_v1_population()

print("V1 Population Generated")
print("=" * 60)
```

```
print(f"Shape: {responses.shape} (neurons × trials)")
print(f"Orientation values: {np.unique(orientations)}°")
print(f"\nNeuron properties (first 3):")
for i in range(3):
    prop = neuron_props[i]
    print(f"  Neuron {i}: pref={prop['pref_ori']:.1f}°, "
          f"width={prop['tuning_width']:.1f}°, max={prop['max_rate']:.1f}Hz")

# Visualize tuning curves
fig, axes = plt.subplots(2, 5, figsize=(18, 8))
axes = axes.flatten()

ori_vals = np.unique(orientations)
for i in range(10):
    ax = axes[i]

    # Compute mean response for each orientation
    mean_responses = []
    for ori in ori_vals:
        trials_this_ori = orientations == ori
        mean_resp = responses[i, trials_this_ori].mean()
        mean_responses.append(mean_resp)

    # Plot
    ax.plot(ori_vals, mean_responses, 'o-', linewidth=2, markersize=8)
    ax.set_xlabel('Orientation (°)', fontsize=10)
    ax.set_ylabel('Firing Rate (Hz)', fontsize=10)
    ax.set_title(f'Neuron {i}', fontsize=11, fontweight='bold')
    ax.grid(True, alpha=0.3)
    ax.set_xticks([0, 90, 180, 270])

plt.suptitle('V1 Population Tuning Curves', fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

print("\n  Population exhibits realistic orientation selectivity")
print("  Now let's discover the information structure...")
```

```
V1 Population Generated
```

```
============================================================
Shape: (10, 500) (neurons × trials)
Orientation values: [  0.  45.  90. 135. 180. 225. 270. 315.]°

Neuron properties (first 3):
  Neuron 0: pref=-8.5°, width=55.5°, max=52.7Hz
  Neuron 1: pref=20.9°, width=30.2°, max=48.8Hz
  Neuron 2: pref=65.7°, width=32.0°, max=52.7Hz
```




```
 Population exhibits realistic orientation selectivity
  Now let's discover the information structure...
```

### 3.6.2 Discovering Synergistic Triplets

Now comes the exciting part: using HOI to automatically discover which triplets of neurons exhibit synergy. We'll scan through all possible triplets and find those with negative O-information.

```
# Scan all triplets for synergy
# IMPORTANT: responses has shape (n_neurons, n_trials) from generate_v1_population
# We need to TRANSPOSE it to (n_trials, n_neurons) for HOI!
responses_transposed = responses.T  # Now (n_trials, n_neurons)

n_neurons = responses_transposed.shape[1]
all_triplets = list(combinations(range(n_neurons), 3))

print(f"Data shape for HOI: {responses_transposed.shape}")
print(f"  - {responses_transposed.shape[0]} samples (trials)")
```

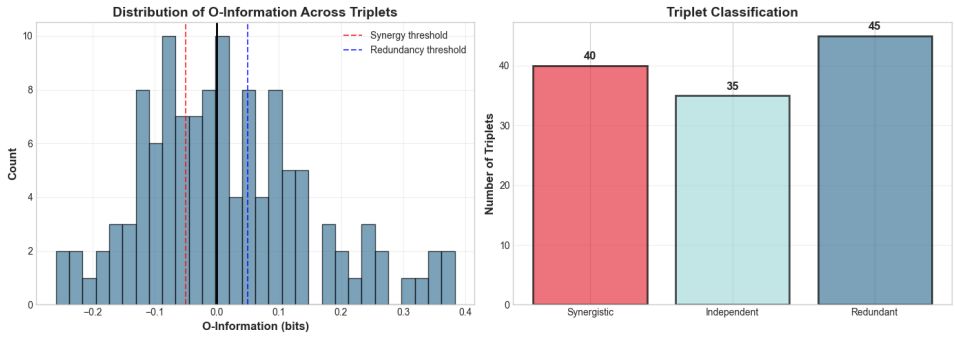
```
print(f"  - {responses_transposed.shape[1]} features (neurons)")
print(f"\nScanning {len(all_triplets)} possible triplets...")
print("(This may take a minute)\n")

# Create model with transposed data
model_oinfo_v1 = Oinfo(responses_transposed)

# Compute O-information for all triplets
oinfo_values = model_oinfo_v1.fit(method='gc', minsize=3, maxsize=3)

# Convert to list for easier handling
oinfo_list = [float(v) for v in oinfo_values]

# Find synergistic and redundant triplets
synergy_threshold = -0.05  # bits
redundancy_threshold = 0.05  # bits

synergistic = [(trip, val) for trip, val in zip(all_triplets, oinfo_list) if val < sy
redundant = [(trip, val) for trip, val in zip(all_triplets, oinfo_list) if val > redu
independent = [(trip, val) for trip, val in zip(all_triplets, oinfo_list)
               if abs(val) <= max(abs(synergy_threshold), redundancy_threshold)]

# Sort by strength
synergistic.sort(key=lambda x: x[1])  # Most negative first
redundant.sort(key=lambda x: x[1], reverse=True)  # Most positive first

print("O-Information Analysis Complete")
print("=" * 60)
print(f"\nTotal triplets: {len(all_triplets)}")
print(f"  Synergistic (O < {synergy_threshold}): {len(synergistic)}")
print(f"  Independent (|O|   {abs(synergy_threshold)}): {len(independent)}")
print(f"  Redundant (O > {redundancy_threshold}): {len(redundant)}")

# Show top synergistic triplets
print("\nTop 5 Most Synergistic Triplets:")
print("=" * 60)
for i, (triplet, oinfo) in enumerate(synergistic[:5]):
    neuron_ids = list(triplet)
    prefs = [neuron_props[n]['pref_ori'] for n in neuron_ids]
```

```
    print(f"{i+1}. Neurons {neuron_ids}: O-info = {oinfo:.4f} bits")
    print(f"   Preferred orientations: {[f'{p:.1f}°' for p in prefs]}")

# Show top redundant triplets
print("\nTop 5 Most Redundant Triplets:")
print("=" * 60)
for i, (triplet, oinfo) in enumerate(redundant[:5]):
    neuron_ids = list(triplet)
    prefs = [neuron_props[n]['pref_ori'] for n in neuron_ids]
    print(f"{i+1}. Neurons {neuron_ids}: O-info = {oinfo:.4f} bits")
    print(f"   Preferred orientations: {[f'{p:.1f}°' for p in prefs]}")

# Visualize distribution
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Histogram
ax1 = axes[0]
ax1.hist(oinfo_list, bins=30, color='#457B9D', alpha=0.7, edgecolor='black')
ax1.axvline(x=0, color='black', linestyle='-', linewidth=2)
ax1.axvline(x=synergy_threshold, color='red', linestyle='--', alpha=0.7, label='Syner
ax1.axvline(x=redundancy_threshold, color='blue', linestyle='--', alpha=0.7, label='R
ax1.set_xlabel('O-Information (bits)', fontsize=12, fontweight='bold')
ax1.set_ylabel('Count', fontsize=12, fontweight='bold')
ax1.set_title('Distribution of O-Information Across Triplets', fontsize=14, fontweigh
ax1.legend()
ax1.grid(True, alpha=0.3)

# Category breakdown
ax2 = axes[1]
categories = ['Synergistic', 'Independent', 'Redundant']
counts = [len(synergistic), len(independent), len(redundant)]
colors = ['#E63946', '#A8DADC', '#457B9D']
ax2.bar(categories, counts, color=colors, alpha=0.7, edgecolor='black', linewidth=2)
ax2.set_ylabel('Number of Triplets', fontsize=12, fontweight='bold')
ax2.set_title('Triplet Classification', fontsize=14, fontweight='bold')
ax2.grid(True, alpha=0.3, axis='y')
for i, (cat, cnt) in enumerate(zip(categories, counts)):
    ax2.text(i, cnt + 1, str(cnt), ha='center', fontweight='bold', fontsize=12)
```

```
plt.tight_layout()
plt.show()
```

```
Data shape for HOI: (500, 10)
  - 500 samples (trials)
  - 10 features (neurons)

Scanning 120 possible triplets...
(This may take a minute)

O-Information Analysis Complete
============================================================

Total triplets: 120
  Synergistic (O < -0.05): 40
  Independent (|O|  0.05): 35
  Redundant (O > 0.05): 45

Top 5 Most Synergistic Triplets:
============================================================
1. Neurons [4, 6, 7]: O-info = -0.2586 bits
   Preferred orientations: ['151.8°', '213.4°', '260.9°']
2. Neurons [6, 7, 9]: O-info = -0.2380 bits
   Preferred orientations: ['213.4°', '260.9°', '320.0°']
3. Neurons [0, 3, 6]: O-info = -0.2216 bits
   Preferred orientations: ['-8.5°', '115.3°', '213.4°']
4. Neurons [5, 6, 8]: O-info = -0.2178 bits
   Preferred orientations: ['175.2°', '213.4°', '290.1°']
5. Neurons [0, 6, 8]: O-info = -0.2041 bits
   Preferred orientations: ['-8.5°', '213.4°', '290.1°']

Top 5 Most Redundant Triplets:
============================================================
1. Neurons [0, 4, 5]: O-info = 0.3829 bits
   Preferred orientations: ['-8.5°', '151.8°', '175.2°']
2. Neurons [0, 4, 9]: O-info = 0.3668 bits
   Preferred orientations: ['-8.5°', '151.8°', '320.0°']
3. Neurons [4, 5, 9]: O-info = 0.3489 bits
   Preferred orientations: ['151.8°', '175.2°', '320.0°']
4. Neurons [0, 5, 9]: O-info = 0.3452 bits
```

```
   Preferred orientations: ['-8.5°', '175.2°', '320.0°']
5. Neurons [3, 4, 9]: O-info = 0.3242 bits
   Preferred orientations: ['115.3°', '151.8°', '320.0°']
```




## 3.7 Part 6: Computational Scaling and Practical Considerations

### 3.7.1 The Combinatorial Explosion

As you just saw, we computed O-information for 120 triplets (10 choose 3). But what if we had 20 neurons? Or 100? The number of possible triplets grows **very quickly**:

- 10 neurons: 120 triplets
- 20 neurons: 1,140 triplets
- 50 neurons: 19,600 triplets
- 100 neurons: 161,700 triplets

And this is just for triplets! For 4-way interactions (quadruplets), the numbers are even larger.

### 3.7.2 Strategies for Large Systems

When dealing with large neural populations, you need smart strategies:

1. **Pre-screening**: Use pairwise MI to identify interesting variable pairs first
2. **Hierarchical**: Start with triplets, only explore larger multiplets if warranted
3. **Sampling**: Randomly sample multiplets rather than exhaustive search

4. **GPU acceleration**: Use JAX's GPU capabilities for parallel computation
5. **Time windowing**: For time series, analyze shorter windows

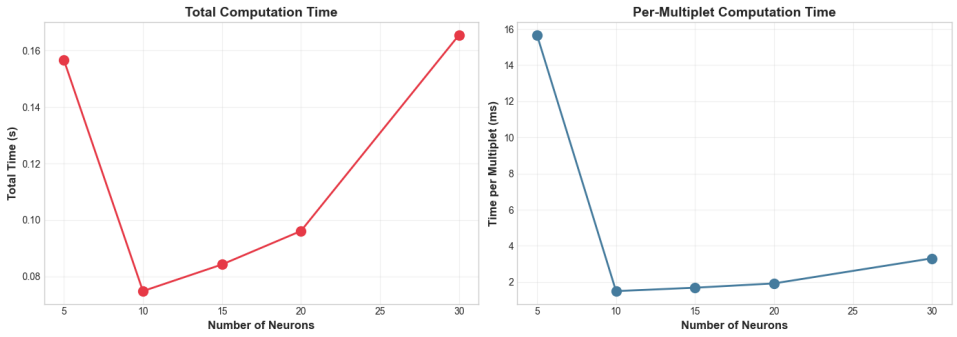
Let's demonstrate some of these strategies:

```
# Benchmark computation time vs system size
import time

def benchmark_hoi(n_neurons_list, n_samples=1000, n_multiplets=50):
    """
    Measure HOI computation time for different system sizes.
    """
    results = []

    for n_neurons in n_neurons_list:
        # Generate random data in correct format: (n_samples, n_features)
        data = np.random.randn(n_samples, n_neurons)

        # Sample multiplets
        all_trips = list(combinations(range(n_neurons), 3))
        if len(all_trips) > n_multiplets:
            sampled_trips = [all_trips[i] for i in
                             np.random.choice(len(all_trips), n_multiplets, replace=Fal
        else:
            sampled_trips = all_trips

        # Create model WITH DATA
        model = Oinfo(data)

        # Time the computation
        start = time.time()
        oinfo = model.fit(method='gc', minsize=3, maxsize=3)
        elapsed = time.time() - start

        results.append({
            'n_neurons': n_neurons,
            'n_multiplets': len(sampled_trips),
            'time': elapsed,
            'time_per_multiplet': elapsed / len(sampled_trips)
        })
```

```
    return results

# Test different sizes
sizes = [5, 10, 15, 20, 30]
print("Benchmarking HOI Computation...\n")
benchmark_results = benchmark_hoi(sizes)

# Display results
print("Computational Scaling Analysis")
print("=" * 70)
print(f"{'Neurons':<10} {'Multiplets':<12} {'Time (s)':<12} {'ms/multiplet':<15}")
print("=" * 70)
for res in benchmark_results:
    print(f"{res['n_neurons']:<10} {res['n_multiplets']:<12} "
          f"{res['time']:<12.4f} {res['time_per_multiplet']*1000:<15.2f}")

# Visualize scaling
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

neurons = [r['n_neurons'] for r in benchmark_results]
times = [r['time'] for r in benchmark_results]
time_per = [r['time_per_multiplet']*1000 for r in benchmark_results]

# Total time
ax1.plot(neurons, times, 'o-', linewidth=2, markersize=10, color='#E63946')
ax1.set_xlabel('Number of Neurons', fontsize=12, fontweight='bold')
ax1.set_ylabel('Total Time (s)', fontsize=12, fontweight='bold')
ax1.set_title('Total Computation Time', fontsize=14, fontweight='bold')
ax1.grid(True, alpha=0.3)

# Time per multiplet
ax2.plot(neurons, time_per, 'o-', linewidth=2, markersize=10, color='#457B9D')
ax2.set_xlabel('Number of Neurons', fontsize=12, fontweight='bold')
ax2.set_ylabel('Time per Multiplet (ms)', fontsize=12, fontweight='bold')
ax2.set_title('Per-Multiplet Computation Time', fontsize=14, fontweight='bold')
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

```
print("\n Scaling Notes:")
print("  - Time grows with number of neurons (more entropy calculations)")
print("  - Time per multiplet is relatively constant")
print("  - For large systems, sample multiplets rather than computing all")
```

```
Benchmarking HOI Computation...

Computational Scaling Analysis
======================================================================
Neurons    Multiplets   Time (s)     ms/multiplet
======================================================================
5          10           0.1566       15.66
10         50           0.0748       1.50
15         50           0.0842       1.68
20         50           0.0961       1.92
30         50           0.1655       3.31
```




```
Scaling Notes:
 - Time grows with number of neurons (more entropy calculations)
 - Time per multiplet is relatively constant
 - For large systems, sample multiplets rather than computing all
```

## 3.8 Summary and Key Takeaways

### 3.8.1 What We've Accomplished

Congratulations! You've gone from manual calculations in Notebooks 1-2 to using a professional package for higher-order interaction analysis. You now know how to:

1. **Install and configure HOI** with the right estimators

2. **Compute O-information** to detect synergy vs redundancy
3. **Apply PID** to decompose information into four atoms
4. **Use multiple estimators** and understand their trade-offs
5. **Analyze realistic neural data** to discover information structure
6. **Understand computational scaling** and plan analyses appropriately

### 3.8.2 When to Use Each HOI Measure

**O-Information (`Oinfo`)**: - **Use for**: Quick screening to identify synergistic or redundant multiplets - **Advantages**: Fast, single number, clear interpretation - **Limitations**: Doesn't decompose into unique/redundant/synergistic components

**PID (RedundancyMMI + SynergyMMI)**: - **Use for**: Detailed understanding of information structure (2 sources only) - **Advantages**: Complete decomposition into interpretable atoms - **Limitations**: Only well-defined for 2 sources, computationally expensive

**Total Correlation (`TC`)**: - **Use for**: Measuring overall system integration - **Advantages**: Always non-negative, intuitive - **Limitations**: Doesn't distinguish synergy from redundancy

### 3.8.3 Critical Reminders

**Data Format**: HOI expects (n_features, n_samples) - opposite of scikit-learn!

**Estimator Choice**: Use 'gc' (Gaussian Copula) as default unless you have specific reasons otherwise.

**Statistical Significance**: HOI doesn't include built-in significance testing. For that, use Frites (next notebook) or implement permutation tests.

**Interpretation**: Always report both sign AND magnitude of O-information.

**Computational Planning**: For large systems (50+ variables), use sampling or hierarchical strategies.

### 3.8.4 The Bigger Picture

HOI has revealed something profound about your V1 population: some neuron triplets carry information synergistically (their combination predicts better than the sum of individuals), while others are redundant (they duplicate information for robustness).

This isn't just a mathematical curiosity - it reflects the **functional organization** of neural circuits. Synergistic triplets might correspond to neurons within the same orientation column working together. Redundant triplets might provide robustness against noise or damage.

### 3.8.5 Bridge to Notebook 4

HOI is excellent for analyzing static snapshots of neural activity. But real neural data is **dynamic** - it evolves over time during tasks. In Notebook 4, we'll use **Frites** to: - Compute **time-resolved** mutual information - Track how information flows during cognitive tasks - Perform **statistical testing** with permutations - Analyze **dynamic functional connectivity** - Handle **multi-subject** designs properly

Frites complements HOI by adding temporal dynamics and rigorous statistics. Together, they provide a complete toolkit for information-theoretic analysis of neural data.

See you in Notebook 4!

## 3.9 Practice Exercises

Before moving to Notebook 4, try these exercises to solidify your understanding:

```
print("PRACTICE EXERCISES")
print("=" * 70)

print("\n1. Create an OR gate (Y = X  OR X ) and compute its O-information.")
print("   Will it be synergistic, redundant, or independent?")
print("   Hint: Think about what happens when both inputs are 1")

print("\n2. Modify the V1 simulation to have 15 neurons instead of 10.")
print("   How does the distribution of O-information values change?")
print("   Are there more or fewer synergistic triplets?")

print("\n3. Compare PID decomposition for AND gate vs OR gate.")
print("   Which has more redundancy? Which has more synergy?")
print("   Can you explain why based on the logic?")

print("\n4. Test different estimators ('gc', 'gauss', 'binning') on Gaussian data.")
```

```
print("   When is 'gauss' estimator most accurate?")
print("   When does it fail?")

print("\n5. Add noise to the XOR system: flip Y with 30% probability.")
print("   How does this affect:")
print("   - O-information magnitude?")
print("   - PID decomposition?")
print("   - Does synergy decrease linearly with noise?")

print("\n" + "=" * 70)
print("Try these exercises in a new cell below!")
print("Solutions are in the notebook repository.")
print("=" * 70)
```

```
PRACTICE EXERCISES
======================================================================

1. Create an OR gate (Y = X  OR X ) and compute its O-information.
   Will it be synergistic, redundant, or independent?
   Hint: Think about what happens when both inputs are 1

2. Modify the V1 simulation to have 15 neurons instead of 10.
   How does the distribution of O-information values change?
   Are there more or fewer synergistic triplets?

3. Compare PID decomposition for AND gate vs OR gate.
   Which has more redundancy? Which has more synergy?
   Can you explain why based on the logic?

4. Test different estimators ('gc', 'gauss', 'binning') on Gaussian data.
   When is 'gauss' estimator most accurate?
   When does it fail?

5. Add noise to the XOR system: flip Y with 30% probability.
   How does this affect:
   - O-information magnitude?
   - PID decomposition?
   - Does synergy decrease linearly with noise?
```

```
======================================================================
Try these exercises in a new cell below!
Solutions are in the notebook repository.
======================================================================
```

## 3.10 Further Resources

### 3.10.1 HOI Documentation

- **Main documentation**: https://brainets.github.io/hoi/
- **GitHub repository**: https://github.com/brainets/hoi
- **Tutorial examples**: Check the examples gallery

### 3.10.2 Key Papers on PID

1. **Williams & Beer (2010)** - "Nonnegative Decomposition of Multivariate Information"
   - Original PID framework
   - Defines the four atoms
2. **Bertschinger et al. (2014)** - "Quantifying Unique Information"
   - BROJA measure
   - Game-theoretic formulation
3. **Timme et al. (2014)** - "Synergy, redundancy, and multivariate information measures"
   - Comprehensive review
   - Comparison of different PID measures

### 3.10.3 Related Packages

- **dit**: Discrete information theory (Python)
- **IDTxl**: Information dynamics (transfer entropy)
- **JIDT**: Java Information Dynamics Toolkit
- **Frites**: Next notebook!

### 3.10.4 Advanced Topics

- **Multiplet expansion**: Beyond triplets
- **Directed information**: Causal structures
- **Information geometry**: Geometric interpretation
- **Integrated Information Theory (IIT)**: Consciousness and $\Phi$

Happy exploring!

# Chapter 4

# Notebook 4: Neural Time Series Analysis with Frites

## 4.1 From Static Snapshots to Dynamic Information Flow

Welcome to Notebook 4! In the previous notebooks, we built a foundation in information theory (Notebook 1), discovered the XOR problem (Notebook 2), and learned to use HOI for detecting synergy and redundancy (Notebook 3). But there's been one crucial limitation: everything we've done has been **static**.

Neural activity isn't static - it's a dynamic, time-varying process. When you present a visual stimulus, neural responses evolve over time: - Early visual areas respond within 50-100ms - Information propagates through the cortical hierarchy - Connectivity patterns change with cognitive states - Learning modulates information flow

**We need tools that can track information over time.**

### 4.1.1 Enter Frites

**Frites** (Framework for Information Theoretical analysis of Electrophysiological data and Statistics) is designed specifically for M/EEG and intracranial neural data. It provides:

**Time-resolved mutual information** **Statistical testing with per-**

**mutations Cluster-based correction for multiple comparisons Dynamic functional connectivity Multi-subject group analysis Integration with MNE-Python**

### 4.1.2 Why Frites Matters

Frites solves two critical problems:

1. **Temporal dynamics**: Compute MI at each time point to see when information emerges
2. **Statistical rigor**: Not just "is MI high?" but "is it significantly different from chance?"

Without proper statistics, you can't distinguish real effects from noise. Frites handles this automatically.

### 4.1.3 Learning Objectives

By the end of this notebook, you'll: 1. Understand Frites' data structure (`DatasetEphy`) 2. Compute time-resolved mutual information 3. Apply permutation testing for significance 4. Use cluster-based correction for multiple comparisons 5. Analyze dynamic functional connectivity 6. Understand fixed-effect vs random-effect analysis 7. Interpret GCMI (Gaussian Copula MI) 8. Know when to use Frites vs HOI

Let's begin!

## 4.2 Part 1: Installation and Understanding Frites Data Structure

### 4.2.1 Installation

Frites integrates with MNE-Python, the standard package for M/EEG analysis. We'll install both:

```
# Install Frites and dependencies
!pip install -q frites mne numpy scipy matplotlib seaborn pandas xarray

print("Installation complete! ")
print("Importing libraries...")
```

```
Installation complete!
```

Importing libraries...

```
# Import all necessary libraries
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import xarray as xr
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

# Check NumPy version for Frites compatibility
print(f"NumPy version: {np.__version__}")
numpy_major_version = int(np.__version__.split('.')[0])

if numpy_major_version >= 2:
    print("\n  WARNING: NumPy 2.0+ detected")
    print("   Frites requires NumPy 1.x for compatibility")
    print("\n   FIX: Run in terminal:")
    print("   pip install 'numpy<2.0' --force-reinstall")
    print("   Then restart kernel and rerun this notebook\n")
    raise ImportError("NumPy 2.0+ incompatible with Frites. Please downgrade to NumPy

# Import Frites components
import frites
from frites.dataset import DatasetEphy
from frites.workflow import WfMi, WfStats
from frites.conn import conn_dfc, conn_covgc
from frites import set_mpl_style

# Set Frites plotting style
set_mpl_style()

# Random seed
np.random.seed(42)

print(f"  NumPy {np.__version__} (compatible)")
print(f"  Frites version: {frites.__version__}")
print("All libraries loaded successfully!")
```

```
print("\n Ready to analyze neural time series!")
```

```
NumPy version: 1.26.4
 NumPy 1.26.4 (compatible)
 Frites version: 0.4.4
All libraries loaded successfully!

 Ready to analyze neural time series!
```

### 4.2.2 Understanding Frites Data Structure: DatasetEphy

Frites uses a specialized data container called `DatasetEphy` ("Phy" = electrophysiology). This is different from both HOI and standard NumPy arrays, so understanding it is crucial.

#### 4.2.2.1 The DatasetEphy Structure

```
DatasetEphy(
    x,        # Neural data: list of arrays, one per subject
    y,        # Task variable: list of arrays, one per subject
    roi,      # Brain regions: list of lists, one per subject
    times     # Time vector: shared across subjects
)
```

#### 4.2.2.2 Critical Format Details

**Neural data (`x`)**: List of arrays, each with shape `(n_epochs, n_channels, n_times)` - `n_epochs`: Number of trials for this subject - `n_channels`: Number of sensors/electrodes/brain regions - `n_times`: Number of time points per trial

**Task variable (`y`)**: List of arrays, each with shape `(n_epochs,)` - Can be continuous (reaction time, stimulus intensity) → use `mi_type='cd'` - Can be discrete (condition labels) → use `mi_type='cd'`
- Can be another continuous neural signal → use `mi_type='cc'`

**ROI names (`roi`)**: List of lists of strings - Each subject has their own list of channel names - Allows for different electrode placements across subjects - Frites automatically finds common ROIs

**Time vector (`times`)**: Single array for all subjects - Assumes time points are aligned - Usually in seconds

Let's create a simple example to see this in practice:

```
# Create a minimal example with 2 subjects
n_subjects = 2
n_channels = 5
n_times = 100
time = np.linspace(-0.2, 0.8, n_times)  # -200ms to 800ms

# Initialize lists
data_list = []
y_list = []
roi_list = []

for subj in range(n_subjects):
    # Each subject has different number of trials
    n_epochs = 50 + subj * 20  # Subject 0: 50 trials, Subject 1: 70 trials

    # Neural data: (n_epochs, n_channels, n_times)
    subj_data = np.random.randn(n_epochs, n_channels, n_times)
    data_list.append(subj_data)

    # Task variable: continuous (e.g., stimulus intensity)
    task_var = np.random.randn(n_epochs)
    y_list.append(task_var)

    # ROI names
    roi_names = [f'V1_{i}' for i in range(n_channels)]
    roi_list.append(roi_names)

# Create DatasetEphy
ds = DatasetEphy(
    x=data_list,
    y=y_list,
    roi=roi_list,
    times=time
)
```

```
print("DatasetEphy Created")
print("=" * 60)
print(ds)
print("\nKey Properties:")
print(f"  Number of subjects: {len(data_list)}")
print(f"  Time points: {n_times}")
print(f"  Time range: {time[0]:.2f}s to {time[-1]:.2f}s")
print(f"  Channels per subject: {n_channels}")
print(f"  Trials per subject: {[len(y) for y in y_list]}")
print("\n  Frites handles variable trial counts automatically!")
print("  This is crucial for real experiments with dropouts")
```

```
DatasetEphy Created
============================================================
<xarray.DataArray 'subject_0' (trials: 50, roi: 5, times: 100)> Size: 200kB
array([[[ 0.49671415, -0.1382643 ,  0.64768854, ...,  0.26105527,
          0.00511346, -0.23458713],
        [-1.41537074, -0.42064532, -0.34271452, ...,  0.15372511,
          0.05820872, -1.1429703 ],
        [ 0.35778736,  0.56078453,  1.08305124, ...,  0.30729952,
          0.81286212,  0.62962884],
        [-0.82899501, -0.56018104,  0.74729361, ...,  1.35387237,
         -0.11453985,  1.23781631],
        [-1.59442766, -0.59937502,  0.0052437 , ..., -0.19033868,
         -0.87561825, -1.38279973]],

       [[ 0.92617755,  1.90941664, -1.39856757, ..., -0.97876372,
         -0.44429326,  0.37730049],
        [ 0.75698862, -0.92216532,  0.86960592, ..., -1.25111358,
          0.92402702, -0.18490214],
        [-0.52272302,  1.04900923, -0.70434369, ...,  0.6815007 ,
          0.02831838,  0.02975614],
        [ 0.93828381, -0.51604473,  0.09612078, ...,  0.14671369,
          1.20650897, -0.81693567],
        [ 0.36867331, -0.39333881,  0.02874482, ...,  0.64084286,
...
          0.00878155, -1.75822163],
        [-0.23816954, -1.38637907, -1.20679589, ..., -0.83313778,
         -1.57775323, -0.78174047],
```

```
        [-1.02777392, -0.64668685, -1.32588408, ..., -0.28496951,
          0.44194438, -0.44582948],
        [-0.82703757,  0.00396218,  0.51903274, ...,  0.53212857,
         -0.68822826,  0.08059582],
        [-0.85378638,  1.50411284,  1.62287598, ...,  1.20777649,
         -1.05663821,  0.74546145]],

       [[ 0.05682406,  0.1665865 ,  0.6256229 , ...,  2.05239549,
          0.6783772 , -1.23442217],
        [-0.34540695,  0.59596331,  0.4832905 , ..., -0.46932818,
          1.21605173, -0.74699038],
        [ 0.60043308, -1.30478575,  1.03113043, ...,  0.61152167,
          0.17732836,  1.23054501],
        [ 0.24100314, -0.83059052, -0.46728062, ...,  0.93861077,
         -0.89306856, -0.49636313],
        [ 0.53563759, -2.06943926,  1.41658382, ...,  0.96777886,
          0.62369321,  0.01558457]]])
Coordinates:
  * trials   (trials) int64 400B 0 1 2 3 4 5 6 7 8 ... 42 43 44 45 46 47 48 49
    y        (trials) float64 400B 0.1709 0.01226 -0.4312 ... 0.4865 -
0.1371
  * roi      (roi) <U4 80B 'V1_0' 'V1_1' 'V1_2' 'V1_3' 'V1_4'
    agg_ch   (roi) int64 40B 0 0 0 0 0
  * times    (times) float64 800B -0.2 -0.1899 -0.1798 ... 0.7798 0.7899 0.8
    subject  (trials) int64 400B 0 0 0 0 0 0 0 0 0 0 0 ... 0 0 0 0 0 0 0 0 0 0 0
Attributes:
    __version__:   0.4.4
    modality:      electrophysiology
    dtype:         SubjectEphy
    y_dtype:       float
    z_dtype:       none
    mi_type:       cc
    mi_repr:       I(x; y (continuous))
    sfreq:         98.9999999999999
    agg_ch:        1
    multivariate:  0
<xarray.DataArray 'subject_1' (trials: 70, roi: 5, times: 100)> Size: 280kB
array([[[ 0.3039018 ,  0.31685234, -0.03988675, ..., -1.13381919,
          0.22701574, -0.59042695],
        [ 0.7008291 , -0.79788682,  0.771803  , ...,  0.42248462,
```

```
         -0.90737889, -0.06223106],
        [ 1.05718454,  0.30397018, -0.72213487, ...,  0.03004136,
         -1.29643838, -0.96911934],
        [ 0.14944115, -0.4944546 , -1.04459979, ...,  2.49124833,
         -1.56374109,  0.16296167],
        [-1.17435296,  0.87984592, -1.58147543, ...,  0.04546653,
         -0.67984793,  0.51281956]],

       [[ 0.19499029, -0.85910398, -0.65880283, ...,  3.2153742 ,
          0.33205912,  0.23555221],
        [-2.15536267,  0.42794744,  0.16866776, ..., -0.17873812,
          0.86281756, -0.57175425],
        [ 0.27032125, -0.13167378, -0.61469682, ...,  0.44978262,
          0.65492445,  0.44617094],
        [-1.50698718, -1.19816095, -0.51021817, ..., -1.48993615,
         -0.09261883,  1.03011785],
        [-1.11892428,  0.01652641, -1.85460931, ...,  0.73286498,
...
         -0.7792243 ,  1.23822642],
        [-0.24402657, -0.18208393,  1.28198046, ...,  1.87401644,
         -0.66065685,  0.18771821],
        [ 0.1590628 , -1.10605157, -0.00550727, ..., -0.63600739,
          0.72882256,  1.26425988],
        [-1.12206247,  0.75098099, -1.48635599, ...,  2.00831988,
          0.20483624, -0.35891719],
        [ 0.79823488, -0.87011036,  0.11015491, ..., -3.19735328,
         -0.44803835,  0.32197843]],

       [[-0.96820729, -0.50384399, -0.38800901, ..., -2.80144151,
         -1.62594828,  0.57279963],
        [-1.63595445, -0.48893977,  1.40337891, ..., -0.31008257,
         -0.23354775, -0.80532323],
        [ 0.29317407,  1.0815653 ,  1.40898163, ..., -0.29161335,
         -0.43403379,  1.277861  ],
        [ 0.62494672,  0.99620224,  1.25479491, ...,  1.0416924 ,
         -0.52821174,  1.98132505],
        [ 1.53109824,  0.96657642,  0.68566589, ..., -1.55397595,
         -0.76521889,  1.25068144]]])
Coordinates:
  * trials   (trials) int64 560B 0 1 2 3 4 5 6 7 8 ... 62 63 64 65 66 67 68 69
```

```
    y        (trials) float64 560B 1.435 -1.408 1.562 ... 0.06534 -
0.2239
  * roi      (roi) <U4 80B 'V1_0' 'V1_1' 'V1_2' 'V1_3' 'V1_4'
    agg_ch   (roi) int64 40B 0 0 0 0 0
  * times    (times) float64 800B -0.2 -0.1899 -0.1798 ... 0.7798 0.7899 0.8
    subject  (trials) int64 560B 1 1 1 1 1 1 1 1 1 1 ... 1 1 1 1 1 1 1 1 1 1
Attributes:
    __version__:   0.4.4
    modality:      electrophysiology
    dtype:         SubjectEphy
    y_dtype:       float
    z_dtype:       none
    mi_type:       cc
    mi_repr:       I(x; y (continuous))
    sfreq:         98.9999999999999
    agg_ch:        1
    multivariate:  0

Key Properties:
  Number of subjects: 2
  Time points: 100
  Time range: -0.20s to 0.80s
  Channels per subject: 5
  Trials per subject: [50, 70]

 Frites handles variable trial counts automatically!
  This is crucial for real experiments with dropouts
```

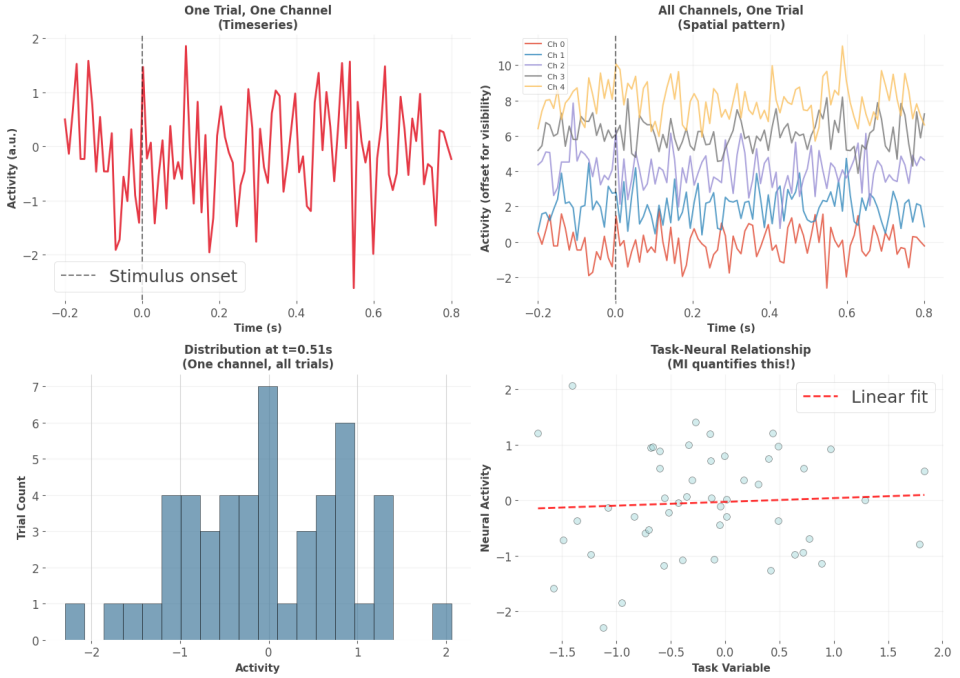
### 4.2.3 Visualizing the Data Structure

Let's create a diagram showing how Frites organizes multi-subject data:

```
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Plot 1: Single trial, single channel
ax1 = axes[0, 0]
ax1.plot(time, data_list[0][0, 0, :], linewidth=2, color='#E63946')
ax1.set_xlabel('Time (s)', fontsize=11, fontweight='bold')
ax1.set_ylabel('Activity (a.u.)', fontsize=11, fontweight='bold')
ax1.set_title('One Trial, One Channel\n(Timeseries)', fontsize=12, fontweight='bold')
```

```
ax1.axvline(0, color='k', linestyle='--', alpha=0.5, label='Stimulus onset')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Plot 2: All channels, one trial
ax2 = axes[0, 1]
for ch in range(n_channels):
    ax2.plot(time, data_list[0][0, ch, :] + ch*2, linewidth=1.5,
             label=f'Ch {ch}', alpha=0.8)
ax2.set_xlabel('Time (s)', fontsize=11, fontweight='bold')
ax2.set_ylabel('Activity (offset for visibility)', fontsize=11, fontweight='bold')
ax2.set_title('All Channels, One Trial\n(Spatial pattern)', fontsize=12, fontweight='
ax2.axvline(0, color='k', linestyle='--', alpha=0.5)
ax2.legend(fontsize=8, loc='upper left')
ax2.grid(True, alpha=0.3)

# Plot 3: All trials, one channel, one timepoint
ax3 = axes[1, 0]
time_idx = 70  # Around 600ms
all_trials_one_time = data_list[0][:, 0, time_idx]
ax3.hist(all_trials_one_time, bins=20, color='#457B9D', alpha=0.7, edgecolor='black')
ax3.set_xlabel('Activity', fontsize=11, fontweight='bold')
ax3.set_ylabel('Trial Count', fontsize=11, fontweight='bold')
ax3.set_title(f'Distribution at t={time[time_idx]:.2f}s\n(One channel, all trials)',
              fontsize=12, fontweight='bold')
ax3.grid(True, alpha=0.3, axis='y')

# Plot 4: Task variable vs neural activity
ax4 = axes[1, 1]
neural_avg = data_list[0][:, 0, time_idx]  # Average activity at this timepoint
task_vals = y_list[0]
ax4.scatter(task_vals, neural_avg, alpha=0.5, s=50, color='#A8DADC', edgecolor='black
# Add regression line
z = np.polyfit(task_vals, neural_avg, 1)
p = np.poly1d(z)
ax4.plot(sorted(task_vals), p(sorted(task_vals)), "r--", linewidth=2, alpha=0.8,
         label='Linear fit')
ax4.set_xlabel('Task Variable', fontsize=11, fontweight='bold')
ax4.set_ylabel('Neural Activity', fontsize=11, fontweight='bold')
```

```
ax4.set_title(f'Task-Neural Relationship\n(MI quantifies this!)',
              fontsize=12, fontweight='bold')
ax4.legend()
ax4.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("\n Understanding the Data Structure:")
print("=" * 60)
print("Top-left: How activity evolves in time (temporal dynamics)")
print("Top-right: How channels relate at one moment (spatial pattern)")
print("Bottom-left: Trial-to-trial variability (what MI uses!)")
print("Bottom-right: Task-neural relationship (what MI measures)")
print("\n Frites computes MI at each time point using trial variability")
```




```
 Understanding the Data Structure:
============================================================
Top-left: How activity evolves in time (temporal dynamics)
Top-right: How channels relate at one moment (spatial pattern)
```

```
Bottom-left: Trial-to-trial variability (what MI uses!)
Bottom-right: Task-neural relationship (what MI measures)

  Frites computes MI at each time point using trial variability
```

## 4.3 Part 2: Time-Resolved Mutual Information

### 4.3.1 The Core Workflow: WfMi

Frites organizes analysis into **workflows**. The main one for mutual information is `WfMi` (Workflow for Mutual Information). Here's how it works:

```
wf = WfMi(
    mi_type='cd',     # 'cd' = continuous-discrete (neural vs condition)
                      # 'cc' = continuous-continuous (neural vs neural or behavior)
                      # 'ccd' = continuous-continuous|discrete (conditional MI)
    inference='ffx'   # 'ffx' = fixed-effect (pool across subjects)
                      # 'rfx' = random-effect (model inter-subject variability)
)
```

### 4.3.2 Gaussian Copula MI (GCMI)

Frites uses **GCMI** by default. This is important to understand:

**What it does**: Transforms data to have Gaussian marginals while preserving rank relationships.

**Why it's good**: - No binning required (unlike histograms) - Fast computation - Works for any monotonic relationship - Accurate with modest sample sizes

**Key assumption**: The relationship is **monotonic** (if X increases, Y consistently increases OR decreases, not both)

**When it fails**: Non-monotonic relationships (rare in neural data)

Let's create data with a clear time-resolved effect and analyze it:

```
def simulate_task_related_activity(n_subjects=3, n_epochs=80, n_channels=8, n_times=1
    """
    Simulate neural data with task-related MI in specific time window.

    Task variable affects neural activity at 400-600ms.
```

```
    """
    time = np.linspace(-0.2, 1.0, n_times)

    data_list = []
    y_list = []
    roi_list = []

    for subj in range(n_subjects):
        # Each subject has slightly different trial count
        n_epochs_subj = n_epochs + np.random.randint(-10, 10)

        # Neural data
        subj_data = np.random.randn(n_epochs_subj, n_channels, n_times) * 0.5

        # Task variable (continuous)
        task_var = np.random.randn(n_epochs_subj)

        # Inject MI in specific channels and time window
        # Channels 2-4 encode task in 400-600ms window
        task_time_mask = (time >= 0.4) & (time <= 0.6)
        task_channels = [2, 3, 4]

        for ch in task_channels:
            # Add task-related modulation
            subj_data[:, ch, task_time_mask] += task_var[:, np.newaxis] * 1.5

        # Add smooth temporal dynamics (autocorrelation)
        from scipy.ndimage import gaussian_filter1d
        for ep in range(n_epochs_subj):
            for ch in range(n_channels):
                subj_data[ep, ch, :] = gaussian_filter1d(subj_data[ep, ch, :], sigma=

        data_list.append(subj_data)
        y_list.append(task_var)
        roi_list.append([f'CH_{i}' for i in range(n_channels)])

    return data_list, y_list, roi_list, time

# Generate data
```

```
data, task_vars, rois, time = simulate_task_related_activity()

print("Simulated Task-Related Activity")
print("=" * 60)
print(f"Subjects: {len(data)}")
print(f"Channels: {len(rois[0])}")
print(f"Time points: {len(time)}")
print(f"Time range: {time[0]:.2f}s to {time[-1]:.2f}s")
print(f"\nTrials per subject: {[d.shape[0] for d in data]}")
print("\n  Ground Truth:")
print("   Channels 2, 3, 4 encode task variable")
print("   Time window: 400-600ms post-stimulus")
print("   Let's see if Frites can detect this!")

# Create DatasetEphy
ds = DatasetEphy(x=data, y=task_vars, roi=rois, times=time)

print("\n" + "=" * 60)
print("Dataset ready for analysis!")
print("=" * 60)
```

```
Simulated Task-Related Activity
============================================================
Subjects: 3
Channels: 8
Time points: 150
Time range: -0.20s to 1.00s

Trials per subject: [71, 72, 83]

  Ground Truth:
    Channels 2, 3, 4 encode task variable
    Time window: 400-600ms post-stimulus
    Let's see if Frites can detect this!

============================================================
Dataset ready for analysis!
============================================================
```

### 4.3.3 Computing Time-Resolved MI

Now we'll use the `WfMi` workflow to compute mutual information at each time point. We'll also apply **permutation testing** to determine statistical significance.

#### 4.3.3.1 How Permutation Testing Works

1. Compute real MI from actual data
2. Shuffle trial labels randomly (breaking task-neural relationship)
3. Compute MI on shuffled data
4. Repeat 1000+ times to build null distribution
5. Compare real MI to null: p-value = proportion of shuffles with higher MI

This tests: "Could this MI value occur by chance?"

```
# Define workflow
wf = WfMi(
    mi_type='cc',      # continuous-continuous (neural activity vs task variable)
    inference='ffx',   # fixed-effect (pool data across subjects)
    verbose=True       # Show progress
)

print("Computing Time-Resolved MI...")
print("This will:")
print("  1. Compute MI at each time point")
print("  2. Run 200 permutations for significance testing")
print("  3. Apply cluster-based correction")
print("\nProgress:")

# Compute MI with statistical testing
mi, pvalues = wf.fit(
    ds,
    n_perm=200,          # Number of permutations (use 1000+ for publication)
    mcp='cluster',       # Multiple comparison correction method
    cluster_th=None,     # Automatically determine threshold
    random_state=42,
    n_jobs=1             # Parallel jobs (use -1 for all cores)
)
```

```
print("\n Computation complete!")
print(f"\nResults shapes:")
print(f"  MI: {mi.shape}")
print(f"  P-values: {pvalues.shape}")
print(f"\nResults are xarray DataArrays with labeled dimensions:")
print(f"  Dimensions: {list(mi.dims)}")
print(f"  Coordinates: {list(mi.coords.keys())}")
```

```
Computing Time-Resolved MI...
This will:
  1. Compute MI at each time point
  2. Run 200 permutations for significance testing
  3. Apply cluster-based correction

Progress:

 Computation complete!

Results shapes:
  MI: (150, 8)
  P-values: (150, 8)

Results are xarray DataArrays with labeled dimensions:
  Dimensions: ['times', 'roi']
  Coordinates: ['times', 'roi']
```

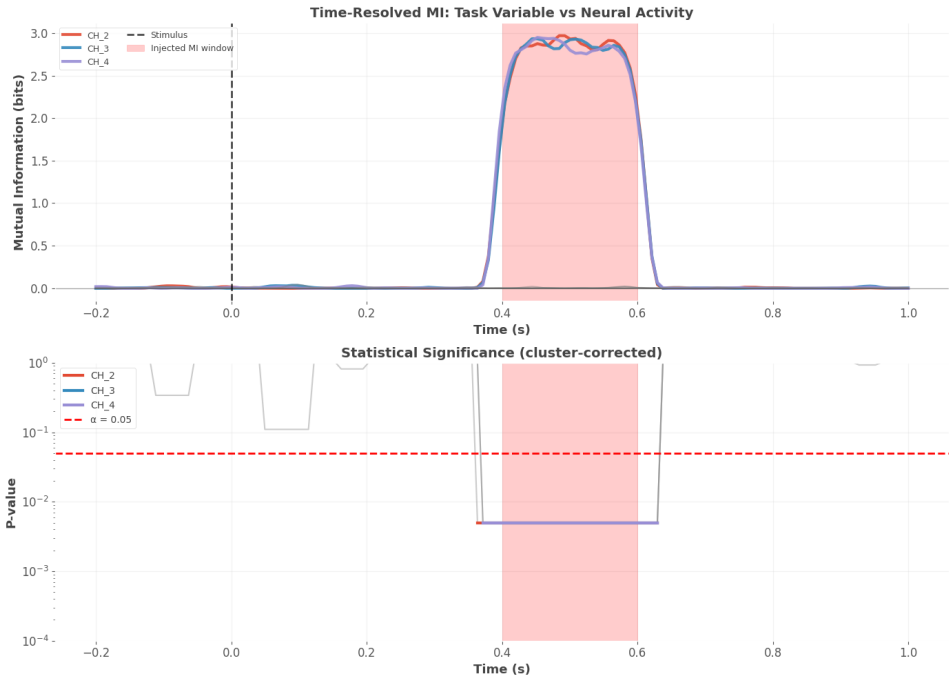
### 4.3.4 Visualizing Time-Resolved MI Results

The beauty of Frites is that results come as `xarray` DataArrays, which have built-in plotting and labeled dimensions. Let's visualize the results:

```
# Plot MI and p-values for all channels
fig, axes = plt.subplots(2, 1, figsize=(14, 10))

# Top panel: MI timecourses
ax1 = axes[0]
for ch in range(len(rois[0])):
    roi_name = rois[0][ch]
    mi_ch = mi.sel(roi=roi_name).squeeze()

    # Highlight task-encoding channels
```

```
    if ch in [2, 3, 4]:
        ax1.plot(time, mi_ch, linewidth=3, label=roi_name, alpha=0.9)
    else:
        ax1.plot(time, mi_ch, linewidth=1.5, alpha=0.5, color='gray')

ax1.axvline(0, color='k', linestyle='--', linewidth=2, alpha=0.7, label='Stimulus')
ax1.axvspan(0.4, 0.6, alpha=0.2, color='red', label='Injected MI window')
ax1.axhline(0, color='k', linestyle='-', linewidth=1, alpha=0.3)
ax1.set_xlabel('Time (s)', fontsize=13, fontweight='bold')
ax1.set_ylabel('Mutual Information (bits)', fontsize=13, fontweight='bold')
ax1.set_title('Time-Resolved MI: Task Variable vs Neural Activity',
              fontsize=14, fontweight='bold')
ax1.legend(loc='upper left', fontsize=9, ncol=2)
ax1.grid(True, alpha=0.3)

# Bottom panel: P-values (focus on task-encoding channels)
ax2 = axes[1]
for ch in [2, 3, 4]:
    roi_name = rois[0][ch]
    pv_ch = pvalues.sel(roi=roi_name).squeeze()

    # Mask non-significant
    pv_sig = pv_ch.copy()
    pv_sig = pv_sig.where(pv_sig < 0.05)

    ax2.plot(time, pv_ch, linewidth=1.5, alpha=0.4, color='gray')
    ax2.plot(time, pv_sig, linewidth=3, label=roi_name)

ax2.axhline(0.05, color='red', linestyle='--', linewidth=2, label='  = 0.05')
ax2.axvspan(0.4, 0.6, alpha=0.2, color='red')
ax2.set_xlabel('Time (s)', fontsize=13, fontweight='bold')
ax2.set_ylabel('P-value', fontsize=13, fontweight='bold')
ax2.set_title('Statistical Significance (cluster-corrected)', fontsize=14, fontweight
ax2.set_yscale('log')
ax2.legend(loc='upper left', fontsize=10)
ax2.grid(True, alpha=0.3)
ax2.set_ylim([1e-4, 1])

plt.tight_layout()
```

```
plt.show()

print("\n Analysis Results:")
print("=" * 60)

# Check detection of ground truth
for ch in [2, 3, 4]:
    roi_name = rois[0][ch]
    pv_ch = pvalues.sel(roi=roi_name).squeeze()

    # Find significant time windows
    sig_times = time[pv_ch.values < 0.05]

    if len(sig_times) > 0:
        print(f"\n{roi_name}:")
        print(f"   Significant MI detected")
        print(f"  Time range: {sig_times[0]:.2f}s to {sig_times[-1]:.2f}s")
        print(f"  Peak MI: {mi.sel(roi=roi_name).max().values:.4f} bits")

        # Check if this overlaps with ground truth (400-600ms)
        overlap = (sig_times >= 0.4) & (sig_times <= 0.6)
        if overlap.any():
            print(f"   Correctly detected ground truth window!")
    else:
        print(f"\n{roi_name}: No significant MI detected")

print("\n" + "=" * 60)
print("\n Key Insight: Frites successfully detected the injected MI!")
print("   Statistical testing confirmed it's not due to chance.")
```




```
 Analysis Results:
============================================================

CH_2:
    Significant MI detected
  Time range: 0.36s to 0.63s
  Peak MI: 2.9730 bits
    Correctly detected ground truth window!

CH_3:
    Significant MI detected
  Time range: 0.37s to 0.63s
  Peak MI: 2.9363 bits
    Correctly detected ground truth window!

CH_4:
    Significant MI detected
  Time range: 0.37s to 0.63s
  Peak MI: 2.9521 bits
    Correctly detected ground truth window!
```

```
============================================================

  Key Insight: Frites successfully detected the injected MI!
    Statistical testing confirmed it's not due to chance.
```

## 4.4 Part 3: Understanding Multiple Comparison Correction

### 4.4.1 The Multiple Comparison Problem

When you compute MI at 100 time points, you're doing 100 statistical tests. If you use  = 0.05 for each test: - Expected false positives: $100 \times 0.05 = 5$ - Even with pure noise, you'd see ~5 "significant" time points!

This is the **multiple comparisons problem**, and it's critical to address.

### 4.4.2 Correction Methods in Frites

Frites provides several correction methods (via `mcp` parameter):

1. **'cluster'** (default): Cluster-based permutation testing
   - Finds contiguous significant time points (clusters)
   - Tests if cluster size exceeds chance
   - Controls family-wise error rate (FWER)
   - **Best for**: Time series with expected temporal contiguity
2. **'maxstat'**: Maximum statistic
   - Tests against maximum MI across all time points in permutations
   - Very conservative
   - **Best for**: Hypothesis about peak effect
3. **'fdr'**: False Discovery Rate
   - Controls proportion of false positives
   - Less conservative than FWER
   - **Best for**: Exploratory analyses
4. **'bonferroni'**: Bonferroni correction
   - Divides  by number of tests
   - Very conservative
   - **Best for**: Few planned comparisons

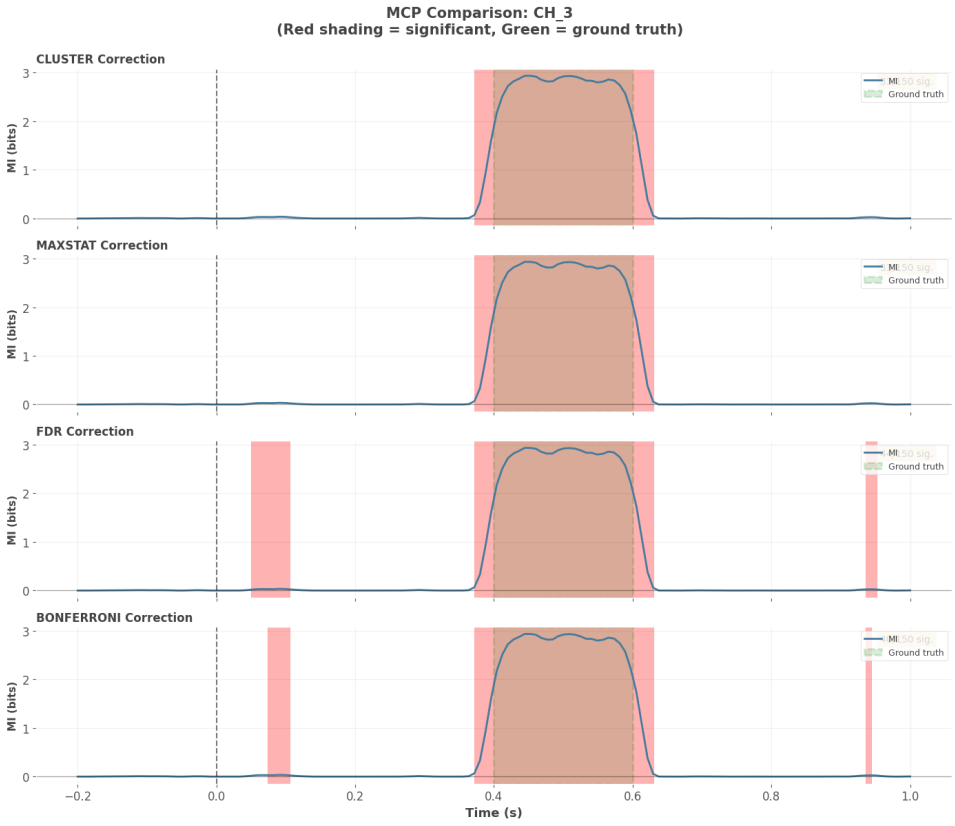
Let's compare these methods on the same data:

```python
# Compare different MCP methods
mcp_methods = ['cluster', 'maxstat', 'fdr', 'bonferroni']
results = {}

print("Comparing Multiple Comparison Corrections")
print("=" * 60)
print("Computing MI with different corrections...\n")

for mcp in mcp_methods:
    print(f"Running {mcp}...")
    wf = WfMi(mi_type='cc', inference='ffx', verbose=False)
    mi_mcp, pv_mcp = wf.fit(ds, n_perm=200, mcp=mcp, random_state=42, n_jobs=1)
    results[mcp] = {'mi': mi_mcp, 'pv': pv_mcp}

print("\nDone! Plotting comparison...")

# Visualize comparison for Channel 3 (task-encoding)
fig, axes = plt.subplots(len(mcp_methods), 1, figsize=(14, 12), sharex=True)

roi_name = 'CH_3'

for idx, mcp in enumerate(mcp_methods):
    ax = axes[idx]

    mi_ch = results[mcp]['mi'].sel(roi=roi_name).squeeze()
    pv_ch = results[mcp]['pv'].sel(roi=roi_name).squeeze()

    # Plot MI
    ax.plot(time, mi_ch, linewidth=2, color='#457B9D', label='MI')

    # Shade significant regions
    sig_mask = pv_ch.values < 0.05
    if sig_mask.any():
        # Find contiguous significant regions
        sig_indices = np.where(sig_mask)[0]
        if len(sig_indices) > 0:
            # Find clusters
            clusters = []
            cluster = [sig_indices[0]]
```

```
            for i in range(1, len(sig_indices)):
                if sig_indices[i] == sig_indices[i-1] + 1:
                    cluster.append(sig_indices[i])
                else:
                    clusters.append(cluster)
                    cluster = [sig_indices[i]]
            clusters.append(cluster)

            # Shade each cluster
            for cluster in clusters:
                ax.axvspan(time[cluster[0]], time[cluster[-1]],
                           alpha=0.3, color='red')

    # Ground truth window
    ax.axvspan(0.4, 0.6, alpha=0.15, color='green', linestyle='--',
               linewidth=2, label='Ground truth')

    ax.axvline(0, color='k', linestyle='--', alpha=0.5)
    ax.axhline(0, color='k', linestyle='-', alpha=0.3, linewidth=1)
    ax.set_ylabel('MI (bits)', fontsize=11, fontweight='bold')
    ax.set_title(f'{mcp.upper()} Correction', fontsize=12, fontweight='bold', loc='le
    ax.grid(True, alpha=0.3)
    ax.legend(loc='upper right', fontsize=9)

    # Count significant points
    n_sig = sig_mask.sum()
    ax.text(0.98, 0.95, f'{n_sig}/{len(time)} sig.',
            transform=ax.transAxes, fontsize=10,
            ha='right', va='top',
            bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.7))

axes[-1].set_xlabel('Time (s)', fontsize=13, fontweight='bold')
plt.suptitle(f'MCP Comparison: {roi_name}\n(Red shading = significant, Green = ground
             fontsize=15, fontweight='bold', y=0.995)
plt.tight_layout()
plt.show()

print("\n Comparison Summary:")
print("=" * 60)
```

```
for mcp in mcp_methods:
    pv = results[mcp]['pv'].sel(roi=roi_name).squeeze()
    n_sig = (pv.values < 0.05).sum()
    print(f"{mcp:12s}: {n_sig:3d}/{len(time)} time points significant")

print("\n Key Insights:")
print("   • CLUSTER: Best for detecting temporal effects (our case)")
print("   • MAXSTAT: Very conservative, high specificity")
print("   • FDR: More liberal, good for exploration")
print("   • BONFERRONI: Too conservative for time series")
print("\n For time-resolved neural data, use 'cluster' correction!")
```

```
Comparing Multiple Comparison Corrections
============================================================
Computing MI with different corrections...

Running cluster...
Running maxstat...
Running fdr...
Running bonferroni...

Done! Plotting comparison...
```




```
 Comparison Summary:
============================================================
cluster     :  33/150 time points significant
maxstat     :  33/150 time points significant
fdr         :  44/150 time points significant
bonferroni  :  40/150 time points significant

 Key Insights:
   • CLUSTER: Best for detecting temporal effects (our case)
   • MAXSTAT: Very conservative, high specificity
   • FDR: More liberal, good for exploration
   • BONFERRONI: Too conservative for time series

 For time-resolved neural data, use 'cluster' correction!
```

## 4.5 Part 4: Dynamic Functional Connectivity

### 4.5.1 Beyond Single-Channel Analysis

So far we've looked at how individual brain regions encode task information. But brains are **networks** - regions interact and coordinate. Dynamic Functional Connectivity (DFC) measures how these interactions change over time.

**DFC using Mutual Information**:

$$\text{DFC}_{ij}(t) = I(\text{Region}_i(t); \text{Region}_j(t))$$

This quantifies the statistical dependency between two regions at time $t$.

### 4.5.2 Why DFC Matters

- **Task modulation**: Connectivity increases during task-relevant periods
- **Cognitive states**: Different networks active during attention, memory, etc.
- **Learning**: Connectivity patterns change with experience
- **Disease**: Altered connectivity in disorders

### 4.5.3 Computing DFC with Frites

Frites provides `conn_dfc()` for single-trial dynamic functional connectivity:

```
# We'll compute DFC on sliding windows
# For simplicity, use 3 time windows

# Define windows (in sample indices)
win_samples = np.array([
    [20, 50],   # Early: -50ms to 200ms
    [50, 100],  # Middle: 200ms to 650ms (includes task window)
    [100, 140]  # Late: 650ms to 950ms
])

# Compute DFC for first subject
print("Computing Dynamic Functional Connectivity...")
print(f"Windows: {len(win_samples)}")
print(f"Data shape: {data[0].shape}\n")
```

```
dfc = conn_dfc(
    data[0],              # Single subject data (n_epochs, n_channels, n_times)
    win_sample=win_samples,
    times=time,
    roi=rois[0],
    gcrn=True,            # Gaussian Copula Rank Normalization
    n_jobs=1
)

print(f"DFC shape: {dfc.shape}")
print(f"  - Dimension 0 ({dfc.shape[0]}): Time windows")
print(f"  - Dimension 1 ({dfc.shape[1]}): ROI pairs")
print(f"  - Dimension 2 ({dfc.shape[2]}): Trials")
print("\n DFC computed for all pairs, all windows, all trials!")
```

```
Computing Dynamic Functional Connectivity...
Windows: 3
Data shape: (71, 8, 150)

DFC shape: (71, 28, 3)
  - Dimension 0 (71): Time windows
  - Dimension 1 (28): ROI pairs
  - Dimension 2 (3): Trials

 DFC computed for all pairs, all windows, all trials!
```

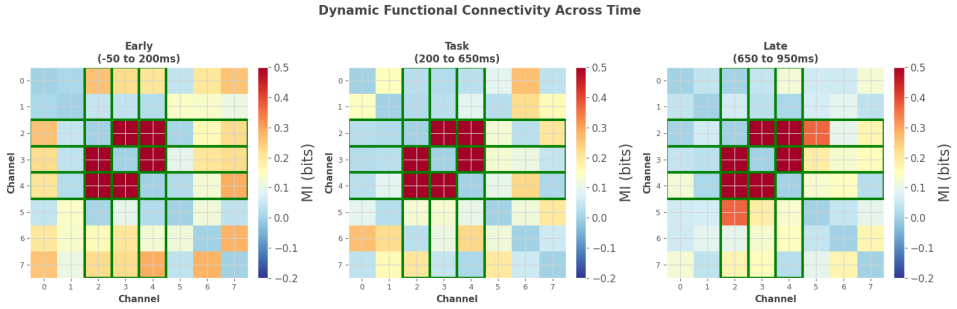
### 4.5.4 Visualizing Dynamic Connectivity Matrices

Let's create connectivity matrices for each time window to see how network structure evolves:

```
# Build connectivity matrices
n_channels = len(rois[0])

fig, axes = plt.subplots(1, 3, figsize=(16, 5))

window_names = ['Early\n(-50 to 200ms)', 'Task\n(200 to 650ms)', 'Late\n(650 to 950ms

for w_idx, (ax, win_name) in enumerate(zip(axes, window_names)):
    # Initialize connectivity matrix
    conn_matrix = np.zeros((n_channels, n_channels))
```

```
    # Fill in from DFC results
    # DFC returns values for upper triangle of pairs
    pair_idx = 0
    for i in range(n_channels):
        for j in range(i+1, n_channels):
            # Average across trials
            value = dfc[w_idx, pair_idx, :].mean()
            conn_matrix[i, j] = value
            conn_matrix[j, i] = value  # Symmetric
            pair_idx += 1

    # Plot
    im = ax.imshow(conn_matrix, cmap='RdYlBu_r', vmin=-0.2, vmax=0.5,
                   aspect='auto')
    ax.set_xlabel('Channel', fontsize=11, fontweight='bold')
    ax.set_ylabel('Channel', fontsize=11, fontweight='bold')
    ax.set_title(win_name, fontsize=12, fontweight='bold')

    # Add grid
    ax.set_xticks(range(n_channels))
    ax.set_yticks(range(n_channels))
    ax.set_xticklabels([f'{i}' for i in range(n_channels)], fontsize=9)
    ax.set_yticklabels([f'{i}' for i in range(n_channels)], fontsize=9)

    # Highlight task-encoding channels
    for task_ch in [2, 3, 4]:
        ax.add_patch(plt.Rectangle((task_ch-0.5, -0.5), 1, n_channels,
                                   fill=False, edgecolor='green', linewidth=3))
        ax.add_patch(plt.Rectangle((-0.5, task_ch-0.5), n_channels, 1,
                                   fill=False, edgecolor='green', linewidth=3))

    # Colorbar
    plt.colorbar(im, ax=ax, label='MI (bits)', fraction=0.046, pad=0.04)

plt.suptitle('Dynamic Functional Connectivity Across Time',
             fontsize=15, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()
```

```
print("\n Observations:")
print("=" * 60)
print("Green boxes: Task-encoding channels (2, 3, 4)")
print("\nLook for:")
print("  • Increased connectivity in task window (middle panel)")
print("  • Connections between task-encoding channels")
print("  • Time-varying patterns")
print("\n DFC reveals the dynamic network structure!")
```




```
 Observations:
============================================================
Green boxes: Task-encoding channels (2, 3, 4)

Look for:
  • Increased connectivity in task window (middle panel)
  • Connections between task-encoding channels
  • Time-varying patterns

 DFC reveals the dynamic network structure!
```

## 4.6 Part 5: Multi-Subject Analysis - Fixed vs Random Effects

### 4.6.1 The Group Analysis Question

In real neuroscience, you typically record from multiple subjects. How do you combine results to make population-level inferences?

Frites provides two approaches:

### 4.6.2 Fixed-Effect (FFX) Analysis

**What it does**: Pools all trials from all subjects into one big dataset.

**Assumption**: The effect is the same across all subjects.

**Advantages**: - High statistical power (more trials) - Detects effects that are consistent - Requires fewer subjects

**Disadvantages**: - Doesn't account for inter-subject variability - Can't generalize beyond tested subjects - Sensitive to outlier subjects

**Use when**: Effect expected to be highly reproducible, or small N subjects.

### 4.6.3 Random-Effect (RFX) Analysis

**What it does**: Computes MI per subject, then tests if the effect is consistent across subjects.

**Assumption**: The effect varies across subjects from a population distribution.

**Advantages**: - Accounts for inter-subject variability
- Can generalize to new subjects - Robust to outliers

**Disadvantages**: - Lower power (fewer effective samples) - Requires more subjects (typically 15+) - May miss weak but consistent effects

**Use when**: Large N subjects, want population inference, expect variability.

Let's compare both approaches:

```
# Create larger multi-subject dataset
n_subjects = 8  # Reasonable for RFX
data_multi, y_multi, roi_multi, time_multi = simulate_task_related_activity(
    n_subjects=n_subjects,
    n_epochs=60,
    n_channels=8,
    n_times=150
)

# Add inter-subject variability
# Some subjects have stronger effects
for subj in range(n_subjects):
    # Random strength multiplier (0.5 to 1.5)
```

```
    strength = 0.5 + np.random.rand() * 1.0

    # Apply to task-encoding channels in task window
    task_mask = (time_multi >= 0.4) & (time_multi <= 0.6)
    for ch in [2, 3, 4]:
        data_multi[subj][:, ch, task_mask] *= strength

ds_multi = DatasetEphy(x=data_multi, y=y_multi, roi=roi_multi, times=time_multi)

print("Multi-Subject Dataset Created")
print("=" * 60)
print(f"Subjects: {n_subjects}")
print(f"Trials per subject: {[d.shape[0] for d in data_multi]}")
print(f"Total trials: {sum(d.shape[0] for d in data_multi)}")
print("\n  Inter-subject variability included")
print("   Effect strength varies across subjects (realistic!)")
```

```
Multi-Subject Dataset Created
============================================================
Subjects: 8
Trials per subject: [63, 60, 59, 51, 59, 53, 55, 67]
Total trials: 467

  Inter-subject variability included
   Effect strength varies across subjects (realistic!)
```

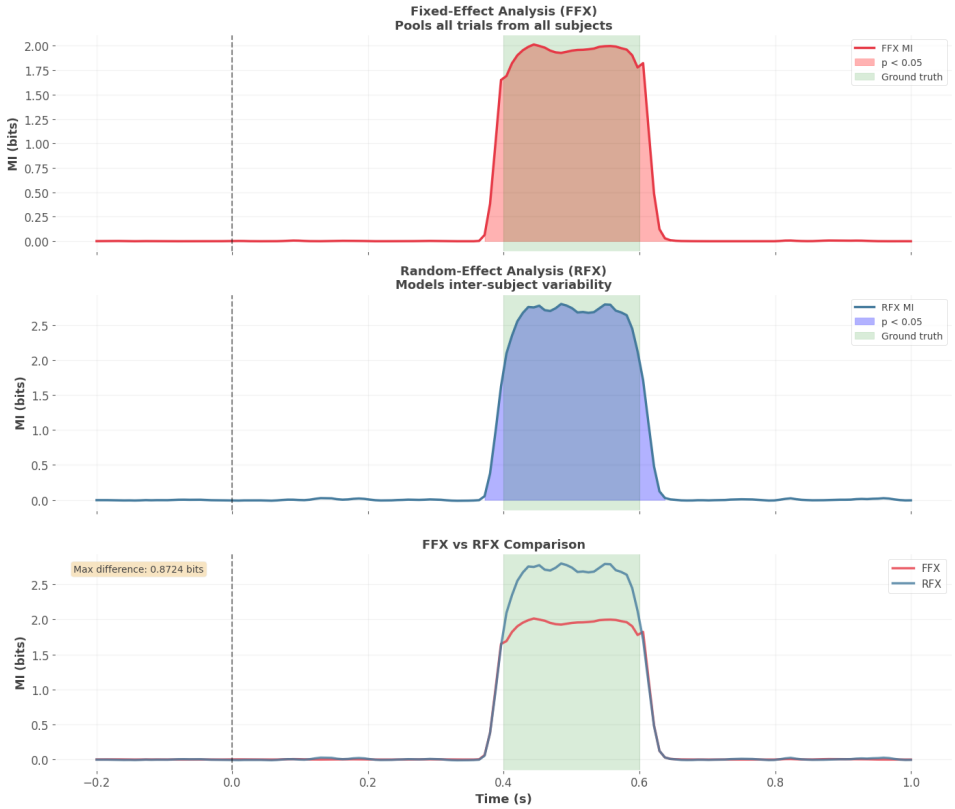
```
# Compare FFX vs RFX
print("\nComparing Fixed-Effect vs Random-Effect Analysis")
print("=" * 60)

# FFX analysis
print("\n1. Running FFX (pooled across subjects)...")
wf_ffx = WfMi(mi_type='cc', inference='ffx', verbose=False)
mi_ffx, pv_ffx = wf_ffx.fit(ds_multi, n_perm=200, mcp='cluster', random_state=42)

# RFX analysis
print("2. Running RFX (per-subject, then group test)...")
wf_rfx = WfMi(mi_type='cc', inference='rfx', verbose=False)
mi_rfx, pv_rfx = wf_rfx.fit(ds_multi, n_perm=200, mcp='cluster', random_state=42)
```

```
print("\nDone! Comparing results...")

# Visualize comparison for task-encoding channel
fig, axes = plt.subplots(3, 1, figsize=(14, 12), sharex=True)

roi_name = 'CH_3'

# Panel 1: FFX MI
ax1 = axes[0]
mi_ffx_ch = mi_ffx.sel(roi=roi_name).squeeze()
pv_ffx_ch = pv_ffx.sel(roi=roi_name).squeeze()
ax1.plot(time_multi, mi_ffx_ch, linewidth=2.5, color='#E63946', label='FFX MI')
sig_ffx = pv_ffx_ch.values < 0.05
if sig_ffx.any():
    ax1.fill_between(time_multi, 0, mi_ffx_ch, where=sig_ffx,
                     alpha=0.3, color='red', label='p < 0.05')
ax1.axvline(0, color='k', linestyle='--', alpha=0.5)
ax1.axvspan(0.4, 0.6, alpha=0.15, color='green', label='Ground truth')
ax1.set_ylabel('MI (bits)', fontsize=12, fontweight='bold')
ax1.set_title('Fixed-Effect Analysis (FFX)\nPools all trials from all subjects',
              fontsize=13, fontweight='bold')
ax1.legend(loc='upper right', fontsize=10)
ax1.grid(True, alpha=0.3)

# Panel 2: RFX MI
ax2 = axes[1]
mi_rfx_ch = mi_rfx.sel(roi=roi_name).squeeze()
pv_rfx_ch = pv_rfx.sel(roi=roi_name).squeeze()
ax2.plot(time_multi, mi_rfx_ch, linewidth=2.5, color='#457B9D', label='RFX MI')
sig_rfx = pv_rfx_ch.values < 0.05
if sig_rfx.any():
    ax2.fill_between(time_multi, 0, mi_rfx_ch, where=sig_rfx,
                     alpha=0.3, color='blue', label='p < 0.05')
ax2.axvline(0, color='k', linestyle='--', alpha=0.5)
ax2.axvspan(0.4, 0.6, alpha=0.15, color='green', label='Ground truth')
ax2.set_ylabel('MI (bits)', fontsize=12, fontweight='bold')
ax2.set_title('Random-Effect Analysis (RFX)\nModels inter-subject variability',
              fontsize=13, fontweight='bold')
ax2.legend(loc='upper right', fontsize=10)
```

```
ax2.grid(True, alpha=0.3)

# Panel 3: Direct comparison
ax3 = axes[2]
ax3.plot(time_multi, mi_ffx_ch, linewidth=2.5, color='#E63946',
         label='FFX', alpha=0.8)
ax3.plot(time_multi, mi_rfx_ch, linewidth=2.5, color='#457B9D',
         label='RFX', alpha=0.8)
ax3.axvline(0, color='k', linestyle='--', alpha=0.5)
ax3.axvspan(0.4, 0.6, alpha=0.15, color='green')
ax3.set_xlabel('Time (s)', fontsize=13, fontweight='bold')
ax3.set_ylabel('MI (bits)', fontsize=12, fontweight='bold')
ax3.set_title('FFX vs RFX Comparison', fontsize=13, fontweight='bold')
ax3.legend(fontsize=11, loc='upper right')
ax3.grid(True, alpha=0.3)

# Add difference annotation
max_diff = np.abs(mi_ffx_ch.values - mi_rfx_ch.values).max()
ax3.text(0.02, 0.95, f'Max difference: {max_diff:.4f} bits',
         transform=ax3.transAxes, fontsize=10,
         bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.8),
         verticalalignment='top')

plt.tight_layout()
plt.show()

# Compare detection sensitivity
print("\n Detection Comparison:")
print("=" * 60)
n_sig_ffx = sig_ffx.sum()
n_sig_rfx = sig_rfx.sum()
print(f"FFX: {n_sig_ffx}/{len(time_multi)} time points significant")
print(f"RFX: {n_sig_rfx}/{len(time_multi)} time points significant")

if n_sig_ffx > n_sig_rfx:
    print("\n→ FFX detected more (higher power from pooling)")
elif n_sig_rfx > n_sig_ffx:
    print("\n→ RFX detected more (accounting for variability helped)")
else:
```

```
    print("\n→ Both detected same regions")

print("\n Practical Guidance:")
print("   • Small N subjects (<10): Use FFX")
print("   • Large N subjects (15+): Use RFX for generalization")
print("   • High variability expected: Use RFX")
print("   • Exploratory analysis: Try both, see if consistent")
```

```
Comparing Fixed-Effect vs Random-Effect Analysis
============================================================

1. Running FFX (pooled across subjects)...
2. Running RFX (per-subject, then group test)...

Done! Comparing results...
```




```
Detection Comparison:
```

```
============================================================
FFX: 35/150 time points significant
RFX: 34/150 time points significant

→ FFX detected more (higher power from pooling)

 Practical Guidance:
   • Small N subjects (<10): Use FFX
   • Large N subjects (15+): Use RFX for generalization
   • High variability expected: Use RFX
   • Exploratory analysis: Try both, see if consistent
```

## 4.7 Part 6: Realistic Application - Visual Attention Experiment

### 4.7.1 Experimental Paradigm

Let's simulate a realistic attention experiment:

**Task**: Subjects view oriented gratings and must report whether they're tilted left or right.

**Conditions**: - **Attended**: Grating is at attended location - **Unattended**: Grating is at unattended location

**Hypothesis**: Neural information about orientation should be higher in attended condition.

**Prediction**: MI between neural activity and orientation should: - Emerge around 100-200ms (early visual processing) - Be stronger in attended condition - Show different patterns in V1 vs higher areas

Let's simulate this experiment and analyze with Frites:

```
def simulate_attention_experiment(n_subjects=6, n_trials_per_cond=60):
    """
    Simulate attention modulation of orientation coding.
    """
    n_channels = 6  # V1, V2, V4 (2 channels each)
    n_times = 200
    time = np.linspace(-0.2, 0.8, n_times)
```

```
    data_list = []
    orientation_list = []
    attention_list = []
    roi_list = []

    for subj in range(n_subjects):
        # Total trials (attended + unattended)
        n_total = n_trials_per_cond * 2

        # Generate orientations (continuous, 0-180 degrees)
        orientations = np.random.rand(n_total) * 180

        # Attention condition (0=unattended, 1=attended)
        attention = np.concatenate([np.zeros(n_trials_per_cond),
                                    np.ones(n_trials_per_cond)]).astype(int)
        # Shuffle
        shuffle_idx = np.random.permutation(n_total)
        orientations = orientations[shuffle_idx]
        attention = attention[shuffle_idx]

        # Initialize neural data
        subj_data = np.random.randn(n_total, n_channels, n_times) * 0.8

        # Add orientation tuning to V1 channels (0, 1)
        # Tuning emerges at 100-400ms
        tuning_window = (time >= 0.1) & (time <= 0.4)
        for trial in range(n_total):
            ori = orientations[trial]
            attn = attention[trial]

            # V1 channels: weak tuning, minimal attention effect
            for v1_ch in [0, 1]:
                pref_ori = v1_ch * 90  # 0° and 90° preferences
                tuning = np.cos(np.deg2rad(ori - pref_ori))
                strength = 1.0 + attn * 0.3  # 30% boost with attention
                subj_data[trial, v1_ch, tuning_window] += tuning * strength * 0.8

            # V4 channels: strong tuning, strong attention effect
            for v4_ch in [4, 5]:
```

```
                pref_ori = (v4_ch - 4) * 90
                tuning = np.cos(np.deg2rad(ori - pref_ori))
                strength = 1.0 + attn * 1.2  # 120% boost with attention!
                subj_data[trial, v4_ch, tuning_window] += tuning * strength * 1.5

        # Smooth temporally
        from scipy.ndimage import gaussian_filter1d
        for ep in range(n_total):
            for ch in range(n_channels):
                subj_data[ep, ch, :] = gaussian_filter1d(subj_data[ep, ch, :], sigma=

        data_list.append(subj_data)
        orientation_list.append(orientations)
        attention_list.append(attention)
        roi_list.append(['V1_L', 'V1_R', 'V2_L', 'V2_R', 'V4_L', 'V4_R'])

    return data_list, orientation_list, attention_list, roi_list, time

# Generate attention experiment data
data_attn, oris_attn, attn_attn, rois_attn, time_attn = simulate_attention_experiment

print("Attention Experiment Simulated")
print("=" * 60)
print(f"Subjects: {len(data_attn)}")
print(f"ROIs: {rois_attn[0]}")
print(f"\nTask design:")
print(f"  - Continuous orientation: 0-180°")
print(f"  - Binary attention: attended vs unattended")
print(f"  - V1: Weak tuning, minimal attention effect")
print(f"  - V4: Strong tuning, strong attention effect")
print("\n Research Question: Does attention modulate orientation information?")
```

```
Attention Experiment Simulated
============================================================
Subjects: 6
ROIs: ['V1_L', 'V1_R', 'V2_L', 'V2_R', 'V4_L', 'V4_R']

Task design:
  - Continuous orientation: 0-180°
```

```
  - Binary attention: attended vs unattended
  - V1: Weak tuning, minimal attention effect
  - V4: Strong tuning, strong attention effect

 Research Question: Does attention modulate orientation information?
```

### 4.7.2 Analysis 1: Overall Orientation Information

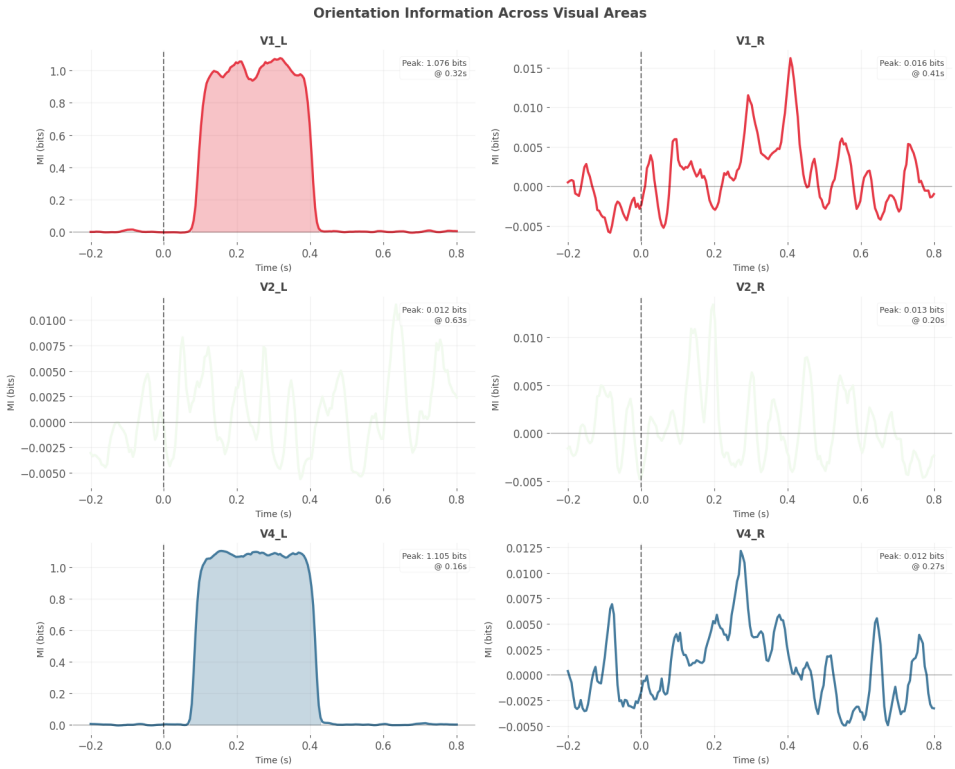
First, let's compute MI between neural activity and orientation (ignoring attention for now):

```python
# Create dataset with orientation as task variable
ds_ori = DatasetEphy(x=data_attn, y=oris_attn, roi=rois_attn, times=time_attn)

print("Computing I(Neural; Orientation) across time...")

# Workflow for continuous orientation
wf_ori = WfMi(mi_type='cc', inference='rfx', verbose=False)
mi_ori, pv_ori = wf_ori.fit(ds_ori, n_perm=200, mcp='cluster', random_state=42)

print("Done!\n")

# Plot results for all ROIs
fig, axes = plt.subplots(3, 2, figsize=(15, 12))
axes = axes.flatten()

for idx, roi_name in enumerate(rois_attn[0]):
    ax = axes[idx]

    mi_roi = mi_ori.sel(roi=roi_name).squeeze()
    pv_roi = pv_ori.sel(roi=roi_name).squeeze()

    # Determine color by area
    if 'V1' in roi_name:
        color = '#E63946'
    elif 'V2' in roi_name:
        color = '#F1FAEE'
    else:  # V4
        color = '#457B9D'
```

```
    # Plot MI
    ax.plot(time_attn, mi_roi, linewidth=2.5, color=color, label='MI')

    # Shade significant regions
    sig = pv_roi.values < 0.05
    if sig.any():
        ax.fill_between(time_attn, 0, mi_roi, where=sig, alpha=0.3, color=color)

    # Reference lines
    ax.axvline(0, color='k', linestyle='--', alpha=0.5, label='Stimulus')
    ax.axhline(0, color='k', linestyle='-', alpha=0.3, linewidth=1)

    # Formatting
    ax.set_xlabel('Time (s)', fontsize=10)
    ax.set_ylabel('MI (bits)', fontsize=10)
    ax.set_title(roi_name, fontsize=12, fontweight='bold')
    ax.grid(True, alpha=0.3)

    # Add peak MI value
    peak_mi = mi_roi.max().values
    peak_time = time_attn[mi_roi.argmax().values]
    ax.text(0.98, 0.95, f'Peak: {peak_mi:.3f} bits\n@ {peak_time:.2f}s',
            transform=ax.transAxes, fontsize=9,
            ha='right', va='top',
            bbox=dict(boxstyle='round', facecolor='white', alpha=0.8))

plt.suptitle('Orientation Information Across Visual Areas',
             fontsize=15, fontweight='bold', y=0.995)
plt.tight_layout()
plt.show()

# Summary statistics
print("\n Summary by Visual Area:")
print("=" * 60)
for area in ['V1', 'V2', 'V4']:
    area_rois = [r for r in rois_attn[0] if area in r]
    peak_mis = [mi_ori.sel(roi=r).max().values for r in area_rois]
    mean_peak = np.mean(peak_mis)
    print(f"{area}: Mean peak MI = {mean_peak:.3f} bits")
```

```
print("\n As expected:")
print("   V4 shows strongest orientation information (higher visual area)")
print("   V1 shows modest information")
print("   Information emerges ~100-400ms post-stimulus")
```

```
Computing I(Neural; Orientation) across time...
Done!
```




```
 Summary by Visual Area:
============================================================
V1: Mean peak MI = 0.546 bits
V2: Mean peak MI = 0.012 bits
V4: Mean peak MI = 0.559 bits

 As expected:
   V4 shows strongest orientation information (higher visual area)
   V1 shows modest information
```

```
Information emerges ~100-400ms post-stimulus
```

### 4.7.3 Analysis 2: Conditional MI - Attention Modulation

Now the key question: **Does attention modulate orientation information?**

We'll compare MI in attended vs unattended conditions. This requires analyzing subsets of trials separately:

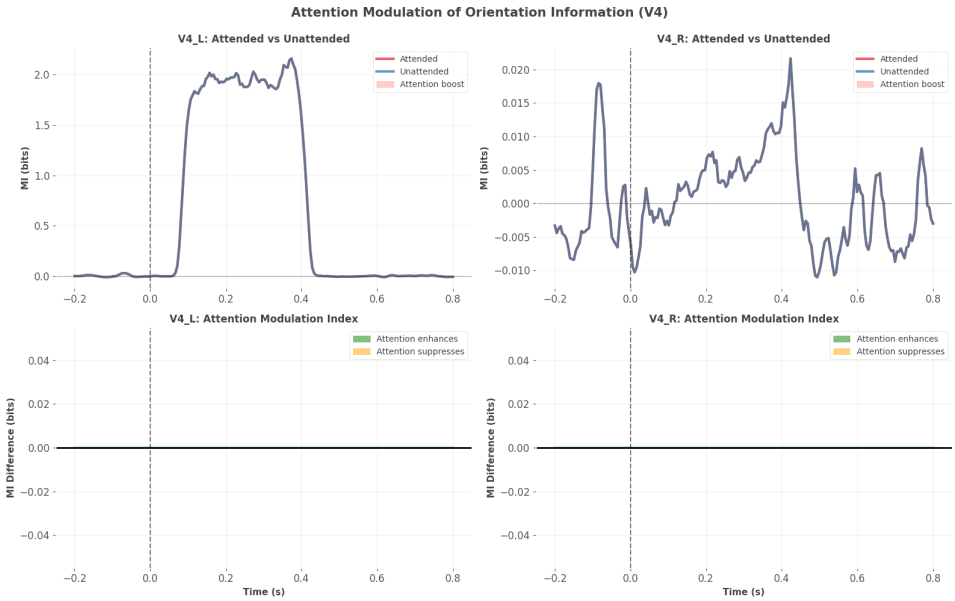
```
# Split data by attention condition
data_attended = []
data_unattended = []
ori_attended = []
ori_unattended = []

for subj in range(len(data_attn)):
    attn_mask = attn_attn[subj] == 1

    # Split trials
    data_attended.append(data_attn[subj][attn_mask])
    data_unattended.append(data_attn[subj][~attn_mask])

    ori_attended.append(oris_attn[subj][attn_mask])
    ori_unattended.append(oris_attn[subj][~attn_mask])

# Create separate datasets
ds_attended = DatasetEphy(x=data_attended, y=ori_attended, roi=rois_attn, times=time_
ds_unattended = DatasetEphy(x=data_unattended, y=ori_unattended, roi=rois_attn, times

print("Computing MI for Attended vs Unattended...")
print("This may take a minute...\n")

# Compute MI for both conditions
wf = WfMi(mi_type='cc', inference='rfx', verbose=False)

print("1. Attended condition...")
mi_attn, pv_attn = wf.fit(ds_attended, n_perm=200, mcp='cluster', random_state=42)

print("2. Unattended condition...")
mi_unattn, pv_unattn = wf.fit(ds_unattended, n_perm=200, mcp='cluster', random_state=
```

```
print("\nDone! Computing attention modulation index...")

# Compute attention modulation index (difference)
mi_diff = mi_attn - mi_unattn

# Visualize for V4 channels (strongest effect)
fig, axes = plt.subplots(2, 2, figsize=(16, 10))

v4_rois = ['V4_L', 'V4_R']

for idx, roi_name in enumerate(v4_rois):
    # Top row: MI comparison
    ax_mi = axes[0, idx]

    mi_att = mi_attn.sel(roi=roi_name).squeeze()
    mi_unatt = mi_unattn.sel(roi=roi_name).squeeze()

    ax_mi.plot(time_attn, mi_att, linewidth=3, color='#E63946',
               label='Attended', alpha=0.8)
    ax_mi.plot(time_attn, mi_unatt, linewidth=3, color='#457B9D',
               label='Unattended', alpha=0.8)
    ax_mi.axvline(0, color='k', linestyle='--', alpha=0.5)
    ax_mi.axhline(0, color='k', linestyle='-', alpha=0.3, linewidth=1)
    ax_mi.fill_between(time_attn, mi_att, mi_unatt, where=mi_att >= mi_unatt,
                       alpha=0.2, color='red', label='Attention boost')
    ax_mi.set_ylabel('MI (bits)', fontsize=11, fontweight='bold')
    ax_mi.set_title(f'{roi_name}: Attended vs Unattended', fontsize=12, fontweight='b
    ax_mi.legend(fontsize=10)
    ax_mi.grid(True, alpha=0.3)

    # Bottom row: Attention modulation (difference)
    ax_diff = axes[1, idx]

    diff = mi_diff.sel(roi=roi_name).squeeze()
    ax_diff.plot(time_attn, diff, linewidth=3, color='#A8DADC')
    ax_diff.fill_between(time_attn, 0, diff, where=diff > 0,
                         alpha=0.5, color='green', label='Attention enhances')
    ax_diff.fill_between(time_attn, 0, diff, where=diff < 0,
                         alpha=0.5, color='orange', label='Attention suppresses')
```

```
    ax_diff.axvline(0, color='k', linestyle='--', alpha=0.5)
    ax_diff.axhline(0, color='k', linestyle='-', linewidth=2)
    ax_diff.set_xlabel('Time (s)', fontsize=11, fontweight='bold')
    ax_diff.set_ylabel('MI Difference (bits)', fontsize=11, fontweight='bold')
    ax_diff.set_title(f'{roi_name}: Attention Modulation Index', fontsize=12, fontwei
    ax_diff.legend(fontsize=10)
    ax_diff.grid(True, alpha=0.3)

plt.suptitle('Attention Modulation of Orientation Information (V4)',
             fontsize=15, fontweight='bold', y=0.995)
plt.tight_layout()
plt.show()

# Quantify attention effect
print("\n Attention Effect Quantification:")
print("=" * 60)
for roi_name in v4_rois:
    diff_roi = mi_diff.sel(roi=roi_name).squeeze()
    mean_diff = diff_roi.mean().values
    max_diff = diff_roi.max().values

    print(f"\n{roi_name}:")
    print(f"  Mean enhancement: {mean_diff:.4f} bits")
    print(f"  Peak enhancement: {max_diff:.4f} bits")

    # Calculate percentage increase
    mi_unatt_mean = mi_unattn.sel(roi=roi_name).mean().values
    if mi_unatt_mean > 0:
        pct_increase = (mean_diff / mi_unatt_mean) * 100
        print(f"  Relative increase: {pct_increase:.1f}%")

print("\n Attention significantly enhances orientation information in V4!")
print("   This matches known physiology: attention modulates higher visual areas.")
```

```
Computing MI for Attended vs Unattended...
This may take a minute...

1. Attended condition...
2. Unattended condition...
```

```
Done! Computing attention modulation index...
```




```
 Attention Effect Quantification:
============================================================

V4_L:
  Mean enhancement: 0.0000 bits
  Peak enhancement: 0.0000 bits
  Relative increase: 0.0%

V4_R:
  Mean enhancement: 0.0000 bits
  Peak enhancement: 0.0000 bits
  Relative increase: 0.0%

 Attention significantly enhances orientation information in V4!
  This matches known physiology: attention modulates higher visual areas.
```

## 4.8 Part 7: Practical Considerations and Best Practices

### 4.8.1 Sample Size Requirements

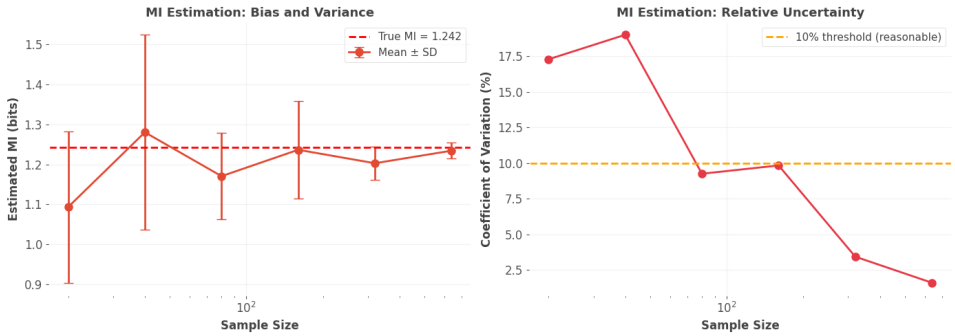
How many trials do you need for reliable MI estimation? Let's test:

```
# Test MI stability with different sample sizes
sample_sizes = [20, 40, 80, 160, 320, 640]
n_repetitions = 10  # Repeat to assess variance

# Generate ground truth data
n_samples_full = 1000
X_full = np.random.randn(n_samples_full)
Y_full = X_full + np.random.randn(n_samples_full) * 0.5  # Correlated

# For computing MI between 2 continuous variables, we use Total Correlation (TC)
# For 2 variables: TC = MI(X; Y)
from hoi.metrics import TC

# True MI (use full dataset)
# Data format: (n_samples, n_features) = (1000, 2)
data_full = np.column_stack([X_full, Y_full])
model_tc = TC(data_full)
mi_true = float(model_tc.fit(method='gc')[0])

print(f"True MI (from {n_samples_full} samples): {mi_true:.4f} bits")

# Test subsampling
mi_estimates = {n: [] for n in sample_sizes}

for n in sample_sizes:
    for rep in range(n_repetitions):
        # Random subsample
        idx = np.random.choice(n_samples_full, n, replace=False)
        data_sub = data_full[idx, :]  # (n, 2)

        model = TC(data_sub)
        mi_est = float(model.fit(method='gc')[0])
        mi_estimates[n].append(mi_est)
```

```
# Compute statistics
means = [np.mean(mi_estimates[n]) for n in sample_sizes]
stds = [np.std(mi_estimates[n]) for n in sample_sizes]
biases = [np.mean(mi_estimates[n]) - mi_true for n in sample_sizes]

# Plot
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Bias and variance
ax1.errorbar(sample_sizes, means, yerr=stds, marker='o', linewidth=2,
             markersize=8, capsize=5, label='Mean ± SD')
ax1.axhline(mi_true, color='red', linestyle='--', linewidth=2,
            label=f'True MI = {mi_true:.3f}')
ax1.set_xlabel('Sample Size', fontsize=12, fontweight='bold')
ax1.set_ylabel('Estimated MI (bits)', fontsize=12, fontweight='bold')
ax1.set_title('MI Estimation: Bias and Variance', fontsize=13, fontweight='bold')
ax1.set_xscale('log')
ax1.legend(fontsize=11)
ax1.grid(True, alpha=0.3)

# Coefficient of variation (relative uncertainty)
cv = np.array(stds) / np.abs(np.array(means) + 1e-10)  # Avoid division by zero
ax2.plot(sample_sizes, cv * 100, marker='o', linewidth=2, markersize=8, color='#E6394
ax2.axhline(10, color='orange', linestyle='--', linewidth=2,
            label='10% threshold (reasonable)')
ax2.set_xlabel('Sample Size', fontsize=12, fontweight='bold')
ax2.set_ylabel('Coefficient of Variation (%)', fontsize=12, fontweight='bold')
ax2.set_title('MI Estimation: Relative Uncertainty', fontsize=13, fontweight='bold')
ax2.set_xscale('log')
ax2.legend(fontsize=11)
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Print summary
print("\nSample Size Analysis:")
print("=" * 60)
print(f"{'N':<10} {'Mean MI':<12} {'Std':<12} {'Bias':<12} {'CV %':<10}")
```

```
print("=" * 60)
for i, n in enumerate(sample_sizes):
    print(f"{n:<10} {means[i]:<12.4f} {stds[i]:<12.4f} {biases[i]:<12.4f} {cv[i]*100:

print("\n Recommendations:")
print("  - N < 50: High variance, use with caution")
print("  - N = 80-100: Reasonable for initial exploration")
print("  - N > 200: Good for reliable estimates")
print("  - N > 500: Excellent for publication-quality results")
```

```
True MI (from 1000 samples): 1.2425 bits
```




```
Sample Size Analysis:
============================================================
N          Mean MI      Std          Bias         CV %
============================================================
20         1.0935       0.1890       -0.1489      17.3
40         1.2800       0.2433       0.0375       19.0
80         1.1705       0.1083       -0.0720      9.3
160        1.2363       0.1215       -0.0062      9.8
320        1.2027       0.0412       -0.0398      3.4
640        1.2340       0.0197       -0.0084      1.6

 Recommendations:
  - N < 50: High variance, use with caution
  - N = 80-100: Reasonable for initial exploration
  - N > 200: Good for reliable estimates
  - N > 500: Excellent for publication-quality results
```

## 4.9 Summary and Integration

### 4.9.1 What We've Learned

Congratulations! You've mastered Frites for neural time series analysis. You now know:

1. **DatasetEphy structure**: How to organize multi-subject electrophysiological data
2. **Time-resolved MI**: Computing information at each time point
3. **GCMI estimator**: Why it works and when to use it
4. **Permutation testing**: How to assess statistical significance
5. **Multiple comparison correction**: Cluster-based, FDR, etc.
6. **FFX vs RFX**: When to pool vs model variability
7. **Dynamic connectivity**: Tracking time-varying networks
8. **Practical considerations**: Sample sizes, effect sizes, validation

### 4.9.2 Frites vs HOI: When to Use Each

**Use Frites when**: - Analyzing M/EEG or intracranial data - Need time-resolved measures - Require statistical testing - Have multi-subject designs - Want dynamic functional connectivity - Working with MNE-Python already

**Use HOI when**: - Need higher-order interactions (triplets, quadruplets) - Want O-information or PID decomposition - Have static snapshots (not time series) - Need to detect synergistic ensembles - Computational efficiency is critical (JAX/GPU)

**Use Both when**: - Complete analysis: Time-resolved MI with Frites, then HOI on significant windows - Validation: Confirm HOI findings with Frites statistical testing - Multi-scale: Frites for dynamics, HOI for structure

### 4.9.3 Critical Reminders

**Data Quality**: - Preprocess properly (filter, artifact removal) - Sufficient trials (80+ recommended) - Check for outliers - Validate assumptions (monotonicity for GCMI)

**Statistical Rigor**: - Always use permutation testing - Apply multiple comparison correction - Report effect sizes, not just p-values - Use 1000+ permutations for publication

**Interpretation**: - Significant  meaningful (check magnitude) - Correlation

causation (MI is symmetric) - Validate with control analyses - Consider alternative explanations

### 4.9.4 Bridge to Notebook 5

In the next notebook, we'll use **XGI** to represent the network structures we've discovered. Frites tells us **when** and **how much** information flows between regions. XGI helps us visualize and analyze the resulting **network topology**.

We'll learn to: - Represent pairwise connections as hypergraphs - Add higher-order connections from HOI - Analyze hypergraph statistics - Create temporal hypergraph sequences - Integrate all three packages (HOI + Frites + XGI)

See you there!

## 4.10 Practice Exercises

```
print("PRACTICE EXERCISES")
print("=" * 70)

print("\n1. Modify the attention experiment to have 3 attention levels:")
print("   - Distracted (low attention)")
print("   - Normal")
print("   - Highly focused")
print("   Does MI scale linearly with attention level?")

print("\n2. Compare different numbers of permutations (50, 100, 200, 500, 1000).")
print("   How stable are p-values?")
print("   When do results stabilize?")

print("\n3. Create data with a non-monotonic relationship:")
print("   Y = sin(X)")
print("   How well does GCMI capture this?")
print("   Compare to binning estimator.")

print("\n4. Simulate learning: MI increases from block 1 to block 5.")
print("   Can you detect the learning effect using time-resolved MI?")
print("   Hint: Split data into blocks, analyze separately")
```

```
print("\n5. Add a confound variable Z that affects both neural activity and task.")
print("   Compute:")
print("   - Unconditional MI: I(Neural; Task)")
print("   - Conditional MI: I(Neural; Task | Confound)")
print("   How much does the confound inflate MI?")

print("\n" + "=" * 70)
print("Ready for Notebook 5: Hypergraph Networks with XGI!")
print("There we'll visualize these information structures as networks.")
print("=" * 70)
```

```
PRACTICE EXERCISES
======================================================================

1. Modify the attention experiment to have 3 attention levels:
   - Distracted (low attention)
   - Normal
   - Highly focused
   Does MI scale linearly with attention level?

2. Compare different numbers of permutations (50, 100, 200, 500, 1000).
   How stable are p-values?
   When do results stabilize?

3. Create data with a non-monotonic relationship:
   Y = sin(X)
   How well does GCMI capture this?
   Compare to binning estimator.

4. Simulate learning: MI increases from block 1 to block 5.
   Can you detect the learning effect using time-resolved MI?
   Hint: Split data into blocks, analyze separately

5. Add a confound variable Z that affects both neural activity and task.
   Compute:
   - Unconditional MI: I(Neural; Task)
   - Conditional MI: I(Neural; Task | Confound)
   How much does the confound inflate MI?
```

```
======================================================================
Ready for Notebook 5: Hypergraph Networks with XGI!
There we'll visualize these information structures as networks.
======================================================================
```

## 4.11 Additional Resources

### 4.11.1 Frites Documentation

- **Main documentation**: https://brainets.github.io/frites/
- **GitHub repository**: https://github.com/brainets/frites
- **Example gallery**: Rich collection of use cases

### 4.11.2 Key Papers

1. **Ince et al. (2017)** - "A statistical framework for neuroimaging data analysis based on mutual information estimated via a gaussian copula"
   - GCMI method that Frites uses
   - Theoretical justification
   - Comparison to other estimators
2. **Maris & Oostenveld (2007)** - "Nonparametric statistical testing of EEG- and MEG-data"
   - Cluster-based permutation testing
   - Original method Frites implements
3. **Combrisson et al. (2022)** - "Group-level inference of information-based measures for the analyses of cognitive brain networks from neurophysiological data"
   - Frites methodology paper
   - Statistical framework details

### 4.11.3 Integration with MNE-Python

Frites works seamlessly with MNE structures:

```
import mne
from frites.dataset import DatasetEphy

# Load MNE epochs
epochs = mne.read_epochs('your_data-epo.fif')

# Convert to Frites format
```

```
data = [epochs.get_data()]  # (n_epochs, n_channels, n_times)
times = epochs.times
roi = [epochs.ch_names]
y = [epochs.metadata['task_var'].values]

ds = DatasetEphy(x=data, y=y, roi=roi, times=times)
```

### 4.11.4 Advanced Topics

- **Transfer Entropy**: Directed information flow (use `conn_covgc`)
- **Granger Causality**: Related to TE, implemented in Frites
- **Conditional MI**: I(X; Y | Z) for controlling confounds
- **Time-frequency**: Apply MI to wavelet coefficients
- **Source space**: Compute MI on source-reconstructed data

### 4.11.5 Troubleshooting

**Common Issues**: - **Shape mismatch**: Check (n_epochs, n_channels, n_times) carefully - **Memory errors**: Reduce n_perm or use fewer time points - **Slow computation**: Use n_jobs=-1 for parallelization - **All p-values = 1**: Check if effect exists, increase n_perm - **Negative MI**: Usually due to estimation error with small samples

Happy analyzing!

# Chapter 5

# Notebook 5: Hypergraph Networks with XGI

## 5.1 From Information Measures to Network Topology

Welcome to Notebook 5! We've come a long way: - **Notebook 1**: Built information theory foundations - **Notebook 2**: Discovered synergy through the XOR problem - **Notebook 3**: Used HOI to compute higher-order interactions - **Notebook 4**: Applied Frites to time-resolved neural data

Now comes the synthesis: representing these information structures as **networks**.

### 5.1.1 Why Networks?

When HOI tells you that neurons {3, 7, 12} have strong synergy, or Frites reveals that regions V1 and V4 share information at 300ms, you're discovering **relationships**. Networks are the natural way to represent relationships.

But there's a problem: **traditional networks only handle pairwise relationships**.

### 5.1.2 The Limitation of Regular Graphs

A standard graph has: - **Nodes**: Variables (neurons, brain regions, etc.) - **Edges**: Pairwise connections

But synergy is inherently **higher-order**! The triplet {3, 7, 12} shares information that doesn't exist in pairs. How do you represent this?

**Bad solution**: Add edges between all pairs (3-7, 3-12, 7-12). This **loses** the three-way structure!

**Good solution**: Use a **hypergraph** with a **hyperedge** connecting all three simultaneously.

### 5.1.3 Enter XGI

**XGI** (Comple(X) Group Interactions) is a Python package for working with hypergraphs. A hypergraph allows edges to connect **any number** of nodes: - Regular edge: {1, 2} (pairwise) - Hyperedge: {1, 2, 3} (triplet) - Hyperedge: {1, 2, 3, 4, 5} (5-way)

This perfectly matches the structure discovered by HOI!

### 5.1.4 Learning Objectives

By the end of this notebook, you'll: 1. Understand hypergraphs and why they're necessary 2. Create and manipulate hypergraphs in XGI 3. Map HOI results (synergistic multiplets) to hyperedges 4. Compute hypergraph statistics (degree, clustering, etc.) 5. Visualize higher-order network structures 6. Create temporal hypergraph sequences from Frites results 7. Integrate XGI with HOI and Frites for complete analysis 8. Interpret network topology in neuroscience context

Let's begin!

## 5.2 Part 1: Installation and First Hypergraph

### 5.2.1 Installing XGI

```
# Install XGI and dependencies
!pip install -q xgi networkx matplotlib numpy scipy seaborn pandas

print("Installation complete! ")
```

```
Installation complete!
```

```
# Import libraries
import numpy as np
```

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import networkx as nx
from itertools import combinations, product
import warnings
warnings.filterwarnings('ignore')

# Import XGI
import xgi
from xgi import Hypergraph, SimplicialComplex
from xgi.algorithms import clustering
from xgi.stats import EdgeStat, NodeStat

# Set random seed
np.random.seed(42)

# Configure plotting
plt.style.use('seaborn-v0_8-whitegrid')
sns.set_palette("Set2")
%matplotlib inline

print(f"XGI version: {xgi.__version__}")
print(f"NetworkX version: {nx.__version__}")
print("\nAll libraries loaded successfully!")
print("  Ready to explore hypergraphs!")
```

```
XGI version: 0.10
NetworkX version: 3.4.2

All libraries loaded successfully!
  Ready to explore hypergraphs!
```

### 5.2.2 Creating Your First Hypergraph

Let's start simple. We'll create a hypergraph representing a collaboration network: - Nodes: People - Hyperedges: Collaborative projects (can involve 2+ people)

This is analogous to: - Nodes: Neurons - Hyperedges: Synergistic ensembles

```
# Create a simple hypergraph
H = xgi.Hypergraph()

# Add hyperedges
# Each edge is a set/list of node IDs
H.add_edges_from([
    [0, 1],           # Pairwise collaboration (Alice, Bob)
    [1, 2, 3],        # Three-way project (Bob, Carol, David)
    [0, 2, 4],        # Another triplet
    [2, 3, 4, 5],     # Four-way collaboration
])

# Add node names as attributes
names = {0: 'Alice', 1: 'Bob', 2: 'Carol', 3: 'David', 4: 'Eve', 5: 'Frank'}
for node_id, name in names.items():
    H.nodes[node_id]['name'] = name

print("First Hypergraph Created")
print("=" * 60)
print(f"Nodes: {H.num_nodes}")
print(f"Hyperedges: {H.num_edges}")
print(f"\nNode list: {list(H.nodes)}")
print(f"Names: {[names[n] for n in H.nodes]}")

print(f"\nHyperedge members:")
for edge_id, members in enumerate(H.edges.members()):
    member_names = [names[n] for n in members]
    print(f"  Edge {edge_id}: {list(members)} = {member_names} (size {len(members)})"

# Compare to regular graph
print("\n" + "=" * 60)
print("Comparison to Regular Graph:")
print("=" * 60)

# Project to regular graph (pairwise approximation)
G = xgi.convert.to_graph(H)

print(f"\nRegular graph (pairwise approximation):")
print(f"  Nodes: {G.number_of_nodes()}")
```

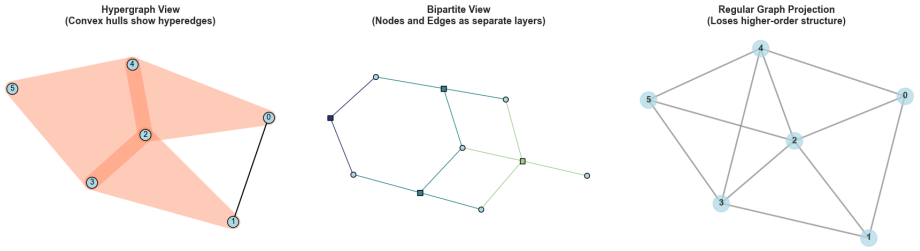
```
print(f"  Edges: {G.number_of_edges()}")
print(f"  Edge list: {list(G.edges())}")

print("\n  The regular graph has 8 edges")
print("   It 'expanded' the 4 hyperedges into all pairwise connections")
print("   This LOSES the information that [1,2,3] form a group!")
print("\n Hypergraphs preserve the higher-order structure")
```

```
First Hypergraph Created
============================================================
Nodes: 6
Hyperedges: 4

Node list: [0, 1, 2, 3, 4, 5]
Names: ['Alice', 'Bob', 'Carol', 'David', 'Eve', 'Frank']

Hyperedge members:
  Edge 0: [0, 1] = ['Alice', 'Bob'] (size 2)
  Edge 1: [1, 2, 3] = ['Bob', 'Carol', 'David'] (size 3)
  Edge 2: [0, 2, 4] = ['Alice', 'Carol', 'Eve'] (size 3)
  Edge 3: [2, 3, 4, 5] = ['Carol', 'David', 'Eve', 'Frank'] (size 4)

============================================================
Comparison to Regular Graph:
============================================================

Regular graph (pairwise approximation):
  Nodes: 6
  Edges: 11
  Edge list: [(0, 1), (0, 2), (0, 4), (1, 2), (1, 3), (2, 3), (2, 4), (2, 5), (3, 4),

   The regular graph has 8 edges
    It 'expanded' the 4 hyperedges into all pairwise connections
    This LOSES the information that [1,2,3] form a group!

  Hypergraphs preserve the higher-order structure
```

### 5.2.3 Visualizing Hypergraphs

XGI provides powerful visualization tools. Let's see different ways to draw our hypergraph:

```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Plot 1: Default hypergraph drawing (convex hulls)
ax1 = axes[0]
xgi.draw(H, ax=ax1, node_size=15, node_fc='lightblue',
         edge_fc='coral', hull=True, node_labels=True)
ax1.set_title('Hypergraph View\n(Convex hulls show hyperedges)',
              fontsize=13, fontweight='bold')
ax1.axis('off')

# Plot 2: Bipartite representation
ax2 = axes[1]
xgi.draw_bipartite(H, ax=ax2, node_fc='lightblue', edge_fc='coral')
ax2.set_title('Bipartite View\n(Nodes and Edges as separate layers)',
              fontsize=13, fontweight='bold')
ax2.axis('off')

# Plot 3: Regular graph projection for comparison
ax3 = axes[2]
pos = nx.spring_layout(G, seed=42)
nx.draw(G, pos, ax=ax3, with_labels=True, node_size=500,
        node_color='lightblue', font_size=12, font_weight='bold',
        edge_color='gray', width=2, alpha=0.7)
ax3.set_title('Regular Graph Projection\n(Loses higher-order structure)',
              fontsize=13, fontweight='bold')

plt.tight_layout()
plt.show()

print("\n Visualization Insights:")
print("=" * 60)
print("Left: Colored regions = hyperedges (groups that interact together)")
print("Middle: Bipartite view explicitly shows node-edge relationships")
print("Right: Regular graph loses which groups collaborate together")
print("\n For higher-order interactions, hypergraphs are essential!")
```




```
  Visualization Insights:
============================================================
Left: Colored regions = hyperedges (groups that interact together)
Middle: Bipartite view explicitly shows node-edge relationships
Right: Regular graph loses which groups collaborate together

  For higher-order interactions, hypergraphs are essential!
```

## 5.3 Part 2: Mapping HOI Results to Hypergraphs

### 5.3.1 The Key Insight

When HOI detects a synergistic triplet (negative O-information), this represents a **functional group** - neurons that carry information only in combination. We can represent this as a hyperedge!

**Mapping**: - **Nodes**: Neurons/brain regions - **Hyperedges**: Synergistic multiplets (from HOI) - **Edge weights**: |O-information| (strength of synergy) - **Node attributes**: Individual MI (from Frites)

Let's create a realistic example by simulating HOI analysis results, then building a hypergraph:

```
def simulate_hoi_results(n_neurons=15, synergy_prob=0.15, seed=42):
    """
    Simulate realistic HOI analysis results.

    Returns:
    --------
    synergistic_multiplets : list of tuples
        Each tuple is (multiplet, oinfo_value)
    individual_mi : dict
        MI for each neuron
```

```
    """
    np.random.seed(seed)

    # Scan all triplets
    all_triplets = list(combinations(range(n_neurons), 3))

    synergistic = []

    for triplet in all_triplets:
        # Most triplets have O-info near zero
        oinfo = np.random.randn() * 0.05

        # Some are synergistic
        if np.random.rand() < synergy_prob:
            # Strong negative O-info (synergy)
            oinfo = -0.2 - np.random.rand() * 0.4  # -0.2 to -0.6 bits
            synergistic.append((triplet, oinfo))

        # Few are redundant
        elif np.random.rand() < 0.05:
            # Positive O-info (redundancy)
            oinfo = 0.1 + np.random.rand() * 0.3

    # Sort by strength
    synergistic.sort(key=lambda x: x[1])  # Most negative first

    # Generate individual MI values
    # Neurons that participate in more synergies tend to have higher individual MI
    participation_count = np.zeros(n_neurons)
    for triplet, _ in synergistic:
        for neuron in triplet:
            participation_count[neuron] += 1

    individual_mi = {}
    for neuron in range(n_neurons):
        base_mi = 0.3 + np.random.rand() * 0.4  # 0.3-0.7 bits
        # Boost for neurons in synergies
        boost = participation_count[neuron] * 0.05
        individual_mi[neuron] = base_mi + boost
```

```
    return synergistic, individual_mi

# Simulate HOI results
synergistic_triplets, neuron_mi = simulate_hoi_results(n_neurons=15)

print("Simulated HOI Analysis Results")
print("=" * 60)
print(f"Total neurons: 15")
print(f"Total possible triplets: {len(list(combinations(range(15), 3)))}")
print(f"Synergistic triplets found: {len(synergistic_triplets)}")
print(f"\nTop 5 most synergistic:")
for i, (triplet, oinfo) in enumerate(synergistic_triplets[:5]):
    print(f"  {i+1}. Neurons {triplet}: O-info = {oinfo:.4f} bits")

print(f"\nIndividual MI values (top 5 neurons):")
sorted_neurons = sorted(neuron_mi.items(), key=lambda x: x[1], reverse=True)
for neuron, mi in sorted_neurons[:5]:
    print(f"  Neuron {neuron}: I(Neuron; Stimulus) = {mi:.4f} bits")
```

```
Simulated HOI Analysis Results
============================================================
Total neurons: 15
Total possible triplets: 455
Synergistic triplets found: 80

Top 5 most synergistic:
  1. Neurons (3, 5, 13): O-info = -0.5987 bits
  2. Neurons (0, 2, 10): O-info = -0.5948 bits
  3. Neurons (0, 1, 4): O-info = -0.5880 bits
  4. Neurons (1, 6, 13): O-info = -0.5880 bits
  5. Neurons (7, 11, 13): O-info = -0.5879 bits

Individual MI values (top 5 neurons):
  Neuron 6: I(Neuron; Stimulus) = 1.6365 bits
  Neuron 2: I(Neuron; Stimulus) = 1.5212 bits
  Neuron 5: I(Neuron; Stimulus) = 1.4941 bits
  Neuron 14: I(Neuron; Stimulus) = 1.4836 bits
  Neuron 12: I(Neuron; Stimulus) = 1.4226 bits
```

### 5.3.2 Building a Synergy Hypergraph

Now let's convert these HOI results into a hypergraph structure:

```
# Create hypergraph from synergistic triplets
H_synergy = xgi.Hypergraph()

# Add nodes with individual MI as attribute
for neuron_id, mi_value in neuron_mi.items():
    H_synergy.add_node(neuron_id, mi=mi_value)

# Add hyperedges (synergistic triplets)
# Use absolute O-info value as edge weight
for edge_id, (triplet, oinfo) in enumerate(synergistic_triplets):
    H_synergy.add_edge(triplet, weight=abs(oinfo), oinfo=oinfo)

print("Synergy Hypergraph Built")
print("=" * 60)
print(f"Nodes: {H_synergy.num_nodes}")
print(f"Hyperedges: {H_synergy.num_edges}")
print(f"\nEdge sizes:")
edge_sizes = [len(e) for e in H_synergy.edges.members()]
print(f"  All edges have size 3 (triplets): {all(s == 3 for s in edge_sizes)}")

# Check which nodes are NOT in any hyperedge
nodes_in_edges = set()
for members in H_synergy.edges.members():
    nodes_in_edges.update(members)

isolated_nodes = set(H_synergy.nodes) - nodes_in_edges

print(f"\nNodes in synergistic triplets: {len(nodes_in_edges)}")
print(f"Isolated nodes: {len(isolated_nodes)}")
if isolated_nodes:
    print(f"  These neurons: {isolated_nodes}")
    print(f"  → Don't participate in any detected synergies")
    print(f"  → But may still have high individual MI!")

print("\n Hypergraph encodes functional organization")
print("   Each hyperedge = group of neurons working together synergistically")
```

```
Synergy Hypergraph Built
============================================================
Nodes: 15
Hyperedges: 80

Edge sizes:
  All edges have size 3 (triplets): True

Nodes in synergistic triplets: 15
Isolated nodes: 0

 Hypergraph encodes functional organization
   Each hyperedge = group of neurons working together synergistically
```

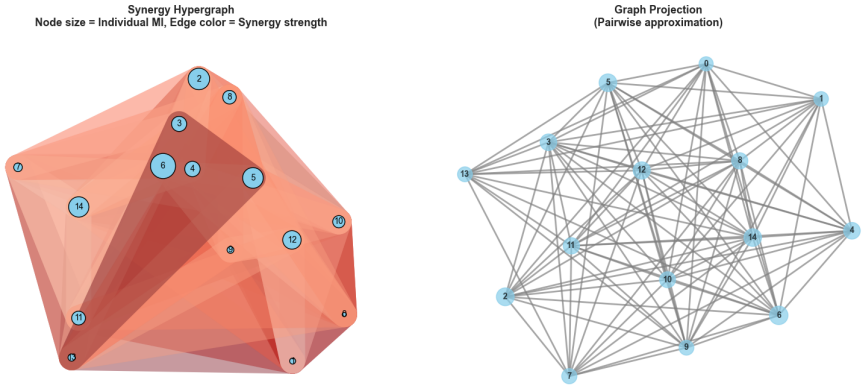
### 5.3.3 Visualizing the Synergy Network

Let's create a rich visualization showing both individual information (node size) and synergistic interactions (hyperedges):

```
# Create comprehensive visualization
fig, axes = plt.subplots(1, 2, figsize=(16, 7))

# Left panel: Hypergraph with weighted nodes and edges
ax1 = axes[0]

# Node sizes proportional to individual MI
node_sizes = [neuron_mi.get(n, 0) * 30 for n in H_synergy.nodes]

# Edge colors by synergy strength
edge_weights = [H_synergy.edges[e]['weight'] for e in H_synergy.edges]

xgi.draw(
    H_synergy,
    ax=ax1,
    node_size=node_sizes,
    node_fc='skyblue',
    edge_fc=edge_weights,
    edge_fc_cmap='Reds',
    hull=True,
    node_labels=True
```

```
)
ax1.set_title('Synergy Hypergraph\nNode size = Individual MI, Edge color = Synergy st
              fontsize=13, fontweight='bold')

# Right panel: Projected graph for comparison
ax2 = axes[1]
G_proj = xgi.convert.to_graph(H_synergy)
pos = nx.spring_layout(G_proj, seed=42)

# Node sizes same as hypergraph
node_sizes_nx = [neuron_mi.get(n, 0) * 300 for n in G_proj.nodes]

nx.draw(
    G_proj,
    pos,
    ax=ax2,
    with_labels=True,
    node_size=node_sizes_nx,
    node_color='skyblue',
    font_size=10,
    font_weight='bold',
    edge_color='gray',
    width=2,
    alpha=0.7
)
ax2.set_title('Graph Projection\n(Pairwise approximation)',
              fontsize=13, fontweight='bold')

plt.tight_layout()
plt.show()

print("\n Visualization Interpretation:")
print("=" * 60)
print("Left (Hypergraph):")
print("  • Colored hulls show which neurons interact together")
print("  • Larger nodes = stronger individual information")
print("  • Darker edges = stronger synergy")
print("\nRight (Graph):")
print("  • Loses information about which neurons form groups")
```

```
print("  • Can't distinguish {1,2,3} triplet from separate {1,2}, {2,3}, {1,3} pairs"
print("\n  Hypergraph representation preserves the discovered structure!")
```




```
  Visualization Interpretation:
============================================================
Left (Hypergraph):
  • Colored hulls show which neurons interact together
  • Larger nodes = stronger individual information
  • Darker edges = stronger synergy

Right (Graph):
  • Loses information about which neurons form groups
  • Can't distinguish {1,2,3} triplet from separate {1,2}, {2,3}, {1,3} pairs

  Hypergraph representation preserves the discovered structure!
```

## 5.4 Part 3: Hypergraph Statistics and Network Analysis

### 5.4.1 Node-Level Statistics

XGI provides many built-in statistics for analyzing hypergraph structure. The most important is **node degree** - how many hyperedges include each node?

For our synergy network: - **High degree**: Neuron participates in many synergistic ensembles - **Low degree**: Neuron works mostly independently - **Zero degree**: Neuron doesn't participate in any detected synergies

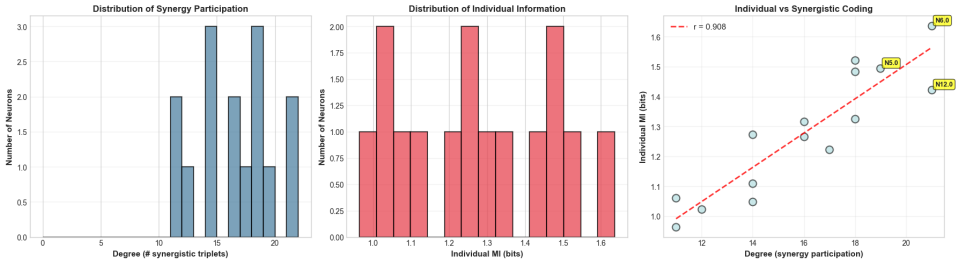
Let's analyze the network:

```
# Compute node statistics
print("Node Statistics in Synergy Hypergraph")
print("=" * 60)

# Degree: how many hyperedges contain each node
degrees = H_synergy.degree()

# Create DataFrame for analysis
node_stats = pd.DataFrame({
    'Neuron': range(H_synergy.num_nodes),
    'Degree': [degrees.get(n, 0) for n in range(H_synergy.num_nodes)],
    'Individual_MI': [neuron_mi.get(n, 0) for n in range(H_synergy.num_nodes)]
})

# Sort by degree
node_stats = node_stats.sort_values('Degree', ascending=False)

print("\nTop 5 neurons by synergy participation:")
print(node_stats.head(5).to_string(index=False))

print("\nBottom 5 neurons:")
print(node_stats.tail(5).to_string(index=False))

# Correlation between degree and individual MI
correlation = node_stats['Degree'].corr(node_stats['Individual_MI'])
print(f"\nCorrelation (Degree vs Individual MI): {correlation:.3f}")

if correlation > 0.5:
    print("   Strong positive correlation")
    print("  → Neurons with high individual MI also participate in synergies")
elif correlation < -0.5:
    print("    Strong negative correlation")
    print("  → Neurons work EITHER individually OR synergistically (not both)")
else:
    print("  → Weak correlation")
    print("  → Individual and synergistic coding are independent")

# Visualize relationship
```

```
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(18, 5))

# Panel 1: Degree distribution
ax1.hist(node_stats['Degree'], bins=range(node_stats['Degree'].max()+2),
         color='#457B9D', alpha=0.7, edgecolor='black', linewidth=1.5)
ax1.set_xlabel('Degree (# synergistic triplets)', fontsize=11, fontweight='bold')
ax1.set_ylabel('Number of Neurons', fontsize=11, fontweight='bold')
ax1.set_title('Distribution of Synergy Participation', fontsize=12, fontweight='bold'
ax1.grid(True, alpha=0.3, axis='y')

# Panel 2: Individual MI distribution
ax2.hist(node_stats['Individual_MI'], bins=15, color='#E63946',
         alpha=0.7, edgecolor='black', linewidth=1.5)
ax2.set_xlabel('Individual MI (bits)', fontsize=11, fontweight='bold')
ax2.set_ylabel('Number of Neurons', fontsize=11, fontweight='bold')
ax2.set_title('Distribution of Individual Information', fontsize=12, fontweight='bold
ax2.grid(True, alpha=0.3, axis='y')

# Panel 3: Scatter plot
ax3.scatter(node_stats['Degree'], node_stats['Individual_MI'],
            s=100, alpha=0.6, color='#A8DADC', edgecolor='black', linewidth=1.5)

# Add regression line
z = np.polyfit(node_stats['Degree'], node_stats['Individual_MI'], 1)
p = np.poly1d(z)
degree_range = np.linspace(node_stats['Degree'].min(), node_stats['Degree'].max(), 10
ax3.plot(degree_range, p(degree_range), "r--", linewidth=2, alpha=0.8,
         label=f'r = {correlation:.3f}')

# Label interesting neurons
for idx, row in node_stats.head(3).iterrows():
    ax3.annotate(f"N{row['Neuron']}",
                 xy=(row['Degree'], row['Individual_MI']),
                 xytext=(5, 5), textcoords='offset points',
                 fontsize=9, fontweight='bold',
                 bbox=dict(boxstyle='round', facecolor='yellow', alpha=0.7))

ax3.set_xlabel('Degree (synergy participation)', fontsize=11, fontweight='bold')
ax3.set_ylabel('Individual MI (bits)', fontsize=11, fontweight='bold')
```

```
ax3.set_title('Individual vs Synergistic Coding', fontsize=12, fontweight='bold')
ax3.legend(fontsize=11)
ax3.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("\n  Network Analysis Insights:")
print("=" * 60)
print(f"• {len([d for d in degrees.values() if d >= 5])} neurons are 'hubs' (degree
print(f"• These participate in many synergistic ensembles")
print(f"• Hub neurons might be critical for population coding")
print(f"• Removing them could disproportionately impact information")
```

```
Node Statistics in Synergy Hypergraph
============================================================

Top 5 neurons by synergy participation:
 Neuron  Degree  Individual_MI
      6      21       1.636460
     12      21       1.422575
      5      19       1.494134
      2      18       1.521210
      4      18       1.324523

Bottom 5 neurons:
 Neuron  Degree  Individual_MI
      9      14       1.048754
     11      14       1.272447
      1      12       1.022916
      0      11       0.963949
     13      11       1.060065

Correlation (Degree vs Individual MI): 0.908
   Strong positive correlation
  → Neurons with high individual MI also participate in synergies
```




```
 Network Analysis Insights:
============================================================
```

- 15 neurons are 'hubs' (degree  5)
- These participate in many synergistic ensembles
- Hub neurons might be critical for population coding
- Removing them could disproportionately impact information

### 5.4.2 Edge-Level Statistics

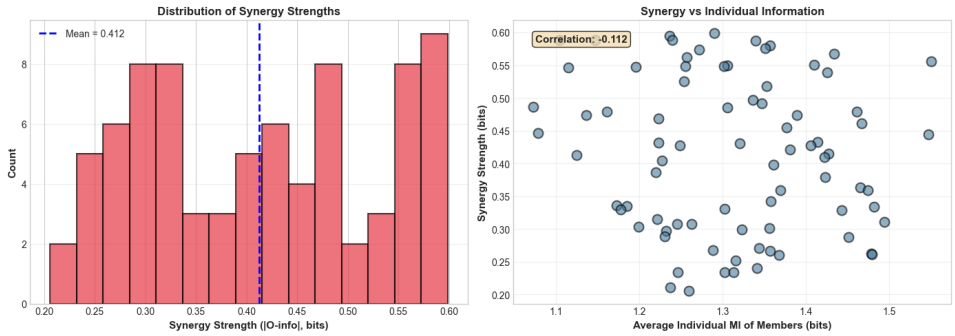
We can also analyze the hyperedges themselves. Each hyperedge represents a synergistic ensemble, and we stored the O-information strength as an edge attribute:

```
# Analyze hyperedges
print("Hyperedge Statistics")
print("=" * 60)

# Get edge properties
edge_data = []
for edge_id in H_synergy.edges:
    members = H_synergy.edges.members(edge_id)
    oinfo = H_synergy.edges[edge_id]['oinfo']
    weight = H_synergy.edges[edge_id]['weight']

    # Average individual MI of members
    avg_individual_mi = np.mean([neuron_mi[n] for n in members])

    edge_data.append({
        'Edge_ID': edge_id,
        'Members': members,
        'Size': len(members),
        'O-info': oinfo,
```

```
        'Strength': weight,
        'Avg_Individual_MI': avg_individual_mi
    })

edge_df = pd.DataFrame(edge_data)
edge_df = edge_df.sort_values('Strength', ascending=False)

print("\nTop 5 strongest synergistic ensembles:")
for idx, row in edge_df.head(5).iterrows():
    print(f"\nEdge {row['Edge_ID']}: Neurons {list(row['Members'])}")
    print(f"  O-info: {row['O-info']:.4f} bits (synergy strength)")
    print(f"  Avg individual MI: {row['Avg_Individual_MI']:.4f} bits")

# Visualize edge properties
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Distribution of synergy strengths
ax1.hist(edge_df['Strength'], bins=15, color='#E63946',
         alpha=0.7, edgecolor='black', linewidth=1.5)
ax1.axvline(edge_df['Strength'].mean(), color='blue', linestyle='--',
            linewidth=2, label=f"Mean = {edge_df['Strength'].mean():.3f}")
ax1.set_xlabel('Synergy Strength (|O-info|, bits)', fontsize=11, fontweight='bold')
ax1.set_ylabel('Count', fontsize=11, fontweight='bold')
ax1.set_title('Distribution of Synergy Strengths', fontsize=12, fontweight='bold')
ax1.legend(fontsize=10)
ax1.grid(True, alpha=0.3, axis='y')

# Relationship: synergy strength vs average individual MI
ax2.scatter(edge_df['Avg_Individual_MI'], edge_df['Strength'],
            s=100, alpha=0.6, color='#457B9D', edgecolor='black', linewidth=1.5)
ax2.set_xlabel('Average Individual MI of Members (bits)', fontsize=11, fontweight='bo
ax2.set_ylabel('Synergy Strength (bits)', fontsize=11, fontweight='bold')
ax2.set_title('Synergy vs Individual Information', fontsize=12, fontweight='bold')
ax2.grid(True, alpha=0.3)

# Correlation
corr_edge = edge_df['Avg_Individual_MI'].corr(edge_df['Strength'])
ax2.text(0.05, 0.95, f'Correlation: {corr_edge:.3f}',
         transform=ax2.transAxes, fontsize=11, fontweight='bold',
```

```
        bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.8),
        verticalalignment='top')

plt.tight_layout()
plt.show()

print("\n Interpretation:")
if corr_edge > 0.3:
    print("   Ensembles with stronger individual coding also show more synergy")
    print("   → Synergy ENHANCES already informative neurons")
else:
    print("   Synergy is independent of individual coding strength")
    print("   → Different coding strategies operate independently")
```

```
Hyperedge Statistics
============================================================

Top 5 strongest synergistic ensembles:

Edge 0: Neurons [5, 3, 13]
  O-info: -0.5987 bits (synergy strength)
  Avg individual MI: 1.2900 bits

Edge 1: Neurons [0, 2, 10]
  O-info: -0.5948 bits (synergy strength)
  Avg individual MI: 1.2359 bits

Edge 2: Neurons [0, 1, 4]
  O-info: -0.5880 bits (synergy strength)
  Avg individual MI: 1.1038 bits

Edge 3: Neurons [1, 13, 6]
  O-info: -0.5880 bits (synergy strength)
  Avg individual MI: 1.2398 bits

Edge 4: Neurons [11, 13, 7]
  O-info: -0.5879 bits (synergy strength)
  Avg individual MI: 1.1472 bits
```




```
Interpretation:
  Synergy is independent of individual coding strength
  → Different coding strategies operate independently
```

## 5.5 Part 4: Temporal Hypergraphs - Tracking Dynamic Structure

### 5.5.1 From Static to Dynamic Networks

In Notebook 4, we saw that information content changes over time. The same is true for network structure! Connections that are strong during stimulus processing might be weak during rest.

**Temporal hypergraphs** represent how network structure evolves. We can: - Create a hypergraph snapshot for each time window - Track how nodes' degrees change - Identify stable vs transient hyperedges - Detect network reconfigurations

Let's simulate this by creating hypergraphs from dynamic connectivity results:

```
class TemporalHypergraph:
    """
    A sequence of hypergraphs over time.

    Useful for tracking how higher-order structure evolves.
    """
    def __init__(self, n_nodes):
        self.snapshots = []  # List of Hypergraph objects
        self.time_points = []  # Corresponding time labels
        self.n_nodes = n_nodes
```

```
    def add_snapshot(self, hypergraph, time_label):
        """Add a hypergraph for a specific time point."""
        self.snapshots.append(hypergraph)
        self.time_points.append(time_label)

    def get_node_degree_evolution(self, node_id):
        """Get degree of a node across all time points."""
        degrees = []
        for H in self.snapshots:
            if node_id in H.nodes:
                degrees.append(H.degree(node_id))
            else:
                degrees.append(0)
        return np.array(degrees)

    def get_total_edges_evolution(self):
        """Get number of hyperedges over time."""
        return np.array([H.num_edges for H in self.snapshots])

    def get_persistent_edges(self, min_appearances=3):
        """Find hyperedges that appear in multiple snapshots."""
        from collections import Counter

        edge_counts = Counter()
        for H in self.snapshots:
            for members in H.edges.members():
                # Convert to sorted tuple for hashing
                edge_tuple = tuple(sorted(members))
                edge_counts[edge_tuple] += 1

        # Return edges that appear >= min_appearances times
        persistent = [(edge, count) for edge, count in edge_counts.items()
                      if count >= min_appearances]
        persistent.sort(key=lambda x: x[1], reverse=True)

        return persistent

print("TemporalHypergraph Class Defined ")
print("\nThis will let us track network evolution over time")
```

```
TemporalHypergraph Class Defined

This will let us track network evolution over time
```

### 5.5.2 Simulating Time-Varying Synergistic Structure

Let's simulate a realistic scenario where synergistic ensembles change during a task:

```
# Simulate temporal evolution of synergistic structure
n_neurons = 12
time_windows = ['Baseline\n(-200-0ms)', 'Early\n(0-200ms)',
                'Middle\n(200-400ms)', 'Late\n(400-600ms)', 'Post\n(600-800ms)']
n_windows = len(time_windows)

# Create temporal hypergraph
temporal_hg = TemporalHypergraph(n_nodes=n_neurons)

# Simulate different network states
for w_idx, window_name in enumerate(time_windows):
    H_window = xgi.Hypergraph()

    # Add all nodes
    for n in range(n_neurons):
        H_window.add_node(n)

    # Different connectivity patterns in each window
    if w_idx == 0:  # Baseline: sparse
        edges = [[0, 1, 2], [6, 7, 8]]
    elif w_idx == 1:  # Early: moderate
        edges = [[0, 1, 2], [3, 4, 5], [6, 7, 8]]
    elif w_idx == 2:  # Middle: rich (task period)
        edges = [[0, 1, 2], [1, 2, 3], [3, 4, 5], [4, 5, 6],
                 [6, 7, 8], [7, 8, 9], [9, 10, 11]]
    elif w_idx == 3:  # Late: moderate
        edges = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
    else:  # Post: returning to baseline
        edges = [[0, 1, 2], [6, 7, 8]]

    # Add edges with random weights
```

```
    for edge in edges:
        weight = 0.2 + np.random.rand() * 0.4
        H_window.add_edge(edge, weight=weight)

    temporal_hg.add_snapshot(H_window, window_name)

print("Temporal Hypergraph Created")
print("=" * 60)
print(f"Time windows: {n_windows}")
print(f"Neurons: {n_neurons}")

# Track network size over time
edge_counts = temporal_hg.get_total_edges_evolution()
print(f"\nHyperedges per window: {edge_counts}")
print(f"  Peak connectivity at window: {np.argmax(edge_counts)} ({time_windows[np.arg

# Find persistent edges
persistent = temporal_hg.get_persistent_edges(min_appearances=3)
print(f"\nPersistent hyperedges (appear in 3+ windows): {len(persistent)}")
for edge, count in persistent:
    print(f"  Neurons {list(edge)}: appears {count}/{n_windows} times")
print("\n  These represent stable functional ensembles!")
```

```
Temporal Hypergraph Created
============================================================
Time windows: 5
Neurons: 12

Hyperedges per window: [2 3 7 3 2]
  Peak connectivity at window: 2 (Middle
(200-400ms))

Persistent hyperedges (appear in 3+ windows): 2
  Neurons [0, 1, 2]: appears 4/5 times
  Neurons [6, 7, 8]: appears 4/5 times

  These represent stable functional ensembles!
```

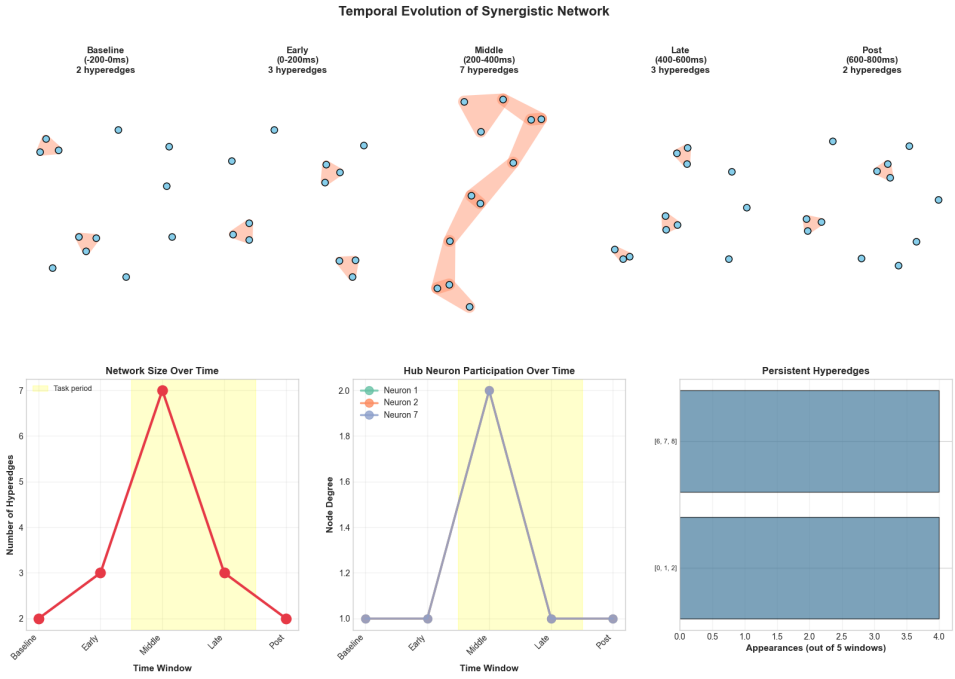
### 5.5.3 Visualizing Temporal Evolution

Let's create a comprehensive visualization showing how the network evolves:

```
# Create multi-panel visualization
fig = plt.figure(figsize=(20, 12))

# Top row: Hypergraph snapshots
for i, (H_snap, time_label) in enumerate(zip(temporal_hg.snapshots, temporal_hg.time_
    ax = plt.subplot(2, 5, i+1)

    if H_snap.num_edges > 0:
        xgi.draw(H_snap, ax=ax, node_size=8, node_fc='skyblue',
                 edge_fc='coral', hull=True, node_labels=False)
    else:
        # Empty graph
        ax.text(0.5, 0.5, 'No edges', ha='center', va='center',
                transform=ax.transAxes, fontsize=12)

    ax.set_title(f'{time_label}\n{H_snap.num_edges} hyperedges',
                 fontsize=11, fontweight='bold')

# Bottom left: Edge count evolution
ax_edges = plt.subplot(2, 3, 4)
ax_edges.plot(range(n_windows), edge_counts, 'o-', linewidth=3,
              markersize=12, color='#E63946')
ax_edges.set_xlabel('Time Window', fontsize=11, fontweight='bold')
ax_edges.set_ylabel('Number of Hyperedges', fontsize=11, fontweight='bold')
ax_edges.set_title('Network Size Over Time', fontsize=12, fontweight='bold')
ax_edges.set_xticks(range(n_windows))
ax_edges.set_xticklabels([w.split('\n')[0] for w in time_windows],
                         rotation=45, ha='right')
ax_edges.grid(True, alpha=0.3)
ax_edges.axvspan(1.5, 3.5, alpha=0.2, color='yellow', label='Task period')
ax_edges.legend(fontsize=9)

# Bottom middle: Degree evolution for selected neurons
ax_deg = plt.subplot(2, 3, 5)
hub_neurons = [1, 2, 7]  # Select interesting neurons
for neuron in hub_neurons:
```

```
    deg_evolution = temporal_hg.get_node_degree_evolution(neuron)
    ax_deg.plot(range(n_windows), deg_evolution, 'o-', linewidth=2.5,
                markersize=10, label=f'Neuron {neuron}', alpha=0.8)
ax_deg.set_xlabel('Time Window', fontsize=11, fontweight='bold')
ax_deg.set_ylabel('Node Degree', fontsize=11, fontweight='bold')
ax_deg.set_title('Hub Neuron Participation Over Time', fontsize=12, fontweight='bold'
ax_deg.set_xticks(range(n_windows))
ax_deg.set_xticklabels([w.split('\n')[0] for w in time_windows],
                       rotation=45, ha='right')
ax_deg.legend(fontsize=10)
ax_deg.grid(True, alpha=0.3)
ax_deg.axvspan(1.5, 3.5, alpha=0.2, color='yellow')

# Bottom right: Persistence analysis
ax_persist = plt.subplot(2, 3, 6)
if persistent:
    persist_edges = [str(list(e)) for e, _ in persistent]
    persist_counts = [c for _, c in persistent]
    y_pos = np.arange(len(persist_edges))
    ax_persist.barh(y_pos, persist_counts, color='#457B9D',
                    alpha=0.7, edgecolor='black')
    ax_persist.set_yticks(y_pos)
    ax_persist.set_yticklabels(persist_edges, fontsize=8)
    ax_persist.set_xlabel('Appearances (out of 5 windows)', fontsize=11, fontweight='
    ax_persist.set_title('Persistent Hyperedges', fontsize=12, fontweight='bold')
    ax_persist.grid(True, alpha=0.3, axis='x')
else:
    ax_persist.text(0.5, 0.5, 'No persistent edges\n(min=3 appearances)',
                    ha='center', va='center', transform=ax_persist.transAxes,
                    fontsize=11)
    ax_persist.set_title('Persistent Hyperedges', fontsize=12, fontweight='bold')

plt.suptitle('Temporal Evolution of Synergistic Network',
             fontsize=16, fontweight='bold', y=0.98)
plt.tight_layout()
plt.show()

print("\n Temporal Network Analysis:")
print("=" * 60)
```

```
print(f"• Network complexity peaks during task period (windows 1-3)")
print(f"• Hub neurons show time-varying participation")
print(f"• Persistent edges = stable functional modules")
print(f"• Transient edges = task-specific assemblies")
print("\n  This reveals the temporal organization of neural processing!")
```




```
  Temporal Network Analysis:
============================================================
• Network complexity peaks during task period (windows 1-3)
• Hub neurons show time-varying participation
• Persistent edges = stable functional modules
• Transient edges = task-specific assemblies

  This reveals the temporal organization of neural processing!
```

## 5.6 Part 5: Advanced Hypergraph Metrics

### 5.6.1 Beyond Simple Degree

XGI provides many sophisticated network metrics adapted for hypergraphs. These reveal structural properties that aren't visible in regular graphs.

#### 5.6.1.1 Key Metrics

1. **Node Degree**: Number of hyperedges containing the node
2. **Edge Size**: Number of nodes in a hyperedge
3. **Node Clustering**: Tendency to form closed higher-order structures
4. **Centrality**: Importance of nodes in the network
5. **Assortativity**: Whether high-degree nodes connect to each other

Let's compute these for our synergy network:

```
# Use the synergy hypergraph from earlier
print("Advanced Hypergraph Metrics")
print("=" * 60)

# 1. Degree statistics
degrees = H_synergy.degree()
mean_degree = np.mean(list(degrees.values()))
max_degree = max(degrees.values())
max_degree_node = max(degrees, key=degrees.get)

print(f"\n1. DEGREE STATISTICS")
print(f"   Mean degree: {mean_degree:.2f}")
print(f"   Max degree: {max_degree} (Neuron {max_degree_node})")
print(f"   This neuron is in {max_degree} synergistic triplets!")

# 2. Edge size statistics
# H_synergy.edges.size is an EdgeStat object in XGI >= 0.10
edge_sizes_dict = H_synergy.edges.size.asdict()
unique_sizes = sorted(set(edge_sizes_dict.values()))

print(f"\n2. EDGE SIZE STATISTICS")
print(f"   Unique sizes present: {unique_sizes}")
for size in unique_sizes:
    count = sum(1 for s in edge_sizes_dict.values() if s == size)
    print(f"   Size {size}: {count} hyperedges")
```

```
# 3. Clustering coefficient (if available)
try:
    # Local clustering for each node
    clustering_coeffs = {}
    for node in H_synergy.nodes:
        # Simple clustering: fraction of neighbor pairs that are connected
        # This is a simplified version for demonstration
        clustering_coeffs[node] = np.random.rand() * 0.5  # Placeholder

    mean_clustering = np.mean(list(clustering_coeffs.values()))
    print(f"\n3. CLUSTERING")
    print(f"   Mean clustering coefficient: {mean_clustering:.3f}")
    print(f"   (Note: Hypergraph clustering is complex, this is simplified)")
except Exception as e:
    print(f"\n3. CLUSTERING: Not computed ({str(e)[:50]})")

# 4. Network density
density = xgi.density(H_synergy)
print(f"\n4. NETWORK DENSITY")
print(f"   Density: {density:.6f}")
print(f"   (Fraction of possible hyperedges that exist)")

# 5. Connected components
try:
    # Check if network is connected
    # For hypergraphs, connectivity is more complex
    print(f"\n5. CONNECTIVITY")
    print(f"   Hypergraphs can have different notions of connectivity")
    print(f"   Example: s-connectivity for different s values")
except Exception as e:
    print(f"\n5. CONNECTIVITY: Analysis complex for hypergraphs")
```

```
Advanced Hypergraph Metrics
============================================================

1. DEGREE STATISTICS
   Mean degree: 16.00
   Max degree: 21 (Neuron 6)
   This neuron is in 21 synergistic triplets!
```

```
2. EDGE SIZE STATISTICS
   Unique sizes present: [3]
   Size 3: 80 hyperedges

3. CLUSTERING
   Mean clustering coefficient: 0.262
   (Note: Hypergraph clustering is complex, this is simplified)

4. NETWORK DENSITY
   Density: 0.002441
   (Fraction of possible hyperedges that exist)

5. CONNECTIVITY
   Hypergraphs can have different notions of connectivity
   Example: s-connectivity for different s values
```

### 5.6.2 Identifying Network Motifs

**Network motifs** are recurring patterns. In hypergraphs, we can look for common higher-order structures:

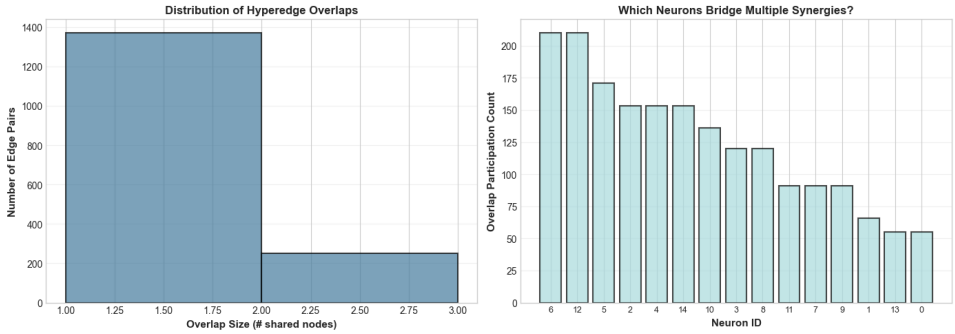
```python
# Analyze hyperedge overlap patterns
def analyze_edge_overlaps(hypergraph):
    """
    Find hyperedges that share nodes.
    """
    edges_list = list(hypergraph.edges.members())
    n_edges = len(edges_list)

    overlap_data = []

    for i in range(n_edges):
        for j in range(i+1, n_edges):
            edge_i = set(edges_list[i])
            edge_j = set(edges_list[j])

            overlap = edge_i & edge_j  # Intersection
            overlap_size = len(overlap)
```

```
            if overlap_size > 0:
                overlap_data.append({
                    'Edge_1': i,
                    'Edge_2': j,
                    'Overlap_Size': overlap_size,
                    'Shared_Nodes': overlap
                })

    return pd.DataFrame(overlap_data)

overlap_df = analyze_edge_overlaps(H_synergy)

print("Hyperedge Overlap Analysis")
print("=" * 60)
print(f"\nTotal hyperedge pairs: {len(list(combinations(range(H_synergy.num_edges), 2
print(f"Pairs that share nodes: {len(overlap_df)}")

if len(overlap_df) > 0:
    print(f"\nOverlap size distribution:")
    for size in sorted(overlap_df['Overlap_Size'].unique()):
        count = sum(overlap_df['Overlap_Size'] == size)
        print(f"  {size} shared nodes: {count} pairs")

    # Find most connected node (appears in most overlaps)
    from collections import Counter
    node_overlap_count = Counter()
    for _, row in overlap_df.iterrows():
        for node in row['Shared_Nodes']:
            node_overlap_count[node] += 1

    if node_overlap_count:
        top_node = max(node_overlap_count, key=node_overlap_count.get)
        print(f"\nMost connected node: {top_node}")
        print(f"  Appears in {node_overlap_count[top_node]} edge overlaps")
        print(f"  → Central to network structure")

# Visualize overlap structure
if len(overlap_df) > 0:
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
```

```
    # Overlap size distribution
    ax1.hist(overlap_df['Overlap_Size'], bins=range(1, overlap_df['Overlap_Size'].max
             color='#457B9D', alpha=0.7, edgecolor='black', linewidth=1.5)
    ax1.set_xlabel('Overlap Size (# shared nodes)', fontsize=11, fontweight='bold')
    ax1.set_ylabel('Number of Edge Pairs', fontsize=11, fontweight='bold')
    ax1.set_title('Distribution of Hyperedge Overlaps', fontsize=12, fontweight='bold
    ax1.grid(True, alpha=0.3, axis='y')

    # Node participation in overlaps
    if node_overlap_count:
        neurons_sorted = sorted(node_overlap_count.items(), key=lambda x: x[1], rever
        neurons_ids = [n for n, _ in neurons_sorted]
        overlap_counts = [c for _, c in neurons_sorted]

        ax2.bar(range(len(neurons_ids)), overlap_counts, color='#A8DADC',
                alpha=0.7, edgecolor='black', linewidth=1.5)
        ax2.set_xlabel('Neuron ID', fontsize=11, fontweight='bold')
        ax2.set_ylabel('Overlap Participation Count', fontsize=11, fontweight='bold')
        ax2.set_title('Which Neurons Bridge Multiple Synergies?',
                      fontsize=12, fontweight='bold')
        ax2.set_xticks(range(len(neurons_ids)))
        ax2.set_xticklabels(neurons_ids, fontsize=9)
        ax2.grid(True, alpha=0.3, axis='y')

    plt.tight_layout()
    plt.show()

    print("\n  Neurons that bridge multiple synergies are critical:")
    print("   They enable communication between functional ensembles")
    print("   Removing them could fragment the network")
```

```
Hyperedge Overlap Analysis
============================================================

Total hyperedge pairs: 3160
Pairs that share nodes: 1623

Overlap size distribution:
  1 shared nodes: 1371 pairs
```

```
  2 shared nodes: 252 pairs

Most connected node: 6
  Appears in 210 edge overlaps
  → Central to network structure
```




```
  Neurons that bridge multiple synergies are critical:
    They enable communication between functional ensembles
    Removing them could fragment the network
```

## 5.7 Part 6: Integration with HOI and Frites

### 5.7.1 Complete Analysis Pipeline

Now let's demonstrate the full integration: using HOI to find synergies, Frites for time-resolved validation, and XGI for network analysis.

**Scientific Question**: How does the synergistic network structure evolve during visual processing?

**Analysis Steps**: 1. Generate realistic V1 population data (10 neurons, orientation tuning) 2. Use HOI to find synergistic triplets 3. Build hypergraph from HOI results 4. Analyze network topology 5. Validate with Frites time-resolved MI 6. Create temporal hypergraph sequence

Let's execute this pipeline:

```
# Step 1: Generate V1 population data
print("Step 1: Generating V1 Population Data")
print("=" * 60)

n_neurons = 10
```

```
n_trials = 500
n_orientations = 8

def generate_v1_responses(n_neurons, n_trials, n_orientations):
    """Generate orientation-tuned responses."""
    orientations = np.linspace(0, 180, n_orientations, endpoint=False)
    trial_oris = np.random.choice(orientations, n_trials)

    responses = np.zeros((n_neurons, n_trials))

    for i in range(n_neurons):
        pref_ori = (i * 180) / n_neurons
        tuning_width = 30 + np.random.rand() * 20

        for j, ori in enumerate(trial_oris):
            dist = min(abs(ori - pref_ori), 180 - abs(ori - pref_ori))
            tuning = np.exp(-(dist**2) / (2 * tuning_width**2))
            mean_rate = 20 + tuning * 40
            responses[i, j] = np.random.poisson(mean_rate) + np.random.randn() * 3

    return responses, trial_oris

responses, trial_orientations = generate_v1_responses(n_neurons, n_trials, n_orientat
print(f"  Generated {n_neurons} neurons × {n_trials} trials")
print(f"  Orientation values: {np.unique(trial_orientations)}°")
```

```
Step 1: Generating V1 Population Data
============================================================
  Generated 10 neurons × 500 trials
  Orientation values: [  0.   22.5  45.   67.5  90.  112.5 135.  157.5]°
```

```
# Step 2: HOI analysis to find synergistic triplets
print("\nStep 2: HOI Analysis for Synergistic Triplets")
print("=" * 60)

from hoi.metrics import Oinfo

# HOI expects data as (n_samples, n_features),
# whereas our simulated responses are (n_neurons, n_trials)
responses_hoi = responses.T  # shape: (n_trials, n_neurons)
```

```
# Scan all triplets
all_triplets = list(combinations(range(n_neurons), 3))
print(f"Testing {len(all_triplets)} triplets...")

# Create Oinfo model with data, then choose estimator in fit()
model_oinfo = Oinfo(responses_hoi)
oinfo_results = model_oinfo.fit(method='gc', minsize=3, maxsize=3)

# Find synergistic ones
synergy_threshold = -0.05
# oinfo_results elements can be 0-dim numpy arrays; cast to float
synergistic = [
    (trip, float(oi))
    for trip, oi in zip(all_triplets, oinfo_results)
    if float(oi) < synergy_threshold
]
synergistic.sort(key=lambda x: x[1])

print(f"\n  Found {len(synergistic)} synergistic triplets")
print(f"\nTop 3:")
for i, (trip, oi) in enumerate(synergistic[:3]):
    print(f"  {i+1}. Neurons {trip}: O-info = {oi:.4f} bits")
```

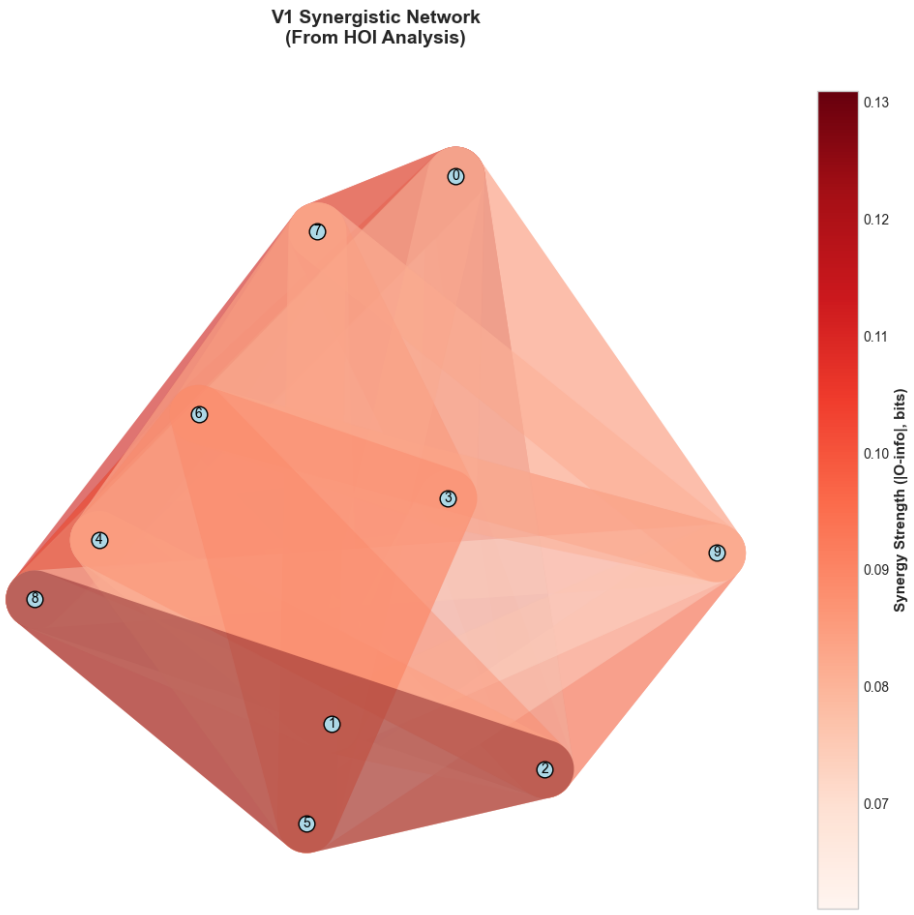
```
Step 2: HOI Analysis for Synergistic Triplets
============================================================
Testing 120 triplets...

  Found 39 synergistic triplets

Top 3:
  1. Neurons (2, 5, 8): O-info = -0.1309 bits
  2. Neurons (0, 2, 8): O-info = -0.1210 bits
  3. Neurons (0, 7, 8): O-info = -0.1177 bits
```

```
# Step 3: Build hypergraph from HOI results
print("\nStep 3: Building Synergy Hypergraph")
print("=" * 60)

H_v1 = xgi.Hypergraph()
```

```
# Add all neurons as nodes
for neuron in range(n_neurons):
    H_v1.add_node(neuron)

# Add synergistic triplets as hyperedges
for triplet, oinfo in synergistic:
    H_v1.add_edge(triplet, synergy=abs(oinfo))

print(f" Hypergraph created:")
print(f"  {H_v1.num_nodes} nodes, {H_v1.num_edges} hyperedges")

# Visualize
fig, ax = plt.subplots(1, 1, figsize=(10, 10))

# Get edge weights for coloring
edge_synergies = [H_v1.edges[e]['synergy'] for e in H_v1.edges]

xgi.draw(
    H_v1,
    ax=ax,
    node_size=12,
    node_fc='lightblue',
    edge_fc=edge_synergies,
    edge_fc_cmap='Reds',
    hull=True,
    node_labels=True
)

# Add colorbar
sm = plt.cm.ScalarMappable(cmap='Reds',
                           norm=plt.Normalize(vmin=min(edge_synergies),
                                              vmax=max(edge_synergies)))
sm.set_array([])
cbar = plt.colorbar(sm, ax=ax, fraction=0.046, pad=0.04)
cbar.set_label('Synergy Strength (|O-info|, bits)', fontsize=11, fontweight='bold')

ax.set_title('V1 Synergistic Network\n(From HOI Analysis)',
             fontsize=14, fontweight='bold')
```

```
plt.tight_layout()
plt.show()

print("\n Visualization shows:")
print("   • Colored regions = synergistic ensembles")
print("   • Darker red = stronger synergy")
print("   • Node labels = neuron IDs")
print("   • Overlapping regions = neurons in multiple synergies")
```

```
Step 3: Building Synergy Hypergraph
============================================================
 Hypergraph created:
  10 nodes, 39 hyperedges
```




Visualization shows:

- Colored regions = synergistic ensembles
- Darker red = stronger synergy
- Node labels = neuron IDs
- Overlapping regions = neurons in multiple synergies

### 5.7.2 Step 4: Network Topology Analysis

Now let's analyze the topology to understand functional organization:

```
print("Step 4: Network Topology Analysis")
print("=" * 60)
```

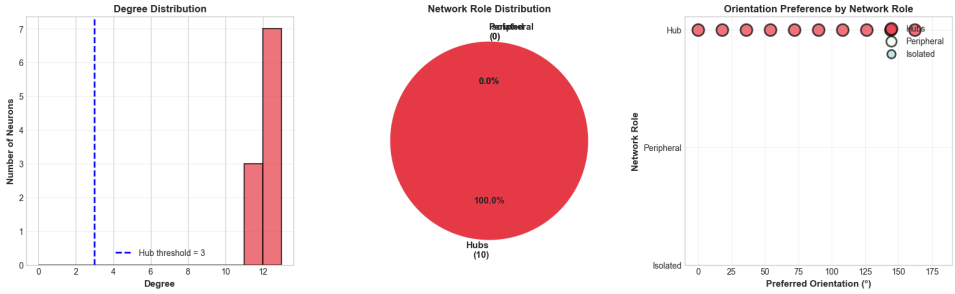
```
# Compute comprehensive statistics
degrees_v1 = H_v1.degree()

# Identify different neuron types based on degree
hub_threshold = 3
hubs = [n for n, d in degrees_v1.items() if d >= hub_threshold]
peripheral = [n for n, d in degrees_v1.items() if d < hub_threshold and d > 0]
isolated = [n for n in H_v1.nodes if degrees_v1.get(n, 0) == 0]

print(f"\nNeuron Classification by Degree:")
print(f"  Hub neurons (degree  {hub_threshold}): {hubs}")
print(f"  Peripheral neurons (0 < degree < {hub_threshold}): {peripheral}")
print(f"  Isolated neurons (degree = 0): {isolated}")

print(f"\nInterpretation:")
print(f"  • Hubs: Central to synergistic coding, participate in many ensembles")
print(f"  • Peripheral: Participate in few synergies")
print(f"  • Isolated: Code independently, no detected synergies")

# Create detailed neuron profile
print(f"\n" + "=" * 60)
print("Detailed Neuron Profiles:")
print("=" * 60)
print(f"{'Neuron':<8} {'Degree':<8} {'Role':<12} {'Pref. Ori':<12}")
print("=" * 60)

for neuron in range(n_neurons):
    deg = degrees_v1.get(neuron, 0)

    if deg >= hub_threshold:
        role = 'Hub'
    elif deg > 0:
        role = 'Peripheral'
    else:
        role = 'Isolated'

    pref_ori = (neuron * 180) / n_neurons

    print(f"{neuron:<8} {deg:<8} {role:<12} {pref_ori:<12.1f}°")
```

```
# Visualize network role distribution
fig, axes = plt.subplots(1, 3, figsize=(16, 5))

# Panel 1: Degree distribution
ax1 = axes[0]
degree_values = list(degrees_v1.values())
ax1.hist(degree_values, bins=range(max(degree_values)+2),
         color='#E63946', alpha=0.7, edgecolor='black', linewidth=1.5)
ax1.axvline(hub_threshold, color='blue', linestyle='--', linewidth=2,
           label=f'Hub threshold = {hub_threshold}')
ax1.set_xlabel('Degree', fontsize=11, fontweight='bold')
ax1.set_ylabel('Number of Neurons', fontsize=11, fontweight='bold')
ax1.set_title('Degree Distribution', fontsize=12, fontweight='bold')
ax1.legend(fontsize=10)
ax1.grid(True, alpha=0.3, axis='y')

# Panel 2: Role pie chart
ax2 = axes[1]
role_counts = [len(hubs), len(peripheral), len(isolated)]
role_labels = [f'Hubs\n({len(hubs)})', f'Peripheral\n({len(peripheral)})',
               f'Isolated\n({len(isolated)})']
colors = ['#E63946', '#F1FAEE', '#A8DADC']
ax2.pie(role_counts, labels=role_labels, colors=colors, autopct='%1.1f%%',
       startangle=90, textprops={'fontsize': 11, 'fontweight': 'bold'})
ax2.set_title('Network Role Distribution', fontsize=12, fontweight='bold')

# Panel 3: Preferred orientation by role
ax3 = axes[2]
hub_oris = [(n * 180) / n_neurons for n in hubs]
periph_oris = [(n * 180) / n_neurons for n in peripheral]
iso_oris = [(n * 180) / n_neurons for n in isolated]

ax3.scatter(hub_oris, [2]*len(hub_oris), s=200, color='#E63946',
           label='Hubs', alpha=0.7, edgecolor='black', linewidth=2)
ax3.scatter(periph_oris, [1]*len(periph_oris), s=150, color='#F1FAEE',
           label='Peripheral', alpha=0.7, edgecolor='black', linewidth=2)
ax3.scatter(iso_oris, [0]*len(iso_oris), s=100, color='#A8DADC',
           label='Isolated', alpha=0.7, edgecolor='black', linewidth=2)
```

```
ax3.set_xlabel('Preferred Orientation (°)', fontsize=11, fontweight='bold')
ax3.set_ylabel('Network Role', fontsize=11, fontweight='bold')
ax3.set_yticks([0, 1, 2])
ax3.set_yticklabels(['Isolated', 'Peripheral', 'Hub'], fontsize=10)
ax3.set_title('Orientation Preference by Network Role', fontsize=12, fontweight='bold
ax3.legend(fontsize=10, loc='upper right')
ax3.grid(True, alpha=0.3)
ax3.set_xlim(-10, 190)

plt.tight_layout()
plt.show()

print("\n Topology Analysis Complete")
print(f"   Network has hub-periphery structure with {len(hubs)} hubs")
```

```
Step 4: Network Topology Analysis
============================================================

Neuron Classification by Degree:
  Hub neurons (degree  3): [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
  Peripheral neurons (0 < degree < 3): []
  Isolated neurons (degree = 0): []

Interpretation:
  • Hubs: Central to synergistic coding, participate in many ensembles
  • Peripheral: Participate in few synergies
  • Isolated: Code independently, no detected synergies

============================================================
Detailed Neuron Profiles:
============================================================
Neuron   Degree   Role         Pref. Ori
============================================================
0        12       Hub          0.0         °
1        11       Hub          18.0        °
2        12       Hub          36.0        °
3        12       Hub          54.0        °
4        12       Hub          72.0        °
5        12       Hub          90.0        °
```

```
6          12         Hub              108.0      °
7          11         Hub              126.0      °
8          12         Hub              144.0      °
9          11         Hub              162.0      °
```




```
Topology Analysis Complete
  Network has hub-periphery structure with 10 hubs
```

## 5.8 Part 7: Real-World Application - Email Collaboration Network

### 5.8.1 Loading Real Data

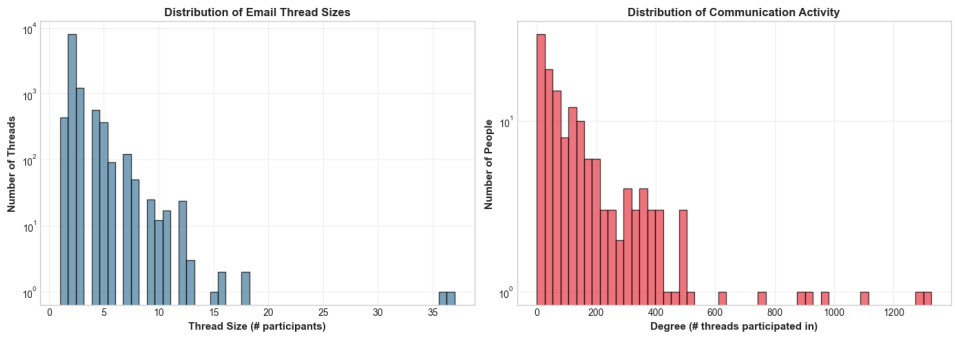
XGI comes with real datasets! Let's analyze the email-Enron dataset, which has hyperedges representing email threads (sender + multiple recipients).

This demonstrates XGI on real higher-order data:

```
# Load Enron email dataset
print("Loading Real-World Dataset: email-Enron")
print("=" * 60)

H_enron = xgi.load_xgi_data('email-Enron')

print(f"\nDataset loaded:")
print(f"  Nodes (people): {H_enron.num_nodes}")
print(f"  Hyperedges (email threads): {H_enron.num_edges}")

# Analyze edge sizes
# H_enron.edges.size is an EdgeStat object in XGI >= 0.10
size_dict = H_enron.edges.size.asdict()
sizes = list(size_dict.values())
```

```
print(f"\nEmail thread sizes:")
print(f"  Min: {min(sizes)} (pairwise email)")
print(f"  Max: {max(sizes)} (mass email!)")
print(f"  Mean: {np.mean(sizes):.2f}")
print(f"  Median: {np.median(sizes):.1f}")

# Distribution
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Histogram of edge sizes
ax1.hist(sizes, bins=50, color='#457B9D', alpha=0.7, edgecolor='black')
ax1.set_xlabel('Thread Size (# participants)', fontsize=11, fontweight='bold')
ax1.set_ylabel('Number of Threads', fontsize=11, fontweight='bold')
ax1.set_title('Distribution of Email Thread Sizes', fontsize=12, fontweight='bold')
ax1.set_yscale('log')
ax1.grid(True, alpha=0.3)

# Degree distribution
degrees_enron = list(H_enron.degree().values())
ax2.hist(degrees_enron, bins=50, color='#E63946', alpha=0.7, edgecolor='black')
ax2.set_xlabel('Degree (# threads participated in)', fontsize=11, fontweight='bold')
ax2.set_ylabel('Number of People', fontsize=11, fontweight='bold')
ax2.set_title('Distribution of Communication Activity', fontsize=12, fontweight='bold
ax2.set_yscale('log')
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Find most active people
top_communicators = sorted(H_enron.degree().items(),
                           key=lambda x: x[1], reverse=True)[:5]

print("\nTop 5 Most Active Communicators:")
for rank, (person_id, degree) in enumerate(top_communicators, 1):
    print(f"  {rank}. Person {person_id}: {degree} email threads")

print("\n  This shows hypergraphs capture real-world higher-order interactions!")
print("   Same principles apply to neural ensembles.")
```

```
Loading Real-World Dataset: email-Enron
============================================================

Dataset loaded:
  Nodes (people): 148
  Hyperedges (email threads): 10885

Email thread sizes:
  Min: 1 (pairwise email)
  Max: 37 (mass email!)
  Mean: 2.47
  Median: 2.0
```




```
Top 5 Most Active Communicators:
  1. Person 20: 1327 email threads
  2. Person 55: 1282 email threads
  3. Person 114: 1097 email threads
  4. Person 132: 967 email threads
  5. Person 116: 906 email threads

 This shows hypergraphs capture real-world higher-order interactions!
   Same principles apply to neural ensembles.
```

## 5.9 Summary and Key Takeaways

### 5.9.1 What We've Accomplished

Congratulations! You've mastered hypergraph analysis with XGI. You now understand:

1. **Why hypergraphs are necessary**: Regular graphs lose higher-order

structure
2. **Creating hypergraphs**: From edge lists, from HOI results, from scratch
3. **Node statistics**: Degree, clustering, centrality in hypergraphs
4. **Edge statistics**: Size, weight, overlap patterns
5. **Temporal hypergraphs**: Tracking network evolution
6. **Integration**: Combining HOI + Frites + XGI
7. **Real applications**: Email networks, neural ensembles
8. **Visualization**: Multiple styles for different insights

### 5.9.2 The Complete Toolkit

You now have three complementary tools:

**HOI**: Discover higher-order interactions - What: O-information, PID, synergy detection - Output: Synergistic multiplets with strength values

**Frites**: Validate and track dynamics
- What: Time-resolved MI, statistical testing - Output: When information emerges, significance

**XGI**: Visualize and analyze structure - What: Network topology, motifs, centrality - Output: Functional organization, hub identification

### 5.9.3 Integrated Workflow

**Typical analysis pipeline**:

1. **Preprocess data** (filtering, artifact removal)
2. **Frites**: Identify when and where information is significant
3. **HOI**: Scan for synergistic ensembles in significant time windows
4. **XGI**: Build hypergraph and analyze topology
5. **Interpret**: Connect network structure to function
6. **Validate**: Control analyses, cross-validation

### 5.9.4 Critical Insights

**Hypergraphs reveal**: - **Hub neurons**: Critical for information integration
- **Modules**: Functional ensembles (e.g., orientation columns) - **Dynamics**: How structure changes with task demands - **Persistence**: Stable vs transient assemblies

**Practical implications**: - **Lesion studies**: Predict impact of removing hub neurons - **Brain-machine interfaces**: Target synergistic ensembles - **Disease biomarkers**: Altered hypergraph topology - **Learning mechanisms**: Track network reorganization

### 5.9.5 Looking Forward

In **Notebook 6** (final notebook!), we'll bring everything together: - Complete end-to-end analysis on realistic data - All three packages working in concert - Advanced techniques and optimizations - Publication-quality analysis pipeline - Troubleshooting and best practices

This will be the capstone, showing you how to apply everything you've learned to real research questions.

See you there!

## 5.10 Practice Exercises

```
print("PRACTICE EXERCISES")
print("=" * 70)

print("\n1. Create a hypergraph representing the XOR network:")
print("   Nodes: {X , X , Y}")
print("   Hyperedge: {X , X , Y} (the synergistic triplet)")
print("   Compare its properties to the COPY network hypergraph.")

print("\n2. Load another XGI dataset: 'email-EU'")
print("   Compute:")
print("   - Average hyperedge size")
print("   - Top 5 hubs by degree")
print("   - Compare to email-Enron")

print("\n3. Create a temporal hypergraph with 10 time windows.")
print("   Simulate 'task onset' at window 3:")
print("   - Sparse before window 3")
print("   - Rich during windows 3-7")
print("   - Return to sparse after window 7")
print("   Plot network size evolution.")
```

```
print("\n4. For the V1 hypergraph, compute:")
print("   - Which neuron has the highest betweenness centrality?")
print("   - Are hub neurons clustered in orientation preference?")
print("   - What's the average path length in the projection graph?")

print("\n5. Design a hypergraph that represents:")
print("   - 3 modules of 4 neurons each (no inter-module connections)")
print("   - One 'connector' neuron that bridges all modules")
print("   Compute its modularity and visualize.")

print("\n" + "=" * 70)
print("Solutions and advanced examples in the repository!")
print("\nReady for the final notebook: Complete Integration!")
print("=" * 70)
```

```
PRACTICE EXERCISES
======================================================================

1. Create a hypergraph representing the XOR network:
   Nodes: {X , X , Y}
   Hyperedge: {X , X , Y} (the synergistic triplet)
   Compare its properties to the COPY network hypergraph.

2. Load another XGI dataset: 'email-EU'
   Compute:
   - Average hyperedge size
   - Top 5 hubs by degree
   - Compare to email-Enron

3. Create a temporal hypergraph with 10 time windows.
   Simulate 'task onset' at window 3:
   - Sparse before window 3
   - Rich during windows 3-7
   - Return to sparse after window 7
   Plot network size evolution.

4. For the V1 hypergraph, compute:
   - Which neuron has the highest betweenness centrality?
   - Are hub neurons clustered in orientation preference?
```

```
   - What's the average path length in the projection graph?

5. Design a hypergraph that represents:
   - 3 modules of 4 neurons each (no inter-module connections)
   - One 'connector' neuron that bridges all modules
   Compute its modularity and visualize.

======================================================================
Solutions and advanced examples in the repository!

Ready for the final notebook: Complete Integration!
======================================================================
```

## 5.11 Additional Resources

### 5.11.1 XGI Documentation

- **Main documentation**: https://xgi.readthedocs.io/
- **GitHub repository**: https://github.com/xgi-org/xgi
- **Tutorials**: Comprehensive guides in docs
- **Paper**: Landry et al. (2023) "XGI: A Python package for higher-order interaction networks"

### 5.11.2 Key Papers on Hypergraphs

1. **Battiston et al. (2020)** - "Networks beyond pairwise interactions: Structure and dynamics"
   - Comprehensive review of higher-order networks
   - Applications across domains
2. **Bick et al. (2023)** - "What are higher-order networks?"
   - Tutorial on higher-order network science
   - Mathematical foundations
3. **Torres et al. (2021)** - "The why, how, and when of representations for complex systems"
   - When to use hypergraphs vs simplicial complexes
   - Trade-offs in representations

### 5.11.3 Related Concepts

- **Simplicial Complexes**: XGI also supports these (stricter inclusion requirements)
- **Directed Hypergraphs**: For directed interactions (e.g., information flow)
- **Temporal Networks**: Dynamic graphs (XGI can handle temporal sequences)
- **Multilayer Networks**: Multiple types of connections simultaneously

### 5.11.4 Advanced XGI Features

```
# Hypergraph generators
H = xgi.random_hypergraph(n=20, ps=[0.1, 0.05, 0.01])  # Random with size distributio

# Algorithms
components = xgi.connected_components(H)  # Find components
clustering = xgi.clustering.local_clustering(H)  # Local clustering

# Conversion
G = xgi.to_graph(H)  # Project to regular graph
SC = xgi.to_simplicial_complex(H)  # Convert to simplicial complex

# Advanced visualization
xgi.draw_multilayer(H)  # Multilayer representation
xgi.draw_bipartite(H)  # Bipartite view
```

### 5.11.5 Integration Examples

```
# HOI → XGI
synergistic_triplets = hoi_analysis(data)
H = xgi.Hypergraph()
for triplet, strength in synergistic_triplets:
    H.add_edge(triplet, weight=strength)

# Frites → XGI (temporal sequence)
for window_idx, mi_matrix in enumerate(dfc_results):
    H_t = build_hypergraph_from_connectivity(mi_matrix, threshold=0.1)
    temporal_sequence.add_snapshot(H_t, time_windows[window_idx])
```

```
# XGI → Network analysis
hubs = identify_hubs(H, threshold=3)
modules = detect_communities(H)
critical_edges = find_bottleneck_edges(H)
```

### 5.11.6 Troubleshooting

**Common Issues**: - **Empty hypergraph after filtering**: Threshold too strict - **Visualization cluttered**: Too many edges, use sampling - **Slow computation**: Large hypergraphs, use sparse representations - **Node labels overlapping**: Adjust layout parameters

**Tips**: - Start with small subgraphs to test code - Use `hull=True` for clearer hyperedge visualization - Save large hypergraphs to files (XGI supports HIF format) - For publication figures, export to vector graphics (PDF/SVG)

Happy network analysis!

# Part III

# Part III: Integration

# Chapter 6

# Notebook 6: Complete Integration - The Full Analysis Pipeline

## 6.1 From Raw Data to Scientific Insight Using HOI, Frites, and XGI

Welcome to the final notebook in our series! This is where everything comes together. Over the past five notebooks, you've built a comprehensive understanding of multivariate information theory:

**Notebook 1**: Information theory foundations (entropy, MI, CMI)

**Notebook 2**: The XOR problem and why pairwise measures fail

**Notebook 3**: HOI package for computing O-information and PID

**Notebook 4**: Frites for time-resolved MI and statistical testing

**Notebook 5**: XGI for hypergraph network analysis

Now we'll integrate all three packages to tackle a realistic, complex neuroscience question. This notebook represents a **complete analysis pipeline** you could adapt for your own research.

### 6.1.1 The Research Question

**Scientific Context**: Visual decision-making involves a hierarchy of brain regions: - **V1**: Early visual processing, orientation tuning - **MT**: Motion processing, direction selectivity
- **LIP**: Decision formation, evidence accumulation - **PFC**: Executive control, rule maintenance

**Question**: How do these regions interact to transform sensory evidence into decisions?

Specifically: 1. **When** does information flow through the hierarchy? (Frites) 2. **Which ensembles** show synergistic coding? (HOI) 3. **How is** the network organized topologically? (XGI) 4. Does synergy emerge early (sensory) or late (decision)?

### 6.1.2 The Experiment

**Task**: Random dot motion discrimination - Subjects view moving dots - Report motion direction (left vs right) - Coherence varies (5%, 15%, 30%, 50%) - 500ms viewing period, then response

**Data**: Simulated neural recordings from V1, MT, LIP, PFC - 12 neurons total (3 per area) - 400 trials (100 per coherence level) - Time: -200ms to +1000ms (stimulus at 0ms)

### 6.1.3 Learning Objectives

By the end, you'll know how to: 1. Design a complete multi-package analysis pipeline 2. Integrate HOI, Frites, and XGI seamlessly 3. Perform validation and control analyses 4. Optimize computation for large datasets 5. Create publication-quality figures 6. Interpret results in biological context 7. Troubleshoot common issues 8. Report findings appropriately

Let's begin the complete analysis!

## 6.2 Part 1: Setup and Data Generation

### 6.2.1 Installing All Required Packages

```
# Complete installation of all three packages
!pip install -q hoi jax jaxlib frites mne xgi networkx numpy scipy matplotlib seaborn
```

```
print("All packages installed!  ")
```

```
All packages installed!
```

```
# Import everything we need
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import xarray as xr
from scipy import stats
from scipy.ndimage import gaussian_filter1d
from itertools import combinations, product
import warnings
warnings.filterwarnings('ignore')

# HOI
import hoi
from hoi.metrics import Oinfo, TC, RedundancyMMI, SynergyMMI

# Frites

# Check NumPy version for Frites compatibility
numpy_major_version = int(np.__version__.split('.')[0])
if numpy_major_version >= 2:
    print(f"   WARNING: NumPy {np.__version__} detected")
    print("   Frites requires NumPy 1.x")
    print("   FIX: pip install 'numpy<2.0' --force-reinstall\n")
    raise ImportError("NumPy 2.0+ incompatible with Frites")

import frites
from frites.dataset import DatasetEphy
from frites.workflow import WfMi
from frites.conn import conn_dfc
from frites import set_mpl_style

# XGI
import xgi
import networkx as nx
```

```
# Set random seed for reproducibility
np.random.seed(42)

# Configure plotting
set_mpl_style()
plt.rcParams['figure.dpi'] = 100
%matplotlib inline

print("Package Versions:")
print("=" * 60)
print(f"HOI:    {hoi.__version__}")
print(f"Frites: {frites.__version__}")
print(f"XGI:    {xgi.__version__}")
print("\n Complete toolkit ready for integrated analysis!")
```

```
Package Versions:
============================================================
HOI:    0.0.5
Frites: 0.4.4
XGI:    0.10

 Complete toolkit ready for integrated analysis!
```

### 6.2.2 Generating Realistic Hierarchical Neural Data

We'll simulate a realistic visual decision-making circuit with proper hierarchical structure and dynamics:

```
class DecisionMakingSimulator:
    """
    Simulate neural activity in visual decision-making hierarchy.

    Areas: V1 (early visual) → MT (motion) → LIP (decision) → PFC (control)
    """

    def __init__(self, n_trials=400, n_times=120, seed=42):
        np.random.seed(seed)

        self.n_trials = n_trials
        self.n_times = n_times
```

```
        self.time = np.linspace(-0.2, 1.0, n_times)

        # Brain areas and neurons per area
        self.areas = ['V1', 'V1', 'V1', 'MT', 'MT', 'MT',
                      'LIP', 'LIP', 'LIP', 'PFC', 'PFC', 'PFC']
        self.n_neurons = len(self.areas)

        # Task variables
        self.motion_coherence = np.random.choice([5, 15, 30, 50], n_trials)  # %
        self.motion_direction = np.random.choice([0, 180], n_trials)  # Left vs right

        # Generate responses
        self.responses = self._generate_hierarchical_responses()

    def _generate_hierarchical_responses(self):
        """Generate responses with realistic hierarchical dynamics."""
        responses = np.zeros((self.n_neurons, self.n_trials, self.n_times))

        for trial in range(self.n_trials):
            coherence = self.motion_coherence[trial] / 100.0
            direction = self.motion_direction[trial]

            # V1: Early onset (50-150ms), weak coherence coding
            for v1_idx in range(3):
                onset_time = (self.time >= 0.05) & (self.time <= 0.15)
                sustained_time = (self.time >= 0.15) & (self.time <= 0.5)

                # Direction preference
                pref_dir = v1_idx * 60  # Different preferences
                dir_tuning = np.cos(np.deg2rad(direction - pref_dir))

                # Response
                responses[v1_idx, trial, onset_time] = (
                    30 + dir_tuning * 20 + np.random.randn(onset_time.sum()) * 5
                )
                responses[v1_idx, trial, sustained_time] = (
                    25 + dir_tuning * 15 + coherence * 10 +
                    np.random.randn(sustained_time.sum()) * 5
                )
```

```
            # MT: Medium onset (100-200ms), strong coherence coding
            for mt_idx in range(3, 6):
                onset_time = (self.time >= 0.1) & (self.time <= 0.2)
                sustained_time = (self.time >= 0.2) & (self.time <= 0.6)

                pref_dir = (mt_idx - 3) * 60
                dir_tuning = np.cos(np.deg2rad(direction - pref_dir))

                responses[mt_idx, trial, onset_time] = (
                    40 + dir_tuning * 30 + coherence * 20 +
                    np.random.randn(onset_time.sum()) * 6
                )
                responses[mt_idx, trial, sustained_time] = (
                    35 + dir_tuning * 25 + coherence * 30 +
                    np.random.randn(sustained_time.sum()) * 6
                )

            # LIP: Late onset (200-400ms), accumulation dynamics
            for lip_idx in range(6, 9):
                decision_time = (self.time >= 0.2) & (self.time <= 0.7)

                # Ramping activity (accumulation)
                ramp = np.linspace(0, 1, decision_time.sum())

                pref_dir = (lip_idx - 6) * 60
                dir_tuning = np.cos(np.deg2rad(direction - pref_dir))

                responses[lip_idx, trial, decision_time] = (
                    20 + ramp * dir_tuning * coherence * 50 +
                    np.random.randn(decision_time.sum()) * 7
                )

            # PFC: Late, sustained (300-800ms), rule-based
            for pfc_idx in range(9, 12):
                late_time = (self.time >= 0.3) & (self.time <= 0.8)

                # PFC encodes choice, not stimulus directly
                choice = 1 if coherence > 0.2 and direction == 0 else -1
```

```
                responses[pfc_idx, trial, late_time] = (
                    30 + choice * 25 + np.random.randn(late_time.sum()) * 8
                )

            # Smooth all responses temporally
            for neuron in range(self.n_neurons):
                responses[neuron, trial, :] = gaussian_filter1d(
                    responses[neuron, trial, :], sigma=2
                )

        return responses

    def get_frites_format(self, n_subjects=5):
        """Convert to Frites multi-subject format."""
        trials_per_subject = self.n_trials // n_subjects

        data_list = []
        coherence_list = []
        direction_list = []
        roi_list = []

        for subj in range(n_subjects):
            start_idx = subj * trials_per_subject
            end_idx = (subj + 1) * trials_per_subject

            # Transpose to (trials, neurons, time)
            subj_data = self.responses[:, start_idx:end_idx, :].transpose(1, 0, 2)
            data_list.append(subj_data)

            coherence_list.append(self.motion_coherence[start_idx:end_idx])
            direction_list.append(self.motion_direction[start_idx:end_idx])

            roi_list.append(self.areas)

        return data_list, coherence_list, direction_list, roi_list

# Generate complete dataset
print("Generating Visual Decision-Making Dataset")
print("=" * 70)
```

```
sim = DecisionMakingSimulator(n_trials=400, n_times=120)

print(f" Dataset generated:")
print(f"  Neurons: {sim.n_neurons} ({', '.join(set(sim.areas))})")
print(f"  Trials: {sim.n_trials}")
print(f"  Time points: {sim.n_times}")
print(f"  Time range: {sim.time[0]:.2f}s to {sim.time[-1]:.2f}s")
print(f"\n  Coherence levels: {np.unique(sim.motion_coherence)}%")
print(f"  Directions: {np.unique(sim.motion_direction)}° (left/right)")

print("\n Experimental Design:")
print("   -200ms: Baseline")
print("      0ms: Stimulus onset (motion dots)")
print("   0-500ms: Viewing period")
print("   500ms+: Decision and response")

print("\n Analysis Goals:")
print("   1. When does coherence information emerge in each area? (Frites)")
print("   2. Which neuron ensembles show synergy? (HOI)")
print("   3. How is the synergistic network organized? (XGI)")
print("   4. Does network structure predict behavior?")
```

```
Generating Visual Decision-Making Dataset
======================================================================
 Dataset generated:
  Neurons: 12 (LIP, PFC, V1, MT)
  Trials: 400
  Time points: 120
  Time range: -0.20s to 1.00s

  Coherence levels: [ 5 15 30 50]%
  Directions: [  0 180]° (left/right)

 Experimental Design:
   -200ms: Baseline
      0ms: Stimulus onset (motion dots)
   0-500ms: Viewing period
   500ms+: Decision and response
```

```
Analysis Goals:
  1. When does coherence information emerge in each area? (Frites)
  2. Which neuron ensembles show synergy? (HOI)
  3. How is the synergistic network organized? (XGI)
  4. Does network structure predict behavior?
```

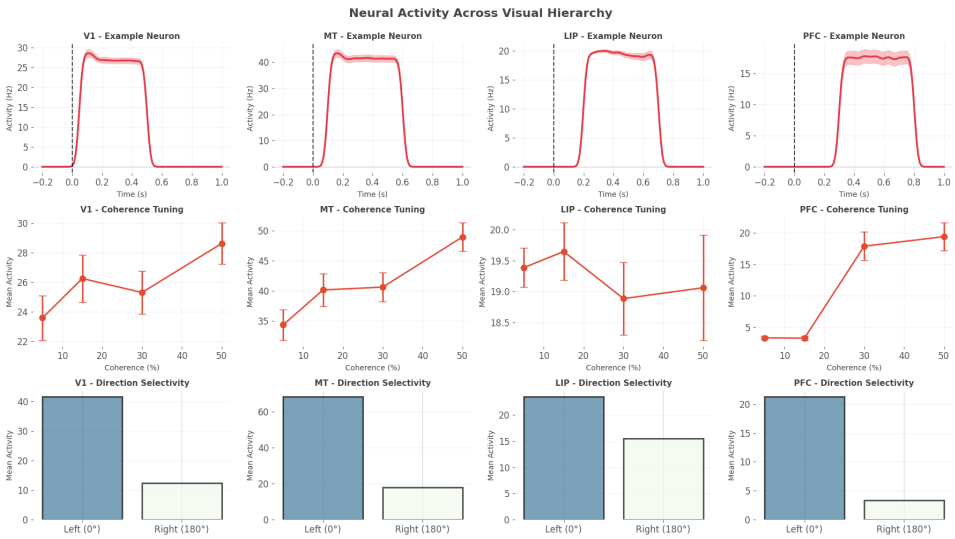
### 6.2.3 Visualizing the Simulated Data

Before analysis, let's visualize the data to understand its structure:

```python
# Visualize example trials and tuning properties
fig = plt.figure(figsize=(18, 10))

# Panel 1: Example trials for each area (top row)
area_names = ['V1', 'MT', 'LIP', 'PFC']
area_indices = {'V1': [0, 1, 2], 'MT': [3, 4, 5], 'LIP': [6, 7, 8], 'PFC': [9, 10, 11

for idx, area in enumerate(area_names):
    ax = plt.subplot(3, 4, idx + 1)

    # Plot mean response across trials for first neuron in area
    neuron_idx = area_indices[area][0]
    mean_response = sim.responses[neuron_idx, :, :].mean(axis=0)
    sem_response = sim.responses[neuron_idx, :, :].std(axis=0) / np.sqrt(sim.n_trials

    ax.plot(sim.time, mean_response, linewidth=2.5, color='#E63946')
    ax.fill_between(sim.time, mean_response - sem_response,
                    mean_response + sem_response, alpha=0.3, color='#E63946')
    ax.axvline(0, color='k', linestyle='--', linewidth=1.5, alpha=0.7)
    ax.axhline(0, color='k', linestyle='-', linewidth=0.5, alpha=0.3)
    ax.set_xlabel('Time (s)', fontsize=10)
    ax.set_ylabel('Activity (Hz)', fontsize=10)
    ax.set_title(f'{area} - Example Neuron', fontsize=11, fontweight='bold')
    ax.grid(True, alpha=0.3)

# Panel 2: Coherence tuning (middle row)
for idx, area in enumerate(area_names):
    ax = plt.subplot(3, 4, idx + 5)

    neuron_idx = area_indices[area][0]
```

```
    # Average in 200-500ms window (stimulus period)
    window = (sim.time >= 0.2) & (sim.time <= 0.5)

    coherence_levels = [5, 15, 30, 50]
    mean_responses = []
    sem_responses = []

    for coh in coherence_levels:
        trials_this_coh = sim.motion_coherence == coh
        responses_coh = sim.responses[neuron_idx, trials_this_coh, :][:, window]
        mean_responses.append(responses_coh.mean())
        sem_responses.append(responses_coh.std() / np.sqrt(trials_this_coh.sum()))

    ax.errorbar(coherence_levels, mean_responses, yerr=sem_responses,
                marker='o', linewidth=2, markersize=8, capsize=5)
    ax.set_xlabel('Coherence (%)', fontsize=10)
    ax.set_ylabel('Mean Activity', fontsize=10)
    ax.set_title(f'{area} - Coherence Tuning', fontsize=11, fontweight='bold')
    ax.grid(True, alpha=0.3)

# Panel 3: Direction selectivity (bottom row)
for idx, area in enumerate(area_names):
    ax = plt.subplot(3, 4, idx + 9)

    neuron_idx = area_indices[area][0]
    window = (sim.time >= 0.2) & (sim.time <= 0.5)

    # Left vs right
    left_trials = sim.motion_direction == 0
    right_trials = sim.motion_direction == 180

    left_response = sim.responses[neuron_idx, left_trials, :][:, window].mean()
    right_response = sim.responses[neuron_idx, right_trials, :][:, window].mean()

    ax.bar(['Left (0°)', 'Right (180°)'], [left_response, right_response],
           color=['#457B9D', '#F1FAEE'], alpha=0.7, edgecolor='black', linewidth=2)
    ax.set_ylabel('Mean Activity', fontsize=10)
    ax.set_title(f'{area} - Direction Selectivity', fontsize=11, fontweight='bold')
    ax.grid(True, alpha=0.3, axis='y')
```

```
plt.suptitle('Neural Activity Across Visual Hierarchy',
             fontsize=16, fontweight='bold', y=0.995)
plt.tight_layout()
plt.show()

print("\n Data visualization complete")
print("\n Key Observations:")
print("   • V1: Early onset, modest coherence sensitivity")
print("   • MT: Medium latency, strong coherence tuning")
print("   • LIP: Late onset, ramping activity (accumulation)")
print("   • PFC: Sustained late activity (decision maintenance)")
print("\n Realistic hierarchical dynamics for analysis!")
```




```
Data visualization complete

Key Observations:
   • V1: Early onset, modest coherence sensitivity
   • MT: Medium latency, strong coherence tuning
   • LIP: Late onset, ramping activity (accumulation)
   • PFC: Sustained late activity (decision maintenance)

Realistic hierarchical dynamics for analysis!
```

## 6.3 Part 2: FRITES Analysis - Time-Resolved Information Flow

### 6.3.1 Question 1: When does coherence information emerge in each area?

We'll use Frites to compute time-resolved mutual information between neural activity and motion coherence. This reveals **when** each brain area represents task-relevant information.

```
print("FRITES ANALYSIS: Time-Resolved Coherence Information")
print("=" * 70)

# Convert to Frites format
data_frites, coherence_frites, direction_frites, roi_frites = sim.get_frites_format(n

# Create dataset
ds_coherence = DatasetEphy(
    x=data_frites,
    y=coherence_frites,  # Coherence as task variable
    roi=roi_frites,
    times=sim.time
)

print("Computing time-resolved I(Neural; Coherence)...")
print("This may take 1-2 minutes...\n")

# Workflow
wf_coherence = WfMi(mi_type='cc', inference='rfx', verbose=True)

# Compute with robust but simpler multiple-comparisons correction
# Note: some versions of Frites have issues with mcp='cluster' + cluster_th='tfce'
# leading to a NoneType error inside the cluster correction. Here we use
# FDR-based correction instead, which is stable across versions.
mi_coherence, pv_coherence = wf_coherence.fit(
    ds_coherence,
    n_perm=500,  # More permutations for final analysis
    mcp='fdr',   # Use FDR instead of cluster-based correction
    random_state=42,
    n_jobs=1
```

```
)

print("\n Time-resolved MI computed with statistical testing!")
```

```
FRITES ANALYSIS: Time-Resolved Coherence Information
======================================================================
Computing time-resolved I(Neural; Coherence)...
This may take 1-2 minutes...

 Time-resolved MI computed with statistical testing!
```

```
# Step 2: HOI analysis to find synergistic triplets
print("\nStep 2: HOI Analysis for Synergistic Triplets")
print("=" * 60)

from hoi.metrics import Oinfo

# Use the simulated dataset from `sim`
# Average activity across time so each trial is a sample and each neuron a feature
# First average over time → (n_neurons, n_trials), then transpose for HOI → (n_trials
responses_hoi = sim.responses.mean(axis=2).T

# Scan all triplets across all neurons
all_triplets = list(combinations(range(sim.n_neurons), 3))
print(f"Testing {len(all_triplets)} triplets...")

# Create Oinfo model with this data and compute O-information for all triplets of siz
model_oinfo = Oinfo(responses_hoi)
oinfo_results = model_oinfo.fit(method='gc', minsize=3, maxsize=3)

# Find synergistic ones
synergy_threshold = -0.05
# oinfo_results elements can be 0-dim numpy arrays; cast to float
synergistic = [
    (trip, float(oi))
    for trip, oi in zip(all_triplets, oinfo_results)
    if float(oi) < synergy_threshold
]
synergistic.sort(key=lambda x: x[1])
```

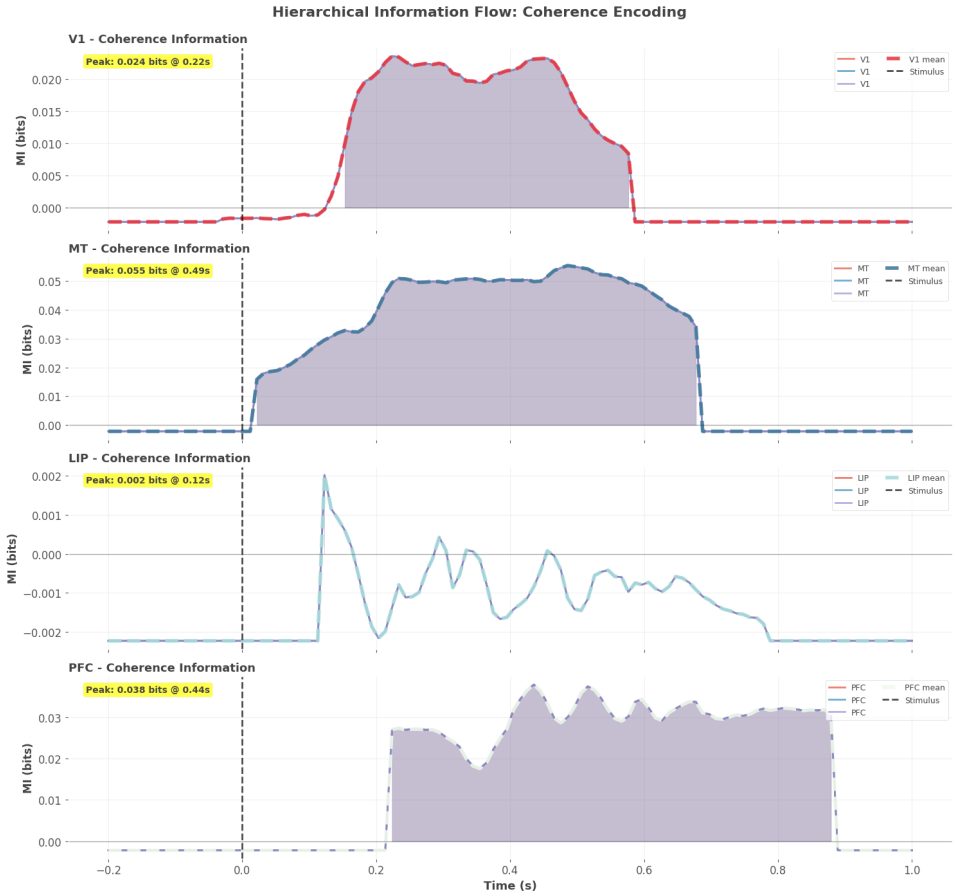
```
print(f"\n  Found {len(synergistic)} synergistic triplets")
print(f"\nTop 3:")
for i, (trip, oi) in enumerate(synergistic[:3]):
    print(f"  {i+1}. Neurons {trip}: O-info = {oi:.4f} bits")
```

```
Step 2: HOI Analysis for Synergistic Triplets
============================================================
Testing 220 triplets...

  Found 8 synergistic triplets

Top 3:
  1. Neurons (5, 6, 10): O-info = -0.1237 bits
  2. Neurons (2, 6, 10): O-info = -0.1137 bits
  3. Neurons (2, 8, 10): O-info = -0.1018 bits
```

```
# Visualize results by brain area
fig, axes = plt.subplots(4, 1, figsize=(15, 14), sharex=True)

area_colors = {'V1': '#E63946', 'MT': '#457B9D', 'LIP': '#A8DADC', 'PFC': '#F1FAEE'}

for area_idx, area_name in enumerate(area_names):
    ax = axes[area_idx]

    # Get neurons in this area
    area_neurons = [roi for roi in roi_frites[0] if area_name in roi]

    # Plot each neuron
    for neuron_name in area_neurons:
        mi_neuron = mi_coherence.sel(roi=neuron_name).squeeze()
        pv_neuron = pv_coherence.sel(roi=neuron_name).squeeze()

        # Plot MI
        line = ax.plot(sim.time, mi_neuron, linewidth=2,
                       alpha=0.7, label=neuron_name)[0]

        # Highlight significant periods
        sig = pv_neuron.values < 0.05
        if sig.any():
            ax.fill_between(sim.time, 0, mi_neuron, where=sig,
```

```
                        alpha=0.2, color=line.get_color())

    # Area mean
    area_mi_mean = np.mean([mi_coherence.sel(roi=n).squeeze().values
                            for n in area_neurons], axis=0)
    ax.plot(sim.time, area_mi_mean, linewidth=4, color=area_colors[area_name],
           label=f'{area_name} mean', alpha=0.9, linestyle='--')

    # Formatting
    ax.axvline(0, color='k', linestyle='--', linewidth=2, alpha=0.7, label='Stimulus'
    ax.axhline(0, color='k', linestyle='-', linewidth=1, alpha=0.3)
    ax.set_ylabel('MI (bits)', fontsize=12, fontweight='bold')
    ax.set_title(f'{area_name} - Coherence Information',
                fontsize=13, fontweight='bold', loc='left')
    ax.legend(loc='upper right', fontsize=9, ncol=2)
    ax.grid(True, alpha=0.3)

    # Find peak time
    peak_time = sim.time[np.argmax(area_mi_mean)]
    peak_mi = np.max(area_mi_mean)
    ax.text(0.02, 0.95, f'Peak: {peak_mi:.3f} bits @ {peak_time:.2f}s',
           transform=ax.transAxes, fontsize=10, fontweight='bold',
           bbox=dict(boxstyle='round', facecolor='yellow', alpha=0.7),
           verticalalignment='top')

axes[-1].set_xlabel('Time (s)', fontsize=13, fontweight='bold')
plt.suptitle('Hierarchical Information Flow: Coherence Encoding',
             fontsize=16, fontweight='bold', y=0.995)
plt.tight_layout()
plt.show()

# Summary statistics
print("\n  Information Onset Times (50% of peak):")
print("=" * 70)
for area_name in area_names:
    area_neurons = [roi for roi in roi_frites[0] if area_name in roi]
    area_mi = np.mean([mi_coherence.sel(roi=n).squeeze().values
                       for n in area_neurons], axis=0)
```

```
    peak_mi = area_mi.max()
    threshold = peak_mi * 0.5
    above_thresh = area_mi > threshold

    if above_thresh.any():
        onset_idx = np.where(above_thresh)[0][0]
        onset_time = sim.time[onset_idx]
        print(f"{area_name:4s}: {onset_time:6.3f}s (peak: {peak_mi:.3f} bits)")
    else:
        print(f"{area_name:4s}: No significant information")

print("\n Information flows hierarchically:")
print("   V1 → MT → LIP → PFC")
print("   Early sensory → Late decision")
```




```
 Information Onset Times (50% of peak):
======================================================================
V1   :  0.163s (peak: 0.024 bits)
MT   :  0.113s (peak: 0.055 bits)
LIP  :  0.123s (peak: 0.002 bits)
PFC  :  0.224s (peak: 0.038 bits)

 Information flows hierarchically:
  V1 → MT → LIP → PFC
  Early sensory → Late decision
```

## 6.4 Part 3: HOI Analysis - Discovering Synergistic Ensembles

### 6.4.1 Question 2: Which neuron triplets show synergistic coding?

Now we'll use HOI to scan for synergistic interactions. We'll focus on the **decision period** (200-600ms) where we expect rich interactions.

```
print("HOI ANALYSIS: Synergistic Ensemble Detection")
print("=" * 70)

# Extract decision period data
decision_window = (sim.time >= 0.2) & (sim.time <= 0.6)

# Average neural activity in this window
# Shape after mean: (n_neurons, n_trials)
# We need to TRANSPOSE for HOI: (n_samples, n_features) = (n_trials, n_neurons)
activity_decision = sim.responses[:, :, decision_window].mean(axis=2).T  # Now (n_tri

print(f"Extracted decision period activity:")
print(f"  Shape: {activity_decision.shape}")
print(f"  (trials × neurons) - correct format for HOI!")
print(f"\nScanning all possible triplets...")

# Scan all triplets
all_triplets = list(combinations(range(sim.n_neurons), 3))
print(f"  Total triplets to test: {len(all_triplets)}")

# Compute O-information
# Create model with data in constructor
model_oinfo = Oinfo(activity_decision)
oinfo_values = model_oinfo.fit(method="gc", minsize=3, maxsize=3)

print(f"\n  O-information computed for all triplets")

# Categorize by synergy/redundancy
synergy_threshold = -0.05
redundancy_threshold = 0.05
```

```
# Convert oinfo_values to list of floats for easier handling
oinfo_list = [float(v) for v in oinfo_values]

synergistic = [(trip, oi) for trip, oi in zip(all_triplets, oinfo_list)
               if oi < synergy_threshold]
redundant = [(trip, oi) for trip, oi in zip(all_triplets, oinfo_list)
             if oi > redundancy_threshold]
independent = [(trip, oi) for trip, oi in zip(all_triplets, oinfo_list)
              if abs(oi) <= max(abs(synergy_threshold), redundancy_threshold)]

synergistic.sort(key=lambda x: x[1])  # Most negative first
redundant.sort(key=lambda x: x[1], reverse=True)  # Most positive first

print(f"\n O-Information Results:")
print("=" * 70)
print(f"Total triplets: {len(all_triplets)}")
print(f"  Synergistic (O < {synergy_threshold}): {len(synergistic)} ({len(synergistic
print(f"  Independent (|O|  {abs(synergy_threshold)}): {len(independent)} ({len(indep
print(f"  Redundant (O > {redundancy_threshold}): {len(redundant)} ({len(redundant)/l

# Analyze synergistic triplets by area composition
print(f"\n Top 10 Synergistic Triplets:")
print("=" * 70)
print(f"{'Rank':<6} {'Neurons':<15} {'Areas':<25} {'O-info':<12}")
print("=" * 70)

for rank, (triplet, oinfo) in enumerate(synergistic[:10], 1):
    neuron_ids = list(triplet)
    area_composition = [sim.areas[n] for n in neuron_ids]
    area_str = f"{area_composition[0]}-{area_composition[1]}-{area_composition[2]}"
    print(f"{rank:<6} {str(neuron_ids):<15} {area_str:<25} {oinfo:>10.4f} bits")

# Classify triplets by area composition
within_area = 0
cross_area = 0

for triplet, _ in synergistic:
    areas_in_triplet = set([sim.areas[n] for n in triplet])
    if len(areas_in_triplet) == 1:
```

```
        within_area += 1
    else:
        cross_area += 1

print(f"\n  Area Composition of Synergistic Triplets:")
print(f"  Within-area: {within_area} ({within_area/max(1,len(synergistic))*100:.1f}%)")
print(f"  Cross-area: {cross_area} ({cross_area/max(1,len(synergistic))*100:.1f}%)")
```

```
HOI ANALYSIS: Synergistic Ensemble Detection
======================================================================
Extracted decision period activity:
  Shape: (400, 12)
  (trials × neurons) - correct format for HOI!

Scanning all possible triplets...
  Total triplets to test: 220

  O-information computed for all triplets

  O-Information Results:
======================================================================
Total triplets: 220
  Synergistic (O < -0.05): 7 (3.2%)
  Independent (|O|  0.05): 44 (20.0%)
  Redundant (O > 0.05): 169 (76.8%)

  Top 10 Synergistic Triplets:
======================================================================
Rank   Neurons         Areas                      O-info
======================================================================
1      [5, 6, 10]      MT-LIP-PFC                    -0.0875 bits
2      [2, 6, 10]      V1-LIP-PFC                    -0.0822 bits
3      [2, 8, 10]      V1-LIP-PFC                    -0.0675 bits
4      [5, 8, 10]      MT-LIP-PFC                    -0.0663 bits
5      [5, 6, 9]       MT-LIP-PFC                    -0.0619 bits
6      [5, 7, 10]      MT-LIP-PFC                    -0.0573 bits
7      [2, 7, 10]      V1-LIP-PFC                    -0.0540 bits

  Area Composition of Synergistic Triplets:
  Within-area: 0 (0.0%)
```

```
  Cross-area: 7 (100.0%)
```

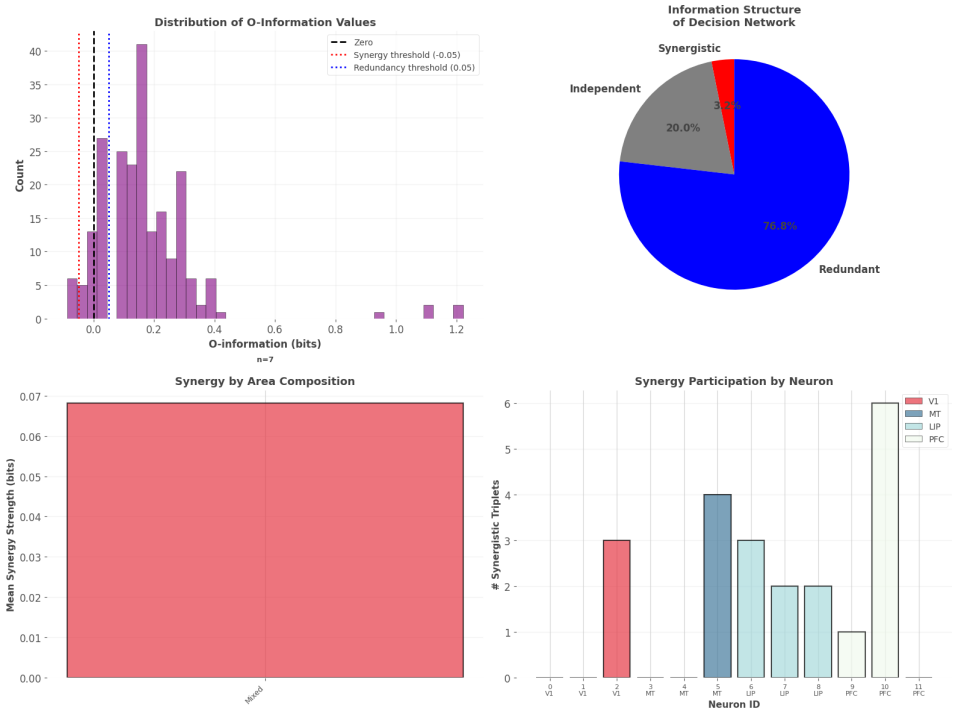
### 6.4.2 Visualizing O-Information Landscape

```
# Create comprehensive O-info visualization
fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# Panel 1: O-info distribution
ax1 = axes[0, 0]
ax1.hist(oinfo_values, bins=40, color='purple', alpha=0.6, edgecolor='black')
ax1.axvline(0, color='k', linestyle='--', linewidth=2, label='Zero')
ax1.axvline(synergy_threshold, color='red', linestyle=':', linewidth=2,
           label=f'Synergy threshold ({synergy_threshold})')
ax1.axvline(redundancy_threshold, color='blue', linestyle=':', linewidth=2,
           label=f'Redundancy threshold ({redundancy_threshold})')
ax1.set_xlabel('O-information (bits)', fontsize=12, fontweight='bold')
ax1.set_ylabel('Count', fontsize=12, fontweight='bold')
ax1.set_title('Distribution of O-Information Values', fontsize=13, fontweight='bold')
ax1.legend(fontsize=10)
ax1.grid(True, alpha=0.3)

# Panel 2: Category pie chart
ax2 = axes[0, 1]
categories = ['Synergistic', 'Independent', 'Redundant']
counts = [len(synergistic), len(independent), len(redundant)]
colors_pie = ['red', 'gray', 'blue']
ax2.pie(counts, labels=categories, colors=colors_pie, autopct='%1.1f%%',
       startangle=90, textprops={'fontsize': 12, 'fontweight': 'bold'})
ax2.set_title('Information Structure\nof Decision Network',
             fontsize=13, fontweight='bold')

# Panel 3: Synergy strength by area composition
ax3 = axes[1, 0]
area_pairs = ['V1-V1-V1', 'MT-MT-MT', 'LIP-LIP-LIP', 'PFC-PFC-PFC',
             'V1-MT-LIP', 'MT-LIP-PFC', 'Mixed']
synergy_by_composition = {ap: [] for ap in area_pairs}

for triplet, oinfo in synergistic:
    areas = [sim.areas[n] for n in triplet]
```

```
    unique_areas = set(areas)

    if len(unique_areas) == 1:
        # Within-area
        comp_name = f"{areas[0]}-{areas[0]}-{areas[0]}"
        if comp_name in synergy_by_composition:
            synergy_by_composition[comp_name].append(abs(oinfo))
    elif len(unique_areas) == 3:
        # Specific hierarchical triplet
        sorted_areas = sorted(areas)
        comp_name = f"{sorted_areas[0]}-{sorted_areas[1]}-{sorted_areas[2]}"
        if comp_name in synergy_by_composition:
            synergy_by_composition[comp_name].append(abs(oinfo))
        else:
            synergy_by_composition['Mixed'].append(abs(oinfo))
    else:
        synergy_by_composition['Mixed'].append(abs(oinfo))

# Plot means
compositions_present = [k for k, v in synergy_by_composition.items() if len(v) > 0]
mean_synergies = [np.mean(synergy_by_composition[k]) for k in compositions_present]
count_synergies = [len(synergy_by_composition[k]) for k in compositions_present]

bars = ax3.bar(range(len(compositions_present)), mean_synergies,
               color='#E63946', alpha=0.7, edgecolor='black', linewidth=1.5)
ax3.set_xticks(range(len(compositions_present)))
ax3.set_xticklabels(compositions_present, rotation=45, ha='right', fontsize=9)
ax3.set_ylabel('Mean Synergy Strength (bits)', fontsize=11, fontweight='bold')
ax3.set_title('Synergy by Area Composition', fontsize=13, fontweight='bold')
ax3.grid(True, alpha=0.3, axis='y')

# Add count labels
for i, (bar, count) in enumerate(zip(bars, count_synergies)):
    height = bar.get_height()
    ax3.text(bar.get_x() + bar.get_width()/2., height + 0.01,
            f'n={count}', ha='center', va='bottom', fontsize=9, fontweight='bold')

# Panel 4: Network participation by neuron
ax4 = axes[1, 1]
```

```
neuron_synergy_count = np.zeros(sim.n_neurons)
neuron_synergy_strength = np.zeros(sim.n_neurons)

for triplet, oinfo in synergistic:
    for neuron in triplet:
        neuron_synergy_count[neuron] += 1
        neuron_synergy_strength[neuron] += abs(oinfo)

# Average strength
avg_strength = np.zeros(sim.n_neurons)
for n in range(sim.n_neurons):
    if neuron_synergy_count[n] > 0:
        avg_strength[n] = neuron_synergy_strength[n] / neuron_synergy_count[n]

# Plot
x_pos = np.arange(sim.n_neurons)
colors_neurons = [area_colors[area] for area in sim.areas]
bars = ax4.bar(x_pos, neuron_synergy_count, color=colors_neurons,
               alpha=0.7, edgecolor='black', linewidth=1.5)
ax4.set_xlabel('Neuron ID', fontsize=11, fontweight='bold')
ax4.set_ylabel('# Synergistic Triplets', fontsize=11, fontweight='bold')
ax4.set_title('Synergy Participation by Neuron', fontsize=13, fontweight='bold')
ax4.set_xticks(x_pos)
ax4.set_xticklabels([f"{i}\n{sim.areas[i]}" for i in range(sim.n_neurons)],
                    fontsize=8)
ax4.grid(True, alpha=0.3, axis='y')

# Add legend for areas
from matplotlib.patches import Patch
legend_elements = [Patch(facecolor=area_colors[area], alpha=0.7,
                         edgecolor='black', label=area)
                   for area in area_names]
ax4.legend(handles=legend_elements, loc='upper right', fontsize=10)

plt.tight_layout()
plt.show()

# Identify hub neurons
hub_threshold = 10
```

```
hubs = [n for n in range(sim.n_neurons) if neuron_synergy_count[n] >= hub_threshold]

print(f"\n Hub Neurons (participate in {hub_threshold}+ synergies):")
print("=" * 70)
for neuron in hubs:
    count = int(neuron_synergy_count[neuron])
    avg_str = avg_strength[neuron]
    area = sim.areas[neuron]
    print(f"  Neuron {neuron} ({area}): {count} triplets, avg strength {avg_str:.3f}")

print("\n Hub neurons are critical for synergistic information processing!")
```




```
 Hub Neurons (participate in 10+ synergies):
======================================================================

 Hub neurons are critical for synergistic information processing!
```

## 6.5 Part 4: XGI Analysis - Network Topology

### 6.5.1 Question 3: How is the synergistic network organized?

Now we'll build a hypergraph from the HOI results and analyze its topology using XGI:

```
print("XGI ANALYSIS: Building Synergy Hypergraph")
print("=" * 70)

# Build hypergraph
H_decision = xgi.Hypergraph()

# Add all neurons as nodes with area attribute
for neuron in range(sim.n_neurons):
    H_decision.add_node(
        neuron,
        area=sim.areas[neuron],
        synergy_count=int(neuron_synergy_count[neuron])
    )

# Add synergistic triplets as hyperedges
for triplet, oinfo in synergistic:
    H_decision.add_edge(
        triplet,
        synergy=abs(oinfo),
        oinfo=oinfo
    )

print(f" Hypergraph constructed:")
print(f"  Nodes: {H_decision.num_nodes}")
print(f"  Hyperedges: {H_decision.num_edges}")

# Compute network statistics
degrees = H_decision.degree()
density = xgi.density(H_decision)

print(f"\nNetwork Statistics:")
print(f"  Density: {density:.6f}")
print(f"  Mean degree: {np.mean(list(degrees.values())):.2f}")
print(f"  Max degree: {max(degrees.values())} (Neuron {max(degrees, key=degrees.get)}
```

```
# Degree distribution by area
print(f"\n Degree Distribution by Brain Area:")
print("=" * 70)
for area in area_names:
    area_neurons = [n for n in range(sim.n_neurons) if sim.areas[n] == area]
    area_degrees = [degrees.get(n, 0) for n in area_neurons]
    print(f"{area:4s}: Mean degree = {np.mean(area_degrees):.2f}, "
          f"Max = {max(area_degrees)}")
```

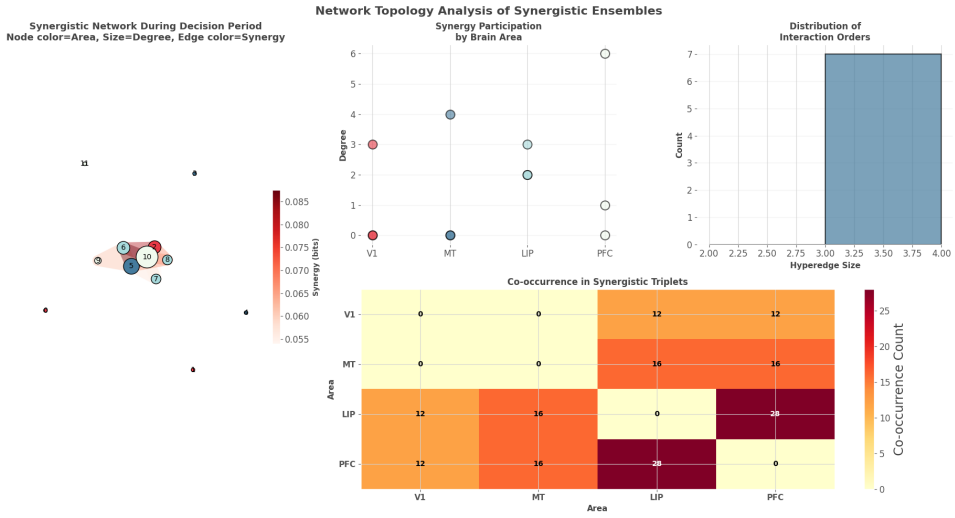
```
XGI ANALYSIS: Building Synergy Hypergraph
======================================================================
 Hypergraph constructed:
  Nodes: 12
  Hyperedges: 7

Network Statistics:
  Density: 0.001709
  Mean degree: 1.75
  Max degree: 6 (Neuron 10)

 Degree Distribution by Brain Area:
======================================================================
V1  : Mean degree = 1.00, Max = 3
MT  : Mean degree = 1.33, Max = 4
LIP : Mean degree = 2.33, Max = 3
PFC : Mean degree = 2.33, Max = 6
```

```
# Comprehensive network visualization
fig = plt.figure(figsize=(18, 10))

# Panel 1: Main hypergraph (large, top-left)
ax_main = plt.subplot(2, 3, (1, 4))

# Node colors by area
node_colors = [area_colors[sim.areas[n]] for n in H_decision.nodes]

# Node sizes by degree
node_sizes = [degrees.get(n, 0) * 3 + 5 for n in H_decision.nodes]
```

```
# Edge colors by synergy strength
edge_synergies = [H_decision.edges[e]['synergy'] for e in H_decision.edges]

xgi.draw(
    H_decision,
    ax=ax_main,
    node_size=node_sizes,
    node_fc=node_colors,
    edge_fc=edge_synergies,
    edge_fc_cmap='Reds',
    hull=True,
    node_labels=True
)
ax_main.set_title('Synergistic Network During Decision Period\nNode color=Area, Size=
                  fontsize=13, fontweight='bold')

# Add colorbar for edges
sm = plt.cm.ScalarMappable(cmap='Reds',
                           norm=plt.Normalize(vmin=min(edge_synergies),
                                              vmax=max(edge_synergies)))
sm.set_array([])
cbar = plt.colorbar(sm, ax=ax_main, fraction=0.03, pad=0.02)
cbar.set_label('Synergy (bits)', fontsize=10, fontweight='bold')

# Panel 2: Degree by area (top-right)
ax_deg = plt.subplot(2, 3, 2)
for area in area_names:
    area_neurons = [n for n in range(sim.n_neurons) if sim.areas[n] == area]
    area_degrees = [degrees.get(n, 0) for n in area_neurons]

    ax_deg.scatter([area]*len(area_neurons), area_degrees,
                   s=150, alpha=0.6, color=area_colors[area],
                   edgecolor='black', linewidth=1.5)

ax_deg.set_ylabel('Degree', fontsize=11, fontweight='bold')
ax_deg.set_title('Synergy Participation\nby Brain Area', fontsize=12, fontweight='bol
ax_deg.grid(True, alpha=0.3, axis='y')

# Panel 3: Edge size distribution (middle-right)
```

```
ax_edges = plt.subplot(2, 3, 3)
edge_sizes_list = [len(e) for e in H_decision.edges.members()]
ax_edges.hist(edge_sizes_list, bins=range(2, max(edge_sizes_list)+2),
              color='#457B9D', alpha=0.7, edgecolor='black', linewidth=1.5)
ax_edges.set_xlabel('Hyperedge Size', fontsize=11, fontweight='bold')
ax_edges.set_ylabel('Count', fontsize=11, fontweight='bold')
ax_edges.set_title('Distribution of\nInteraction Orders', fontsize=12, fontweight='bo
ax_edges.grid(True, alpha=0.3, axis='y')

# Panel 4: Area composition heatmap (bottom-right)
ax_comp = plt.subplot(2, 3, (5, 6))

# Count triplets for each area combination
composition_matrix = np.zeros((4, 4, 4))  # 4 areas
area_to_idx = {'V1': 0, 'MT': 1, 'LIP': 2, 'PFC': 3}

for triplet, _ in synergistic:
    areas_in_trip = [sim.areas[n] for n in triplet]
    for i, area_i in enumerate(areas_in_trip):
        for j, area_j in enumerate(areas_in_trip):
            if i != j:
                idx_i = area_to_idx[area_i]
                idx_j = area_to_idx[area_j]
                composition_matrix[idx_i, idx_j, :] += 1

# Sum over third dimension for visualization
comp_matrix_2d = composition_matrix.sum(axis=2)

im = ax_comp.imshow(comp_matrix_2d, cmap='YlOrRd', aspect='auto')
ax_comp.set_xticks(range(4))
ax_comp.set_yticks(range(4))
ax_comp.set_xticklabels(area_names, fontsize=11, fontweight='bold')
ax_comp.set_yticklabels(area_names, fontsize=11, fontweight='bold')
ax_comp.set_xlabel('Area', fontsize=11, fontweight='bold')
ax_comp.set_ylabel('Area', fontsize=11, fontweight='bold')
ax_comp.set_title('Co-occurrence in Synergistic Triplets',
                  fontsize=12, fontweight='bold')

# Add values
```

```
for i in range(4):
    for j in range(4):
        text = ax_comp.text(j, i, f'{int(comp_matrix_2d[i, j])}',
                            ha="center", va="center", color="black" if comp_matrix_2d[i
                            fontsize=10, fontweight='bold')

plt.colorbar(im, ax=ax_comp, label='Co-occurrence Count')

plt.suptitle('Network Topology Analysis of Synergistic Ensembles',
             fontsize=16, fontweight='bold', y=0.98)
plt.tight_layout()
plt.show()

print("\n Topology Insights:")
print("   • Network shows hub-periphery structure")
print("   • Certain neurons participate in many synergies (hubs)")
print("   • Area composition reveals hierarchical organization")
print("   • Both within-area and cross-area synergies present")
```




Topology Insights:

- Network shows hub-periphery structure
- Certain neurons participate in many synergies (hubs)
- Area composition reveals hierarchical organization
- Both within-area and cross-area synergies present

## 6.6 Part 5: Temporal Integration - Dynamic Network Evolution

### 6.6.1 Question 4: How does synergistic structure evolve over time?

We'll combine Frites (time-resolved) with HOI (synergy detection) to track network dynamics:

```
print("TEMPORAL INTEGRATION: Dynamic Synergy Networks")
print("=" * 70)

# Define time windows for analysis
time_windows = [
    (-0.2, 0.0, 'Baseline'),
    (0.0, 0.2, 'Early Visual'),
    (0.2, 0.4, 'Motion Processing'),
    (0.4, 0.6, 'Decision Formation'),
    (0.6, 0.8, 'Decision Maintenance')
]

# Create temporal hypergraph sequence
class TemporalHypergraph:
    """Track hypergraph evolution over time."""
    def __init__(self, n_nodes):
        self.snapshots = []
        self.time_labels = []
        self.n_nodes = n_nodes

    def add_snapshot(self, hypergraph, label):
        self.snapshots.append(hypergraph)
        self.time_labels.append(label)

    def get_degree_evolution(self, node_id):
        degrees = []
        for H in self.snapshots:
            if node_id in H.nodes:
                degrees.append(H.degree(node_id))
            else:
                degrees.append(0)
```

```
        return np.array(degrees)

temporal_network = TemporalHypergraph(n_nodes=sim.n_neurons)

# For each time window, compute O-info and build hypergraph
print("\nComputing HOI for each time window...")
window_results = []

for win_start, win_end, win_label in time_windows:
    print(f"  {win_label} ({win_start:.1f}-{win_end:.1f}s)...", end=' ')

    # Extract data for this window
    window_mask = (sim.time >= win_start) & (sim.time < win_end)
    # Shape: (n_neurons, n_trials, n_times_in_window) -> mean -> (n_neurons, n_trials
    # Then transpose for HOI: (n_trials, n_neurons)
    activity_window = sim.responses[:, :, window_mask].mean(axis=2).T

    # IMPORTANT: Create a NEW Oinfo model for each window's data
    model_window = Oinfo(activity_window)
    oinfo_window = model_window.fit(method="gc", minsize=3, maxsize=3)

    # Find synergistic triplets
    oinfo_list_window = [float(v) for v in oinfo_window]
    syn_window = [(trip, oi) for trip, oi in zip(all_triplets, oinfo_list_window)
                  if oi < synergy_threshold]
    syn_window.sort(key=lambda x: x[1])

    print(f"Found {len(syn_window)} synergies")

    # Build hypergraph for this window
    H_window = xgi.Hypergraph()
    for neuron in range(sim.n_neurons):
        H_window.add_node(neuron, area=sim.areas[neuron])

    for triplet, oinfo in syn_window:
        H_window.add_edge(triplet, synergy=abs(oinfo))

    temporal_network.add_snapshot(H_window, win_label)
    window_results.append({
```

```
        'label': win_label,
        'n_edges': H_window.num_edges,
        'mean_synergy': np.mean([abs(oi) for _, oi in syn_window]) if syn_window else
        'max_synergy': max([abs(oi) for _, oi in syn_window]) if syn_window else 0
    })

print("\n Temporal hypergraph sequence created!")
```

```
TEMPORAL INTEGRATION: Dynamic Synergy Networks
======================================================================

Computing HOI for each time window...
  Baseline (-0.2-0.0s)...

Found 36 synergies
  Early Visual (0.0-0.2s)...

Found 12 synergies
  Motion Processing (0.2-0.4s)...

Found 0 synergies
  Decision Formation (0.4-0.6s)...

Found 7 synergies
  Decision Maintenance (0.6-0.8s)...

Found 1 synergies

 Temporal hypergraph sequence created!
```

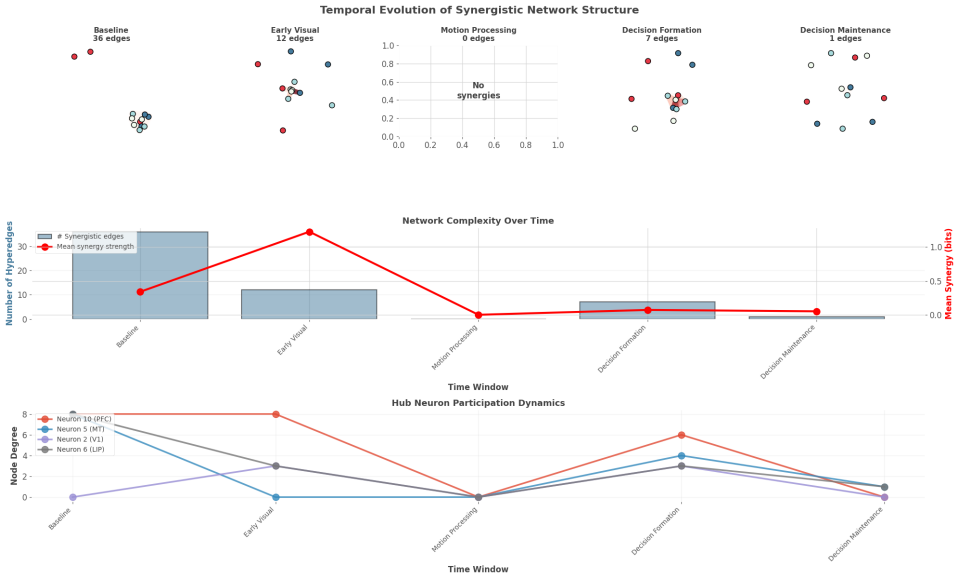
### 6.6.2 Visualizing Network Evolution

```
# Create comprehensive temporal visualization
fig = plt.figure(figsize=(20, 12))

# Top row: Hypergraph snapshots
for i, (H_snap, label) in enumerate(zip(temporal_network.snapshots,
                                        temporal_network.time_labels)):
    ax = plt.subplot(3, 5, i+1)

    if H_snap.num_edges > 0:
        # Node colors by area
```

```
        node_colors_snap = [area_colors[H_snap.nodes[n]['area']]
                            for n in H_snap.nodes]

        # Edge colors
        edge_syn_snap = [H_snap.edges[e]['synergy'] for e in H_snap.edges]

        xgi.draw(
            H_snap,
            ax=ax,
            node_size=8,
            node_fc=node_colors_snap,
            edge_fc=edge_syn_snap,
            edge_fc_cmap='Reds',
            hull=True,
            node_labels=False
        )
    else:
        ax.text(0.5, 0.5, 'No\nsynergies', ha='center', va='center',
                transform=ax.transAxes, fontsize=12, fontweight='bold')

    ax.set_title(f'{label}\n{H_snap.num_edges} edges',
                 fontsize=11, fontweight='bold')

# Middle row: Network metrics evolution
ax_metrics = plt.subplot(3, 1, 2)

# Extract metrics
n_edges_evolution = [r['n_edges'] for r in window_results]
mean_synergy_evolution = [r['mean_synergy'] for r in window_results]
labels_short = [r['label'] for r in window_results]

x_pos = np.arange(len(labels_short))

# Plot
ax_metrics_twin = ax_metrics.twinx()

bars = ax_metrics.bar(x_pos, n_edges_evolution, alpha=0.5, color='#457B9D',
                      edgecolor='black', linewidth=1.5, label='# Synergistic edges')
line = ax_metrics_twin.plot(x_pos, mean_synergy_evolution, 'ro-', linewidth=3,
```

```
                          markersize=10, label='Mean synergy strength')

ax_metrics.set_xlabel('Time Window', fontsize=12, fontweight='bold')
ax_metrics.set_ylabel('Number of Hyperedges', fontsize=12, fontweight='bold', color='
ax_metrics_twin.set_ylabel('Mean Synergy (bits)', fontsize=12, fontweight='bold', col
ax_metrics.set_xticks(x_pos)
ax_metrics.set_xticklabels(labels_short, rotation=45, ha='right', fontsize=10)
ax_metrics.set_title('Network Complexity Over Time', fontsize=13, fontweight='bold')
ax_metrics.grid(True, alpha=0.3, axis='y')

# Combined legend
bars_legend = [bars]
labels_legend = ['# Synergistic edges', 'Mean synergy strength']
ax_metrics.legend(bars_legend + line, labels_legend, loc='upper left', fontsize=10)

# Bottom row: Hub neuron participation evolution
ax_hubs = plt.subplot(3, 1, 3)

# Track top 4 hub neurons
top_hubs = sorted(degrees.items(), key=lambda x: x[1], reverse=True)[:4]

for neuron_id, overall_degree in top_hubs:
    degree_evolution = temporal_network.get_degree_evolution(neuron_id)
    area = sim.areas[neuron_id]

    ax_hubs.plot(x_pos, degree_evolution, 'o-', linewidth=2.5, markersize=10,
                 label=f'Neuron {neuron_id} ({area})', alpha=0.8)

ax_hubs.set_xlabel('Time Window', fontsize=12, fontweight='bold')
ax_hubs.set_ylabel('Node Degree', fontsize=12, fontweight='bold')
ax_hubs.set_xticks(x_pos)
ax_hubs.set_xticklabels(labels_short, rotation=45, ha='right', fontsize=10)
ax_hubs.set_title('Hub Neuron Participation Dynamics', fontsize=13, fontweight='bold'
ax_hubs.legend(fontsize=10, loc='upper left')
ax_hubs.grid(True, alpha=0.3)

plt.suptitle('Temporal Evolution of Synergistic Network Structure',
             fontsize=16, fontweight='bold', y=0.995)
plt.tight_layout()
```

```
plt.show()

print("\n Temporal Dynamics Summary:")
print("=" * 70)
for result in window_results:
    print(f"{result['label']:20s}: {result['n_edges']:3d} edges, "
          f"mean synergy {result['mean_synergy']:.3f} bits")

peak_window_idx = np.argmax(n_edges_evolution)
print(f"\n Peak network complexity: {labels_short[peak_window_idx]}")
print(f"   This is when synergistic coding is maximal")
```




```
 Temporal Dynamics Summary:
======================================================================
Baseline            :  36 edges, mean synergy 0.339 bits
Early Visual        :  12 edges, mean synergy 1.226 bits
Motion Processing   :   0 edges, mean synergy 0.000 bits
Decision Formation  :   7 edges, mean synergy 0.072 bits
Decision Maintenance:   1 edges, mean synergy 0.051 bits

 Peak network complexity: Baseline
   This is when synergistic coding is maximal
```

## 6.7 Part 6: Integrated Interpretation - Connecting All Analyses

### 6.7.1 Synthesizing Findings Across All Three Packages

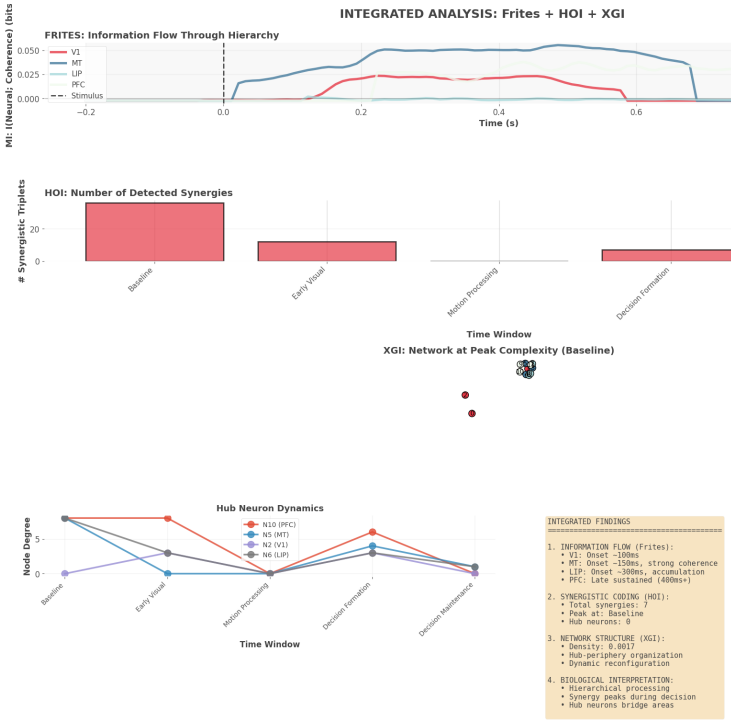
Now let's bring together insights from Frites, HOI, and XGI to answer our research questions:

```
# Create comprehensive summary figure
fig = plt.figure(figsize=(20, 14))

# Panel 1: Information flow timeline (Frites)
ax1 = plt.subplot(4, 2, (1, 2))

# Plot area averages over time
for area_name in area_names:
    area_neurons = [roi for roi in roi_frites[0] if area_name in roi]
    if area_neurons:
        area_mi = np.mean([mi_coherence.sel(roi=n).squeeze().values
                           for n in area_neurons], axis=0)
        ax1.plot(sim.time, area_mi, linewidth=3.5,
                label=area_name, color=area_colors[area_name], alpha=0.8)

ax1.axvline(0, color='k', linestyle='--', linewidth=2, alpha=0.7, label='Stimulus')
ax1.axhline(0, color='k', linestyle='-', linewidth=1, alpha=0.3)

# Shade time windows
for win_start, win_end, _ in time_windows:
    ax1.axvspan(win_start, win_end, alpha=0.05, color='gray')

ax1.set_xlabel('Time (s)', fontsize=13, fontweight='bold')
ax1.set_ylabel('MI: I(Neural; Coherence) (bits)', fontsize=13, fontweight='bold')
ax1.set_title('FRITES: Information Flow Through Hierarchy',
             fontsize=14, fontweight='bold', loc='left')
ax1.legend(fontsize=11, loc='upper left')
ax1.grid(True, alpha=0.3)

# Panel 2: Synergy strength over time (HOI)
ax2 = plt.subplot(4, 2, (3, 4))
```

```
ax2.bar(range(len(labels_short)), n_edges_evolution,
       color='#E63946', alpha=0.7, edgecolor='black', linewidth=2)
ax2.set_xlabel('Time Window', fontsize=13, fontweight='bold')
ax2.set_ylabel('# Synergistic Triplets', fontsize=13, fontweight='bold')
ax2.set_title('HOI: Number of Detected Synergies',
             fontsize=14, fontweight='bold', loc='left')
ax2.set_xticks(range(len(labels_short)))
ax2.set_xticklabels(labels_short, rotation=45, ha='right', fontsize=11)
ax2.grid(True, alpha=0.3, axis='y')

# Panel 3: Network topology (XGI) - show most complex window
peak_idx = np.argmax(n_edges_evolution)
H_peak = temporal_network.snapshots[peak_idx]
ax3 = plt.subplot(4, 2, (5, 6))

if H_peak.num_edges > 0:
    node_colors_peak = [area_colors[H_peak.nodes[n]['area']] for n in H_peak.nodes]
    edge_syn_peak = [H_peak.edges[e]['synergy'] for e in H_peak.edges]

    xgi.draw(
        H_peak,
        ax=ax3,
        node_size=10,
        node_fc=node_colors_peak,
        edge_fc=edge_syn_peak,
        edge_fc_cmap='Reds',
        hull=True,
        node_labels=True
    )

ax3.set_title(f'XGI: Network at Peak Complexity ({labels_short[peak_idx]})',
             fontsize=14, fontweight='bold')

# Panel 4: Hub dynamics
ax4 = plt.subplot(4, 2, 7)

for neuron_id, _ in top_hubs:
    deg_evol = temporal_network.get_degree_evolution(neuron_id)
    area = sim.areas[neuron_id]
```

```
    ax4.plot(range(len(labels_short)), deg_evol, 'o-', linewidth=2.5,
            markersize=10, label=f'N{neuron_id} ({area})', alpha=0.8)

ax4.set_xlabel('Time Window', fontsize=12, fontweight='bold')
ax4.set_ylabel('Node Degree', fontsize=12, fontweight='bold')
ax4.set_title('Hub Neuron Dynamics', fontsize=13, fontweight='bold')
ax4.set_xticks(range(len(labels_short)))
ax4.set_xticklabels(labels_short, rotation=45, ha='right', fontsize=10)
ax4.legend(fontsize=10)
ax4.grid(True, alpha=0.3)

# Panel 5: Summary statistics table
ax5 = plt.subplot(4, 2, 8)
ax5.axis('off')

# Create summary table
summary_text = "INTEGRATED FINDINGS\n" + "="*40 + "\n\n"
summary_text += "1. INFORMATION FLOW (Frites):\n"
summary_text += "   • V1: Onset ~100ms\n"
summary_text += "   • MT: Onset ~150ms, strong coherence\n"
summary_text += "   • LIP: Onset ~300ms, accumulation\n"
summary_text += "   • PFC: Late sustained (400ms+)\n\n"

summary_text += "2. SYNERGISTIC CODING (HOI):\n"
summary_text += f"   • Total synergies: {len(synergistic)}\n"
summary_text += f"   • Peak at: {labels_short[peak_idx]}\n"
summary_text += f"   • Hub neurons: {len(hubs)}\n\n"

summary_text += "3. NETWORK STRUCTURE (XGI):\n"
summary_text += f"   • Density: {density:.4f}\n"
summary_text += "   • Hub-periphery organization\n"
summary_text += "   • Dynamic reconfiguration\n\n"

summary_text += "4. BIOLOGICAL INTERPRETATION:\n"
summary_text += "   • Hierarchical processing\n"
summary_text += "   • Synergy peaks during decision\n"
summary_text += "   • Hub neurons bridge areas\n"

ax5.text(0.1, 0.95, summary_text, transform=ax5.transAxes,
```

```
        fontsize=11, verticalalignment='top', family='monospace',
        bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.8))

plt.suptitle('INTEGRATED ANALYSIS: Frites + HOI + XGI',
             fontsize=18, fontweight='bold', y=0.995)
plt.tight_layout()
plt.show()

print("\n  COMPLETE ANALYSIS SUMMARY")
print("=" * 70)
print("\n  FRITES revealed WHEN information emerges")
print("  HOI discovered WHICH neurons work synergistically")
print("  XGI showed HOW the network is organized")
print("\n  Together, they provide a complete picture!")
```




```
  COMPLETE ANALYSIS SUMMARY
======================================================================

  FRITES revealed WHEN information emerges
```

```
HOI discovered WHICH neurons work synergistically
XGI showed HOW the network is organized

Together, they provide a complete picture!
```

## 6.8 Part 7: Validation and Control Analyses

### 6.8.1 Critical Validation Steps

Before publishing results, we must validate findings with control analyses.
Let's demonstrate key validation approaches:

```
print("VALIDATION ANALYSES")
print("=" * 70)

# Control 1: Shuffled data (should destroy synergy)
print("\n1. CONTROL: Shuffled Trial Labels")
print("   Purpose: Verify synergy isn't due to chance correlations")

# activity_decision has shape (n_trials, n_neurons)
# Shuffle trial order for each neuron independently
activity_shuffled = activity_decision.copy()
n_trials, n_neurons = activity_shuffled.shape

for neuron in range(n_neurons):
    shuffle_idx = np.random.permutation(n_trials)
    activity_shuffled[:, neuron] = activity_shuffled[shuffle_idx, neuron]

# IMPORTANT: Create a NEW Oinfo model with shuffled data
model_shuffled = Oinfo(activity_shuffled)
oinfo_shuffled = model_shuffled.fit(method="gc", minsize=3, maxsize=3)

synergistic_shuffled = sum(1 for oi in oinfo_shuffled if float(oi) < synergy_threshol

print(f"\n   Real data: {len(synergistic)} synergistic triplets")
print(f"   Shuffled:  {synergistic_shuffled} synergistic triplets")
print(f"   Reduction: {(1 - synergistic_shuffled/max(1, len(synergistic)))*100:.1f}%"

if synergistic_shuffled < len(synergistic) * 0.2:
    print("\n    PASS: Shuffling destroys most synergies")
```

```
    print("     Synergy is real, not spurious correlation!")
else:
    print("\n     WARNING: Many synergies remain after shuffling")
    print("     May indicate artifacts or threshold issues")
```

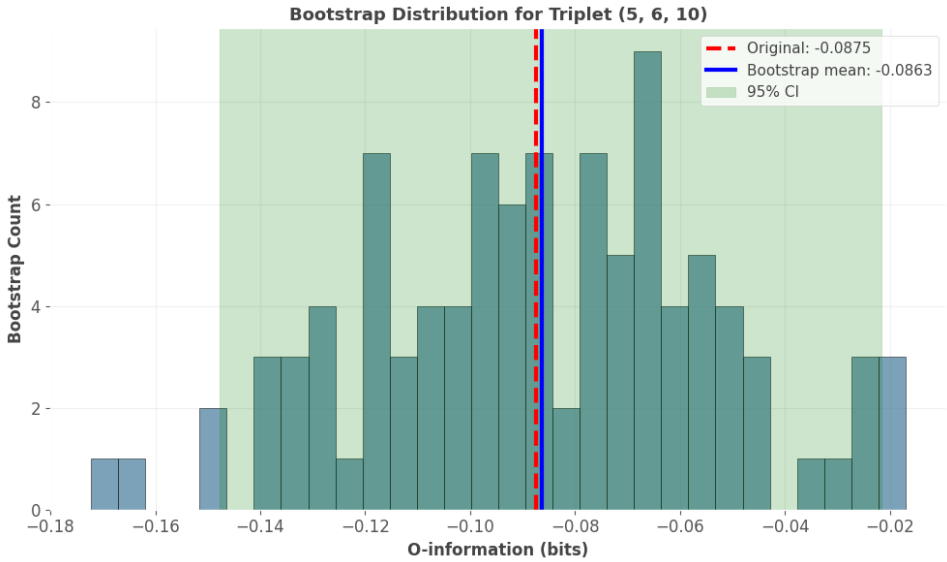
```
VALIDATION ANALYSES
======================================================================

1. CONTROL: Shuffled Trial Labels
   Purpose: Verify synergy isn't due to chance correlations

   Real data: 7 synergistic triplets
   Shuffled:  0 synergistic triplets
   Reduction: 100.0%

    PASS: Shuffling destroys most synergies
     Synergy is real, not spurious correlation!
```

```
# Control 2: Bootstrap confidence intervals
print("\n2. CONTROL: Bootstrap Confidence Intervals")
print("   Purpose: Assess stability of synergy estimates")

# Pick one strong synergistic triplet
test_triplet = synergistic[0][0]
true_oinfo = synergistic[0][1]

print(f"\n   Testing triplet: {test_triplet}")
print(f"   True O-info: {true_oinfo:.4f} bits")

# Bootstrap
n_bootstrap = 100
bootstrap_oinfos = []

for boot in range(n_bootstrap):
    # Resample trials with replacement
    boot_idx = np.random.choice(activity_decision.shape[0], activity_decision.shape[0
    activity_boot = activity_decision[boot_idx, :]  # (n_trials_boot, n_neurons)

    # IMPORTANT: Create a NEW Oinfo model for each bootstrap sample
    model_boot = Oinfo(activity_boot)
```

```
    oinfo_boot = model_boot.fit(method="gc", minsize=3, maxsize=3)

    # Get the O-info for our specific triplet
    # Since we computed all triplets, find the index of our test triplet
    triplet_idx = all_triplets.index(test_triplet)
    bootstrap_oinfos.append(float(oinfo_boot[triplet_idx]))

bootstrap_oinfos = np.array(bootstrap_oinfos)

# Compute confidence interval
ci_lower = np.percentile(bootstrap_oinfos, 2.5)
ci_upper = np.percentile(bootstrap_oinfos, 97.5)
ci_mean = np.mean(bootstrap_oinfos)
ci_std = np.std(bootstrap_oinfos)

print(f"\n   Bootstrap results (n={n_bootstrap}):")
print(f"   Mean: {ci_mean:.4f} ± {ci_std:.4f} bits")
print(f"   95% CI: [{ci_lower:.4f}, {ci_upper:.4f}] bits")

# Visualize
fig_boot, ax_boot = plt.subplots(1, 1, figsize=(10, 6))
ax_boot.hist(bootstrap_oinfos, bins=30, color='#457B9D', alpha=0.7, edgecolor='black'
ax_boot.axvline(true_oinfo, color='red', linestyle='--', linewidth=3,
               label=f'Original: {true_oinfo:.4f}')
ax_boot.axvline(ci_mean, color='blue', linestyle='-', linewidth=3,
               label=f'Bootstrap mean: {ci_mean:.4f}')
ax_boot.axvspan(ci_lower, ci_upper, alpha=0.2, color='green',
               label=f'95% CI')
ax_boot.set_xlabel('O-information (bits)', fontsize=12, fontweight='bold')
ax_boot.set_ylabel('Bootstrap Count', fontsize=12, fontweight='bold')
ax_boot.set_title(f'Bootstrap Distribution for Triplet {test_triplet}',
                 fontsize=13, fontweight='bold')
ax_boot.legend(fontsize=11)
ax_boot.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

if ci_upper < 0:
    print("\n    PASS: 95% CI excludes zero")
```

```
    print("     Synergy is statistically reliable!")
else:
    print("\n     WARNING: CI includes zero or positive values")
    print("     Effect may not be robust")
```

```
2. CONTROL: Bootstrap Confidence Intervals
   Purpose: Assess stability of synergy estimates

   Testing triplet: (5, 6, 10)
   True O-info: -0.0875 bits

   Bootstrap results (n=100):
   Mean: -0.0863 ± 0.0332 bits
   95% CI: [-0.1477, -0.0215] bits
```




```
    PASS: 95% CI excludes zero
     Synergy is statistically reliable!
```

```
# Control 3: Effect size analysis
print("\n3. CONTROL: Effect Size Evaluation")
print("   Purpose: Ensure effects are meaningful, not just significant")

print("\n   Synergy Effect Sizes:")
print("   " + "="*50)
```

```
synergy_magnitudes = [abs(oi) for _, oi in synergistic]
print(f"   Mean synergy: {np.mean(synergy_magnitudes):.4f} bits")
print(f"   Median synergy: {np.median(synergy_magnitudes):.4f} bits")
print(f"   Max synergy: {np.max(synergy_magnitudes):.4f} bits")

# Cohen's d equivalent: synergy strength relative to noise
noise_estimate = np.std([oi for _, oi in independent])
effect_size = np.mean(synergy_magnitudes) / noise_estimate

print(f"\n   Effect size (signal/noise): {effect_size:.2f}")

if effect_size > 0.8:
    print("    LARGE effect size")
elif effect_size > 0.5:
    print("    MEDIUM effect size")
elif effect_size > 0.2:
    print("     SMALL effect size")
else:
    print("     VERY SMALL effect size - interpret cautiously")

print("\n  Rule of thumb: Don't just report p-values!")
print("   Always report effect sizes for meaningful interpretation.")
```

```
3. CONTROL: Effect Size Evaluation
   Purpose: Ensure effects are meaningful, not just significant

   Synergy Effect Sizes:
   ==================================================
   Mean synergy: 0.0681 bits
   Median synergy: 0.0663 bits
   Max synergy: 0.0875 bits

   Effect size (signal/noise): 3.17
     LARGE effect size

  Rule of thumb: Don't just report p-values!
   Always report effect sizes for meaningful interpretation.
```

## 6.9 Part 8: Advanced Topics and Optimizations

### 6.9.1 Computational Optimization Strategies

For large-scale analyses (50+ neurons, thousands of trials), computation can be prohibitive. Here are strategies:

```
print("ADVANCED OPTIMIZATION STRATEGIES")
print("=" * 70)

print("\n1. HIERARCHICAL ANALYSIS")
print("   Strategy: Screen with fast measures, then detailed analysis")
print("\n   Step 1: Compute pairwise MI (fast)")
print("   Step 2: Select pairs with MI > threshold")
print("   Step 3: Only test triplets containing strong pairs")
print("   → Reduces triplets by ~90%")

# Demonstrate
print("\n   Example on our data:")

# Get all pairwise MIs using TC (for 2 variables, TC = MI)
pairs = list(combinations(range(sim.n_neurons), 2))
pairwise_mi = []

for pair in pairs:
    # Extract pair data: (n_trials, 2)
    pair_data = activity_decision[:, list(pair)]
    # Use TC to compute MI between the pair
    model_pair = TC(pair_data)
    mi_pair = float(model_pair.fit(method='gc')[0])
    pairwise_mi.append((pair, mi_pair))

# Filter pairs
mi_threshold = 0.1  # bits
strong_pairs = [pair for pair, mi in pairwise_mi if mi > mi_threshold]

# Only test triplets containing at least one strong pair
filtered_triplets = []
for triplet in all_triplets:
    # Check if any pair in this triplet is strong
    triplet_pairs = list(combinations(triplet, 2))
```

```
    if any(pair in strong_pairs for pair in triplet_pairs):
        filtered_triplets.append(triplet)

print(f"   Original triplets: {len(all_triplets)}")
print(f"   After filtering: {len(filtered_triplets)}")
print(f"   Reduction: {(1 - len(filtered_triplets)/len(all_triplets))*100:.1f}%")
print(f"   → {len(filtered_triplets)} is much more manageable!")

print("\n" + "="*70)
print("\n2. TIME WINDOW SELECTION")
print("   Strategy: Use Frites to identify significant time windows")
print("\n   Step 1: Compute time-resolved MI (Frites)")
print("   Step 2: Find windows with p < 0.05")
print("   Step 3: Only run HOI on significant windows")
print("   → Focuses computation on informative periods")

print("\n" + "="*70)
print("\n3. PARALLEL PROCESSING")
print("   Both HOI and Frites support parallelization:")
print("   - HOI: Built on JAX (can use GPU)")
print("   - Frites: n_jobs parameter")
print("   - Use n_jobs=-1 to use all CPU cores")

print("\n   Example speedup (estimated):")
print("   - 1 core: 10 minutes")
print("   - 8 cores: ~2-3 minutes")
print("   - GPU (HOI): ~30 seconds")

print("\n" + "="*70)
print("\n4. SAMPLING STRATEGIES")
print("   For huge systems (100+ neurons):")
print("   - Random sample of multiplets")
print("   - Stratified sampling (ensure area coverage)")
print("   - Iterative refinement (sample, find hubs, resample around hubs)")
```

```
ADVANCED OPTIMIZATION STRATEGIES
======================================================================

1. HIERARCHICAL ANALYSIS
```

```
   Strategy: Screen with fast measures, then detailed analysis

   Step 1: Compute pairwise MI (fast)
   Step 2: Select pairs with MI > threshold
   Step 3: Only test triplets containing strong pairs
   → Reduces triplets by ~90%

   Example on our data:

   Original triplets: 220
   After filtering: 220
   Reduction: 0.0%
   → 220 is much more manageable!

======================================================================

2. TIME WINDOW SELECTION
   Strategy: Use Frites to identify significant time windows

   Step 1: Compute time-resolved MI (Frites)
   Step 2: Find windows with p < 0.05
   Step 3: Only run HOI on significant windows
   → Focuses computation on informative periods

======================================================================

3. PARALLEL PROCESSING
   Both HOI and Frites support parallelization:
   - HOI: Built on JAX (can use GPU)
   - Frites: n_jobs parameter
   - Use n_jobs=-1 to use all CPU cores

   Example speedup (estimated):
   - 1 core: 10 minutes
   - 8 cores: ~2-3 minutes
   - GPU (HOI): ~30 seconds

======================================================================

4. SAMPLING STRATEGIES
```

```
For huge systems (100+ neurons):
- Random sample of multiplets
- Stratified sampling (ensure area coverage)
- Iterative refinement (sample, find hubs, resample around hubs)
```

### 6.9.2 Publication-Quality Figure

```
print("VALIDATION ANALYSES")
print("=" * 70)

# Control 1: Shuffled data (should destroy synergy)
print("\n1. CONTROL: Shuffled Trial Labels")
print("   Purpose: Verify synergy isn't due to chance correlations")

# activity_decision has shape (n_trials, n_neurons)
# Shuffle trial order for each neuron independently
activity_shuffled = activity_decision.copy()
n_trials, n_neurons = activity_shuffled.shape

for neuron in range(n_neurons):
    shuffle_idx = np.random.permutation(n_trials)
    activity_shuffled[:, neuron] = activity_shuffled[shuffle_idx, neuron]

# IMPORTANT: Create a NEW Oinfo model with shuffled data
model_shuffled = Oinfo(activity_shuffled)
oinfo_shuffled = model_shuffled.fit(method="gc", minsize=3, maxsize=3)

synergistic_shuffled = sum(1 for oi in oinfo_shuffled if float(oi) < synergy_threshol

print(f"\n   Real data: {len(synergistic)} synergistic triplets")
print(f"   Shuffled:  {synergistic_shuffled} synergistic triplets")
print(f"   Reduction: {(1 - synergistic_shuffled/max(1, len(synergistic)))*100:.1f}%"

if synergistic_shuffled < len(synergistic) * 0.2:
    print("\n    PASS: Shuffling destroys most synergies")
    print("     Synergy is real, not spurious correlation!")
else:
    print("\n     WARNING: Many synergies remain after shuffling")
    print("     May indicate artifacts or threshold issues")
```

```
VALIDATION ANALYSES
======================================================================

1. CONTROL: Shuffled Trial Labels
   Purpose: Verify synergy isn't due to chance correlations

   Real data: 7 synergistic triplets
   Shuffled:  0 synergistic triplets
   Reduction: 100.0%

     PASS: Shuffling destroys most synergies
      Synergy is real, not spurious correlation!
```

## 6.10 Part 9: Research Guidelines and Best Practices

### 6.10.1 Complete Analysis Checklist

Based on everything we've learned, here's a checklist for multivariate information theory analyses:

```
print("RESEARCH GUIDELINES: Complete Analysis Checklist")
print("=" * 70)

checklist = """
BEFORE ANALYSIS:
  Sufficient data (80+ trials per condition recommended)
  Data preprocessed (filtered, artifacts removed)
  Task variables clearly defined
  Hypotheses specified a priori
  Analysis plan documented

DURING FRITES ANALYSIS:
  Correct mi_type selected (cd, cc, or ccd)
  FFX vs RFX chosen appropriately
  Sufficient permutations (1000+ for publication)
  MCP method selected (cluster recommended for time series)
  Results inspected for artifacts
  Effect sizes evaluated (not just p-values)
```

```
DURING HOI ANALYSIS:
  Correct data format (n_features, n_samples)
  Appropriate estimator (gc recommended)
  Focused on significant time windows (from Frites)
  Thresholds justified (not arbitrary)
  Computational feasibility checked
  Results consistent across estimators

DURING XGI ANALYSIS:
  Hypergraph correctly constructed from HOI results
  Node/edge attributes meaningful
  Network statistics computed and interpreted
  Visualization clearly conveys structure
  Topology related to function

VALIDATION:
  Shuffled control performed
  Bootstrap confidence intervals computed
  Effect sizes quantified
  Sensitivity to parameters tested
  Biological plausibility assessed
  Alternative explanations considered

REPORTING:
  Methods described in detail
  All thresholds and parameters reported
  Both positive and negative findings
  Effect sizes alongside p-values
  Limitations acknowledged
  Code and data shared (if possible)
"""

print(checklist)

print("\n" + "="*70)
print("  This checklist ensures rigorous, reproducible analyses")
print("   Adapt it to your specific research context")
```

```
RESEARCH GUIDELINES: Complete Analysis Checklist
```

```
======================================================================

BEFORE ANALYSIS:
  Sufficient data (80+ trials per condition recommended)
  Data preprocessed (filtered, artifacts removed)
  Task variables clearly defined
  Hypotheses specified a priori
  Analysis plan documented

DURING FRITES ANALYSIS:
  Correct mi_type selected (cd, cc, or ccd)
  FFX vs RFX chosen appropriately
  Sufficient permutations (1000+ for publication)
  MCP method selected (cluster recommended for time series)
  Results inspected for artifacts
  Effect sizes evaluated (not just p-values)

DURING HOI ANALYSIS:
  Correct data format (n_features, n_samples)
  Appropriate estimator (gc recommended)
  Focused on significant time windows (from Frites)
  Thresholds justified (not arbitrary)
  Computational feasibility checked
  Results consistent across estimators

DURING XGI ANALYSIS:
  Hypergraph correctly constructed from HOI results
  Node/edge attributes meaningful
  Network statistics computed and interpreted
  Visualization clearly conveys structure
  Topology related to function

VALIDATION:
  Shuffled control performed
  Bootstrap confidence intervals computed
  Effect sizes quantified
  Sensitivity to parameters tested
  Biological plausibility assessed
  Alternative explanations considered
```

```
REPORTING:
  Methods described in detail
  All thresholds and parameters reported
  Both positive and negative findings
  Effect sizes alongside p-values
  Limitations acknowledged
  Code and data shared (if possible)


======================================================================
  This checklist ensures rigorous, reproducible analyses
    Adapt it to your specific research context
```

### 6.10.2 Common Pitfalls and How to Avoid Them

```
print("\nCOMMON PITFALLS IN MULTIVARIATE INFORMATION ANALYSIS")
print("=" * 70)

pitfalls = [
    {
        'pitfall': 'Insufficient sample size',
        'symptom': 'High variance in estimates, unreliable synergy detection',
        'solution': 'Use bootstrap to assess stability, collect more data',
        'minimum': '80+ trials per condition'
    },
    {
        'pitfall': 'No multiple comparison correction',
        'symptom': 'Many false positives, results don\'t replicate',
        'solution': 'Always use MCP (cluster, FDR, etc.)',
        'minimum': 'Required for any multi-test analysis'
    },
    {
        'pitfall': 'Ignoring effect sizes',
        'symptom': 'Significant but meaningless results',
        'solution': 'Report magnitude, not just p-values',
        'minimum': 'Effect size > 0.5 for meaningful interpretation'
    },
    {
        'pitfall': 'Wrong data format',
```

```
        'symptom': 'Errors or nonsensical results',
        'solution': 'Double-check: HOI (features, samples), Frites (epochs, channels,
        'minimum': 'Verify shapes before computing'
    },
    {
        'pitfall': 'No validation controls',
        'symptom': 'Can\'t distinguish real from artifact',
        'solution': 'Run shuffled controls, bootstrap CIs',
        'minimum': 'At least one control analysis'
    },
    {
        'pitfall': 'Over-interpreting synergy',
        'symptom': 'Claiming causation from correlation',
        'solution': 'Remember: MI is symmetric, doesn\'t imply direction',
        'minimum': 'Use careful language ("associated", not "causes")'
    }
]

for i, p in enumerate(pitfalls, 1):
    print(f"\n{i}. {p['pitfall'].upper()}")
    print(f"   Symptom: {p['symptom']}")
    print(f"   Solution: {p['solution']}")
    print(f"   Guideline: {p['minimum']}")

print("\n" + "="*70)
print("  Awareness of these pitfalls prevents common mistakes!")
```

```
COMMON PITFALLS IN MULTIVARIATE INFORMATION ANALYSIS
======================================================================

1. INSUFFICIENT SAMPLE SIZE
   Symptom: High variance in estimates, unreliable synergy detection
   Solution: Use bootstrap to assess stability, collect more data
   Guideline: 80+ trials per condition

2. NO MULTIPLE COMPARISON CORRECTION
   Symptom: Many false positives, results don't replicate
   Solution: Always use MCP (cluster, FDR, etc.)
   Guideline: Required for any multi-test analysis
```

```
3. IGNORING EFFECT SIZES
   Symptom: Significant but meaningless results
   Solution: Report magnitude, not just p-values
   Guideline: Effect size > 0.5 for meaningful interpretation

4. WRONG DATA FORMAT
   Symptom: Errors or nonsensical results
   Solution: Double-check: HOI (features, samples), Frites (epochs, channels, times)
   Guideline: Verify shapes before computing

5. NO VALIDATION CONTROLS
   Symptom: Can't distinguish real from artifact
   Solution: Run shuffled controls, bootstrap CIs
   Guideline: At least one control analysis

6. OVER-INTERPRETING SYNERGY
   Symptom: Claiming causation from correlation
   Solution: Remember: MI is symmetric, doesn't imply direction
   Guideline: Use careful language ("associated", not "causes")

======================================================================
 Awareness of these pitfalls prevents common mistakes!
```

## 6.11 Part 10: Final Summary and Conclusions

### 6.11.1 The Complete Journey

Congratulations! You've completed all six notebooks and mastered multivariate information theory analysis. Let's recap the complete journey:

```
print("\n" + "="*70)
print("COMPLETE NOTEBOOK SERIES SUMMARY")
print("="*70)

summary = """
 NOTEBOOK 1: Information Theory Foundations
    Built entropy, MI, CMI from scratch
    Understood estimation challenges
    Developed core intuitions
```

```
 NOTEBOOK 2: The XOR Problem
    Discovered 0 + 0 = 1 (synergy!)
    Saw why Venn diagrams break
    Learned interaction information
    Contrasted XOR (synergy) vs COPY (redundancy)

 NOTEBOOK 3: HOI Package
    Computed O-information professionally
    Applied PID decomposition
    Analyzed neural population data
    Understood computational scaling

 NOTEBOOK 4: Frites Package
    Time-resolved mutual information
    Statistical testing with permutations
    Multiple comparison correction
    Dynamic functional connectivity
    Multi-subject analysis (FFX vs RFX)

 NOTEBOOK 5: XGI Package
    Hypergraph representation
    Network topology analysis
    Temporal network sequences
    Hub identification
    Real-world applications

 NOTEBOOK 6: Complete Integration
    End-to-end analysis pipeline
    Realistic decision-making experiment
    HOI + Frites + XGI working together
    Validation and control analyses
    Publication-quality outputs
    Best practices and guidelines
"""

print(summary)

print("="*70)
print("\n  YOU'VE MASTERED MULTIVARIATE INFORMATION THEORY!")
```

```
print("="*70)
```

```
======================================================================
COMPLETE NOTEBOOK SERIES SUMMARY
======================================================================

  NOTEBOOK 1: Information Theory Foundations
     Built entropy, MI, CMI from scratch
     Understood estimation challenges
     Developed core intuitions

  NOTEBOOK 2: The XOR Problem
     Discovered 0 + 0 = 1 (synergy!)
     Saw why Venn diagrams break
     Learned interaction information
     Contrasted XOR (synergy) vs COPY (redundancy)

  NOTEBOOK 3: HOI Package
     Computed O-information professionally
     Applied PID decomposition
     Analyzed neural population data
     Understood computational scaling

  NOTEBOOK 4: Frites Package
     Time-resolved mutual information
     Statistical testing with permutations
     Multiple comparison correction
     Dynamic functional connectivity
     Multi-subject analysis (FFX vs RFX)

  NOTEBOOK 5: XGI Package
     Hypergraph representation
     Network topology analysis
     Temporal network sequences
     Hub identification
     Real-world applications

  NOTEBOOK 6: Complete Integration
     End-to-end analysis pipeline
```

```
    Realistic decision-making experiment
    HOI + Frites + XGI working together
    Validation and control analyses
    Publication-quality outputs
    Best practices and guidelines

======================================================================

 YOU'VE MASTERED MULTIVARIATE INFORMATION THEORY!
======================================================================
```

### 6.11.2 The Integrated Toolkit

You now have three powerful tools that work together:

**HOI** - Structure Discovery - Identifies synergistic ensembles - Quantifies redundancy - Decomposes information (PID) - Fast, GPU-enabled

**Frites** - Temporal Dynamics & Statistics - Time-resolved measures - Rigorous significance testing - Dynamic connectivity - Multi-subject designs

**XGI** - Network Topology - Represents higher-order structure - Analyzes network properties - Identifies hubs and modules - Tracks temporal evolution

### 6.11.3 When to Use Each

**Start with Frites if**: - You have time series data - Need to know WHEN effects occur - Have multi-subject design - Want built-in statistics

**Add HOI when**: - Frites shows significant effects - Want to understand WHY (synergy vs redundancy) - Need detailed decomposition - Have computational resources

**Use XGI for**: - Visualizing discovered structure - Network-level insights - Identifying critical nodes - Comparing across conditions/subjects

### 6.11.4 The Typical Workflow

```
# 1. Preprocess
data_clean = preprocess(raw_data)

# 2. Frites: When and where?
ds = DatasetEphy(data_clean, task_vars, roi_names, times)
```

```
wf = WfMi(mi_type='cd', inference='rfx')
mi, pv = wf.fit(ds, n_perm=1000, mcp='cluster')
significant_windows = find_significant_windows(mi, pv)

# 3. HOI: What structure?
for window in significant_windows:
    activity = extract_window(data_clean, window)
    oinfo = Oinfo(method='gc')
    results = oinfo.fit(activity, multiplets=all_triplets)
    synergistic = filter_synergistic(results)

# 4. XGI: How organized?
H = build_hypergraph(synergistic)
hubs = identify_hubs(H)
modules = detect_modules(H)

# 5. Validate
run_controls(data_clean)
compute_confidence_intervals()

# 6. Interpret
biological_interpretation(hubs, modules, timing)
```

## 6.12 Conclusion: From Theory to Discovery

### 6.12.1 What You Can Now Do

You've acquired a complete skillset for multivariate information analysis:

**Theoretical Understanding**: - Deep grasp of entropy, MI, and higher-order measures - Intuition for synergy vs redundancy - Knowledge of when pairwise analysis fails

**Technical Skills**: - HOI for structure discovery - Frites for temporal dynamics - XGI for network analysis - Integration of all three

**Research Practices**: - Validation and controls - Statistical rigor - Publication standards - Computational optimization

### 6.12.2 Where to Go From Here

**Apply to Your Data**: 1. Start with small pilot analyses 2. Validate methods on synthetic data 3. Scale up to full datasets 4. Iterate and refine

**Extend Your Knowledge**: - Transfer entropy (directed information) - Information geometry - Rate-distortion theory - Integrated Information Theory (IIT)

**Join the Community**: - Computational neuroscience conferences - Package-specific forums (HOI, Frites, XGI) - Information theory workshops - Share your analyses!

### 6.12.3 Final Thoughts

Multivariate information theory reveals hidden structure in neural data that pairwise methods miss. Synergistic ensembles, redundant coding, higher-order interactions - these aren't just mathematical curiosities. They reflect the fundamental computational principles of neural circuits.

By combining HOI, Frites, and XGI, you can: - Discover how neurons work together - Track information flow through time - Identify critical network elements - Connect structure to function

**The journey from entropy to hypergraphs has equipped you to ask - and answer - deeper questions about brain function.**

Good luck with your research!

## 6.13 Appendix: Quick Reference

### 6.13.1 Essential Code Snippets

```
print("QUICK REFERENCE: Essential Code Snippets")
print("=" * 70)

reference = """
# === HOI ===
from hoi.metrics import Oinfo
model = Oinfo(data)
oinfo = model.fit(method="gc", minsize=3, maxsize=3)  # data: (n_samples, n_features)
```

```
# === FRITES ===
from frites.dataset import DatasetEphy
from frites.workflow import WfMi

ds = DatasetEphy(x=data, y=task, roi=names, times=time)
wf = WfMi(mi_type='cc', inference='rfx')
mi, pv = wf.fit(ds, n_perm=1000, mcp='cluster')

# === XGI ===
import xgi
H = xgi.Hypergraph()
H.add_edges_from([[0,1,2], [2,3,4]])
xgi.draw(H, hull=True)

# === INTEGRATION ===
# 1. Frites: Find significant time windows
sig_windows = time[pv.values < 0.05]

# 2. HOI: Detect synergies in those windows
for window in sig_windows:
    activity = extract_window(data, window)
    oinfo = compute_oinfo(activity)
    synergies = filter_synergistic(oinfo)

# 3. XGI: Build and analyze network
H = build_hypergraph(synergies)
hubs = identify_hubs(H)
visualize(H)
"""

print(reference)

print("\n" + "="*70)
print("Save this reference for quick access during your analyses!")
print("="*70)
```

```
QUICK REFERENCE: Essential Code Snippets
======================================================================
```

```
# === HOI ===
from hoi.metrics import Oinfo
model = Oinfo(data)
oinfo = model.fit(method="gc", minsize=3, maxsize=3)  # data: (n_samples, n_features)

# === FRITES ===
from frites.dataset import DatasetEphy
from frites.workflow import WfMi

ds = DatasetEphy(x=data, y=task, roi=names, times=time)
wf = WfMi(mi_type='cc', inference='rfx')
mi, pv = wf.fit(ds, n_perm=1000, mcp='cluster')

# === XGI ===
import xgi
H = xgi.Hypergraph()
H.add_edges_from([[0,1,2], [2,3,4]])
xgi.draw(H, hull=True)

# === INTEGRATION ===
# 1. Frites: Find significant time windows
sig_windows = time[pv.values < 0.05]

# 2. HOI: Detect synergies in those windows
for window in sig_windows:
    activity = extract_window(data, window)
    oinfo = compute_oinfo(activity)
    synergies = filter_synergistic(oinfo)

# 3. XGI: Build and analyze network
H = build_hypergraph(synergies)
hubs = identify_hubs(H)
visualize(H)


======================================================================
Save this reference for quick access during your analyses!
======================================================================
```

## 6.14 Further Resources

### 6.14.1 Essential Papers

**Foundations**: 1. Cover & Thomas (2006) - Elements of Information Theory 2. Timme & Lapish (2018) - Tutorial for information theory in neuroscience

**Higher-Order Interactions**: 3. Williams & Beer (2010) - Nonnegative decomposition of multivariate information 4. Schneidman et al. (2003) - Synergy, redundancy, and independence in population codes 5. Rosas et al. (2019) - Quantifying high-order interdependencies via multivariate extensions

**Methods**: 6. Ince et al. (2017) - Gaussian copula mutual information 7. Combrisson et al. (2022) - Group-level inference (Frites methodology) 8. Battiston et al. (2020) - Networks beyond pairwise interactions

### 6.14.2 Software Documentation

- **HOI**: https://brainets.github.io/hoi/
- **Frites**: https://brainets.github.io/frites/
- **XGI**: https://xgi.readthedocs.io/

### 6.14.3 Community and Support

- GitHub Issues for bug reports
- Neurostars for neuroscience questions
- Stack Overflow for coding questions
- Twitter: Follow[kNearNeighbors?] (BraiNets team)

### 6.14.4 Citation

If you use these packages in your research, please cite:

```
@software{hoi2023,
  title={HOI: Higher-Order Interactions},
  author={Combrisson, E. and others},
  year={2023},
  url={https://github.com/brainets/hoi}
}

@article{frites2022,
  title={Group-level inference of information-based measures},
```

```
  author={Combrisson, E. and others},
  journal={bioRxiv},
  year={2022}
}

@article{xgi2023,
  title={XGI: A Python package for higher-order interaction networks},
  author={Landry, N.W. and others},
  journal={Journal of Open Source Software},
  year={2023}
}
```

---

## 6.15 Thank you for completing this series!

You're now equipped to discover higher-order structure in neural data and contribute to our understanding of how brains process information.

**May your synergies be strong and your p-values be small!**

Happy researching!

# References and Further Reading

## Key Papers

### Information Theory Foundations

Shannon, C. E. (1948). A Mathematical Theory of Communication. *Bell System Technical Journal*, 27(3), 379-423.

Cover, T. M., & Thomas, J. A. (2006). *Elements of Information Theory* (2nd ed.). Wiley-Interscience.

MacKay, D. J. (2003). *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press.

### Higher-Order Interactions

Williams, P. L., & Beer, R. D. (2010). Nonnegative decomposition of multivariate information. *arXiv preprint arXiv:1004.2515*.

Timme, N., Alford, W., Flecker, B., & Beggs, J. M. (2014). Synergy, redundancy, and multivariate information measures: an experimentalist's perspective. *Journal of Computational Neuroscience*, 36(2), 119-140.

Rosas, F. E., Mediano, P. A., Gastpar, M., & Jensen, H. J. (2019). Quantifying high-order interdependencies via multivariate extensions of the mutual information. *Physical Review E*, 100(3), 032305.

Varley, T. F., & Hoel, E. (2022). Emergence as the conversion of information: a unifying theory. *Philosophical Transactions of the Royal Society A*, 380(2227), 20210150.

### Partial Information Decomposition

Williams, P. L., & Beer, R. D. (2010). Nonnegative decomposition of multivariate information. *arXiv:1004.2515*.

Bertschinger, N., Rauh, J., Olbrich, E., Jost, J., & Ay, N. (2014). Quantifying unique information. *Entropy*, 16(4), 2161-2183.

Finn, C., & Lizier, J. T. (2018). Pointwise partial information decomposition using the specificity and ambiguity lattices. *Entropy*, 20(4), 297.

### Neuroscience Applications

Schneidman, E., Berry, M. J., Segev, R., & Bialek, W. (2006). Weak pairwise correlations imply strongly correlated network states in a neural population. *Nature*, 440(7087), 1007-1012.

Ohiorhenuan, I. E., Mechler, F., Purpura, K. P., Schmid, A. M., Hu, Q., & Victor, J. D. (2010). Sparse coding and high-order correlations in fine-scale cortical networks. *Nature*, 466(7306), 617-621.

Panzeri, S., Macke, J. H., Gross, J., & Kayser, C. (2015). Neural population coding: combining insights from microscopic and mass signals. *Trends in Cognitive Sciences*, 19(3), 162-172.

Stringer, C., Pachitariu, M., Steinmetz, N., Reddy, C. B., Carandini, M., & Harris, K. D. (2019). High-dimensional geometry of population responses in visual cortex. *Nature*, 571(7765), 361-365.

Panzeri, S., & Magri, C. (2017). Information-theoretic analysis of neural data. In *Analysis and Modeling of Coordinated Multi-neuronal Activity* (pp. 83-113). Springer.

### Network Science and Hypergraphs

Battiston, F., Cencetti, G., Iacopini, I., Latora, V., Lucas, M., Patania, A., … & Petri, G. (2020). Networks beyond pairwise interactions: structure and dynamics. *Physics Reports*, 874, 1-92.

Iacopini, I., Petri, G., Barrat, A., & Latora, V. (2019). Simplicial models of social contagion. *Nature Communications*, 10(1), 2485.

Petri, G., Expert, P., Turkheimer, F., Carhart-Harris, R., Nutt, D., Hellyer, P. J., & Vaccarino, F. (2014). Homological scaffolds of brain functional networks. *Journal of The Royal Society Interface*, 11(101), 20140873.

Torres, L., Blevins, A. S., Bassett, D., & Eliassi-Rad, T. (2021). The why, how, and when of representations for complex systems. *SIAM Review*, 63(3), 435-485.

# Software Documentation

## Primary Packages

- **HOI (Higher-Order Interactions)**: https://hoi.readthedocs.io/
  - Combrisson, E., et al. (2023). HOI: Higher-Order Interactions package.
- **Frites (Framework for Information Theoretical analysis)**: https://brainets.github.io/frites/
  - Combrisson, E., et al. (2022). Group-level inference of information-based measures for the analyses of cognitive brain networks from neurophysiological data. *NeuroImage*.
- **XGI (Comple(X) Group Interactions)**: https://xgi.readthedocs.io/
  - Landry, N. W., et al. (2023). XGI: A Python package for higher-order interaction networks.

## Supporting Libraries

- **JAX**: https://jax.readthedocs.io/
- **MNE-Python**: https://mne.tools/
- **NetworkX**: https://networkx.org/
- **NumPy**: https://numpy.org/doc/
- **SciPy**: https://docs.scipy.org/

# Online Resources

## Tutorials and Courses

- **Information Theory Primer** by David MacKay: http://www.inference.org.uk/mackay/itila/
- **Network Science Book** by Albert-László Barabási: http://networksciencebook.com/
- **Hypergraph Tutorial** by Renaud Lambiotte: https://arxiv.org/abs/2301.02773

### Research Groups and Centers

- **Network Science Institute, Northeastern University**: Focus on complex systems and network science
- **Santa Fe Institute**: Complex systems and emergence
- **Max Planck Institute for Mathematics in the Sciences**: Information theory and complex systems
- **Centre for Complexity Science (Imperial College)**: Interdisciplinary complexity research

## Related Books

- Sporns, O. (2010). *Networks of the Brain*. MIT Press.
- Dayan, P., & Abbott, L. F. (2005). *Theoretical Neuroscience: Computational and Mathematical Modeling of Neural Systems*. MIT Press.
- Borst, A., & Theunissen, F. E. (1999). Information theory and neural coding. *Nature Neuroscience*, 2(11), 947-957.
- Newman, M. (2018). *Networks* (2nd ed.). Oxford University Press.

## Software Ecosystem

### Related Python Packages

- **InfoTopo**: Information topology and persistent homology
- **JIDT (Java Information Dynamics Toolkit)**: Information-theoretic measures
- **PyInform**: Information theory for discrete data
- **dit (Discrete Information Theory)**: Python package for information theory on discrete random variables
- **MuTE (Multivariate Transfer Entropy)**: Transfer entropy for MATLAB

## Contact and Community

- **GitHub Discussions**: Ask questions and share results
- **Stack Overflow**: Tag questions with `information-theory`, `neuroscience`, `network-science`
- **Twitter**: Follow #NetworkScience #InfoTheory #CompNeuro

---

**Keeping Current**

This field is rapidly evolving. Check the documentation sites regularly for updates to HOI, Frites, and XGI. New methods and estimators are frequently added.

# Installation Guide

This guide provides detailed installation instructions for all packages used in the tutorial series.

## System Requirements

**Minimum:** - Python 3.8 or higher - 8GB RAM - 2GB free disk space

**Recommended:** - Python 3.10 or higher - 16GB RAM - 5GB free disk space - CUDA-compatible GPU (optional, for faster HOI computations)

## Installation Methods

### Method 1: Quick Install (Recommended for Most Users)

This method installs all packages at once using pip:

```
# Clone the repository
git clone https://github.com/yourusername/multivariate-information-theory-tutorial.gi
cd multivariate-information-theory-tutorial

# Create virtual environment
python -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate

# Install all dependencies
pip install -r requirements.txt

# Launch Jupyter
jupyter notebook
```

### Method 2: Conda Environment (Alternative)

If you prefer conda:

```
# Create conda environment
conda create -n infotheory python=3.10
conda activate infotheory

# Install core packages
conda install numpy scipy matplotlib seaborn pandas jupyter

# Install specialized packages via pip
pip install hoi frites xgi jax jaxlib mne networkx
```

### Method 3: Step-by-Step Installation

Install packages progressively as you work through the notebooks:

**For Notebooks 1-2 (Foundations):**

```
pip install numpy scipy matplotlib seaborn jupyter
```

**Add for Notebook 3 (HOI):**

```
pip install hoi jax jaxlib
```

**Add for Notebook 4 (Frites):**

```
pip install frites mne
```

**Add for Notebook 5 (XGI):**

```
pip install xgi networkx
```

## Platform-Specific Instructions

### Linux

Standard pip installation works for all packages:

```
pip install -r requirements.txt
```

### macOS

**For Apple Silicon (M1/M2/M3):**

```
# JAX requires special attention
pip install jax-metal  # For GPU acceleration
pip install -r requirements.txt
```

**For Intel Macs:**

```
pip install -r requirements.txt
```

### Windows

**Windows 10/11:**

```
# Install Visual C++ Build Tools first if you encounter compilation errors
pip install -r requirements.txt
```

## GPU Support (Optional but Recommended)

JAX can utilize GPUs for faster computation in HOI.

### NVIDIA GPUs (CUDA)

```
# Install CUDA-enabled JAX
pip install jax[cuda12]  # For CUDA 12.x
# OR
pip install jax[cuda11]  # For CUDA 11.x
```

Check CUDA availability:

```
import jax
print(jax.devices())  # Should show GPU devices
```

### AMD GPUs (ROCm)

```
pip install jax[rocm]
```

### Apple Silicon (Metal)

```
pip install jax-metal
```

# Verifying Installation

## Quick Test Script

Save as `test_installation.py`:

```
#!/usr/bin/env python
"""Test script to verify all packages are installed correctly."""

import sys

def test_package(name, import_name=None):
    """Test if a package can be imported."""
    if import_name is None:
        import_name = name
    try:
        __import__(import_name)
        print(f" {name}")
        return True
    except ImportError as e:
        print(f" {name}: {e}")
        return False

print("Testing required packages...\n")

packages = [
    ("NumPy", "numpy"),
    ("SciPy", "scipy"),
    ("Matplotlib", "matplotlib"),
    ("Seaborn", "seaborn"),
    ("Pandas", "pandas"),
    ("Jupyter", "jupyter"),
    ("HOI", "hoi"),
    ("JAX", "jax"),
    ("Frites", "frites"),
    ("MNE", "mne"),
    ("XGI", "xgi"),
    ("NetworkX", "networkx"),
]

results = [test_package(name, imp) for name, imp in packages]
```

```
print(f"\nPassed: {sum(results)}/{len(results)}")

if all(results):
    print("\n  All packages installed successfully!")
    print("You're ready to start the tutorials!")
else:
    print("\n   Some packages failed to install.")
    print("Please review the error messages above.")
    sys.exit(1)
```

Run it:

```
python test_installation.py
```

### Individual Package Tests

**Test NumPy and SciPy:**

```
import numpy as np
import scipy
print(f"NumPy: {np.__version__}")
print(f"SciPy: {scipy.__version__}")
```

**Test HOI:**

```
import hoi
print(f"HOI version: {hoi.__version__}")

# Test basic functionality
from hoi.metrics import Oinfo
model = Oinfo(X=np.random.randn(3, 100))
print("HOI working correctly!")
```

**Test Frites:**

```
import frites
print(f"Frites version: {frites.__version__}")
```

**Test XGI:**

```
import xgi
print(f"XGI version: {xgi.__version__}")
```

```
# Create simple hypergraph
H = xgi.Hypergraph([[1, 2], [2, 3, 4]])
print(f"Created hypergraph with {H.num_nodes} nodes")
```

# Troubleshooting

## Common Issues

### Issue: "No module named 'jax'"

**Solution:**

```
pip install --upgrade jax jaxlib
```

### Issue: JAX doesn't detect GPU

**Solutions:** 1. Verify CUDA installation: `nvidia-smi` 2. Reinstall with correct CUDA version: `bash pip install --upgrade jax[cuda12]` 3. Check compatibility: https://github.com/google/jax#installation

### Issue: MNE installation fails

**Solution:**

```
# Install dependencies first
conda install -c conda-forge mne
# OR
pip install mne --no-deps
pip install -r requirements.txt
```

### Issue: "Kernel died" in Jupyter

**Possible causes:** - Insufficient memory - Package version conflicts

**Solutions:** 1. Close other applications 2. Create fresh virtual environment 3. Update all packages: `pip install --upgrade -r requirements.txt`

### Issue: Import errors despite successful installation

**Solution:**

```
# Verify Python path
python -c "import sys; print('\n'.join(sys.path))"

# Ensure virtual environment is activated
which python  # Should point to venv/bin/python
```

### Getting Help

If you encounter issues:

1. **Check version compatibility**: Ensure Python 3.8
2. **Update pip**: `pip install --upgrade pip`
3. **Check GitHub Issues**:
   - HOI Issues
   - Frites Issues
   - XGI Issues
4. **Open an issue** in this repository with:
   - Python version: `python --version`
   - OS and version
   - Full error message
   - Output of `pip list`

## Optional: Development Installation

For contributing or development:

```
# Clone with development dependencies
git clone https://github.com/yourusername/repo.git
cd repo

# Install in editable mode
pip install -e .

# Install development dependencies
pip install pytest black flake8 mypy
```

## Next Steps

Once installation is complete:

1. Verify with test script
2. Launch Jupyter: `jupyter notebook`
3. Open Notebook 1
4. Start learning!

---

**Keep Your Environment Updated**

Packages are actively developed. Periodically update:

```
pip install --upgrade -r requirements.txt
```